

Starling

Animated renderings of data structures for teaching

Manual for v1.0.0

Contents

1. Introduction	4
1.1. Installation	4
1.2. Quick start	4
1.3. The shape of the API	6
1.4. One contract, nine structures	7
2. The animation model	7
2.1. What a frame carries	8
2.2. Step kinds and the result	8
2.3. Element keys	8
3. Putting frames on the page	9
3.1. Slides with subslides	10
3.2. Auxiliary state	11
3.3. Reading step metadata	11
4. Theming	11
4.1. The two override paths	12
4.2. Performance: state costs a layout pass	13
4.3. Theme references	13
5. The style vocabulary	13
6. The op command stream	15
6.1. On a graph and on a hash map	17
7. Composing with cetz	17
7.1. anchor and the draw backends	17
7.2. Overlaying a callout on a frame	18
8. Extending starling	19
9. Binary search trees	20
9.1. Static display	20
9.2. Search	20
9.3. Insert	22
9.4. Delete	24
9.5. Rotate	26
9.6. Traversals	28
10. Red-black trees	29
10.1. Insert	30
10.2. Delete	32
10.3. Fix-ups from a hand-built tree	35
11. AVL trees	36

11.1.	Insert	37
11.2.	Delete	40
11.3.	Rotate and fix-up	44
12.	2-3-4 trees	47
12.1.	Two strategies	48
12.2.	Search and delete	51
13.	Tries	55
13.1.	Search	56
13.2.	Insert and delete	61
14.	Graphs	68
14.1.	Construction and layout	68
14.2.	Adjacency tables	68
14.3.	Prim's minimum spanning tree	69
14.4.	Kruskal's minimum spanning tree	71
14.5.	Dijkstra's shortest paths	74
14.6.	Traversals	78
14.7.	Node shapes and sizing	79
14.8.	Optional auto-layout with graphviz	79
15.	Hash maps	80
15.1.	The hash box	81
15.2.	Insertion, by strategy	81
15.3.	Search	86
15.4.	Deletion, tombstones, and a deliberate bug	89
15.5.	Resizing and rehashing	91
15.6.	Palette	93
16.	Linear sorts	94
16.1.	Counting sort	94
16.2.	Radix sort	117
17.	Skip lists	142
17.1.	Search	143
17.2.	Insert and delete	144
18.	Git commit graphs	147
19.	Migrating from 0.3.x	149
20.	API reference	150
20.1.	bst — binary search trees	150
20.2.	rbt — red-black trees	164
20.3.	avl — AVL trees	180
20.4.	b24 — 2-3-4 trees	200
20.5.	trie — tries	215
20.6.	Shared binary-tree operations	225
20.7.	graph — weighted graphs	231
20.8.	hashmap — hash maps	242
20.9.	sort — linear sorts	249
20.10.	skiplist — skip lists	254
20.11.	styles — the semantic style vocabulary	259

20.12. Presentation helpers	261
20.13. aux — auxiliary state strips	266
20.14. The op command stream	268
20.15. Frames and renderers	270
20.16. Snapshots	274
20.17. Styles and style keys	276
20.18. Theme	281
20.19. Drawing utilities	282
20.20. Alt-text helpers	285
20.21. Draw backends	286
20.22. Graph auto-layout	297
20.23. git — the commit-graph DSL	298

1. Introduction

Starling draws data structures, one step at a time, so a lecture can show *how* an algorithm works rather than only what it produced. It ships nine structures — binary search trees, red-black trees, AVL trees, 2-3-4 trees, tries, weighted graphs, hash maps, the linear (distribution) sorts, and skip lists — plus a DSL for git commit graphs, and it is built on [cetz](#). Slide decks are a first-class target, but starling has no dependency on [touying](#): every animation is a plain array of values you place however your document needs.

Every animation in starling is the same thing: an ordered array of **frames**. A frame carries a builder that draws one moment of the structure, a caption, step metadata, and alt text. Helpers collapse that array into a final image, a vertical stack, a per-subslide sequence, or whatever you assemble yourself.

Coming from 0.3.x? Version 1.0.0 is a rewrite of the public API: plain dictionaries instead of classes, one namespace per data structure, and one theme. Nothing from 0.3 keeps working. The [migration chapter](#) maps every old name to its replacement.

1.1. Installation

```
#import "@preview/starling:1.0.0" as starling
```

Typst 0.14 or later is required. The graphviz auto-layout helper pulls `@preview/diagraph-layout` — but only if you call it (Section 14.8), so projects that place their graphs by hand never fetch it.

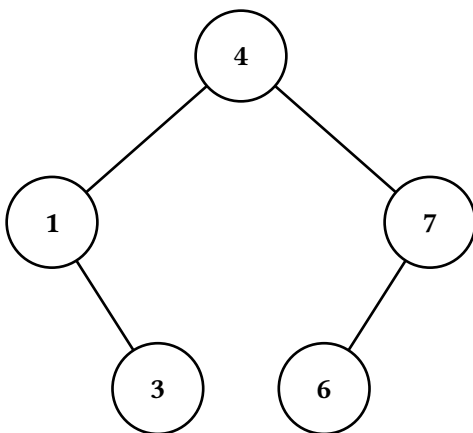
1.2. Quick start

```
#import "@preview/starling:1.0.0" as starling
#import starling: bst, last, stacked

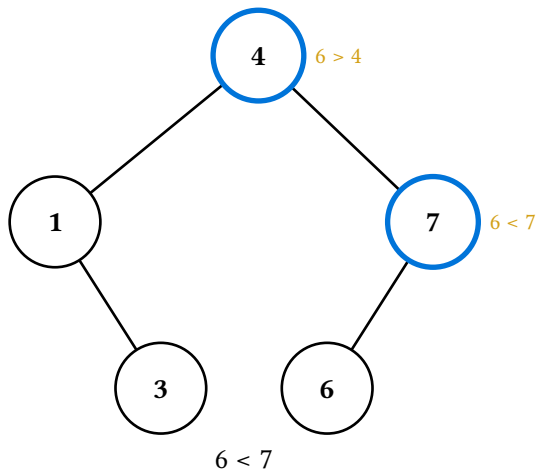
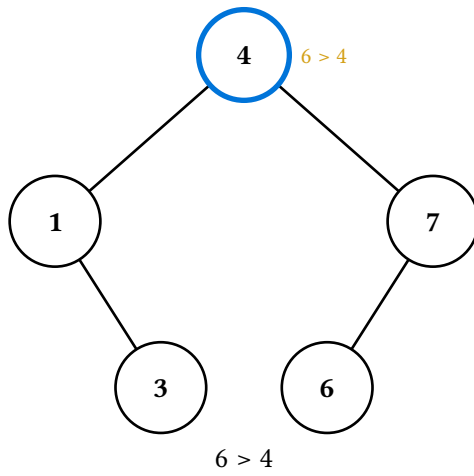
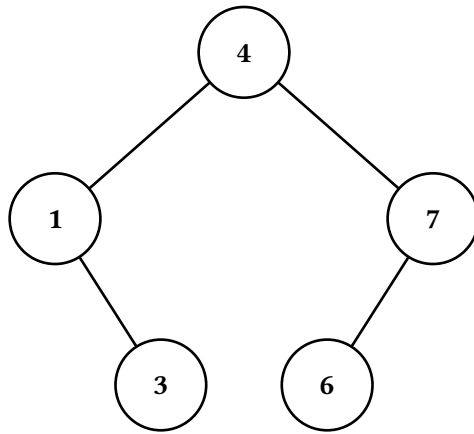
#let t = bst.new(4, 1, 7, 3, 6)
```

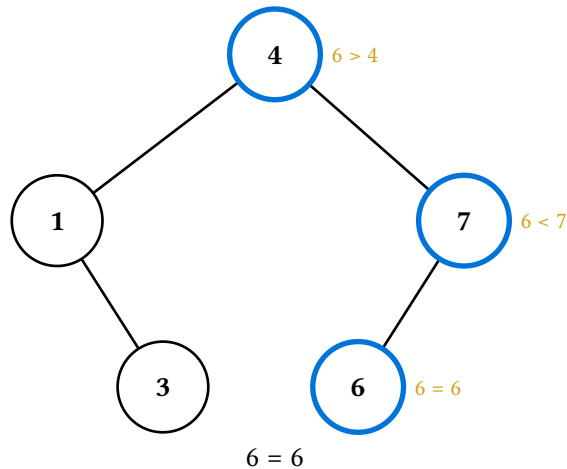
The first value becomes the root; the rest are inserted in order. Every data structure lives in a namespace of plain functions, and the structure is always the first argument.

A static rendering — `display` returns a one-frame array, and `last` takes the final frame of any array:



A search animation, stacked vertically with per-step captions:





And on a slide, one subslide per step:

```
#alternatives(..subslides(bst.search-display(t, 6)))
```

1.3. The shape of the API

Starling exports two kinds of name. **Namespaces** hold everything specific to one structure or vocabulary; the **flat layer** holds what is common to all of them.

Namespace	Holds
bst rbt avl b24 trie	Tree structures: pure operations, literal builders, and the *-display animations.
graph hashmap sort skiplist	The non-tree structures, same contract.
styles	The semantic style vocabulary — attention, success, ghost, ... (Section 5).
aux	Auxiliary-state strips: aux-strip, aux-view-title (Section 3.2).
git	The git commit-graph DSL (Section 18). Its verbs (commit, branch, merge) are too generic to go flat.

The flat layer, grouped by what it is for:

Group	Names
Frames on the page	last, stacked, figures, subslides, canvas, aux-strip
The op stream	style-node, style-edge, annotate, set-alt, set-caption, set-step, commit, apply-ops, blank-snapshot, apply-snapshot
Frames and renderers	make-renderer, render, overlay, result
Theme	default-theme, set-theme, theme-ref, role
Drawing	draw-tree, draw-graph, draw-hashmap, draw-array, draw-skiplist, anchor, auto-layout

That is the whole surface; a conformance test asserts the list exactly, so nothing leaks into it by accident.

1.4. One contract, nine structures

Every data-structure module presents the same names, so knowing one means knowing them all:

Name	Role
<code>new(...)</code>	Build a structure from values. <code>bst.new(4, 1, 7)</code> , <code>trie.new("cat", "car")</code> , <code>hashmap.new(7, strategy: "linear")</code> .
pure operations	<code>insert</code> , <code>delete</code> , <code>contains</code> , ... — each takes the structure first and returns a new structure or a value. Nothing mutates.
<code>describe(s)</code>	A one-line string describing the structure. It is what the animations' alt text opens with.
<code>check-invariants(s)</code>	Returns true, or panics naming the broken invariant.
<code>display(s, ...)</code>	The structure as a single frame.
<code><op>-display(s, ...)</code>	One animation per operation, each returning an array of frames.
<code>renderer(s, ...)</code>	A renderer pre-painted with the structure's own styling, for driving by hand with the op stream (Section 6).
anchor, key helpers	The <code>cetz</code> anchor sanitizer, plus the constructors for that structure's element keys (Section 2.3).

Signature rules hold across all of them: the structure is the first positional argument; `theme`: is always a partial nested theme override (Section 4); `node-style`: and `edge-style`: are the base style layers; and every animation is named `<op>-display` and nothing else is.

2. The animation model

Four shapes do all the work, and all four are plain dictionaries you can inspect, slice, and rebuild.

Shape	Role
Structure	The data — a tree node, a graph, a table. Carries a <code>kind</code> field and nothing else that is starling's business. Every operation returns a new one.
Snapshot	A <i>sparse style overlay</i> for one moment: which elements are filled what colour, which edges are dashed or hidden, which notes hang off which nodes. (<code>nodes: (:)</code> , <code>edges: (:)</code>), keyed by element key. Pure data — no theme, no content.
Renderer	A structure, a draw backend, and an ordered list of snapshots, plus the caption / step / alt metadata for each. What you build up when driving an animation by hand.
Frame	The output record: (<code>builder</code> , <code>caption</code> , <code>step</code> , <code>alt</code> , ...). <code>builder</code> is a function <code>theme => content</code> — not baked content — so one frame array can render against different themes without recomputing the animation. This is what every <code>*-display</code> returns.

The data flow, end to end:

```

structure + snapshots → render(renderer) or a *-display
                      → array(Frame) (builders; theme-agnostic)
                      → a helper resolves the theme once
                      → each (frame.builder)(theme)
                      → document content

```

The shape is load-bearing: because frames are data until the last step, you can pull one out, read its step, swap its caption, thread it into a custom layout, or overlay cetz commands on it, all without rendering anything.

2.1. What a frame carries

Field	Meaning
builder	theme => content. The helpers call it; you rarely do directly.
caption	none or content — the textual narration of this step.
step	Free-form metadata about the step; always has kind (Section 2.2).
alt	Alt text for this step, always an explicitly-set string. The helpers wrap each canvas in an alt-carrying figure, so a deck compiled to PDF/UA keeps its narration.
extra	cetz commands appended inside the frame’s canvas — what overlay (Section 7) appends to.

Captions run on a second channel from the drawing on purpose. A search frame labels the compared node *in the canvas* with the comparison it just made, and *also* carries “6 > 4” as its caption. The inline note makes a single frame readable on its own; the caption is a parallel textual track for layouts that want narration somewhere else on the slide. It is also why `last` hides captions by default while stacked and figures show them.

2.2. Step kinds and the result

`step.kind` says what a frame is showing. A few kinds are universal — `static` (a one-frame display), `init`, `settled` (the terminal success of a mutation), and `found / not-found` (lookup outcomes) — and each structure adds its own where the semantics genuinely differ (`descend`, `probe`, `splice`, `rehash`, `visit`, ...). Each module lists its full set in a comment at the top of its source.

Every animation’s **final** frame also carries `step.result`: the structure the operation produced. That is what `result(frames)` reads, and it is how an animation and the state it leaves behind stay in sync:

```

#let frames = bst.insert-display(t, 5)
#stacked(frames)
#let t = result(frames)    // t now has the 5 in it

```

For searches and traversals — which change nothing — `step.result` is the input structure, so the pattern is uniform.

2.3. Element keys

Every element a backend draws has a **key**: an opaque string that snapshots, op streams, and anchors all use to name it. The alphabet is the structure’s business.

Structure	Key	Meaning
bst rbt avl	<code>"", "L", "RL"</code>	The path from the root in L/R characters. The root is the empty string.
b24	<code>"01", "01#1"</code>	Child indices as digits; an optional <code>#i</code> suffix addresses one key compartment of that node.
trie	<code>"", "c", "ca"</code>	The prefix spelled by the edges down to the node.
graph	<code>"A", graph.edge-key("A", "B")</code>	The node id; edges are <code>"u-v"</code> (undirected) or <code>"u->v"</code> (directed).
hashmap	<code>hashmap.cell-key(3), hashmap.entry-key(3, 1)</code>	Slot 3; entry 1 of bucket 3's chain (which also keys the link into it).
sort	<code>sort.cell-key("count", 5)</code>	Row and column of the cell; <code>sort.entry-key(row, i, j)</code> for a bucket-chain entry.
skiplist	<code>skiplist.box-key(2, 1)</code>	Column 2's lane box at level 1; also <code>data-key(col)</code> and <code>forward-key(col, level)</code> .

An **edge** is keyed by its child in a tree (each child has exactly one parent, so this is unambiguous) and by edge-key in a graph.

Because keys are opaque, an animation written against one structure works on any structure of the same shape: styling is independent of the values in the nodes. And because both the snapshot layer and the anchor layer use the same key, calling out a node in your own cetz code is `anchor(<the key>)` — see Section 7.

3. Putting frames on the page

An animation is an array; these five helpers turn it into content. All of them except `canvas` wrap each canvas in a figure carrying that frame's alt text, which changes nothing visible but keeps the animation narrated for screen readers.

Helper	Gives you
<code>last(frames)</code>	The final frame alone — the static, print form. Takes a lone frame too, so <code>last(frames.at(3))</code> needs no re-wrapping.
<code>stacked(frames)</code>	Every frame stacked down the page with its caption — the handout form.
<code>figures(frames)</code>	An array of figures, one per frame. Splat it into <code>touying's alternatives(...)</code> .

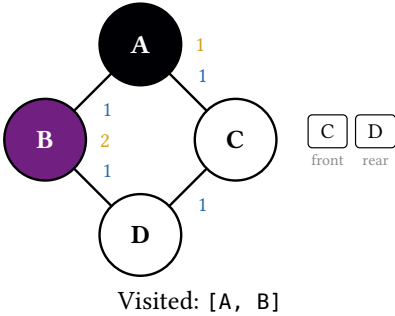
Helper	Gives you
<code>subslides(frames, ..)</code>	An array of compositions — canvas, auxiliary strip, and caption laid out together, one per step (Section 3.1).
<code>canvas(frame)</code>	One frame's bare canvas, with no caption and no alt text , for hand-built layouts where you own the accessibility.

`last`, `stacked`, and `canvas` open one context and resolve the theme once for the whole call. `figures` and `subslides` cannot: touting lays each returned element out independently, so each opens its own.

3.1. Slides with subslides

`subslides` is the one to reach for in a deck. It composes each step's canvas with the algorithm's auxiliary state and the step's caption, so a slide is one call:

```
#alternatives(..subslides(graph.bfs-display(g, "A"), aux: "right"))
```



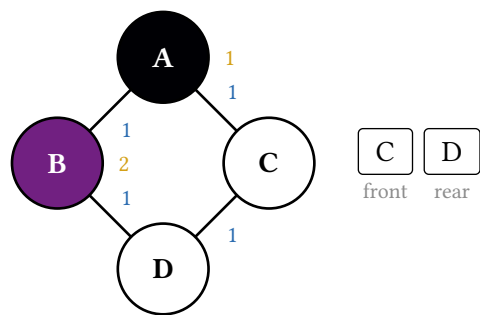
Its arguments: `aux`: places the strip (`none`, `"right"`, `"left"`, or `"below"`), `aux-view`: picks one view when a step carries several, `aux-size`: sets the strip's text size, `caption`: toggles the narration, and `fit`: scales the composition. `fit: 60%` is a plain scale; `fit: (20cm, 12cm)` measures **every** frame in the animation and applies one common factor so the drawing keeps a constant size from subslide to subslide instead of resizing under each step.

Composing that sandwich by hand — a grid of canvas, aux-strip, and `f.caption`, each in its own `alternatives` — is what decks used to do, and it both drops the alt text and lets the three columns drift apart between subslides. If you do want the pieces separately, they still step in lockstep because each `alternatives` gets the same number of children:

```
#let frames = graph.bfs-display(g, "A")
#grid(
  columns: (2fr, 1fr), column-gutter: 2em, align: horizon,
  alternatives(..figures(frames, caption: false)),
  stack(dir: ttb, spacing: 1.2em,
    alternatives(..frames.map(f => aux-strip(f.step))),
    alternatives(..frames.map(f => f.caption)),
  ),
)
```

3.2. Auxiliary state

The graph algorithms carry their bookkeeping — BFS’s queue, DFS’s stack, Prim’s frontier, Kruskal’s sorted edge list and disjoint sets, Dijkstra’s priority queue and its `dist / prev` maps — in each frame’s `step`. `aux-strip(frame.step)` renders it as placeable content beside the canvas:



It is deliberately decoupled from the canvas, so a slide can put the graph on one side and the strip on the other. A frame that carries several views (Kruskal two, Dijkstra three) stacks them all under their headings by default; `view`: selects one, `title`: overrides whether the heading shows, and `labels`: supplies custom display labels per node id. The heading string itself is `aux.aux-view-title(kind)`, for rolling your own.

3.3. Reading step metadata

`frame.step` is ordinary data, so a layout can dispatch on it — colouring the narration by what kind of step it is, for instance:

```
#let kind-colors = (compare: blue, inserted: green)
#let narrated = frames.map(f => stack(dir: ttb, spacing: 0.5em,
  canvas(f),
  text(fill: kind-colors.at(f.step.kind, default: black), f.caption),
))
#alternatives(..narrated)
```

4. Theming

Starling has one theme: a nested dictionary with one section per concern.

Section	Holds
render	Structural defaults for an unstyled structure: <code>node-fill</code> , <code>node-stroke</code> , <code>node-text-fill</code> , <code>edge-stroke</code> , <code>note-fill</code> , <code>edge-tag-fill</code> (persistent edge labels — AVL heights, trie letters, graph weights), <code>note-bg</code> (drawn behind in-canvas annotations so they stay legible over edges; set it to your page colour on a tinted background), and <code>elided-fill</code> / <code>elided-stroke</code> for an elided subtree.
op	Operation-semantic roles shared by every structure: <code>search-stroke</code> , <code>attention-stroke</code> , <code>success-stroke</code> , <code>settled-stroke</code> , <code>success-fill</code> , <code>danger-stroke</code> , <code>reset-stroke</code> , and <code>traversal-palette</code> . BST search and hash-map probing read

Section	Holds
	the same search-stroke — the roles describe operations , not structures.
rbt trie hashmap sort skiplist git	Styling intrinsic to one structure: the red/black palette, the trie's word-end shading, the hash map's tombstones and chains, the sort's count tint, the skip list's sentinels and muted lanes, and the git DSL's branch colours. bst, avl, b24 and graph need none.

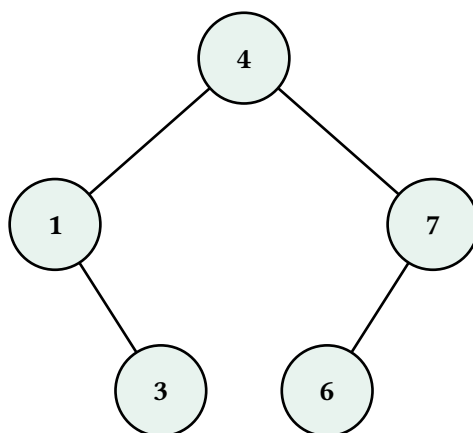
Strokes throughout are full stroke dictionaries — (paint:, thickness:, dash:) — so any aspect can change without the theme growing another key. `default-theme` is the whole thing; read a value out of it with `default-theme.op.attention-stroke`.

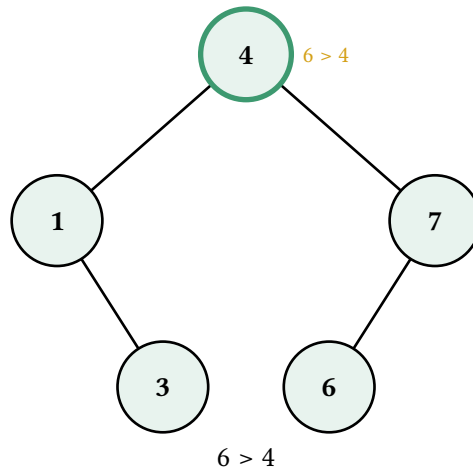
4.1. The two override paths

Both take a **partial** nested dict: only the sections and keys you name change, and an unknown section or key panics so a typo surfaces at once.

```
// Document-wide, from here on:
#set-theme((
  op: (search-stroke: (paint: teal, thickness: 2.5pt)),
  render: (node-fill: yellow.lighten(85%)),
))

// Or for one call only:
#stacked(bst.search-display(t, 6, theme: (op: (search-stroke: (paint: olive,
thickness: 2pt)))))
```





Precedence runs `default-theme ← set-theme state ← a per-call theme: ← the renderer's node-style: / edge-style: layers ← the per-frame snapshot`. A per-call override **layers on** the document's theme rather than replacing it, so naming two keys changes those two and leaves the rest of your palette alone.

4.2. Performance: state costs a layout pass

Typst's state machinery is what makes `set-theme` work: an update later in the document can affect renders earlier in document order, so Typst evaluates, propagates, and re-evaluates everything that observed the state. In practice, **using `set-theme` roughly doubles the compile time of the state-observed part of the document**. The cost is one extra layout pass, not one per read — a thousand reads cost the same as one.

Starling pays that price at most once (there is a single state, where 0.3.x had eight), and only if you call `set-theme` at all. When compile speed matters more than the convenience, hoist a palette into a `let` and pass it per call:

```
#let palette = (op: (search-stroke: (paint: teal, thickness: 2.5pt)))
#stacked(bst.search-display(t, 6, theme: palette))
#stacked(bst.insert-display(t, 5, theme: palette))
```

4.3. Theme references

A style built ahead of time cannot know the theme that will be active when it is drawn — so it stores a **reference** instead of a colour. `role(key)` is a reference into the `op` section and `theme-ref(section, key)` into any section; the frame machinery resolves them just before the backend runs.

```
#style-node("LR", stroke: role("attention-stroke"))
#style-node("LR", fill: theme-ref("rbt", "red-fill"))
```

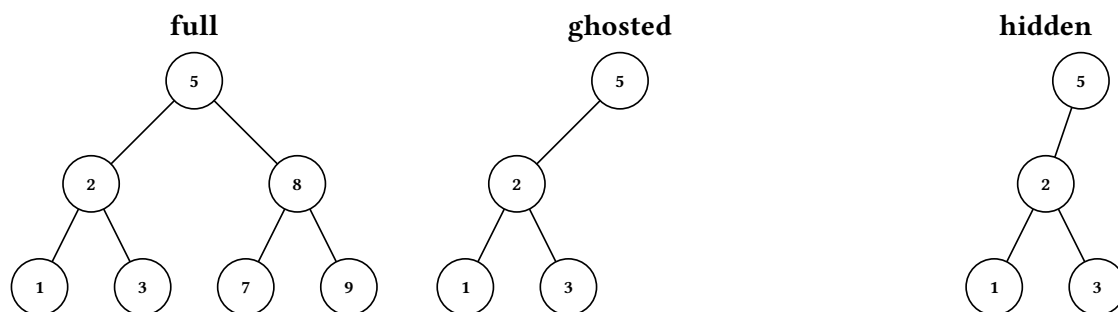
This is what makes the style vocabulary below theme-aware, and it is available to your own `op` streams for the same reason.

5. The style vocabulary

`styles.*` names **intents** rather than colours. Each helper is variadic over element keys and returns an `op` array, so they compose with `+` and drop straight into `apply-ops`:

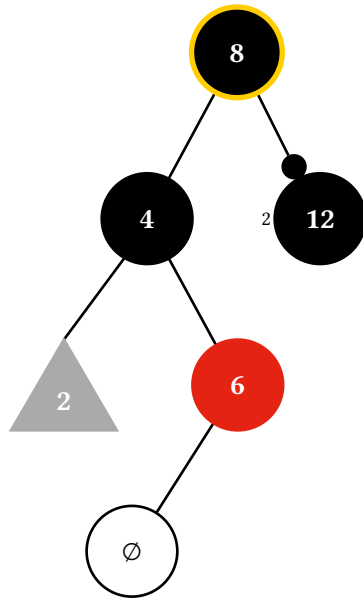
Helper	Effect
<code>styles.attention(..keys)</code>	“Look here” — a rotation pivot, a deletion target, the entry being polled.
<code>styles.search(..keys)</code>	Part of a search or insert walk.
<code>styles.success(..keys)</code>	Settled: the success fill plus the terminal ring.
<code>styles.danger(..keys)</code>	Broken, removed, or rejected.
<code>styles.subtree(..keys)</code>	Elide a subtree: a grey triangle with the incoming edge landing on its apex.
<code>styles.nullify(..keys)</code>	Draw a nil child as a real $\emptyset$ node.
<code>styles.ghost(..keys)</code>	Invisible but space-reserving — the layout is identical to the fully-drawn structure.
<code>styles.hidden(..keys)</code>	Gone entirely, layout space released.
<code>styles.revealed(..keys)</code>	Undo either of those on a sticky frame.
<code>styles.force-show(..keys)</code>	Draw an edge into a phantom (nil) position.

ghost versus hidden is the progressive-reveal decision. Ghosting keeps every element’s exact footprint, so revealing a subtree one node at a time never shifts what is already on the slide; hiding releases the space, so the drawing re-flows around the gap. On a slide you almost always want ghost.



Structure-specific vocabulary lives in that structure’s namespace, where it can name things only that structure has: `rbt.paint-red(..keys)` / `rbt.paint-black(..keys)` recolour nodes from the red-black palette, `rbt.double-black(..keys)` marks the missing-black bookkeeping of a deletion, and `avl.unbalanced(..keys, case: "LL")` rings an imbalanced node with its case.

An elided subtree, a nullified child, and a double-black marker, all on one tree:



6. The op command stream

When an animation doesn't fit any built-in `*-display` — a probe sequence you want narrated your way, a hand-built teaching fixture, a progressive reveal — build the frames yourself. The op stream is a small declarative language for that: each constructor returns an **array** of ops, so streams compose with `+`, and `apply-ops` folds one into a renderer.

Op	Does
<code>style-node(..keys, ..style)</code>	Style one or more nodes. Positional arguments are element keys, named arguments the style — <code>style-node("L", "RR", fill: red)</code> is two ops.
<code>style-edge(..keys, ..style)</code>	The same for edges.
<code>annotate(key, note)</code>	Attach a transient note beside a node.
<code>commit(caption:, step:, alt:)</code>	Close the in-progress frame, attaching metadata, and open a fresh one.
<code>set-caption(c) set-step(s) set-alt(a)</code>	Set one piece of metadata without committing — for the trailing frame, which has no commit after it.

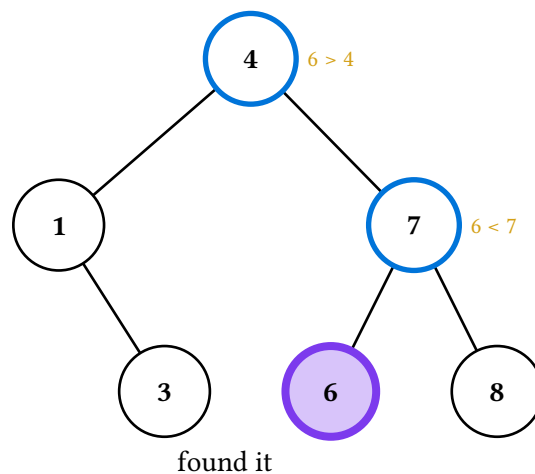
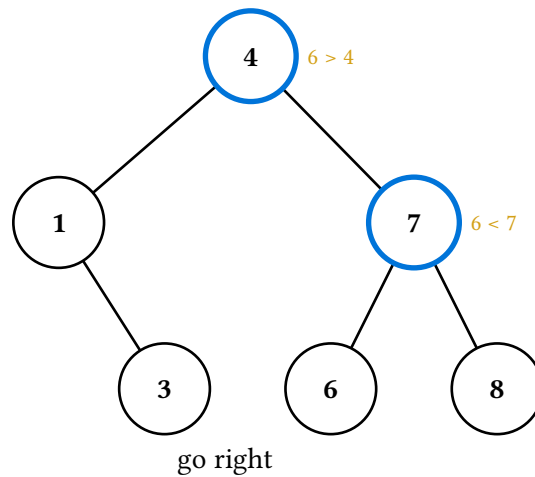
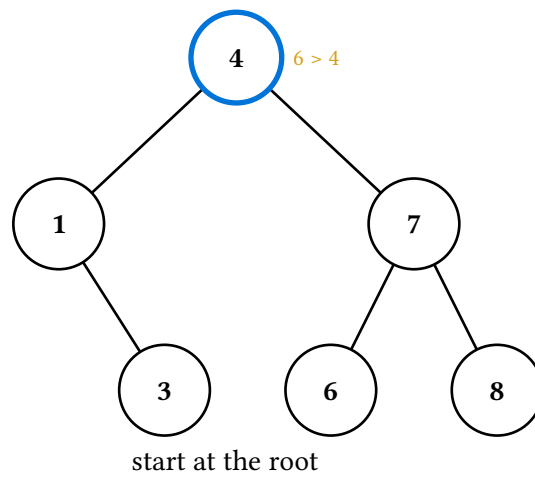
The three pieces: a renderer from the structure's own namespace (so it comes pre-painted with that structure's styling), a stream, and render.

```

#let r = bst.renderer(t, sticky: true)
#let frames = render(apply-ops(r,
  style-node("", stroke: role("search-stroke"))
  + annotate("", [6 > 4])
  + commit(caption: [start at the root], alt: "Comparing 6 against 4."),
))

```

sticky: true is what makes each frame start from the previous one's styling — the accumulate-as-you-go behaviour a walk wants. It is **off** by default, so an animation whose frames each stand alone needs no un-sticking. Captions, steps, and alt text never accumulate; each frame's are its own.



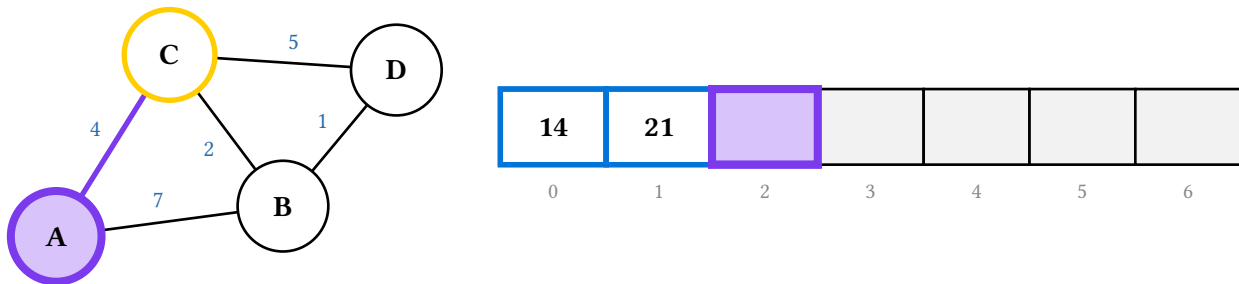
Notice the third batch: `styles.success("RL")` is the vocabulary from Section 5, which is nothing more than a pre-built op array, and the trailing frame closes with `set-caption / set-alt` rather than a `commit`.

A note is transient by design even under `sticky: true` — pass `note: none` in a later batch to clear one that was inherited. There is no “clear everything” op; styling one key at a time is the whole model.

6.1. On a graph and on a hash map

The same stream drives every backend; only the keys change. Each namespace exports the constructors for its own keys, so nothing is spelled by hand.

```
#let r = graph.renderer(g, sticky: true)
#let frames = render(apply-ops(r,
  styles.attention("A")
  + style-edge(graph.edge-key("A", "C"), stroke: role("success-stroke"))
  + commit(caption: [A #sym.arrow C (+4)], alt: "Relaxing A to C."),
))
```



The graph renderer takes the same `positions: / scale: / layout:` arguments its displays do; the hash-map renderer takes `orientation:` and an optional `hash-box:`.

7. Composing with `cetz`

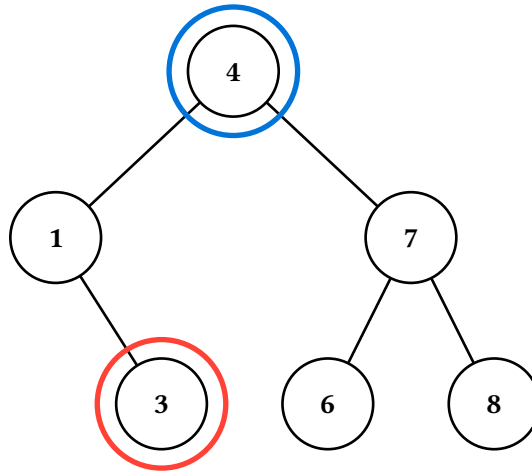
Two escape hatches lead from a starling drawing back to raw `cetz`: drawing a structure inside your own canvas, and appending commands to a frame’s canvas. Both find elements through `anchor(<element key>)`.

7.1. `anchor` and the draw backends

Each `draw-*` backend emits `cetz` drawables **without** wrapping them in a `cetz.canvas`, so you can compose them with your own commands. They read a plain theme: `default` rather than `state`, so no context is needed.

```
#import "@preview/cetz:0.5.2"

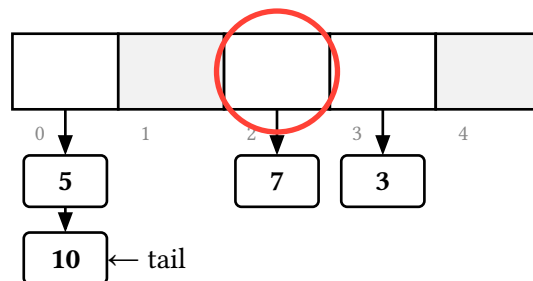
#cetz.canvas({
  starling.draw-tree(t, blank-snapshot())
  // Selective import: cetz.draw has an `anchor` of its own, and a glob
  // import would shadow starling's.
  import cetz.draw: circle
  circle(anchor("LR"), radius: 0.85, stroke: red + 2pt)
})
```



`anchor(key)` is the one sanitizer for every backend: it turns an element key into the cetz element name that backend drew under ("LR" → `e1-LR`, the root → `e1-root`, "c3:1" → `e1-c3-1`). Compass sub-anchors work as usual — `anchor("LR") + ".north"` — and `anchor(key, canvas: "t")` qualifies the name when the drawing sits inside a named group. Pass `name:` to any backend to create that group.

The non-tree backends take the structure's **positioned** form, which is what turns a graph or a table into coordinates:

```
#let h = hashmap.new(5, strategy: "chaining", entries: (5, 10, 7, 3))
#context cetz.canvas({
  starling.draw-hashmap(hashmap.positioned(h), blank-snapshot())
  import cetz.draw: circle, content
  circle(anchor(hashmap.cell-key(2)), radius: 0.8, stroke: red + 2pt)
  content(anchor(hashmap.entry-key(0, 1)) + ".east", anchor: "west", [ #sym.arrow.l
tail])
})
```



7.2. Overlaying a callout on a frame

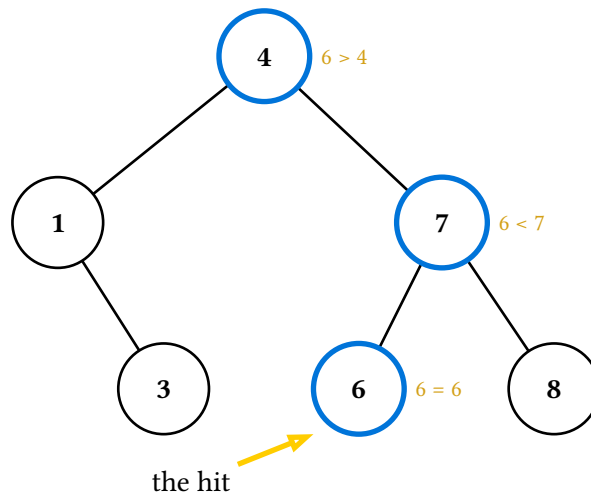
Reaching into an animation to annotate one step used to mean rebuilding its canvas by hand. `overlay(frames, at:, draw:)` appends cetz commands **inside** an existing frame's canvas — so the backend's anchors are in scope — and gives back the frame array:

```
#let frames = overlay(bst.search-display(t, 6), at: -1, draw: theme => {
  import cetz.draw: content, line
  line((rel: (-1.6, -1), to: anchor("RL")), anchor("RL") + ".south-west",
```

```

stroke: theme.op.attention-stroke, mark: (end: ">"))
content((rel: (-1.7, -1.1), to: anchor("RL")), [the hit], anchor: "north-east")
})

```



draw: is either raw cetz commands or a function of the resolved theme, so a callout can wear the same colours the animation does. at: accepts negative indices; -1 is the last frame.

8. Extending starling

A draw backend is a plain function

```

(structure, snapshot, node-style: (:), edge-style: (:), theme: default-theme)
=> cetz commands

```

and `make-renderer(structure, draw, ..)` is the documented way to plug one in. Everything else in the package — the five bundled backends included — goes through the same door: nothing about frames, snapshots, ops, themes, or the presentation helpers knows what a tree is.

```

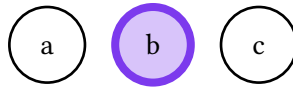
#let draw-blobs(structure, snapshot, node-style: (:), edge-style: (:), theme:
default-theme) = {
  import cetz.draw: circle, content
  for (i, item) in structure.items.enumerate() {
    let style = node-style + snapshot.nodes.at(str(i), default: (:))
    circle((i * 1.4, 0),
      radius: 0.5,
      fill: style.at("fill", default: theme.render.node-fill),
      stroke: style.at("stroke", default: theme.render.node-stroke),
      name: anchor(str(i)))
    content((i * 1.4, 0), item)
  }
}

```

```

#let r = make-renderer((items: ("a", "b", "c")), draw-blobs, sticky: true)
#last(render(apply-ops(r, styles.success("1") + set-alt("The middle blob."))))

```



Two conventions make a custom backend behave like a bundled one. Resolve each element's style as `node-style` $\leftarrow$ the snapshot's entry for that key, falling back to `theme.render` — the frame machinery has already resolved any theme references by the time it calls you. And name each element anchor(<its key>), so callouts and overlay can find it.

For a whole animation rather than a hand-driven renderer, `make-frames` takes a list of specs — each a structure, a `build: theme => snapshot closure`, and the frame's metadata — and is what every `*-display` in the package is built on.

9. Binary search trees

`bst` is the plainest structure in the package and the one to read first: the others differ from it only where their algorithms do.

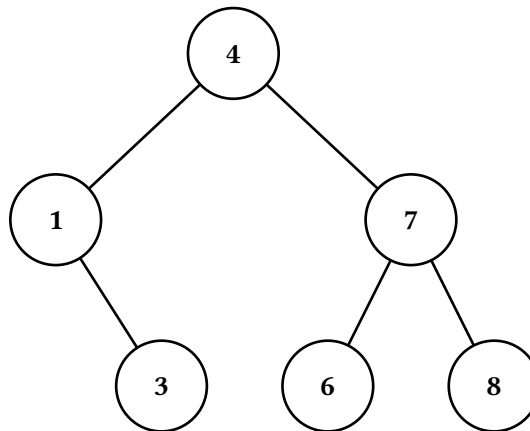
Build one from values with `new` (first value is the root, the rest are inserted in order), or write a tree literal with `node` / `leaf` when the **shape** is the point — a fixture for a specific fix-up case, say:

```
#let t = bst.new(4, 1, 7, 3, 6, 8)
#let fixture = bst.node(4, bst.node(1, none, bst.leaf(3)), bst.leaf(7))
```

Values may carry a display label: pass `(value, label)` in place of a bare value, or `label: on insert / node`. The label is drawn; the value still orders the tree.

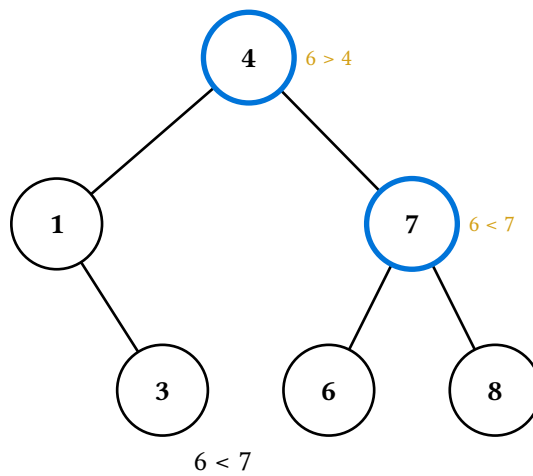
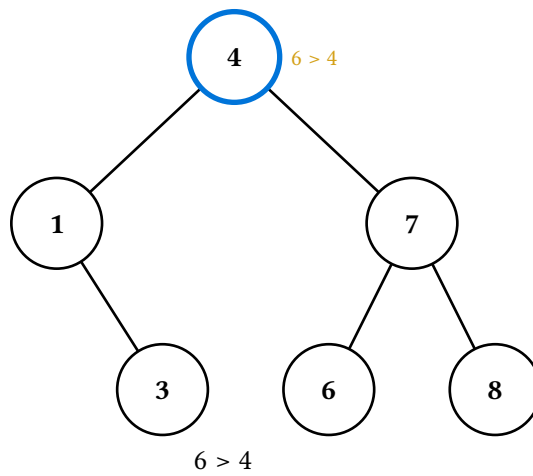
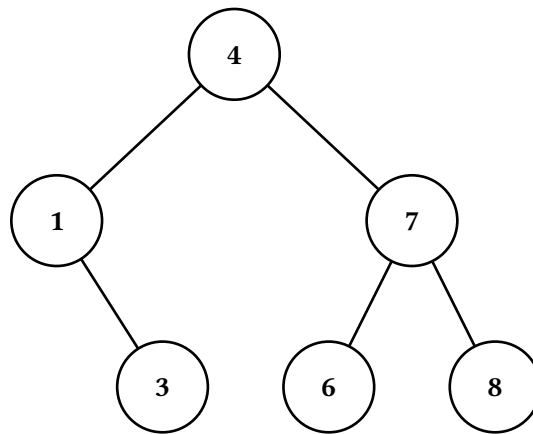
Pure operations — `insert`, `insert-many`, `delete`, `rotate`, `contains`, `by-value`, `path-to`, `resolve`, the four traversals, `describe`, and `check-invariants` — return new trees or plain values, and are what the animations are built on.

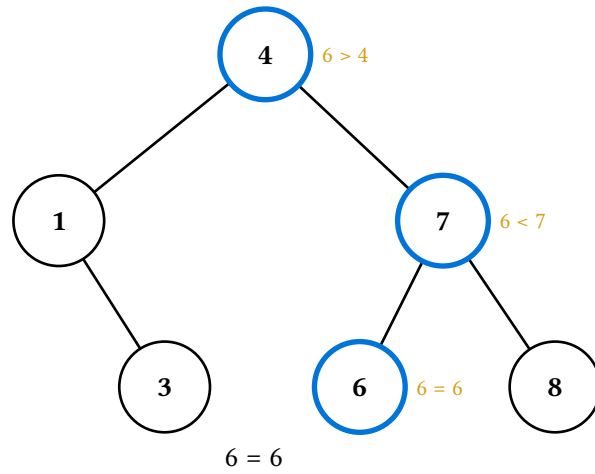
9.1. Static display



9.2. Search

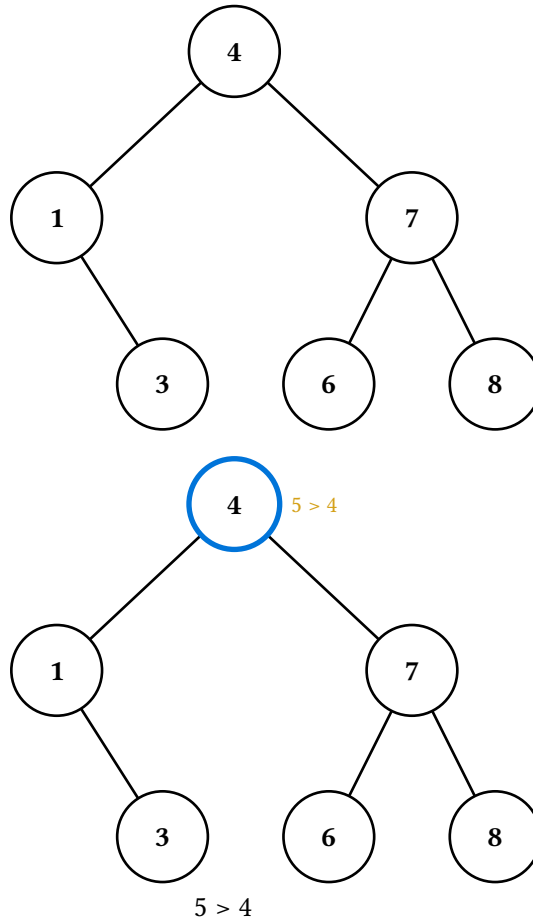
`search-display(t, v)` walks the search path, highlighting each node it visits and labelling it with the comparison it made. The terminal frame is `found` or `not-found`.

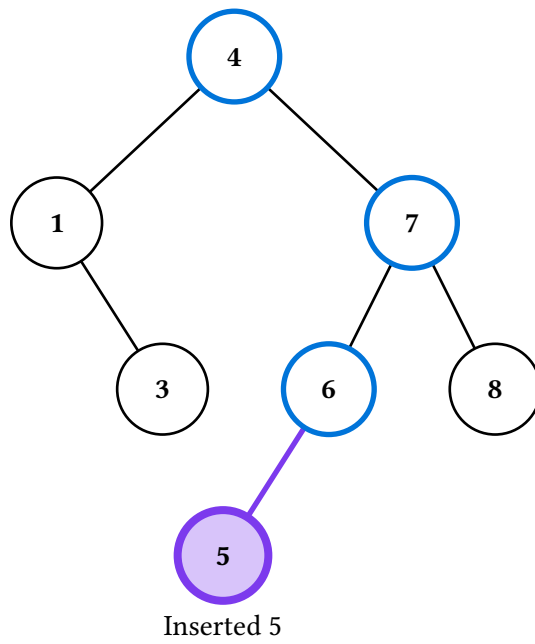
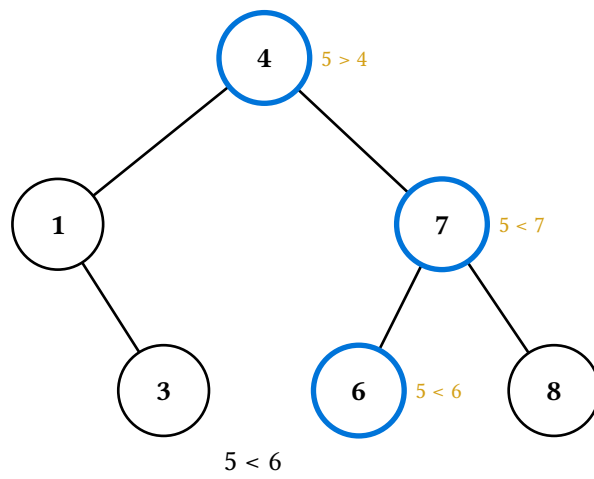
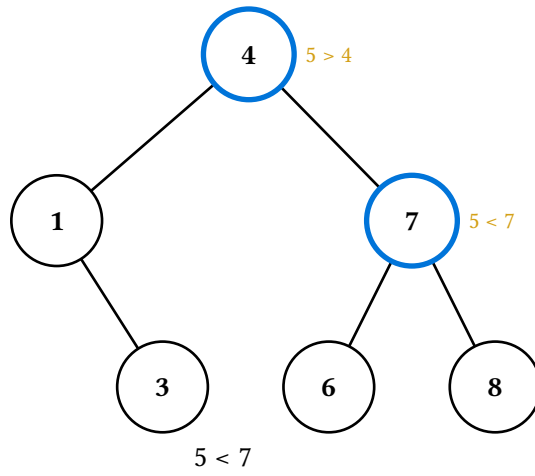




9.3. Insert

`insert-display(t, v, label: auto)` walks the same path, then shows the new node spliced in and settled.

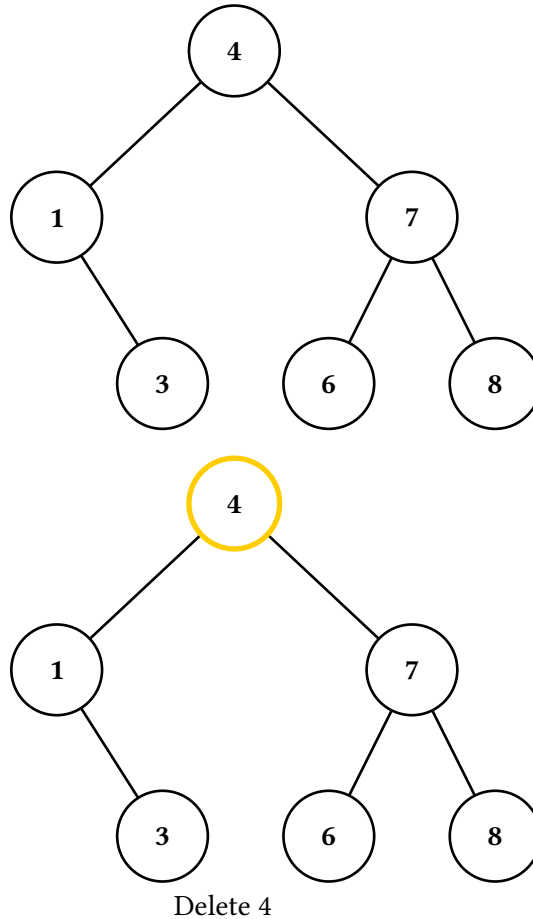


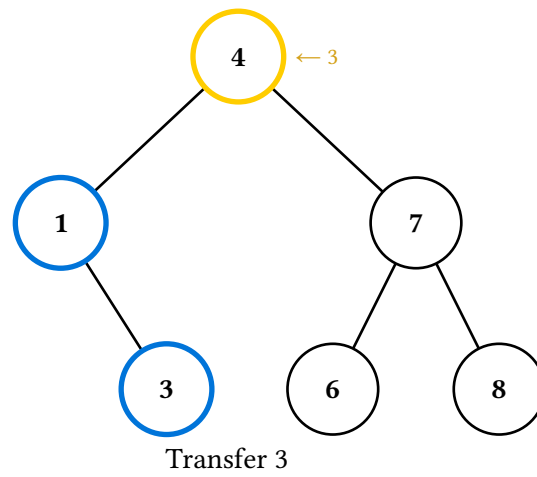
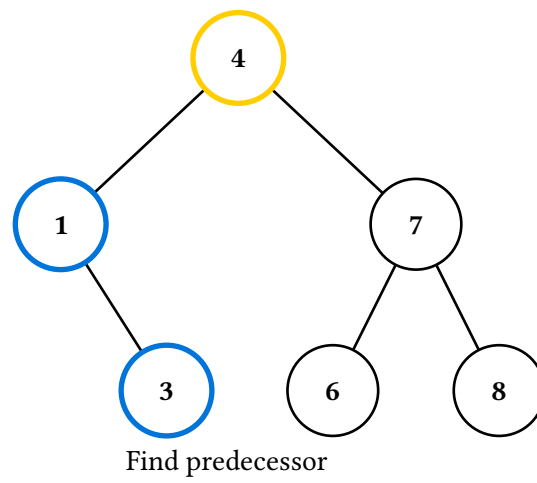
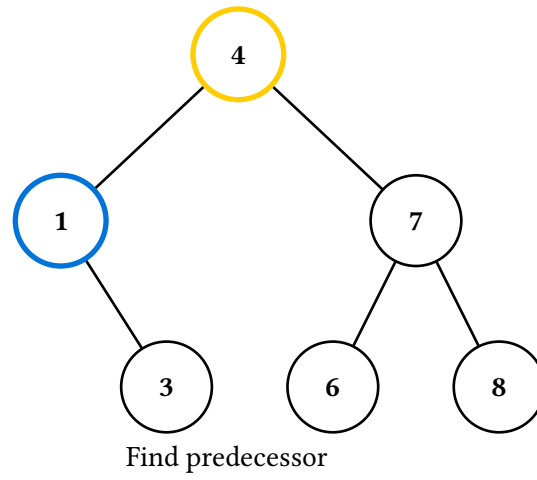


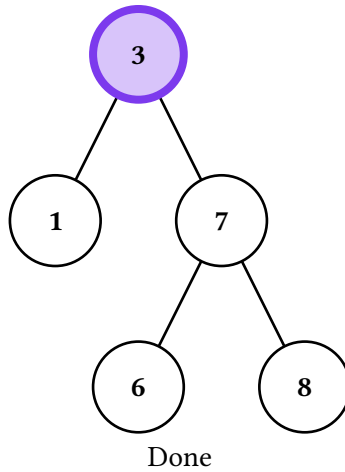
9.4. Delete

`delete-display(t, v)` dispatches on the target's children — leaf, one child, or two. The two-child case is the interesting one: it marks the target, descends to the in-order predecessor, transfers the value, and then excises the emptied node.

Pass `search: true` to precede the deletion with a search-style walk to the target (its comparison notes are cleared on the first deletion frame, so they do not compete with the deletion highlights). The default opens straight on the deletion, which is what you want when that, not the lookup, is the lesson. In `rbt` and `avl` the same flag means one frame per comparison instead of a single frame lighting the whole path.





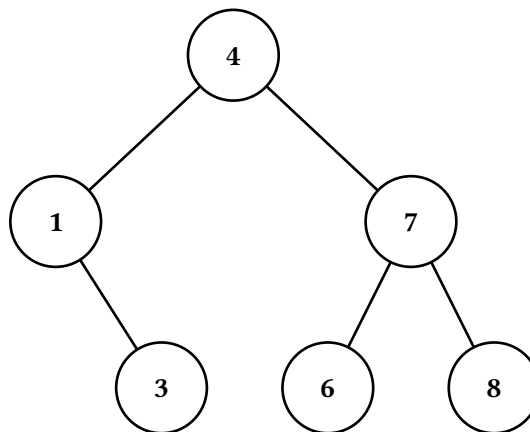


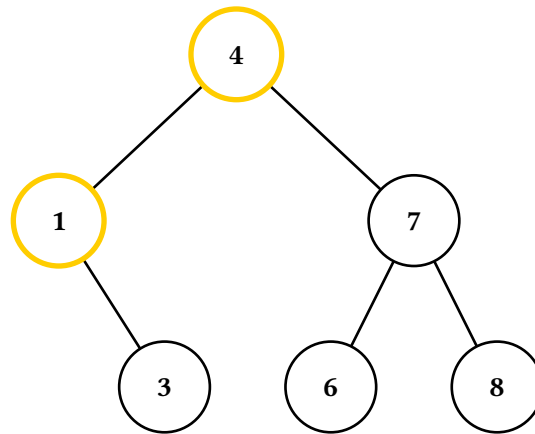
9.5. Rotate

`rotate-display(t, child)` rotates `child` up into its parent's place — anywhere in the tree, not only at the root — inferring the direction from where `child` sits. Six frames, each answering exactly one question:

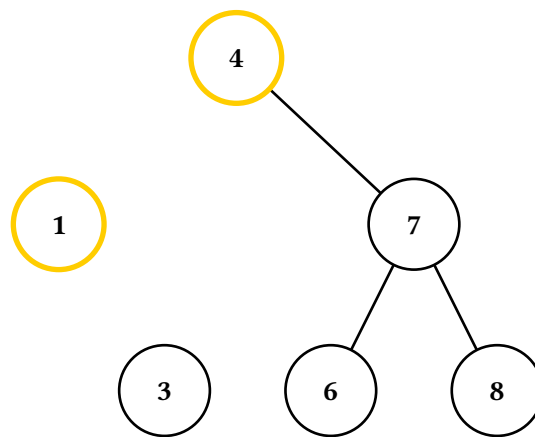
step.kind	Question answered
init	What is the starting tree?
pivots	Which two nodes are rotating?
break	Which edges are about to disappear?
restructure	What is the new shape, edges aside?
connect	Where do the new edges go?
settled	What does the finished tree look like?

`restructure` is the load-bearing frame: it shows the new positions forming while the moved edges are still hidden, so the shape change lands separately from the reconnection.

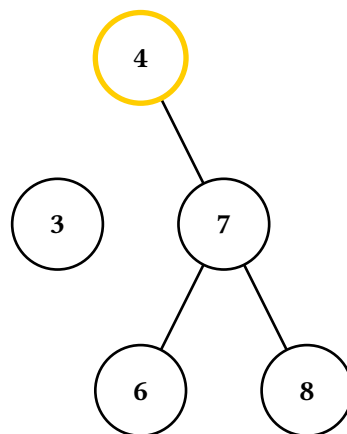




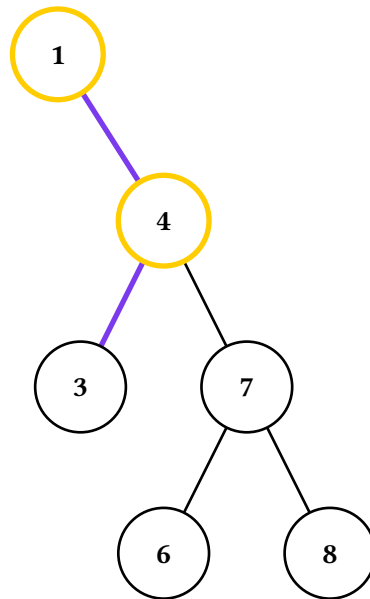
Rotate around 1



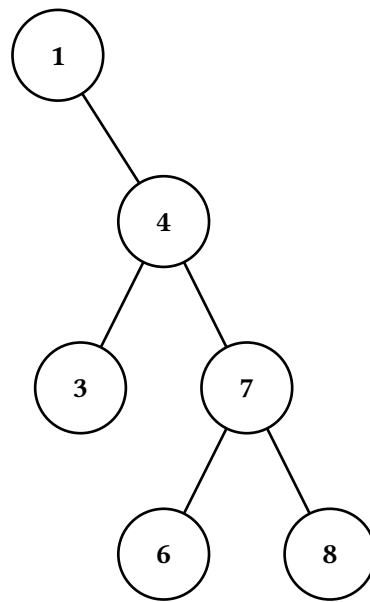
Break edges



Restructure tree

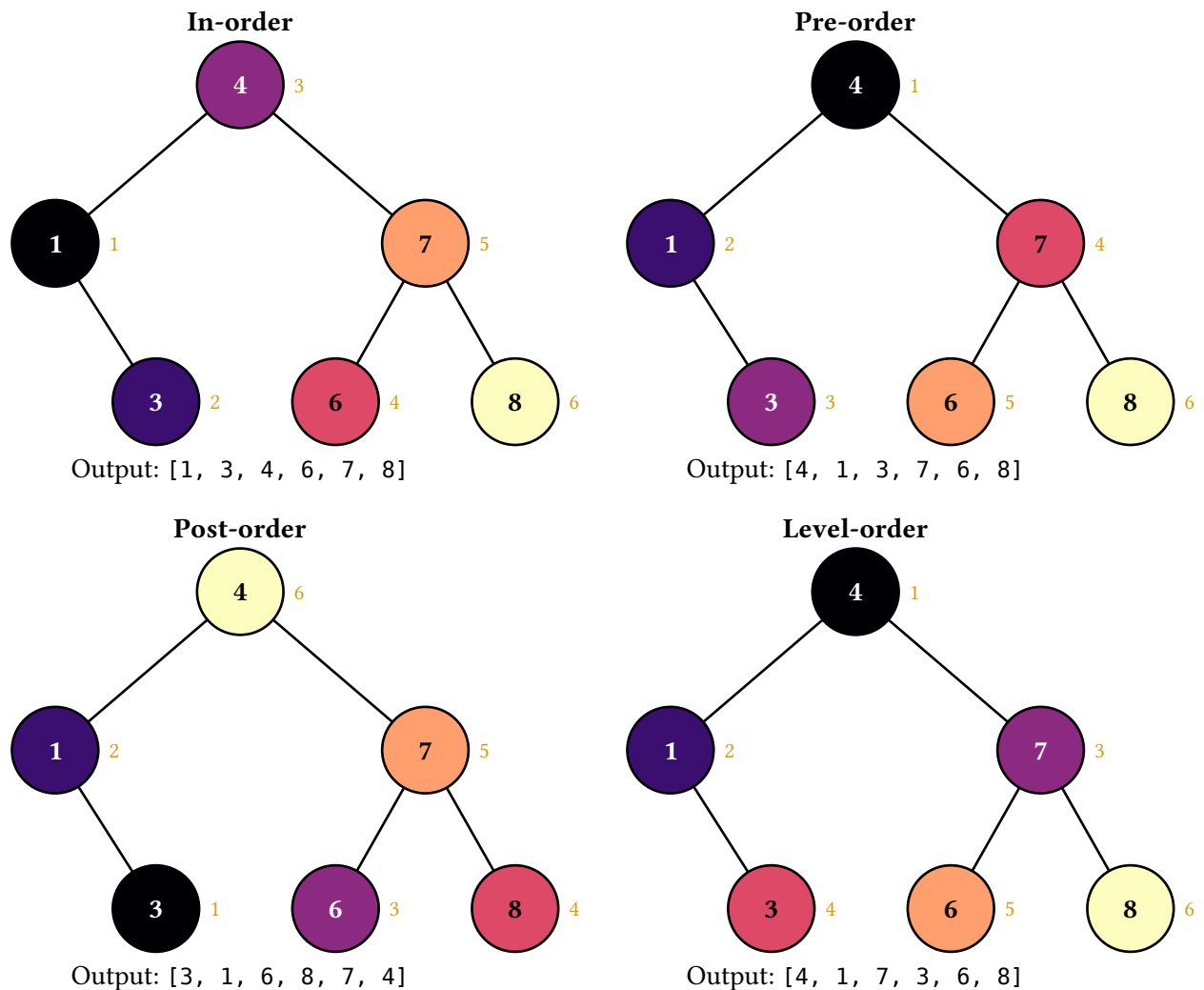


Reconnect edges



9.6. Traversals

The four *-order-display animations emit one frame per visit. The visited node takes the next colour from the op theme's traversal-palette and a numbered badge, and the caption accumulates the output sequence. The final frame therefore carries the algorithm's whole colour signature, which makes the four worth showing side by side:



The pure `bst.in-order(t)` and `friends` return the visit order as an array of paths, for driving a layout of your own.

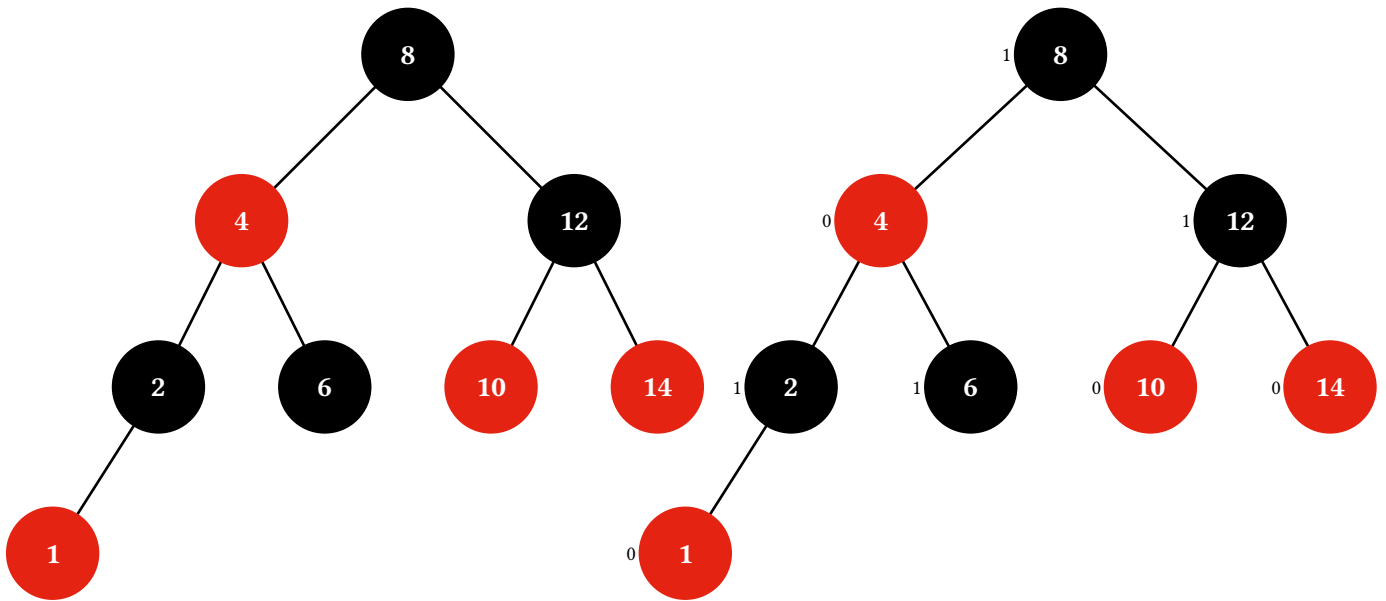
10. Red-black trees

`rbt` adds a colour bit per node and the invariants that make it balanced. Its palette is the theme's `rbt` section, so every frame's nodes wear their semantic colour automatically and the operation highlights compose on top.

Literals here are `rbt.red(v, ..children)` and `rbt.black(v, ..children)` — which is exactly how the textbook fixtures read:

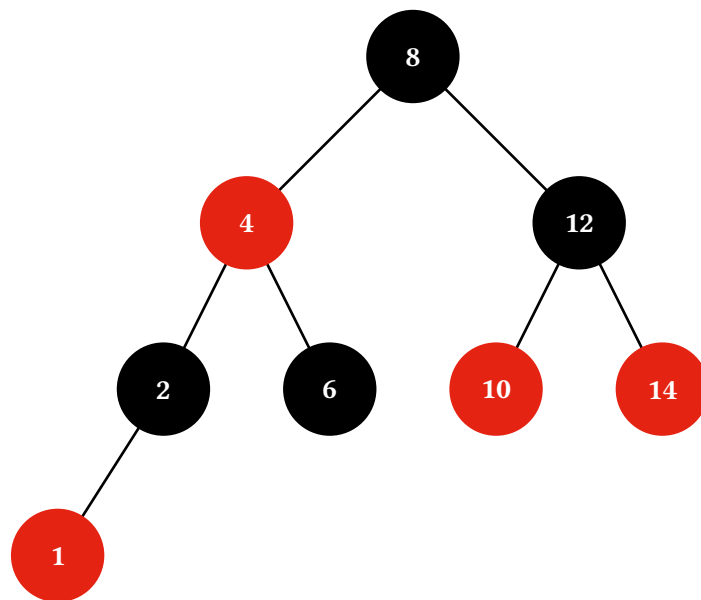
```
#let t = rbt.new(8, 4, 12, 2, 6, 10, 14, 1)
#let violation = rbt.black(8, rbt.red(4, rbt.red(2), none), rbt.black(12))
```

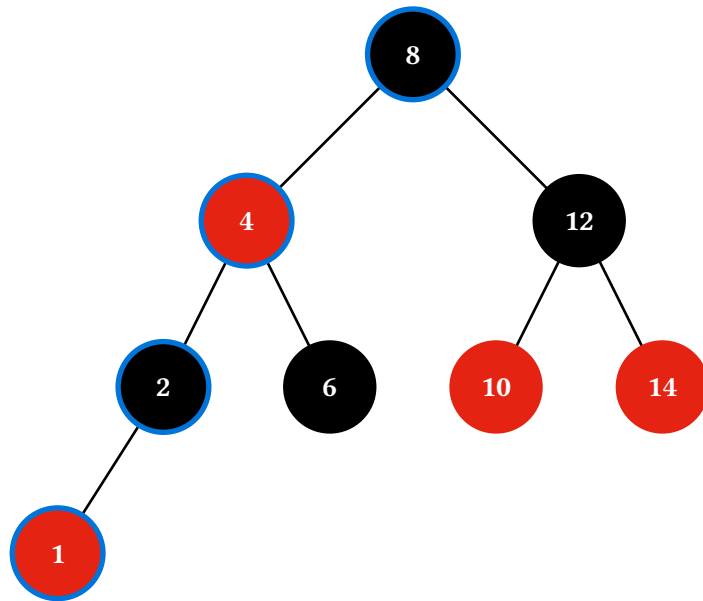
Every display takes bits: `true`, which tags each node with its black-height bit (0 red, 1 black) so a reader can count black height down any path:



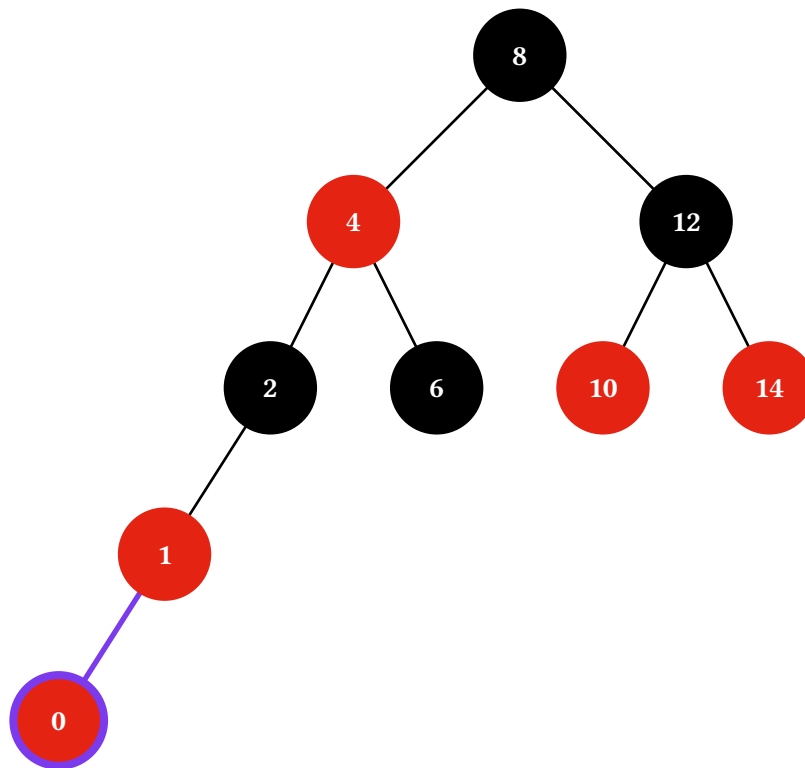
10.1. Insert

`insert-display(t, v)` traces the CLRS insertion: a search descent, the new value spliced in as a red leaf, then the fix-up loop — Case 1 (red uncle: recolour and continue at the grandparent), Case 2 (zigzag: rotate the parent to straighten the red-red pair), Case 3 (straight line: rotate the grandparent and swap colours) — and a final blackening of the root if it ended up red.

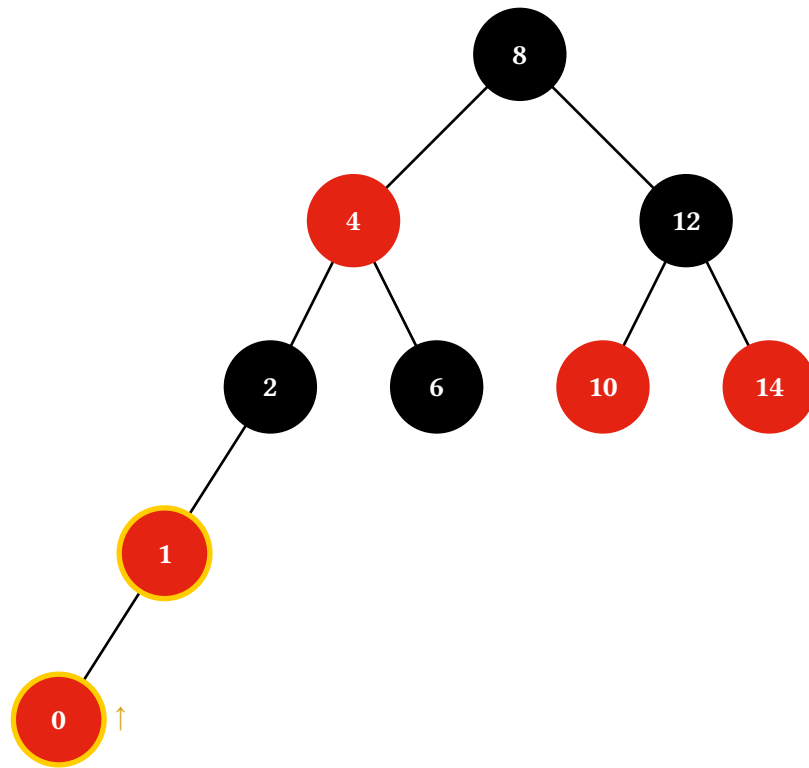




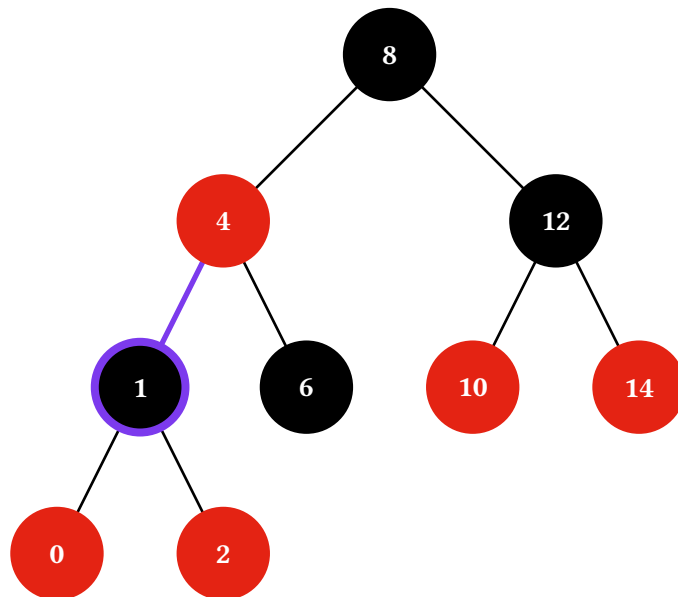
Search for 0



Insert 0 as red leaf



Red-red violation

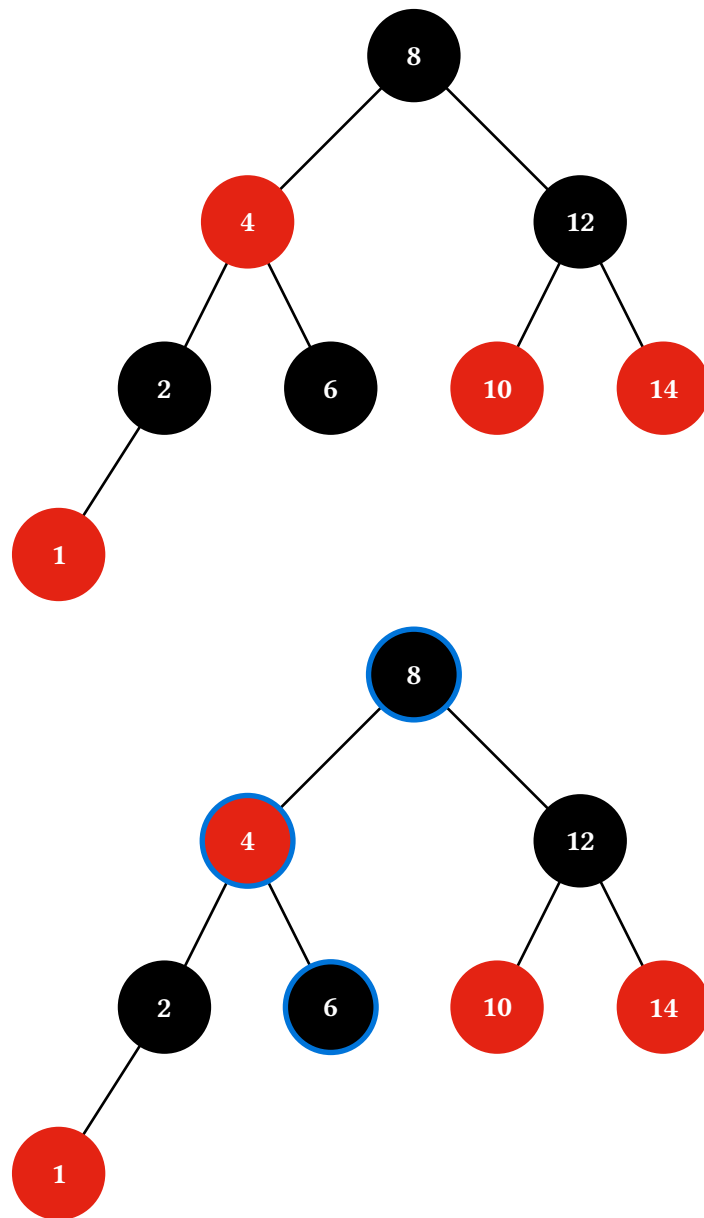


Rotate and color swap

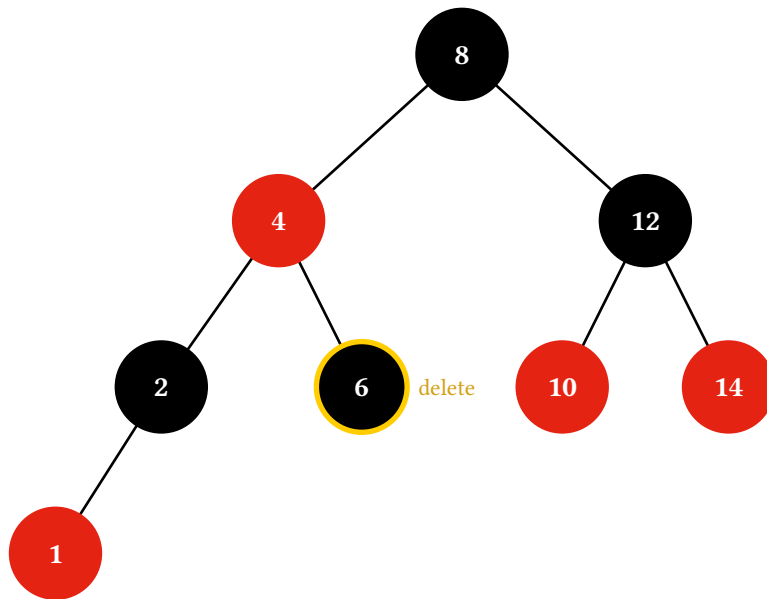
10.2. Delete

`delete-display(t, v)` traces the search, the predecessor walk when the target has two children, the transfer, the excise, and the four-case rebalancing loop. Cases 1, 3, and 4 each emit the rotation and the recolour as separate frames, so the structural pivot lands apart from the colour swap; Case 2 is a pure recolour and emits one.

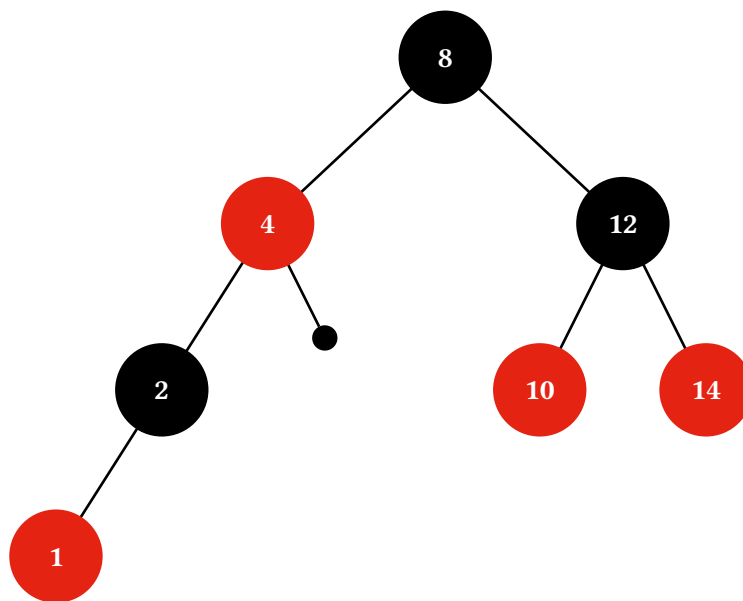
Excising a black node leaves its subtree one black short of the rest of the tree, and the animation marks that with the textbook filled circle at the child end of the affected edge. It persists across every fix-up step that does not resolve it, and disappears when a Case 4 rotation drains it or a red ancestor absorbs it.



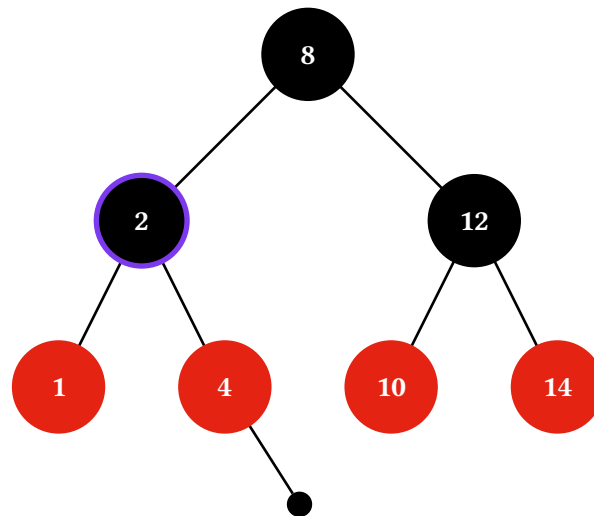
Search for 6



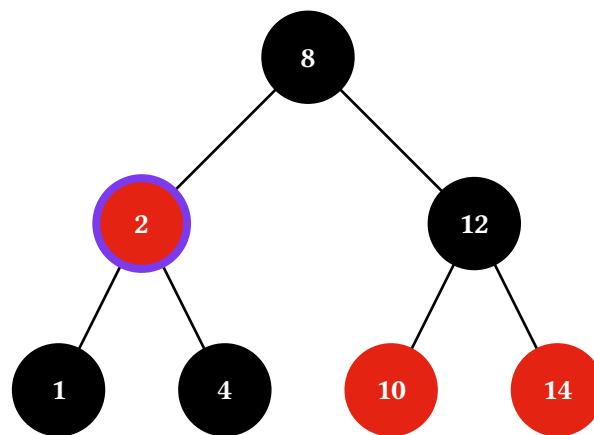
Delete 6



Remove node



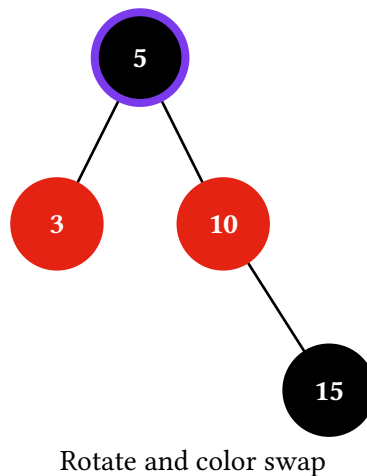
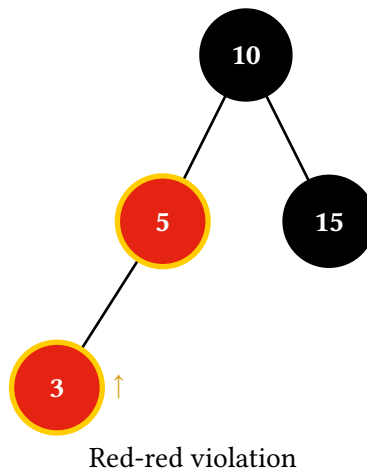
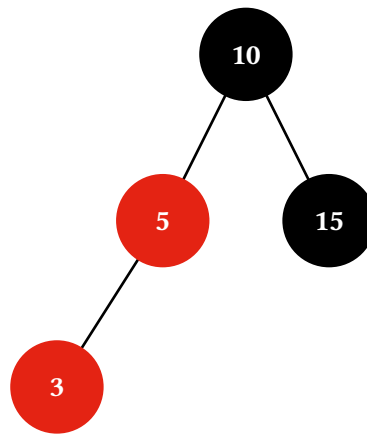
Rotate parent



Recolor

10.3. Fix-ups from a hand-built tree

`fixup-display(t, violation-path)` runs the red-red fix-up on a tree you built yourself, with `violation-path` naming the **lower** of the two reds. The tree is deliberately not validated: the whole point is to show configurations a single insert cannot produce — a red-red in the middle of a tree, or a multi-level Case 1 propagation that ends by blackening the root.

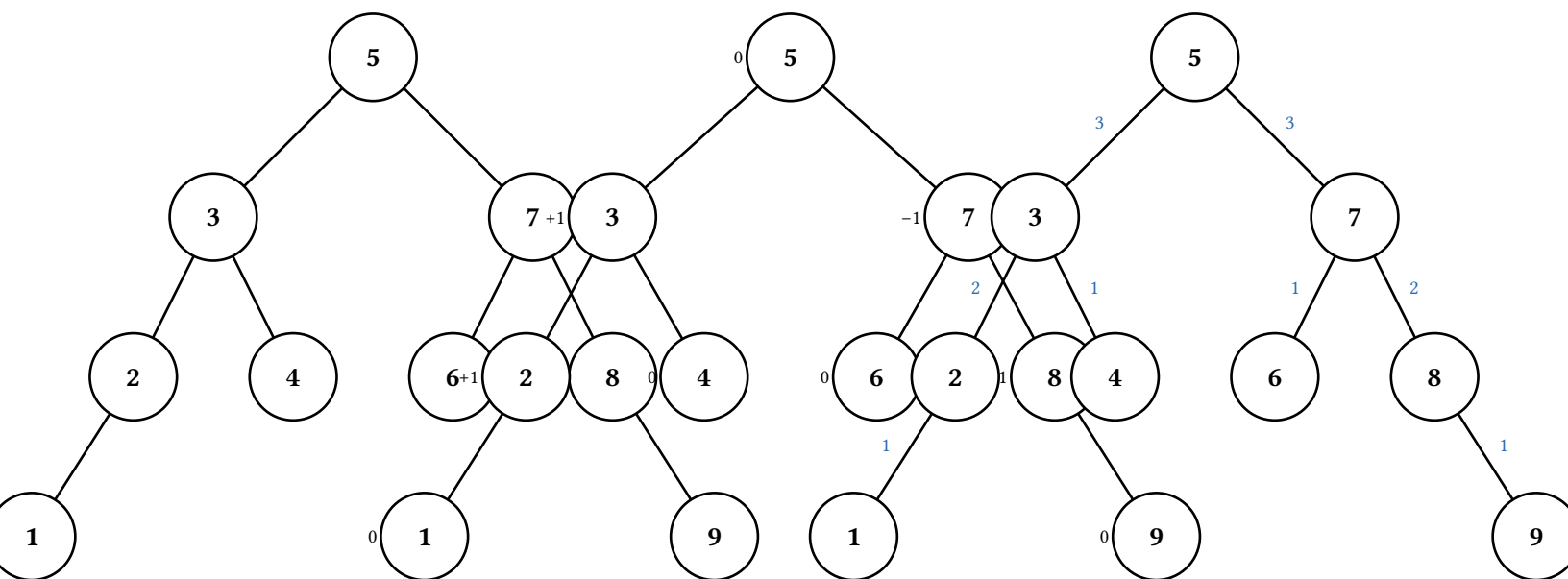


`rbt` also has `search-display` and the four traversals, which behave exactly as the BST's do while keeping the red-black colouring underneath.

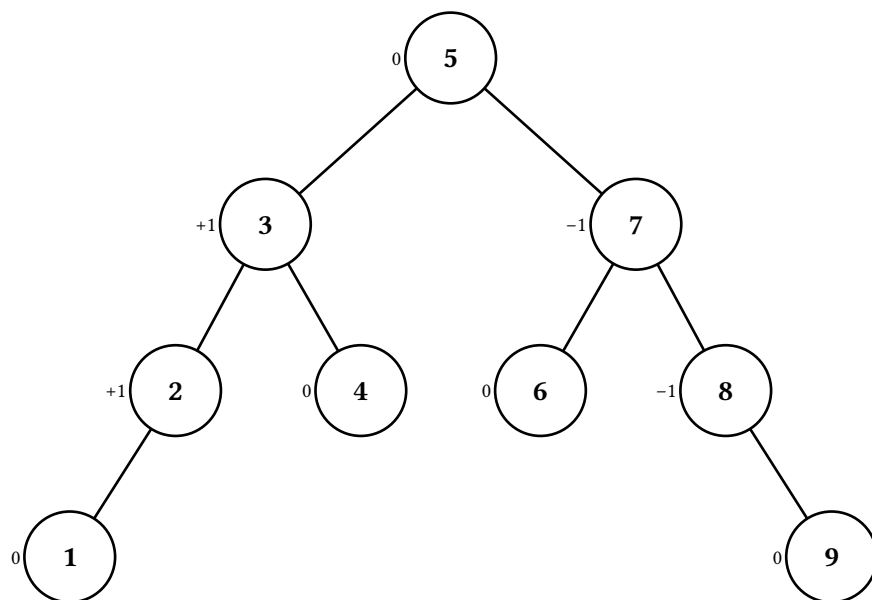
11. AVL trees

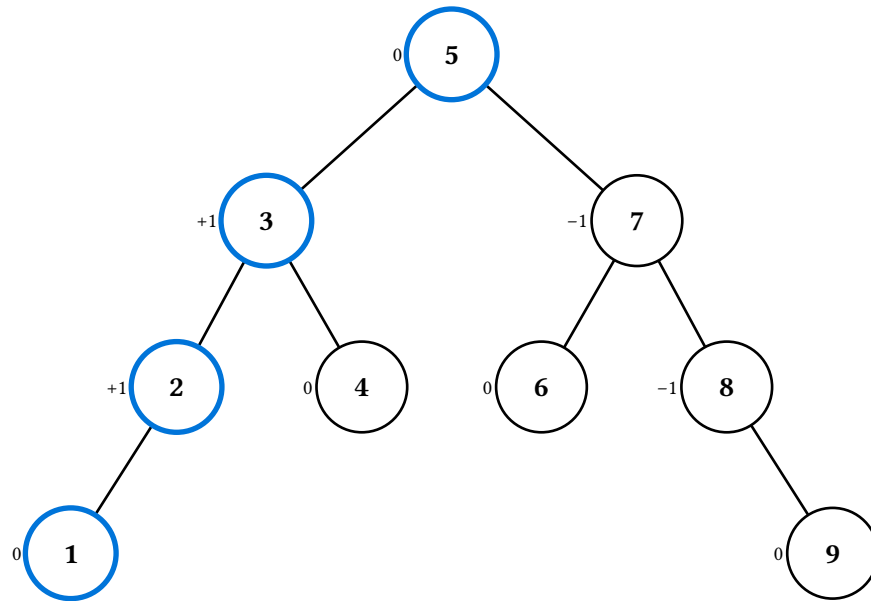
`avl` is a height-balanced BST: every node carries its height, and after any operation no node's two subtrees differ in height by more than one. Imbalances are repaired by the four textbook cases — LL, RR, LR, RL.

Two display flags make the balance visible. `factors: true` tags every node with its signed balance factor; `heights: true` labels each non-root edge with the height of the subtree below it (the root has no incoming edge, and its children's labels make its own factor readable anyway).

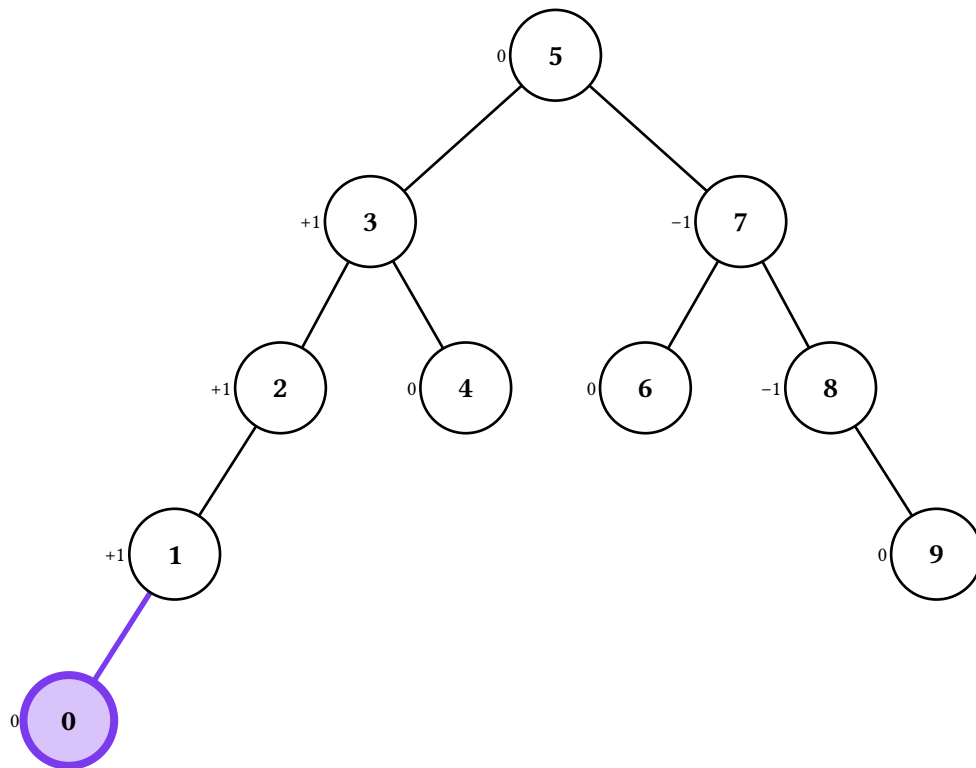


The descent and the splice are the BST's; what follows is AVL's. The climb back to the root recomputes each ancestor's height in turn, and at the first node whose balance factor reaches ± 2 the animation names the case, applies the child rotation if it is a zigzag (LR or RL), and finishes with the rotation at the imbalanced node. One insertion restores the subtree's original height, so the climb stops there.

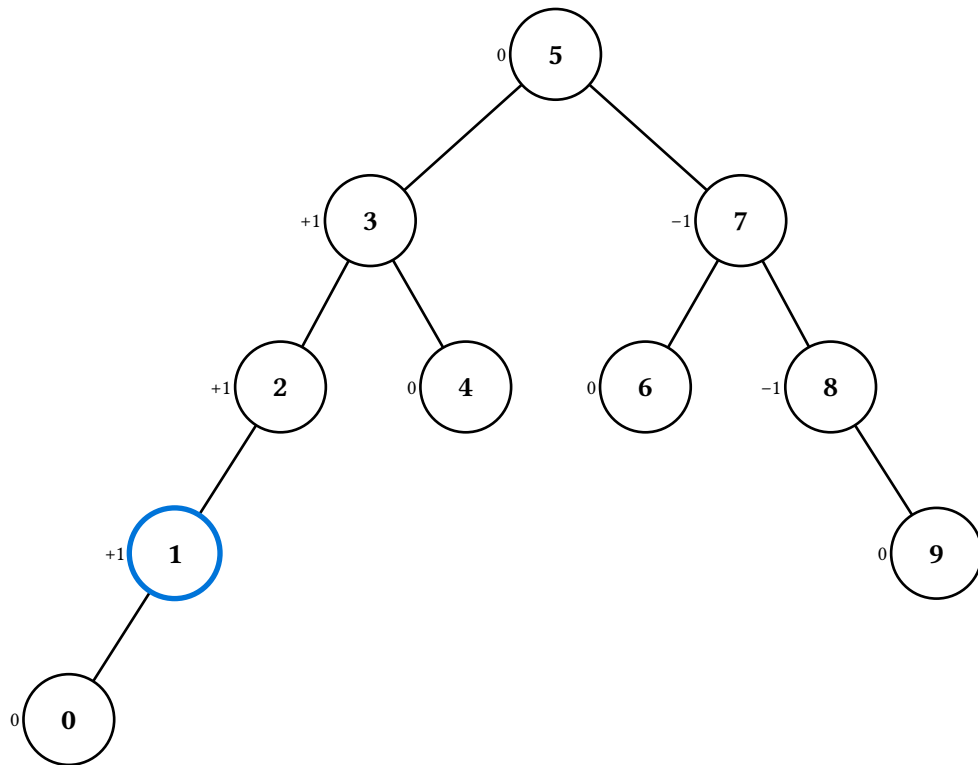




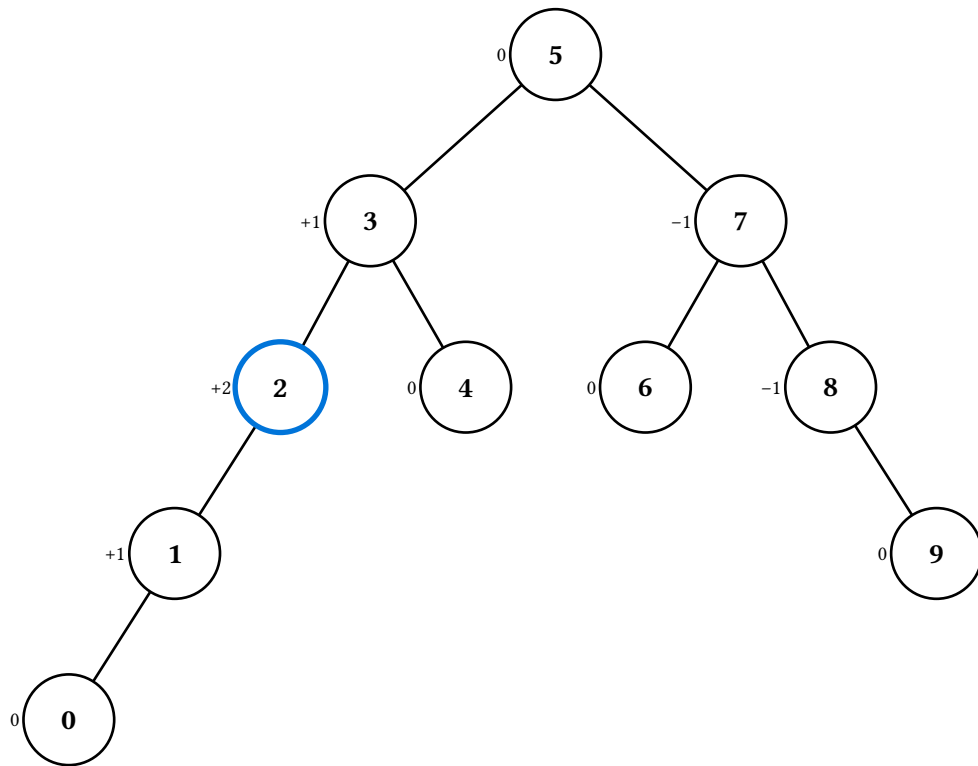
Search for 0



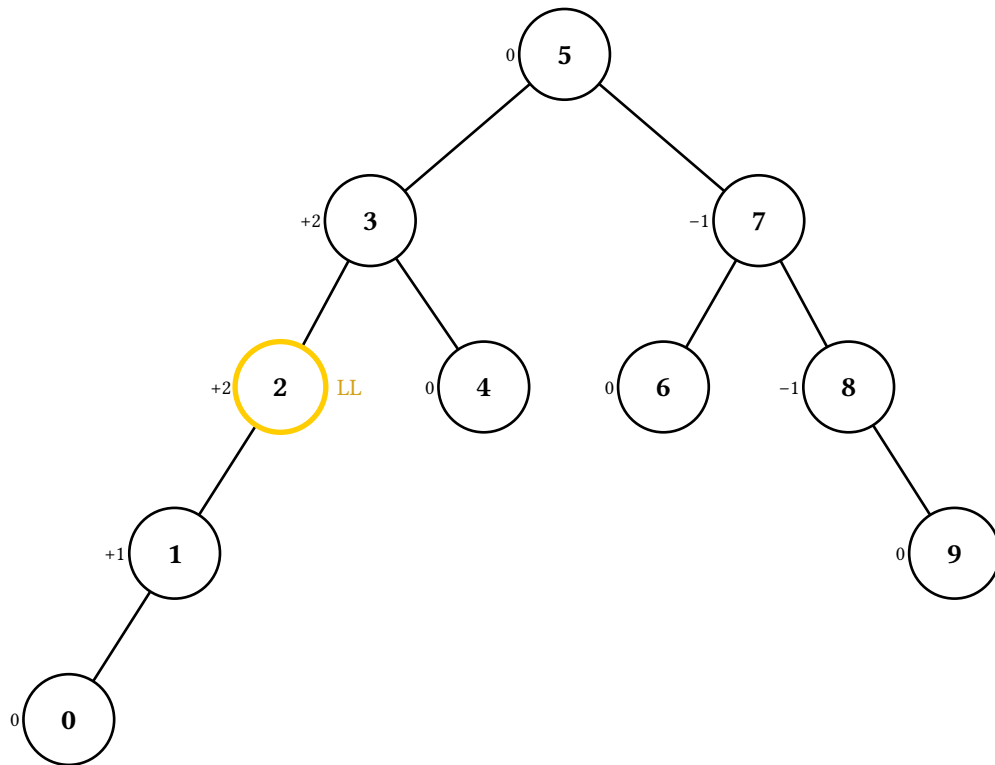
Insert 0 as new leaf



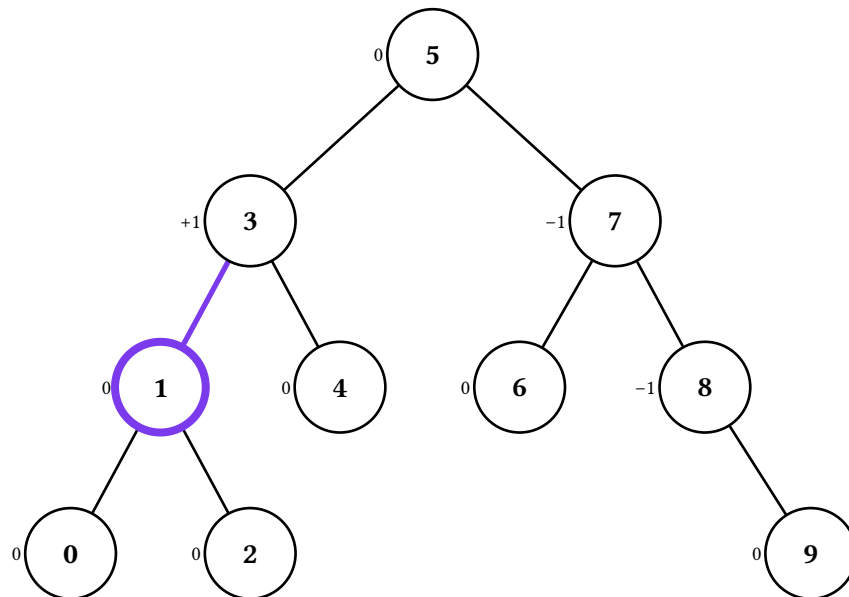
Recompute height



Recompute height



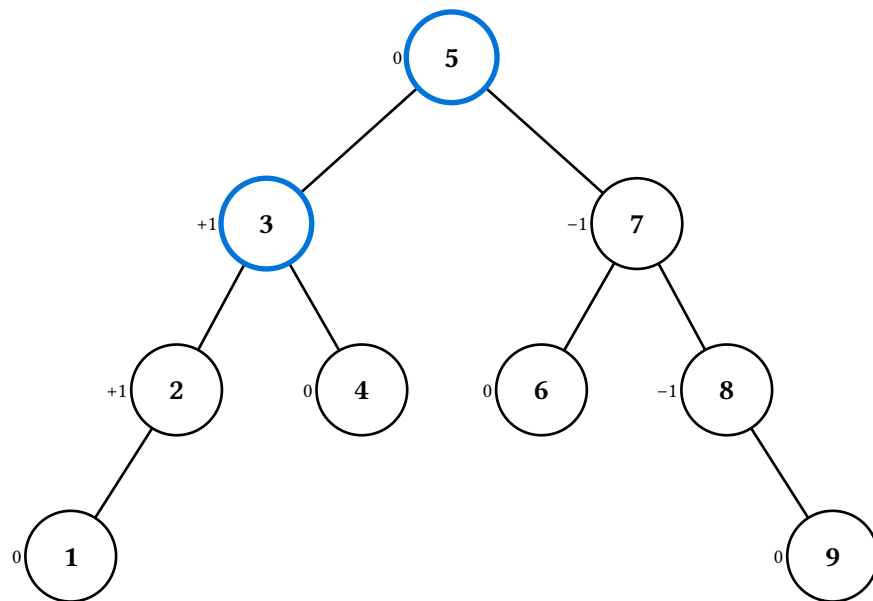
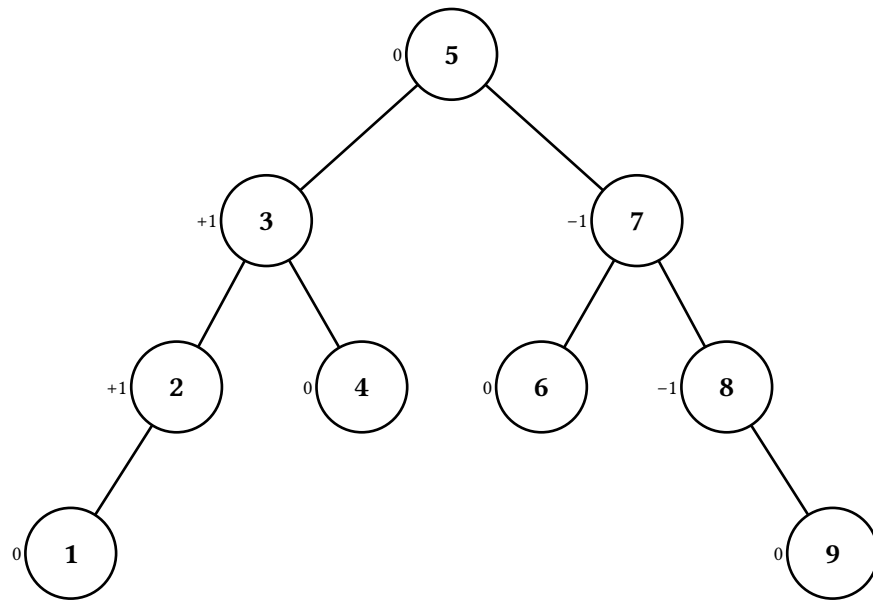
Imbalance: case LL



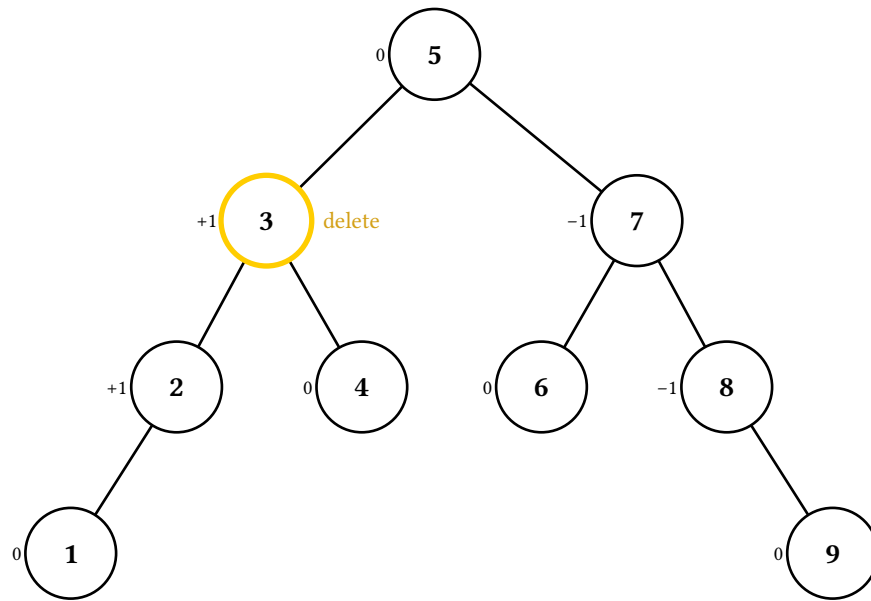
Rotate (case LL)

11.2. Delete

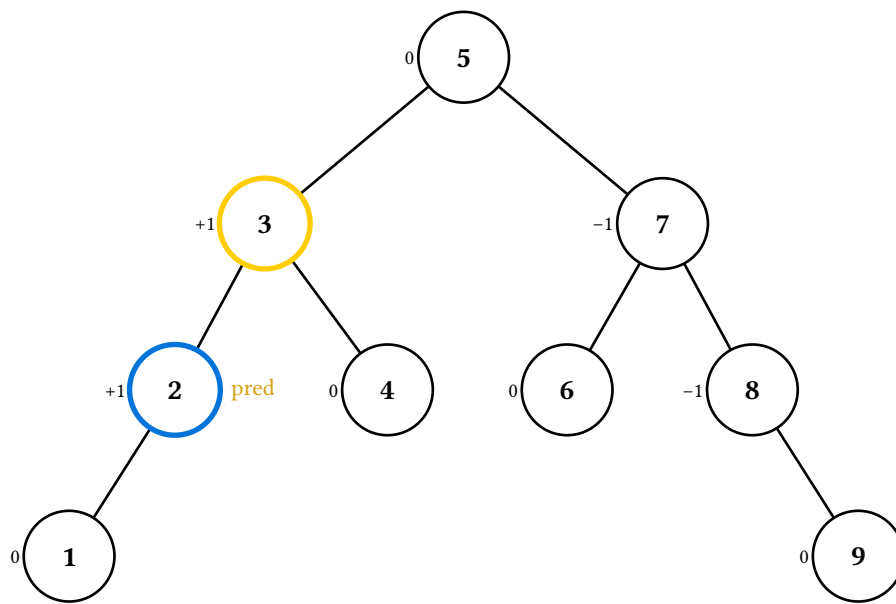
Delete runs the same climb, but a deletion can shorten a subtree — so several ancestors may need rotating, and the loop runs all the way to the root.



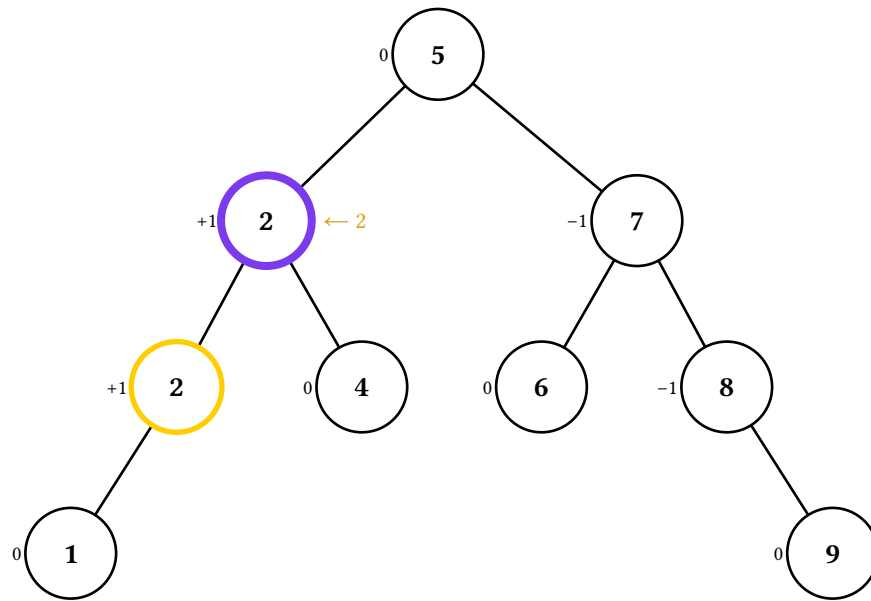
Search for 3



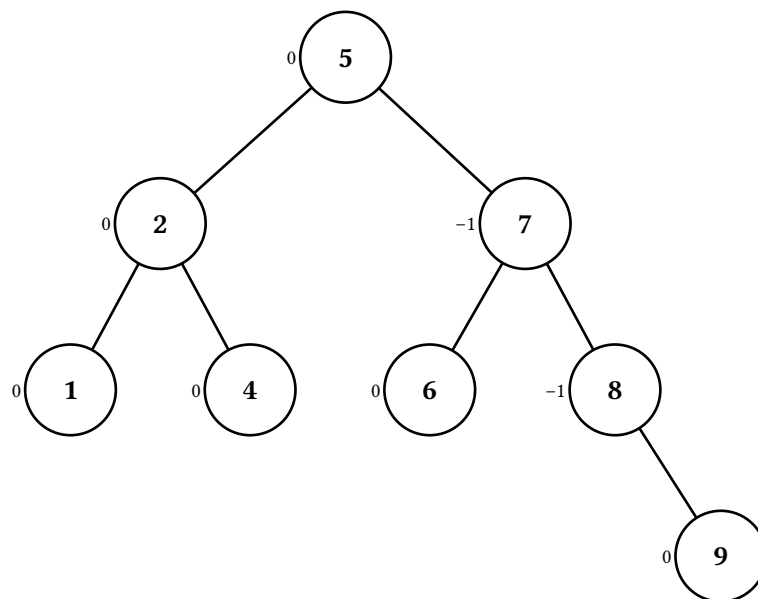
Delete 3



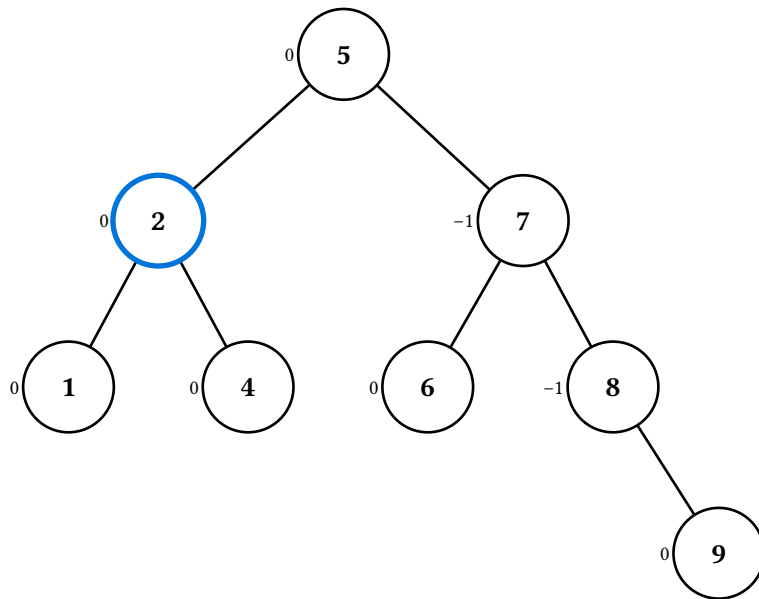
Find predecessor



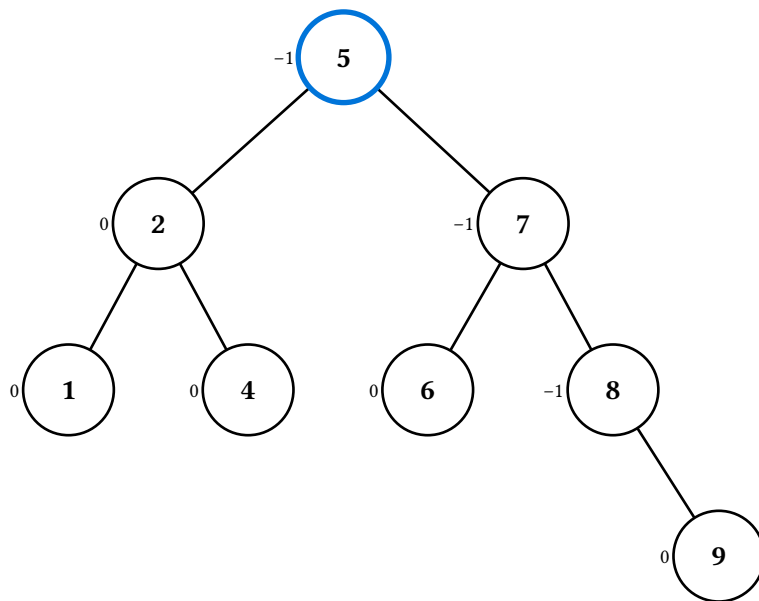
Transfer 2



Remove node



Recompute height

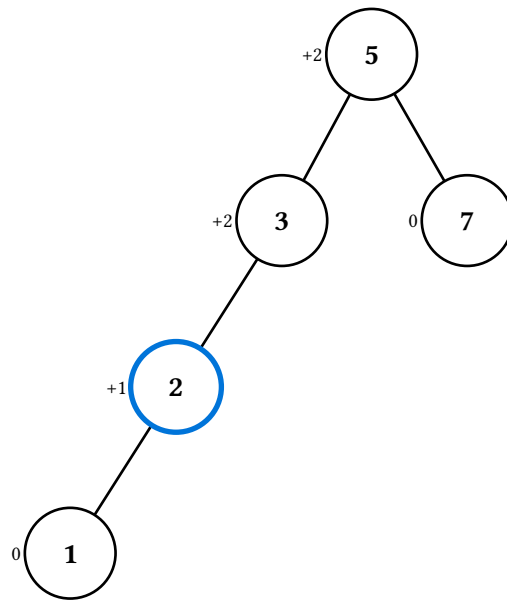
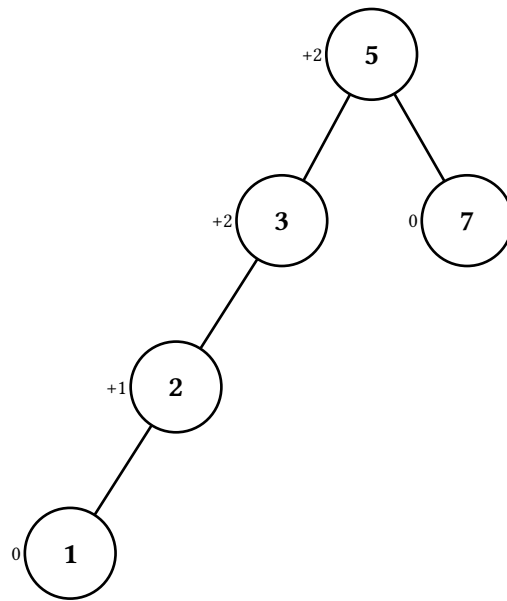


Recompute height

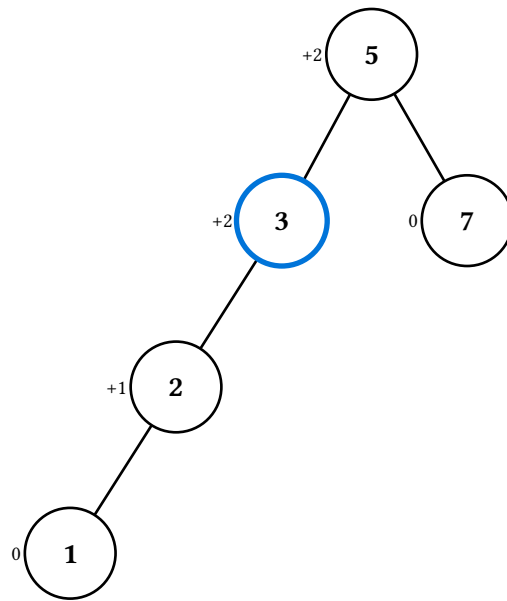
11.3. Rotate and fix-up

`rotate-display(t, child)` is the same six-frame structural rotation the BST has, with heights refreshed afterwards. It is **not** what `insert` and `delete` use internally — those narrate recompute-then-rotate case by case.

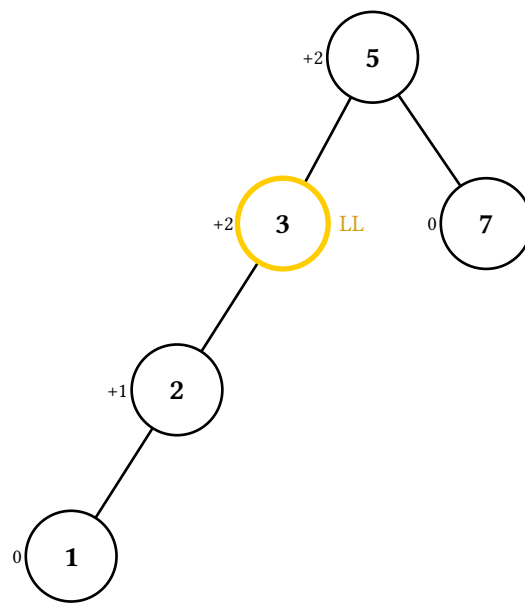
`fixup-display(t, violation-path)` runs the climb on a hand-built tree, walking every proper prefix of `violation-path` deepest-first. With `avl.node`'s `height`: escape hatch you can write down the stale spine a mid-operation tree really has:



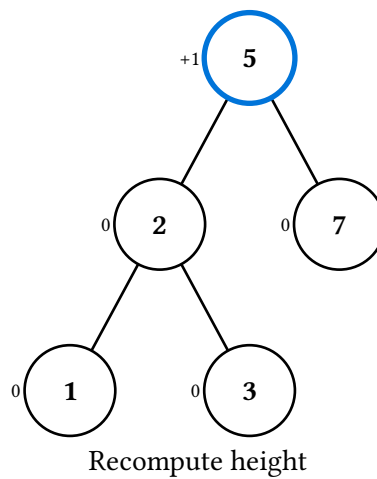
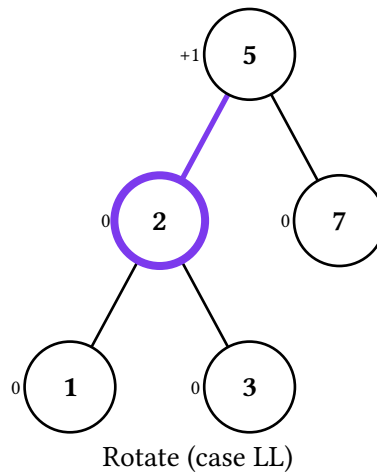
Recompute height



Recompute height



Imbalance: case LL

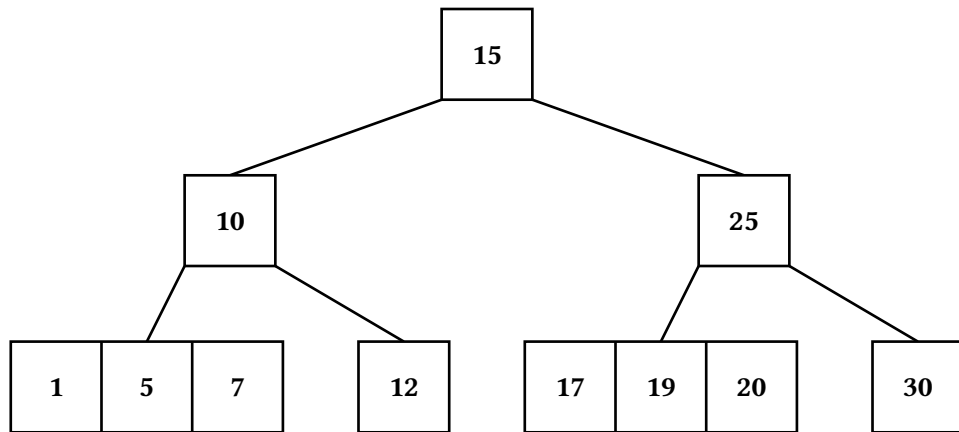


Search and the four traversals come from the same shared implementation the BST uses, and take factors: / heights: like everything else here.

12. 2-3-4 trees

b24 is a B-tree of order 4: every node holds one, two, or three keys (so two, three, or four children) and every leaf sits at the same depth. Nodes draw as subdivided rectangles that widen with their key count, and an individual compartment is addressed by suffixing its node's path with #<i> — "01#1" is the middle key of the leftmost grandchild.

```
#let t = b24.new(10, 5, 15, 1, 7, 12, 20, 25, 30, 17, 19)
#let fixture = b24.node((10, 20), b24.leaf(1, 5), b24.leaf(12), b24.leaf(25, 30))
```

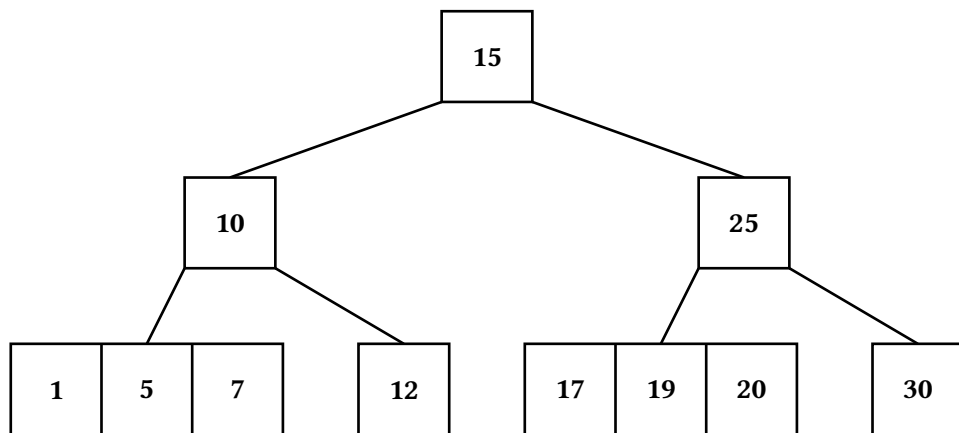


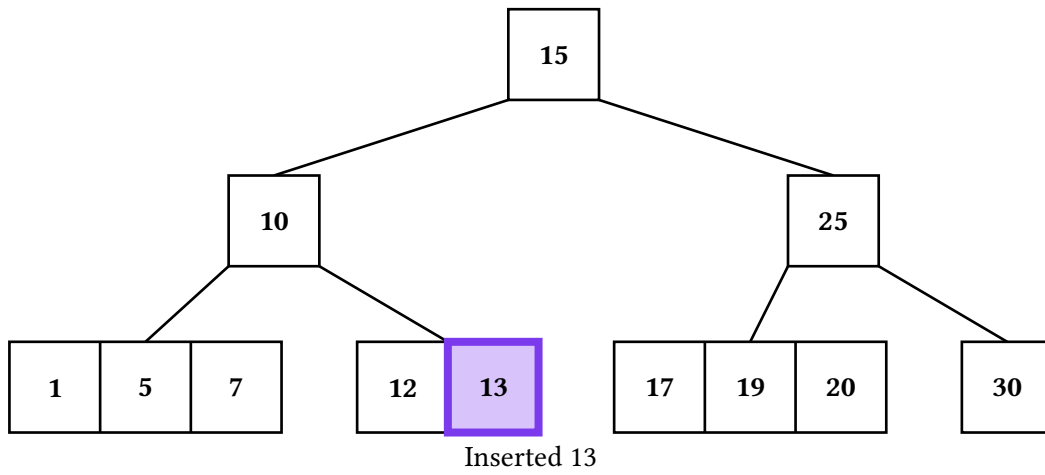
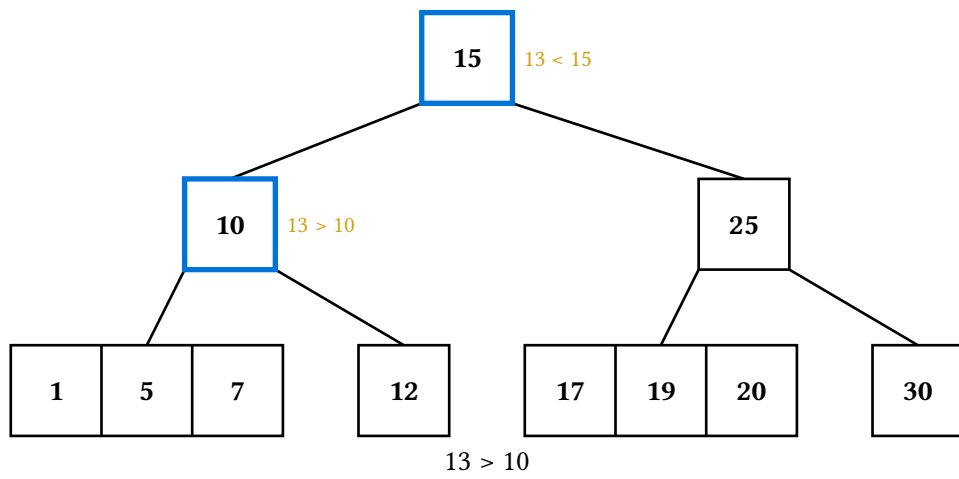
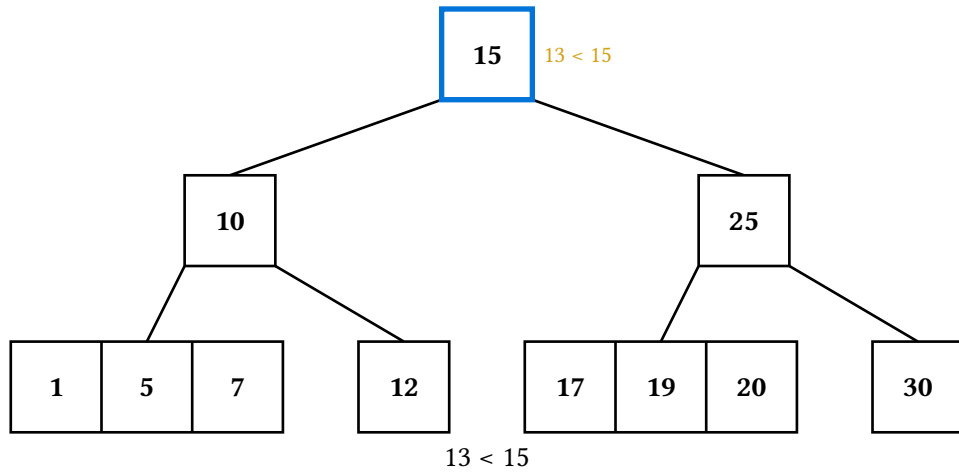
12.1. Two strategies

insert and delete (and their displays) take strategy::

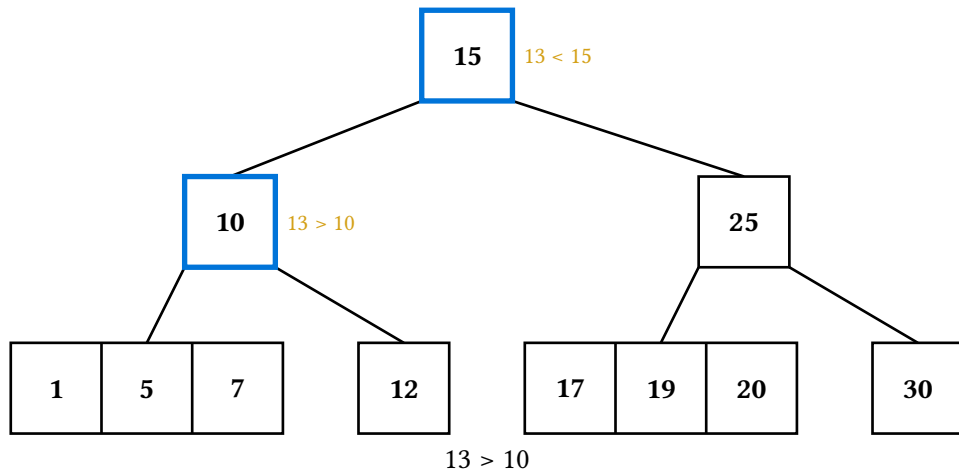
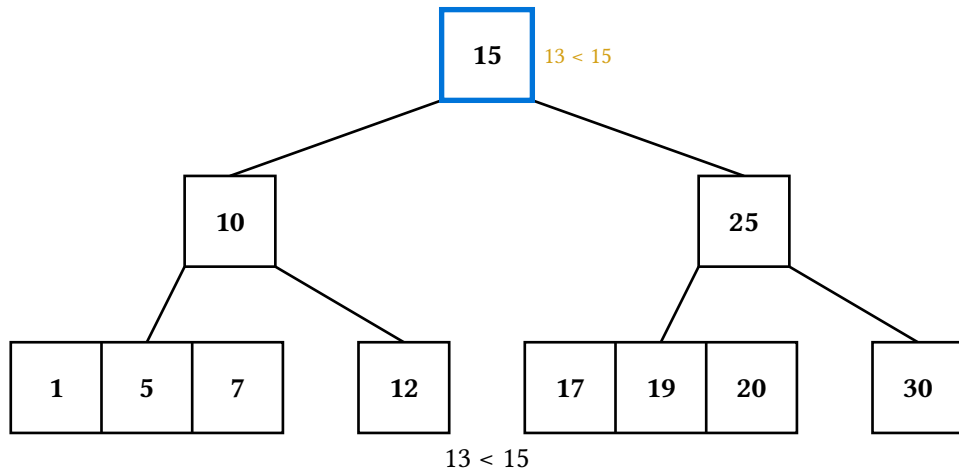
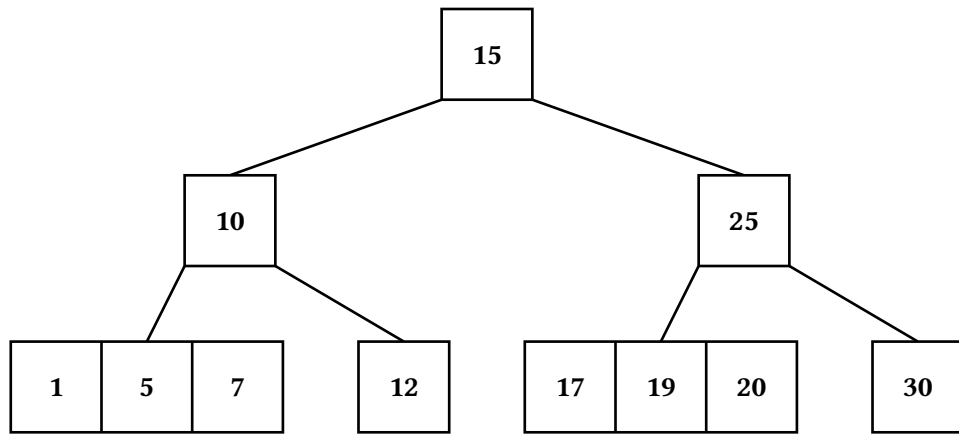
- **"top-down"** (the default) is preventive and single-pass: split any 3-key node on the way down when inserting; refill any 1-key node on the way down when deleting.
- **"bottom-up"** is reactive: walk to the leaf first, then propagate splits or merges back up the descent path.

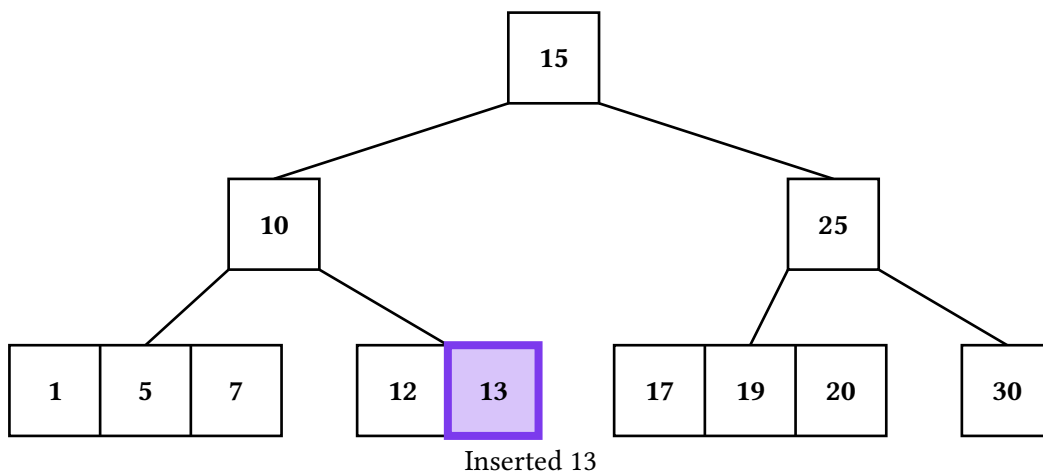
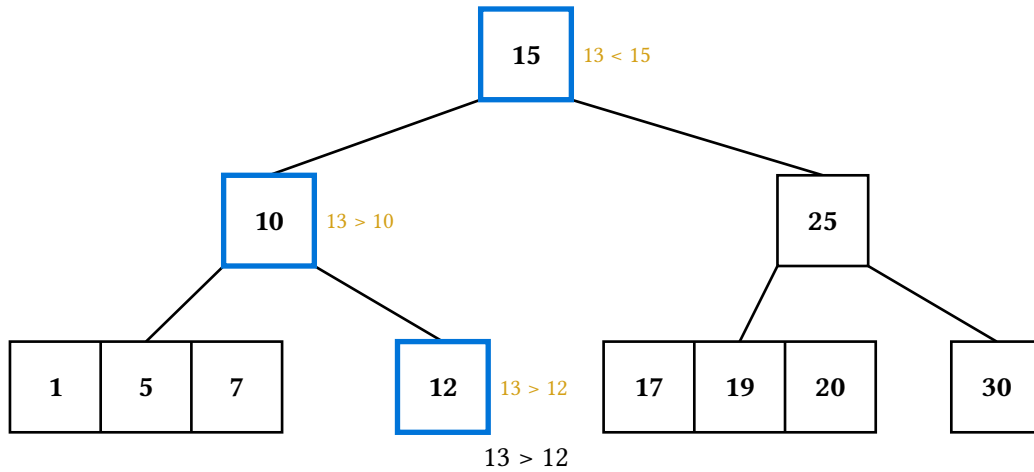
Both produce valid 2-3-4 trees with the same keys, but the **shapes** can legitimately differ — bottom-up promotes from the post-overflow 4-key state, so the freshly inserted key may itself be promoted, while top-down promotes the middle of the pre-insert 3-key state.





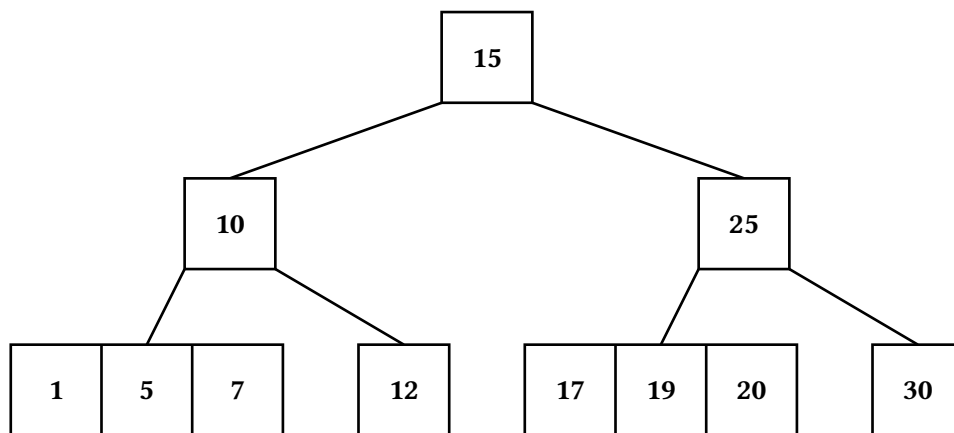
The same insertion, bottom-up:

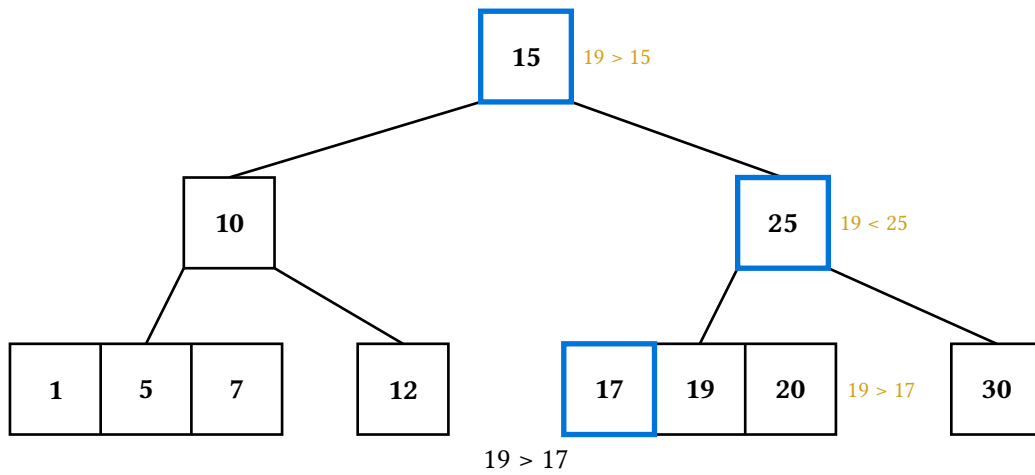
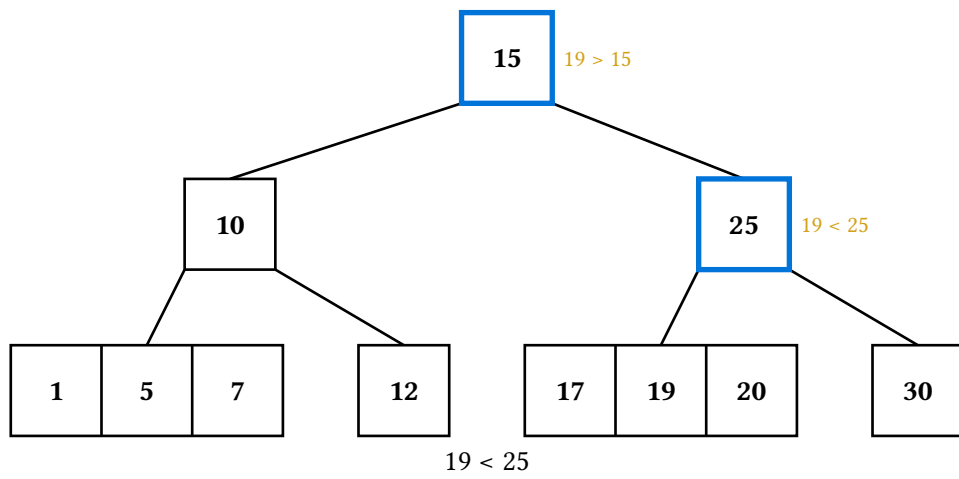
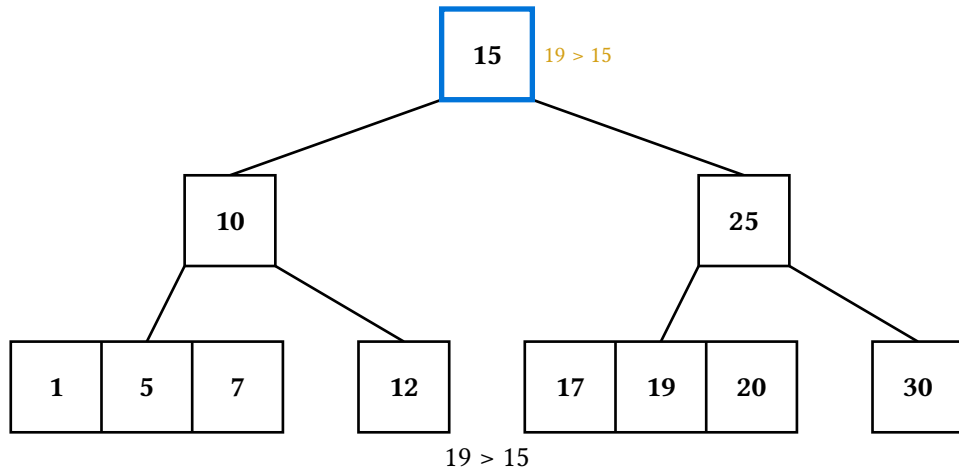


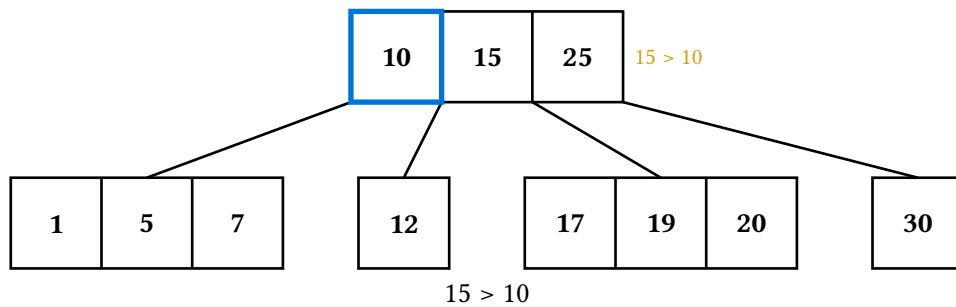
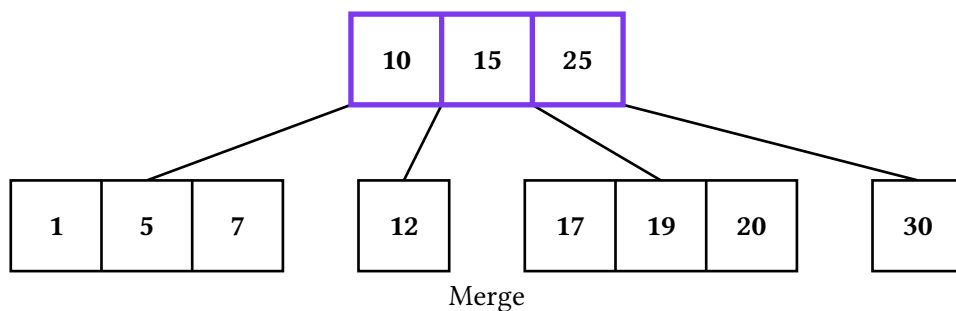
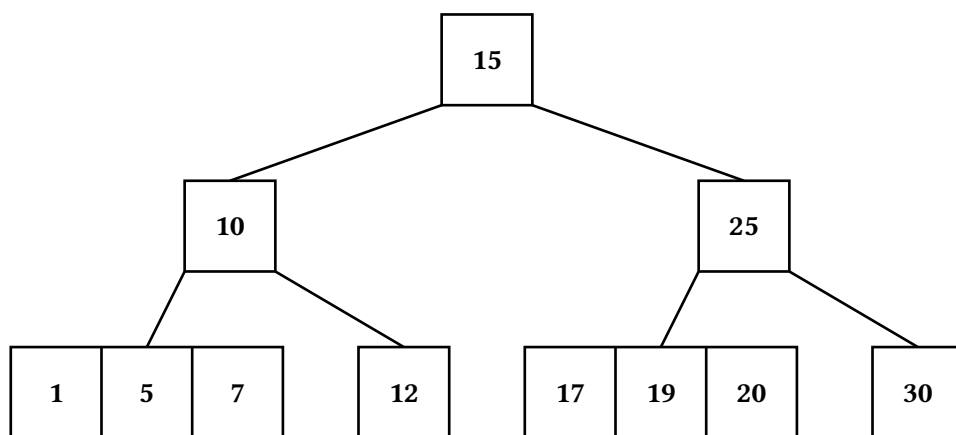
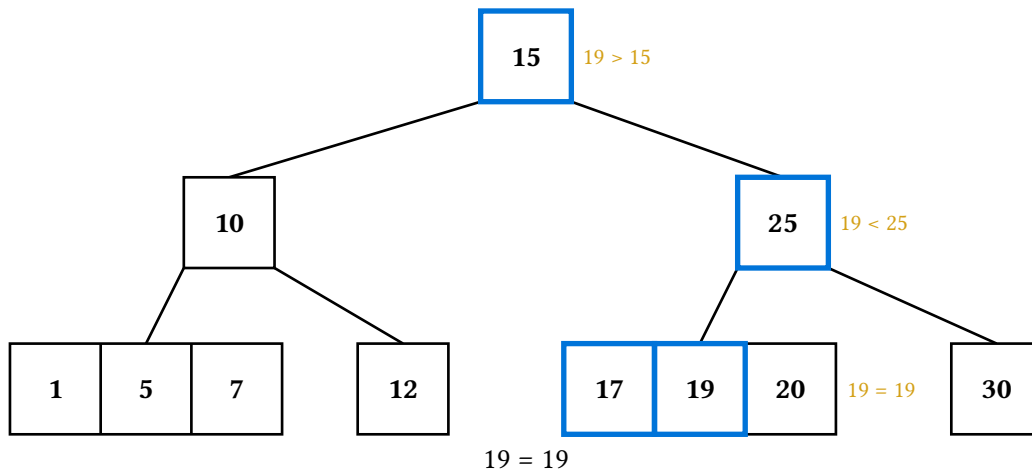


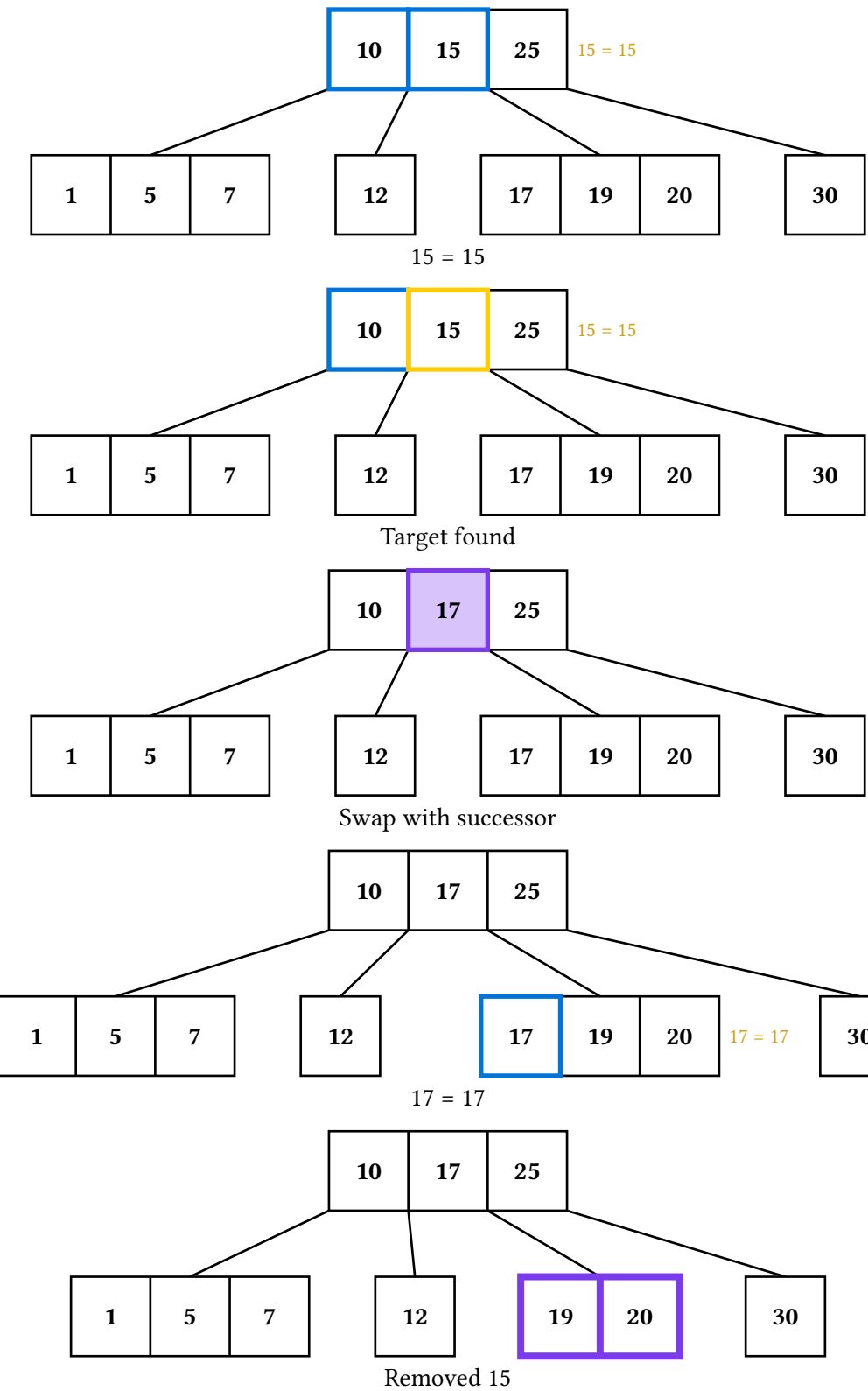
12.2. Search and delete

Search highlights one compartment per comparison and leaves the descent lit behind it. Delete narrates the preventive fix (borrow left, borrow right, or merge) before each step down, and an internal-key deletion swaps with its predecessor and keeps descending to remove that key from its leaf.

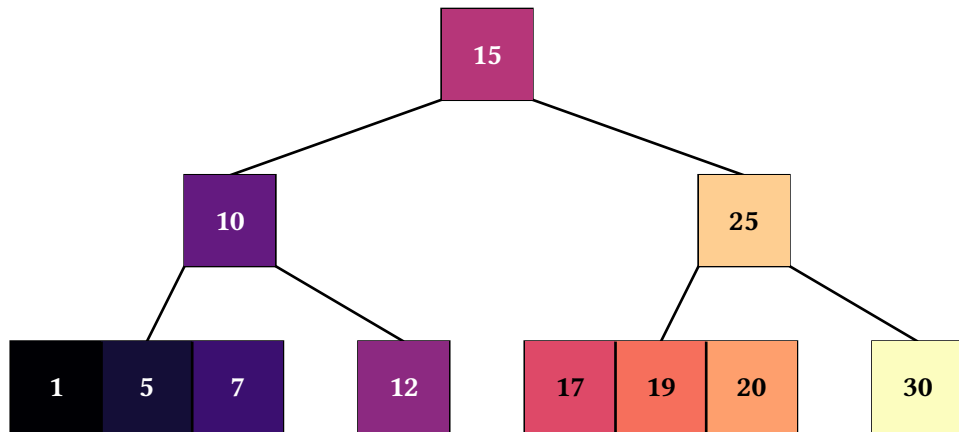








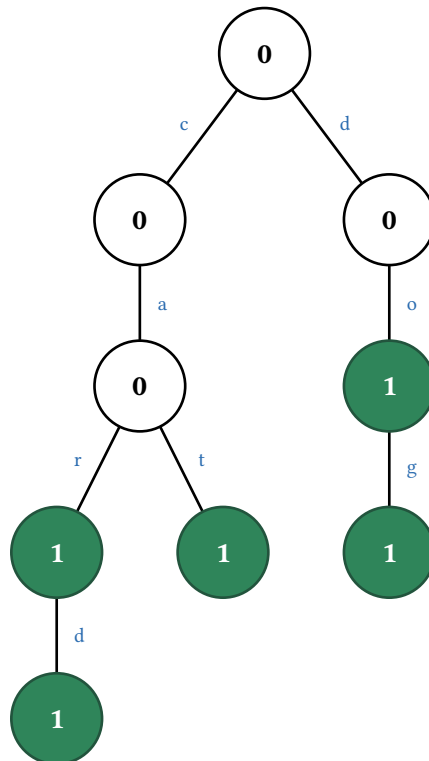
The four traversals sample the traversal palette across the **compartment** visit order — in-order threads each key between its flanking subtrees, while pre- and post-order group a node's keys at the node visit.



13. Tries

trie stores a **set of strings**. A node has no key of its own: its identity is the prefix spelled by the edges from the root down to it, which gives the drawing its two teaching conventions.

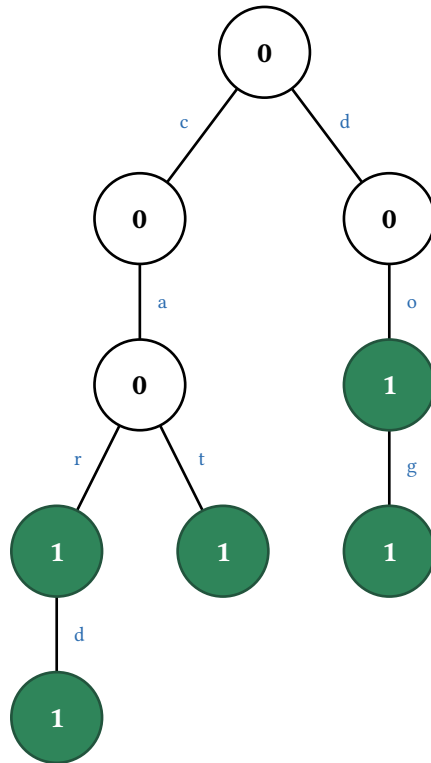
- **Letters live on the edges**, drawn in the render theme's edge-tag-fill — the same persistent-label slot AVL heights and graph weights use. Reading the edges spells the prefix.
- **Nodes show their word-end bit** — 1 when a stored word ends there, 0 for an interior prefix — and word-end nodes are shaded from the theme's trie section.

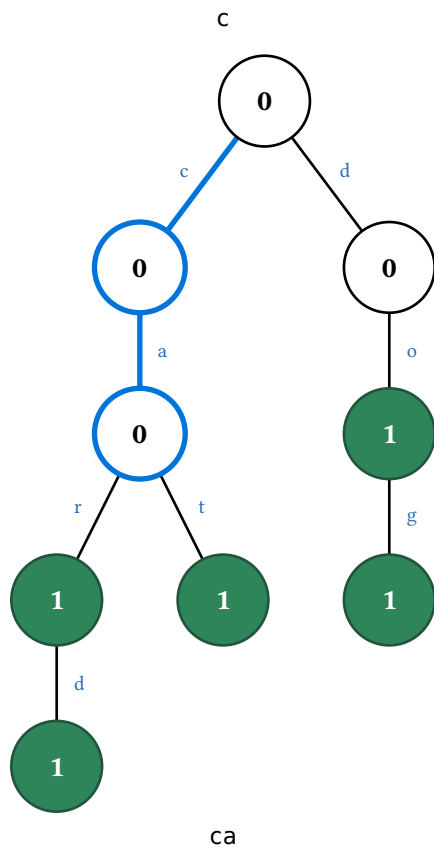
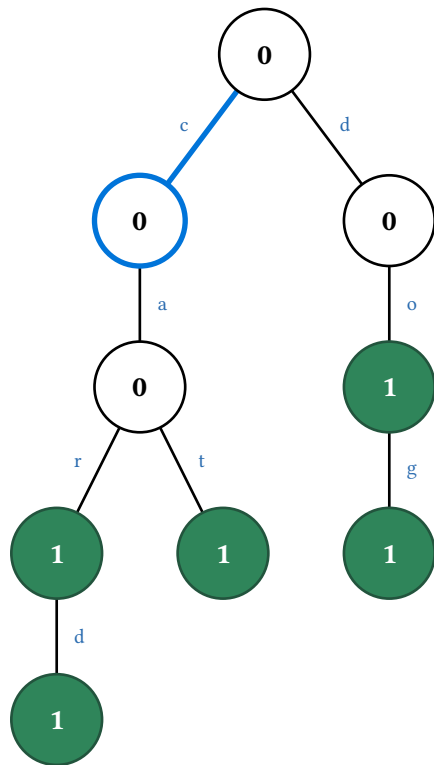


Note that "do" is both a stored word and an interior node on the way to "dog": a word end need not be a leaf.

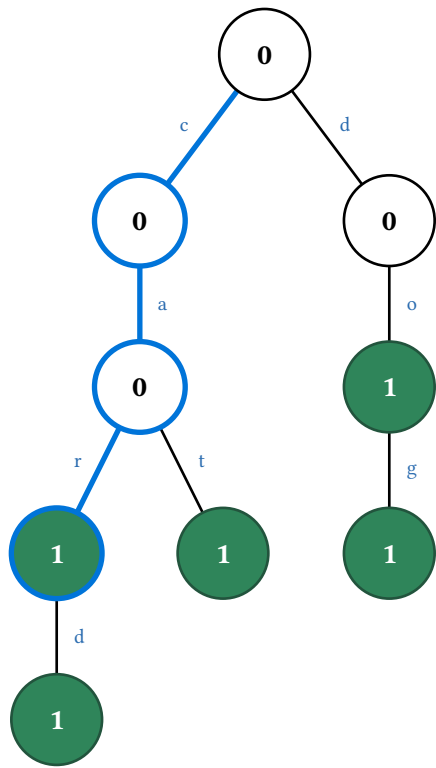
13.1. Search

`search-display(t, word)` walks one character at a time and ends in one of three ways: at a word end (**found**), at an interior node (**a prefix only**), or at a character with no matching edge (**a miss**, ringing the last node it did match).

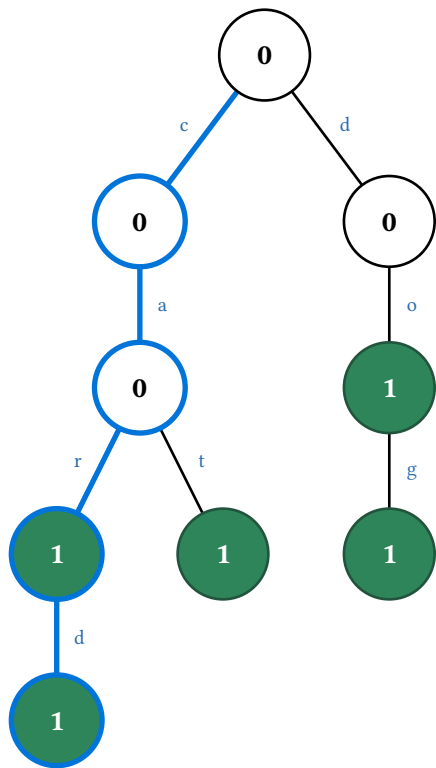




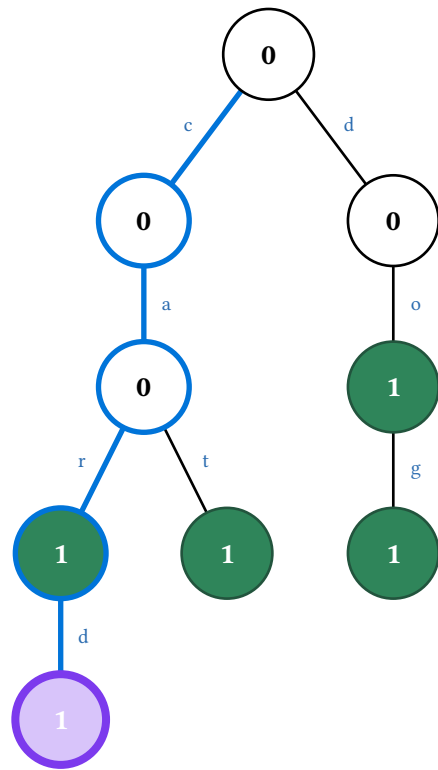
ca



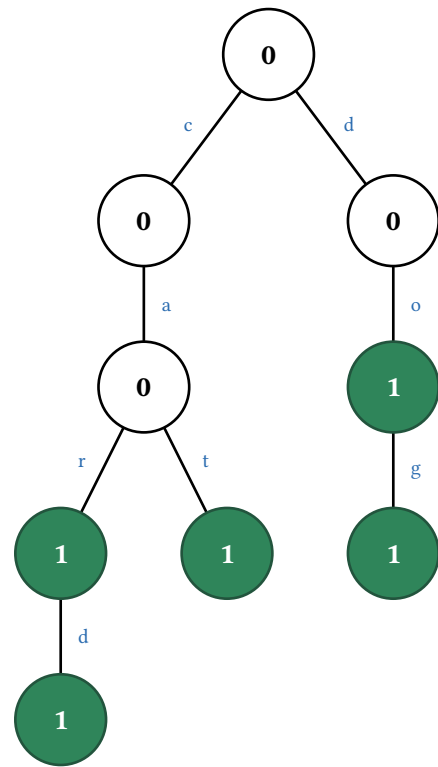
car

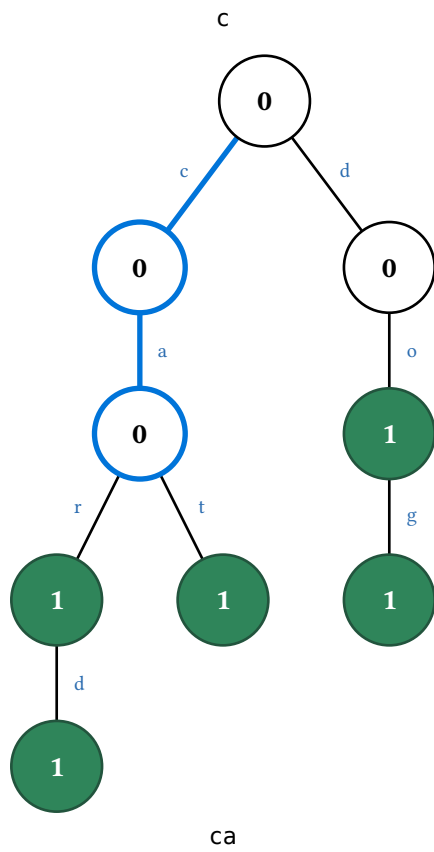
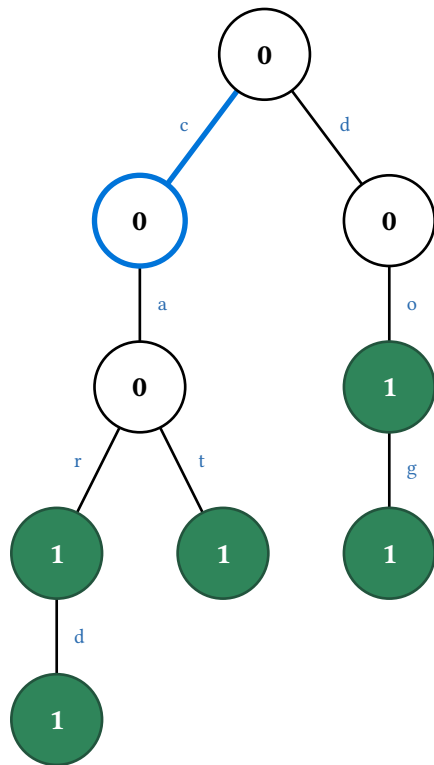


card

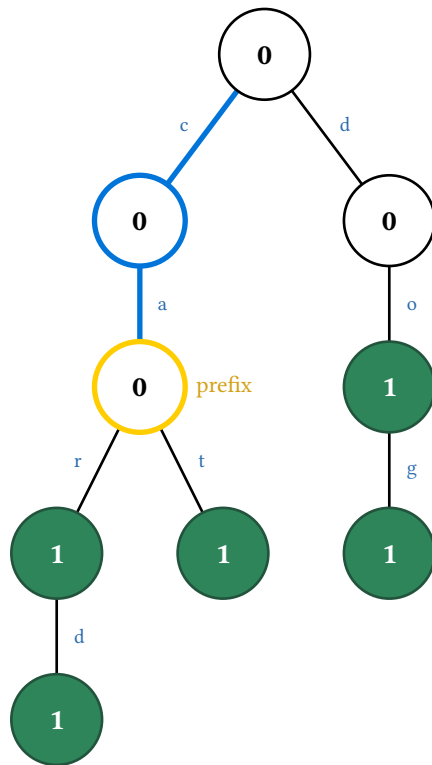


found "card"





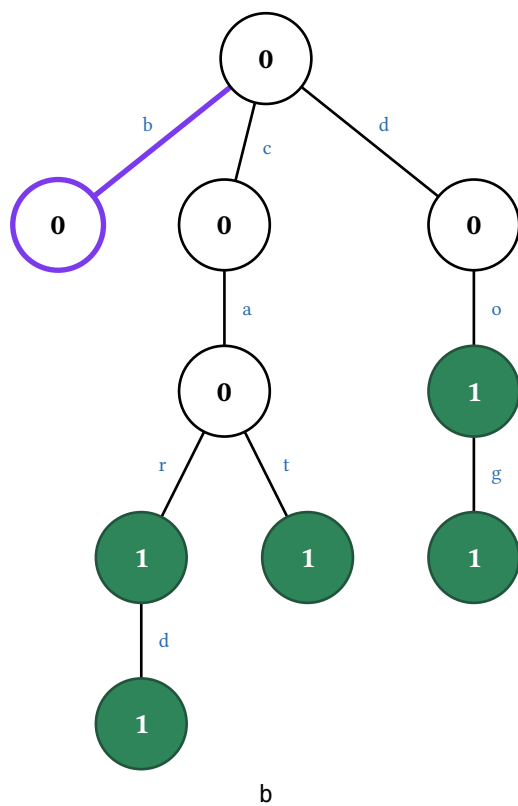
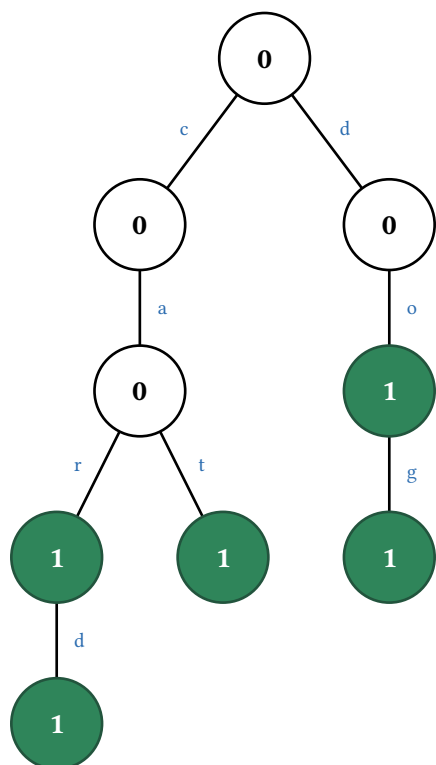
ca

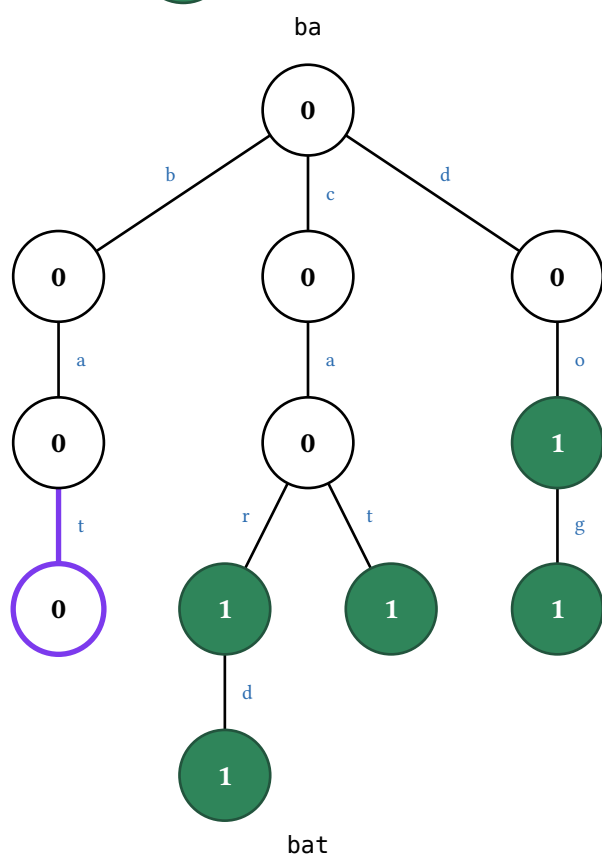
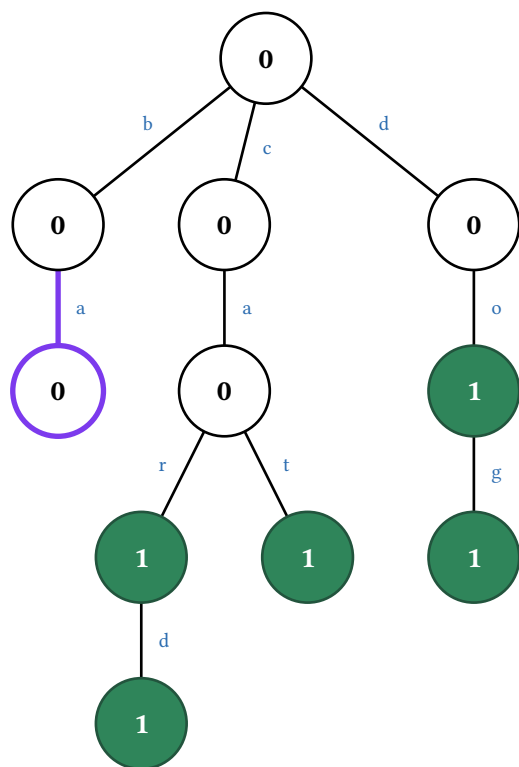


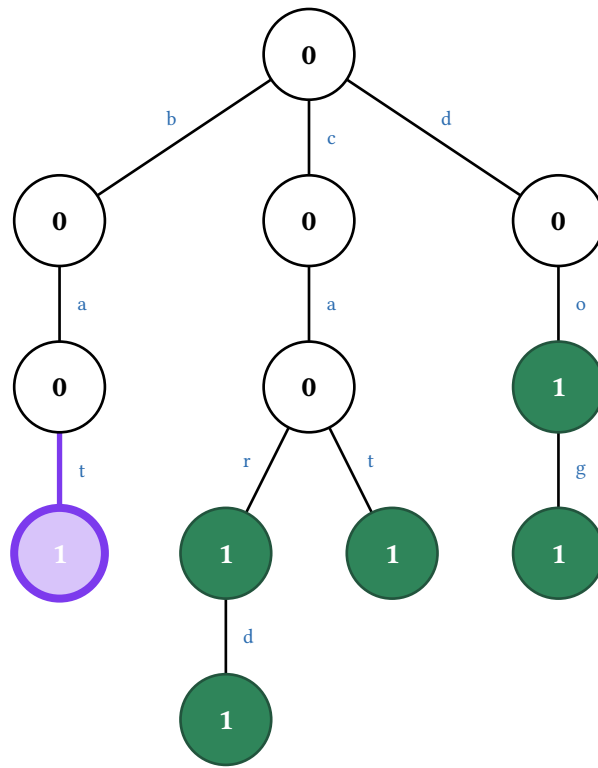
"ca" is a prefix only

13.2. Insert and delete

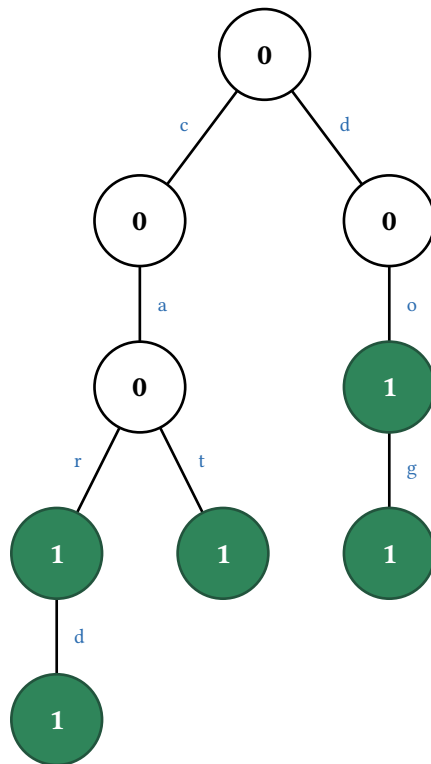
Insert walks the longest existing prefix, then grows the remaining suffix one node per frame before flipping the endpoint's bit. Delete unmarks the word end and then prunes the dead branch one node per frame, stopping at the first ancestor that is itself a word end or has another child.

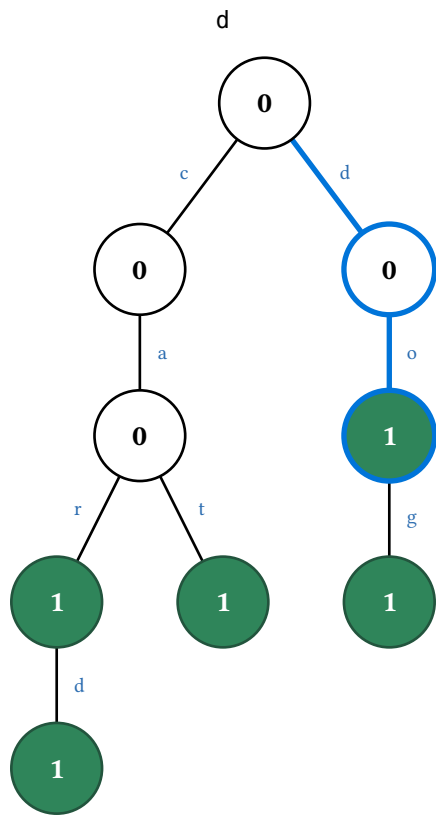
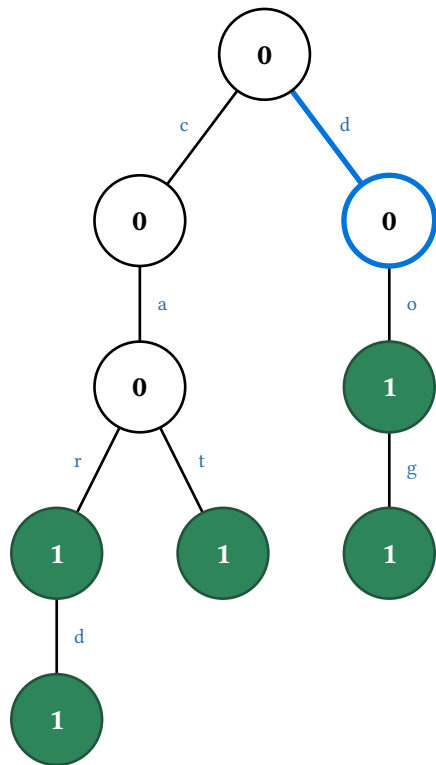




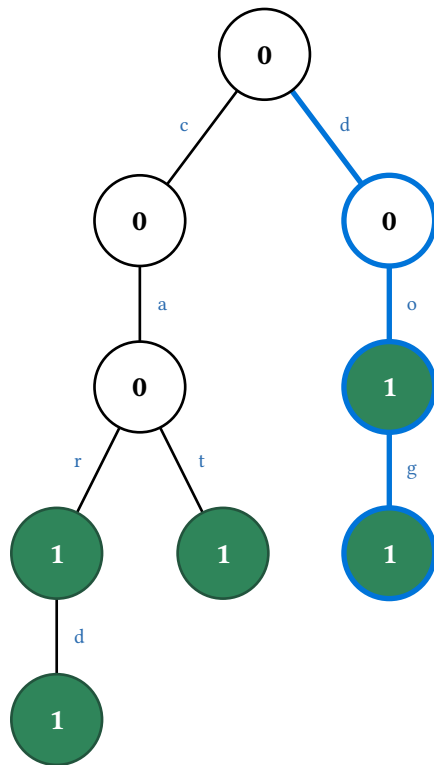


inserted "bat"

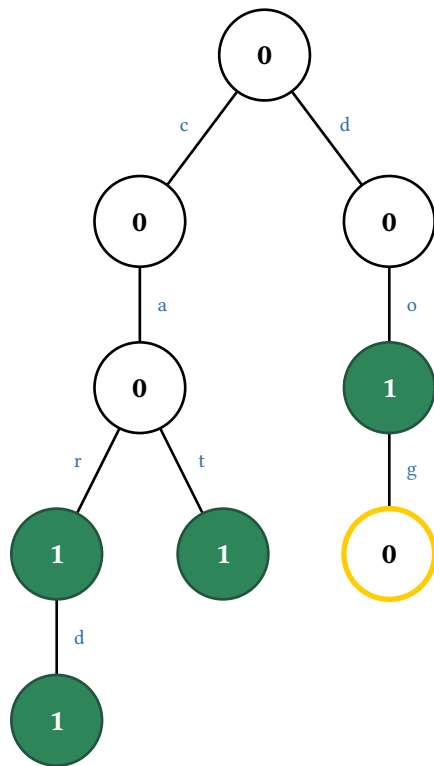




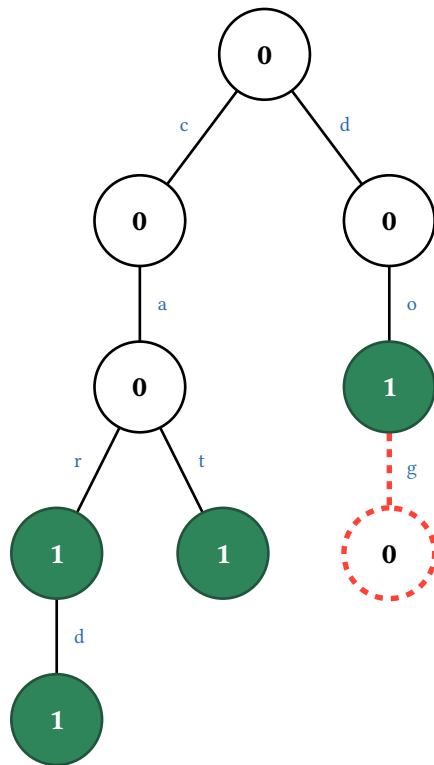
do



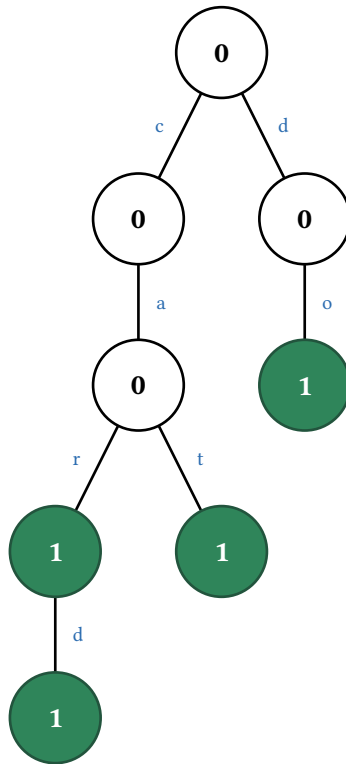
dog



unmark "dog"



prune "dog"



deleted "dog"

Both take the shapes that change nothing gracefully: inserting an existing prefix only flips a bit, and deleting a word that other words extend only unmarks it.

14. Graphs

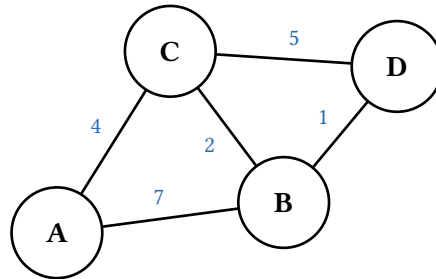
graph is an undirected-or-directed weighted graph with animations for Prim's and Kruskal's minimum spanning trees, Dijkstra's shortest paths, and breadth- and depth-first traversal. It has no root, so nodes are named by plain string ids and edges by `graph.edge-key(u, v)`.

14.1. Construction and layout

Layout is decoupled from drawing: the graph carries node positions in `cetz` units and the backend draws nodes there.

```
#let g = graph.new(  
  (("A", 0, 0), ("B", 3, 0.4), ("C", 1.5, 2.4), ("D", 4.5, 2.2)),  
  edges: (("A", "B", 7), ("B", "C", 2), ("A", "C", 4),  
          ("C", "D", 5), ("B", "D", 1)),  
)
```

A node spec is a bare id, an `(id, label)` pair (label but no position — pair it with `auto-layout`), an `(id, x, y)` triple, or `(id, x, y, label)`. An edge's third slot dispatches on type: a **number** is the weight, a **string or content** is a label drawn in its place, and the four-slot form `(u, v, weight, label)` sets both — so an algorithm can run on the numeric weight while the drawing shows 4 ms.



Hand placement is the first-class path — small teaching graphs read best when you control the layout — and every display takes `scale:` (default 1) to multiply the coordinates while the drawn node size stays fixed. Reach for it when a hand-placed graph is too cramped for a wide slide; to resize everything together, wrap the result in Typst's own `scale` instead.

14.2. Adjacency tables

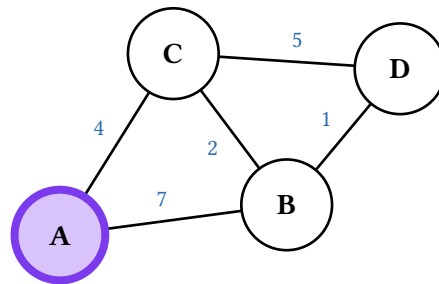
`adjacency-matrix(g)` and `adjacency-list(g)` return placeable Typst tables rather than frames — no `cetz`, no animation. They show 1/0 presence and bare neighbour ids by default; `weights: true` shows each edge's display label instead, marking non-edges with `none-marker` and empty rows with `empty-marker`. A directed graph reads `row = source`, `column = target`.

	A	B	C	D
A	.	7	4	.
B	7	.	2	1
C	4	2	.	5
D	.	1	5	.

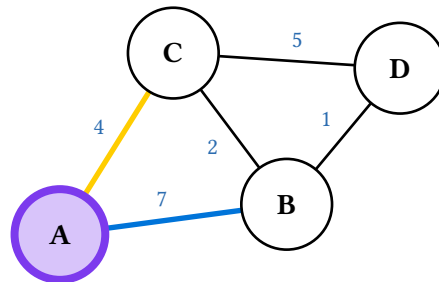
A	B (7), C (4)
B	A (7), C (2), D (1)
C	B (2), A (4), D (5)
D	C (5), B (1)

14.3. Prim's minimum spanning tree

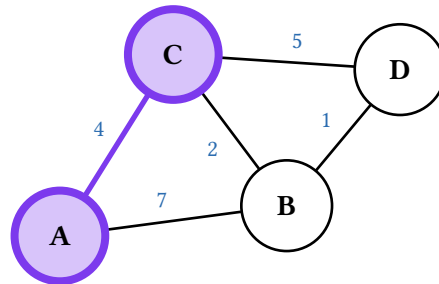
`prim-display(g, start)` grows the tree from `start`. Each round shows the frontier — the crossing edges — with the lightest picked out, commits it, and settles the newly reached node, while the caption tracks the running weight. A terminal **prune** frame then hides every non-tree edge, so the animation ends on the spanning tree alone.



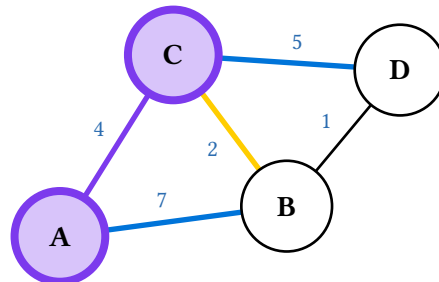
Start at A



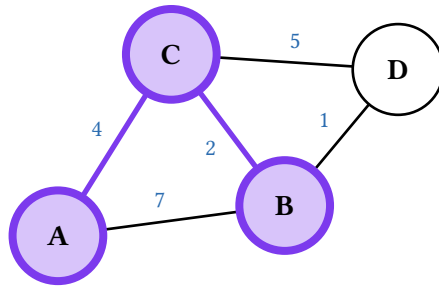
Min crossing edge: A-C (4)



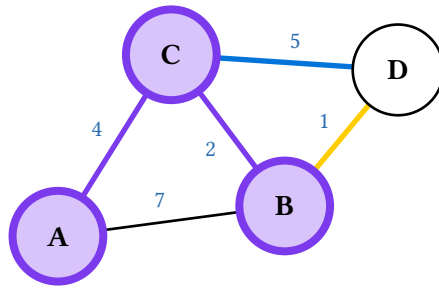
Add A-C tree weight 4



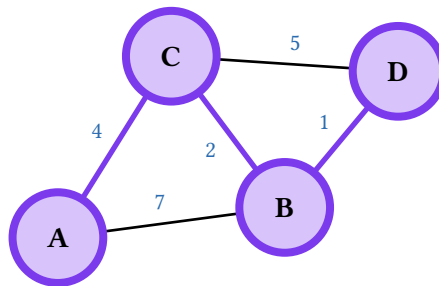
Min crossing edge: B-C (2)



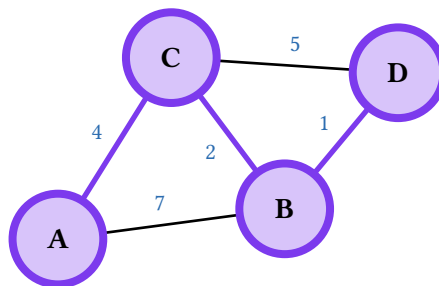
Add B-C tree weight 6



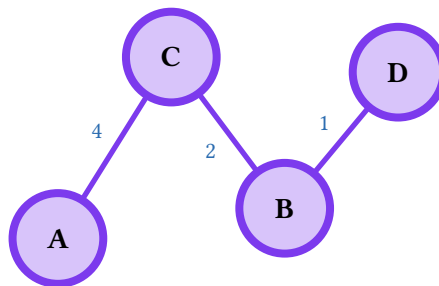
Min crossing edge: B-D (1)



Add B-D tree weight 7

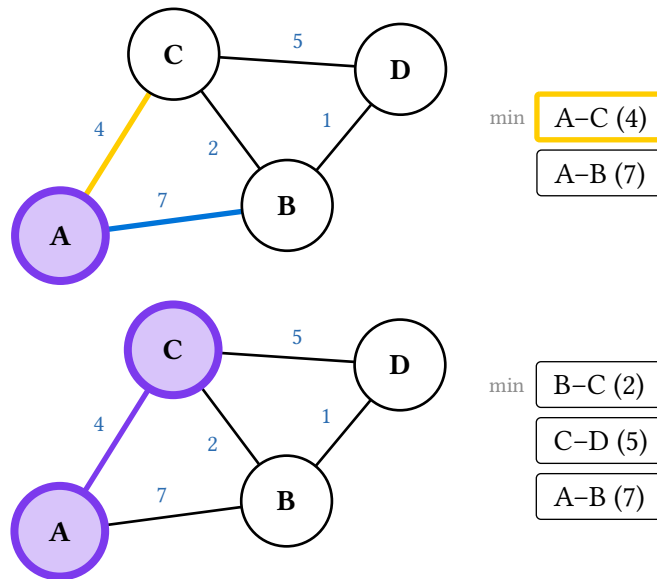


MST weight 7



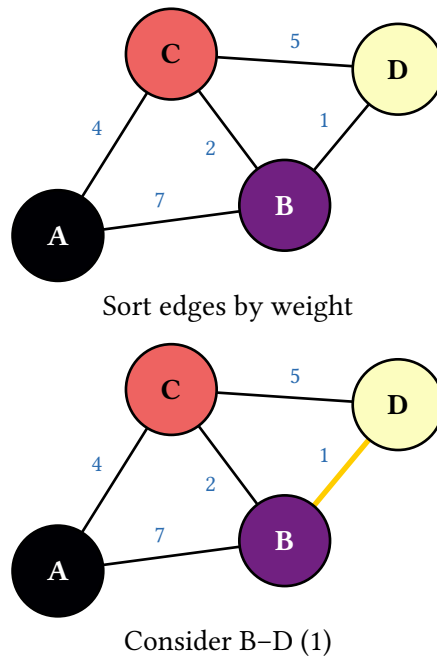
Spanning tree

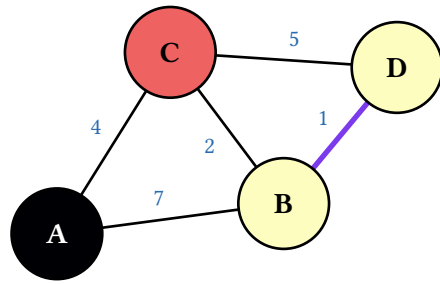
Prim's bookkeeping is a priority queue of crossing edges, and aux-strip renders it beside the canvas — min on top, the chosen edge ringed:



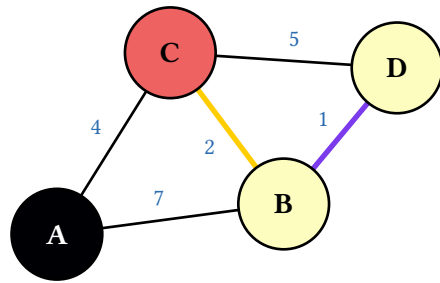
14.4. Kruskal's minimum spanning tree

`kruskal-display(g)` considers edges in weight order, adding each unless it would close a cycle. The union-find forest shows as **node colour**: same colour means same set, and colours merge as components do.

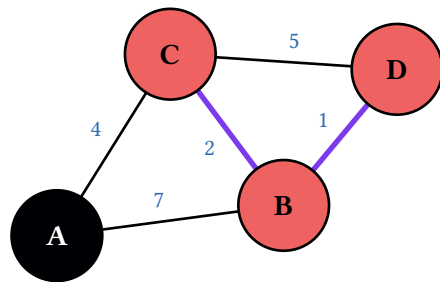




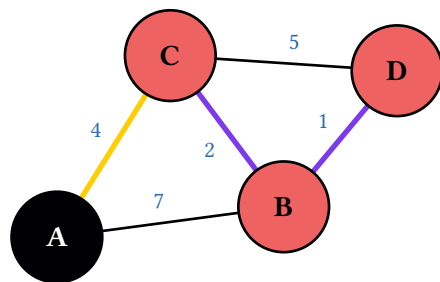
Add B-D weight 1



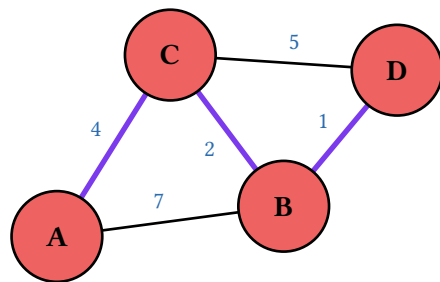
Consider B-C (2)



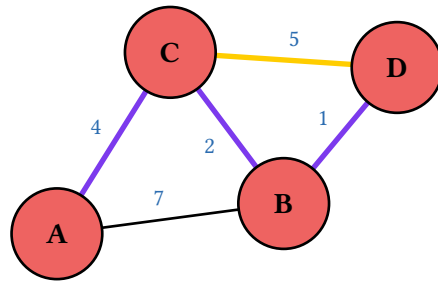
Add B-C weight 3



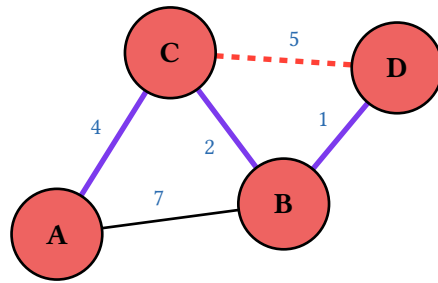
Consider A-C (4)



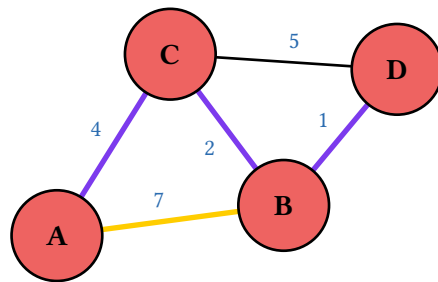
Add A-C weight 7



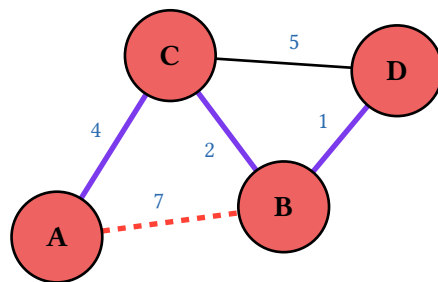
Consider C-D (5)



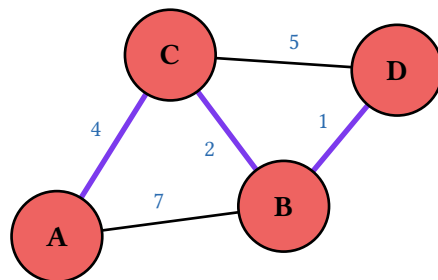
Reject C-D (cycle)



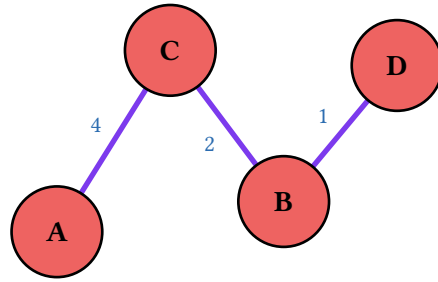
Consider A-B (7)



Reject A-B (cycle)

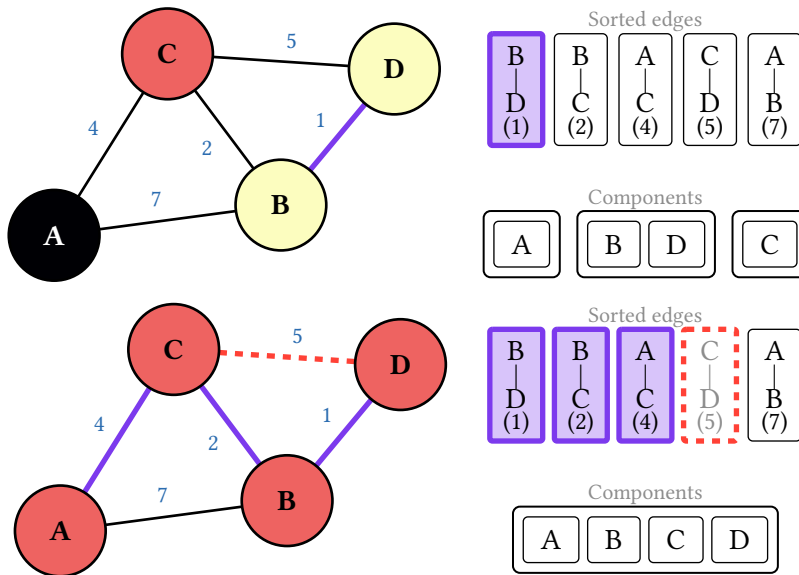


MST weight 7



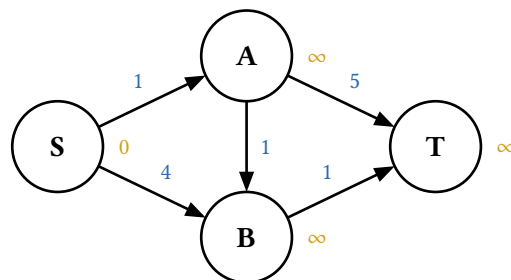
Spanning tree

Kruskal carries two auxiliary views, and `aux-strip` stacks both: the sorted edge list with a cursor under the edge being considered and each edge tagged added / rejected / pending, and the disjoint-set partition. Pass view: "partition" to place one alone.

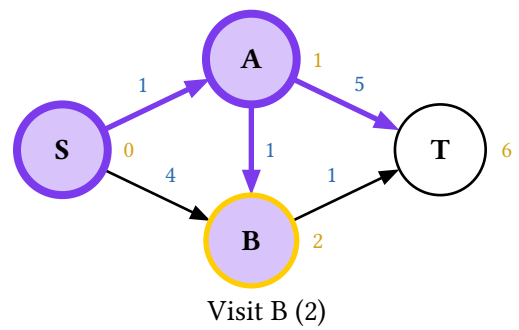
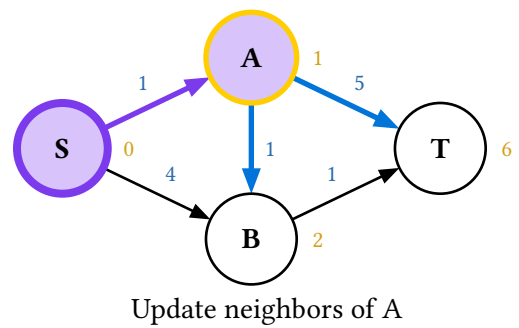
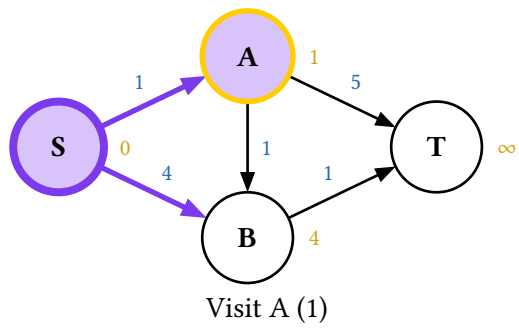
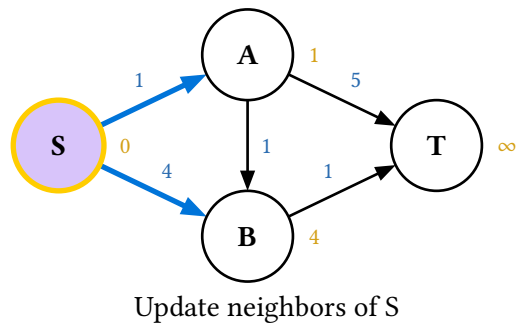
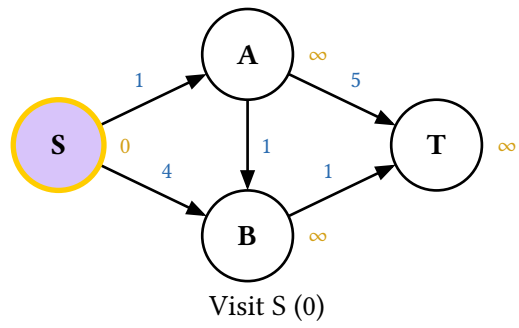


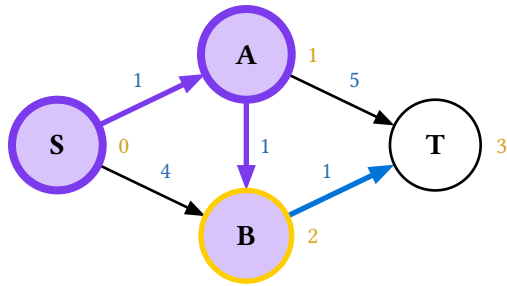
14.5. Dijkstra's shortest paths

`dijkstra-display(g, source, target: none)` models the priority queue the way students implement it: the queue starts with source alone, and each improving relaxation **adds a fresh (node, dist) entry** rather than decreasing a key — so a node can sit in the queue several times. Each round polls the minimum, marks it visited, and relaxes its unvisited neighbours. Polling a stale duplicate for an already-visited node produces a **skip** frame, which is the reason the `if u is not visited` guard exists. Ties break alphabetically.

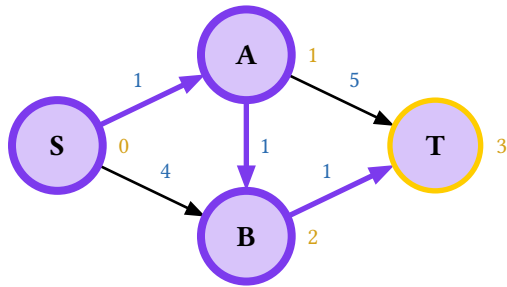


Add S to queue

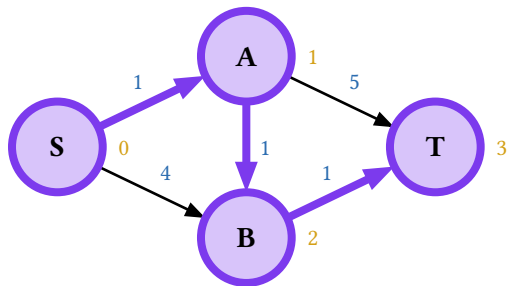




Update neighbors of B

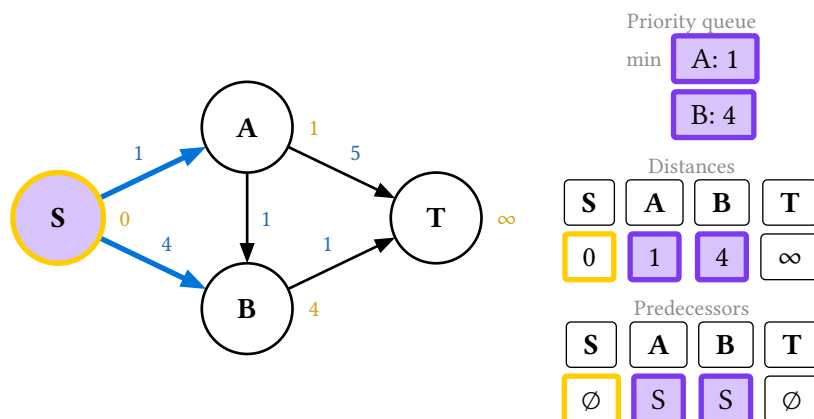


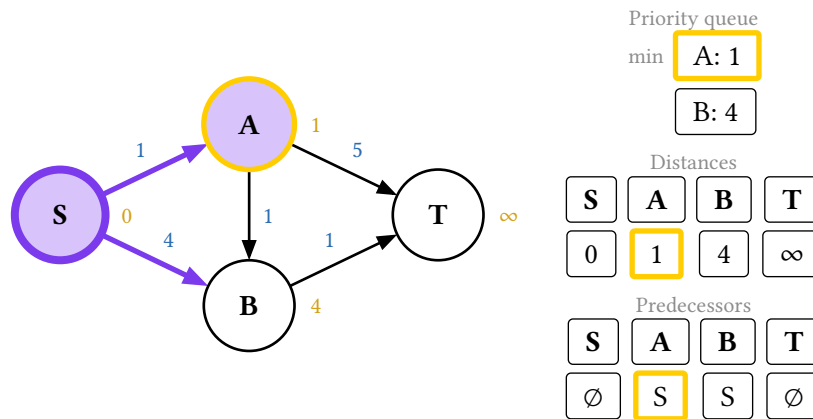
Visit T (3)



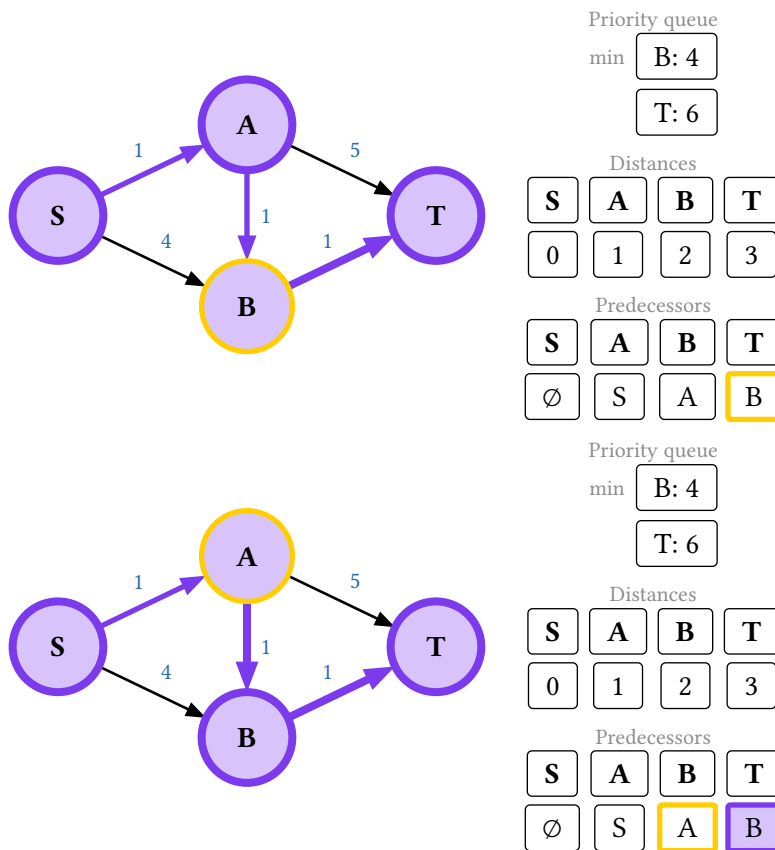
Shortest path to T: 3

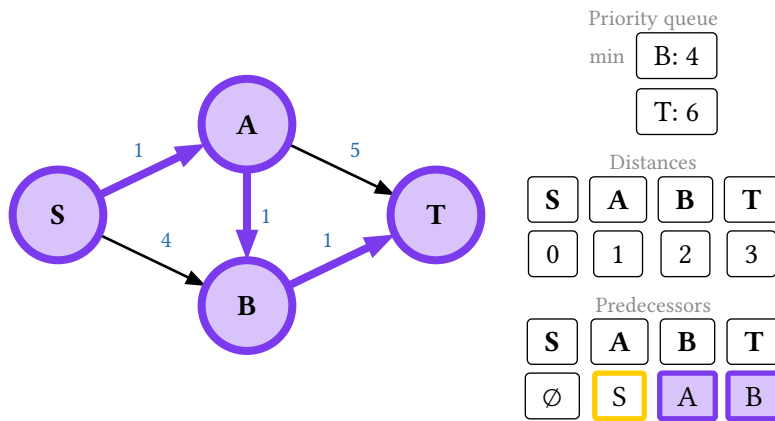
Three auxiliary views ride along — the queue, the dist map, and the prev map — selectable with view: "dist-pq" / "dist-map" / "prev-map":





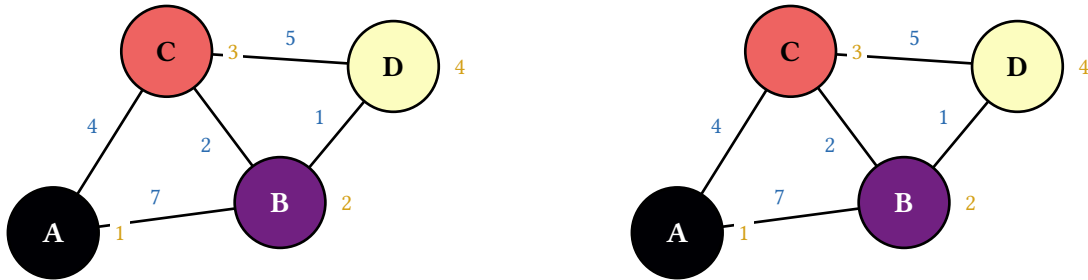
Two flags tailor the canvas. `node-distances: false` drops the tentative distances from the nodes, for when the `dist` view already carries them. `reconstruct: true` (which needs a target) appends the path-reconstruction phase: instead of lighting the whole path at once, it walks `prev` backwards from the target, prepending one node per frame while the `prev` view traces the chain it reads.





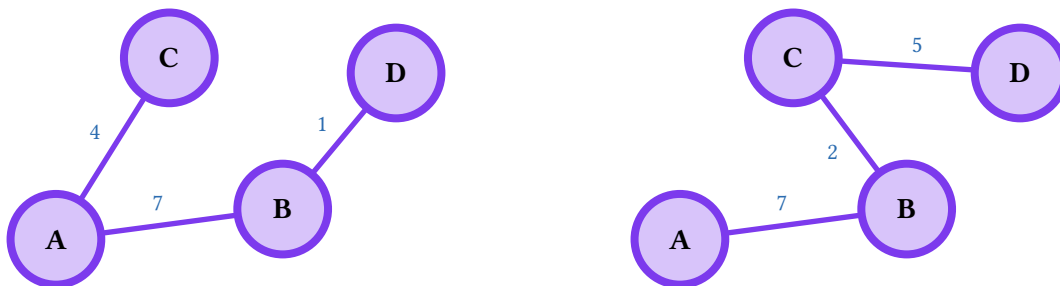
14.6. Traversals

`bfs-display(g, start)` and `dfs-display(g, start)` sample the traversal palette across the visit order, badge each node with its 1-indexed position, and accumulate the sequence in the caption — the same vocabulary as the tree traversals.

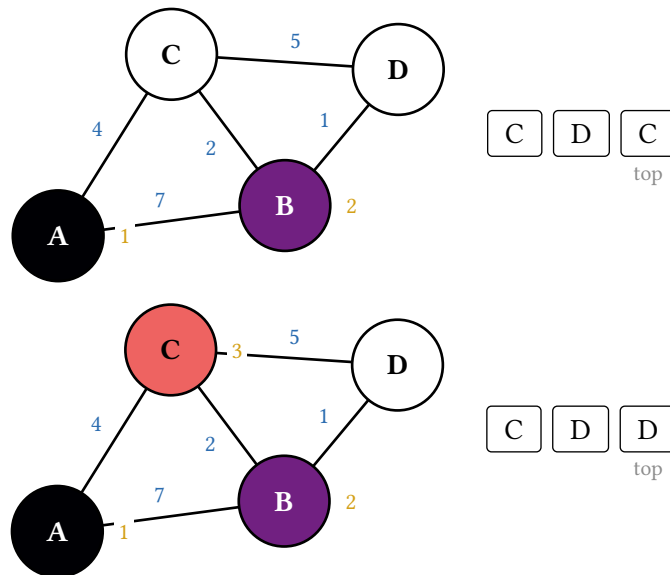


Three flags:

- **target**: turns the traversal into a **search** that stops the moment the target is visited, ending on Found `<t>` — or, if it is unreachable, on `<t> not found` after the whole component.
- **sort-frontier**: true enqueues each node's unseen neighbours in ascending id order instead of edge-declaration order, for a deterministic walk. (The sort is lexicographic, so "10" precedes "2".)
- **spanning-tree**: true renders the **traversal tree** instead of the palette walk: each node joins with a uniform commit style and its discovery edge lights up, accumulating into the BFS or DFS spanning tree, and a final frame prunes the cross edges away. It covers the whole component, so it takes no target:.

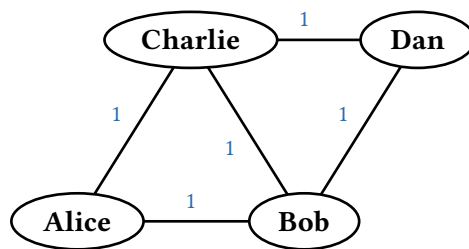


Every frame carries the queue or stack in its step, and the DFS stack is faithful to the iterative algorithm — the same id can appear twice, because a node may be pushed again before an earlier copy is popped:



14.7. Node shapes and sizing

The default circle is sized for short ids. Pass `node-style:` to any `display` to restyle every node at once — it sits beneath the per-frame algorithm styling, so shape and size persist while fills and strokes animate on `top`. `shape` takes "circle", "ellipse", or "rectangle"; `autosize: true` fits each node to its own label; `r` or `rx / ry` pin a size instead. Edges trim to whatever boundary the shape defines, so arrowheads stay flush.



14.8. Optional auto-layout with graphviz

For graphs too large to place by hand, `auto-layout(g, engine:, unit:, sizes:)` computes positions with graphviz through the `diagraph-layout` package — a WASM build, so no external binary is involved. Feed its result to any `display`'s `positions:`, or skip the intermediate map by passing `layout:` and letting the display call it:

```
#let g = graph.new(("A", "B", "C"), edges: (("A", "B", 7), ("B", "C", 2)))

#last(graph.prim-display(g, "A", positions: auto-layout(g)))
#last(graph.display(g, layout: "neato", node-style: (shape: "ellipse", autosize:
true)))
```

The `layout:` form feeds graphviz a per-node size estimate so wide labels get spread apart. That estimate is a cheap label-length heuristic computed without measure, whereas the drawn node is measured exactly — the two only approximate each other, and graphviz's margins absorb the slack. Tune the result with the display's `scale:` or with `layout-unit:`.

diagraph-layout stays an **optional** dependency even though the entrypoint re-exports auto-layout: the import sits inside the function's body, which Typst resolves only when the function is actually called. Projects that hand place their graphs never fetch it.

15. Hash maps

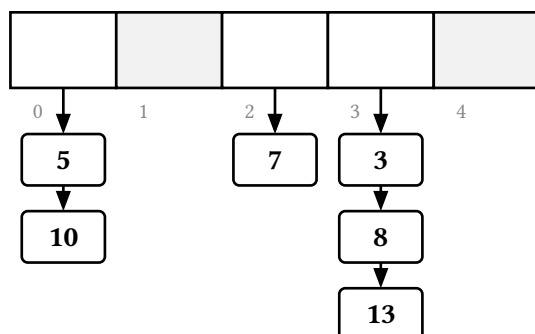
A hashmap is a fixed array of capacity slots, a **pluggable** hash function, and one of four collision strategies:

- "chaining" — each slot holds a list of entries.
- "linear" — open addressing, probing $(h + i) \bmod m$.
- "quadratic" — open addressing, probing $(h + i*i) \bmod m$.
- "double" — open addressing, probing $(h_1 + i * h_2) \bmod m$, with the step from a second hash.

There is no layout to supply — the geometry follows from the capacity — so the displays take an orientation: ("horizontal", the default, or "vertical") instead of positions.

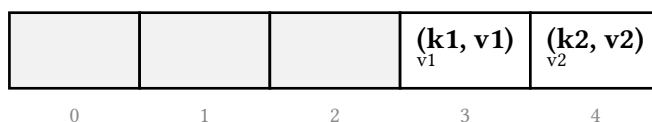
```
#let h = hashmap.new(5, strategy: "chaining", entries: (5, 10, 7, 3, 8, 13))
```

entries: seeds the table by inserting each item in order (a bare key, or a (key, value) pair). hash: is any $(k, m) \Rightarrow \text{index}$ and hash-repr: its on-screen form, with k and m substituted as whole words — so write them standalone $((k * 3) \bmod m)$, since "3k" will not substitute. Double hashing takes a second pair, hash2: / hash2-repr:, defaulting to $1 + (k \bmod (m - 1))$: nonzero, and coprime to a prime m , so the probe visits every slot.



Empty slots are muted, a bucket's chain hangs below it, and a deleted open-addressing slot shows a tombstone. Entries carry a key and an optional value, drawn as a smaller second line.

By default the displays **fit** each cell to the widest label in the whole animation — in both width and height, so entries stay legible at slide font sizes, and so the table never resizes mid-animation as a label grows. That is cell-width: "fit"; a number pins an exact width in cetz units, and auto keeps a fixed footprint without measuring (which is what positioned defaults to, since that path can run outside a layout context).



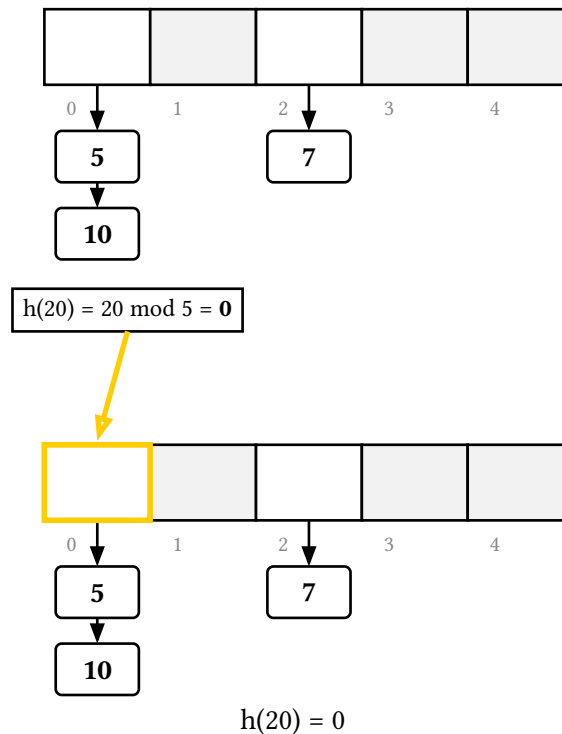
15.1. The hash box

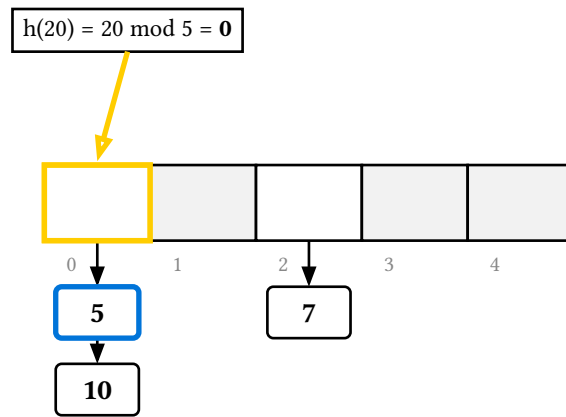
Every insert / search / delete opens by computing the hash, and the animation draws a **hash box** — $h(\text{key}) = \text{<formula>} = \text{<index>}$ — above the target slot with an arrow into it, so the key $\rightarrow$ bucket mapping is explicit before any probing starts.

The opening frame invisibly reserves that box's exact footprint, and the terminal frames that drop it reserve it again, so every frame of the walk renders at the same canvas size and the table does not jump when the box appears or disappears on the next subslide. Top-align the canvas in your deck and the whole animation stays pinned.

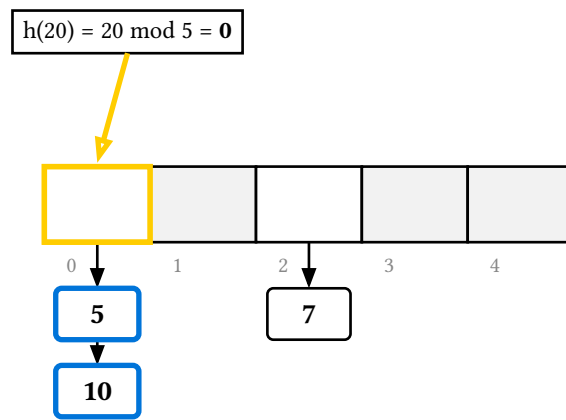
15.2. Insertion, by strategy

Chaining hashes to a bucket, walks the chain comparing keys — so a repeated key updates in place — and appends at the tail:

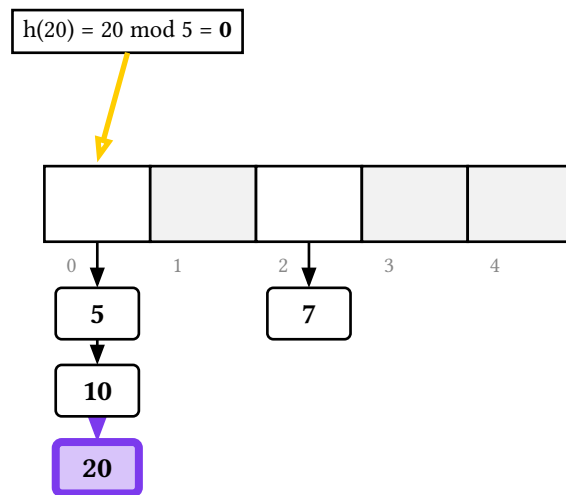




compare 5



compare 10



insert 20

Linear probing steps to the next slot until it finds a free one:

14	21	7				
0	1	2	3	4	5	6

$$h(28) = 28 \bmod 7 = 0$$



14	21	7				
0	1	2	3	4	5	6

$$h(28) = 0$$

$$h(28) = 28 \bmod 7 = 0$$



14	21	7				
0	1	2	3	4	5	6

probe 2

$$h(28) = 28 \bmod 7 = 0$$



14	21	7				
0	1	2	3	4	5	6

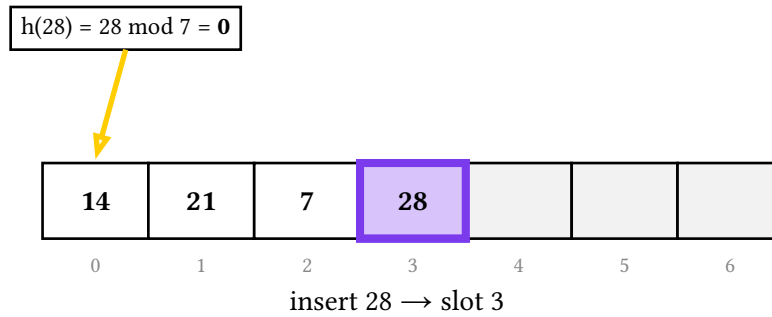
probe 3

$$h(28) = 28 \bmod 7 = 0$$

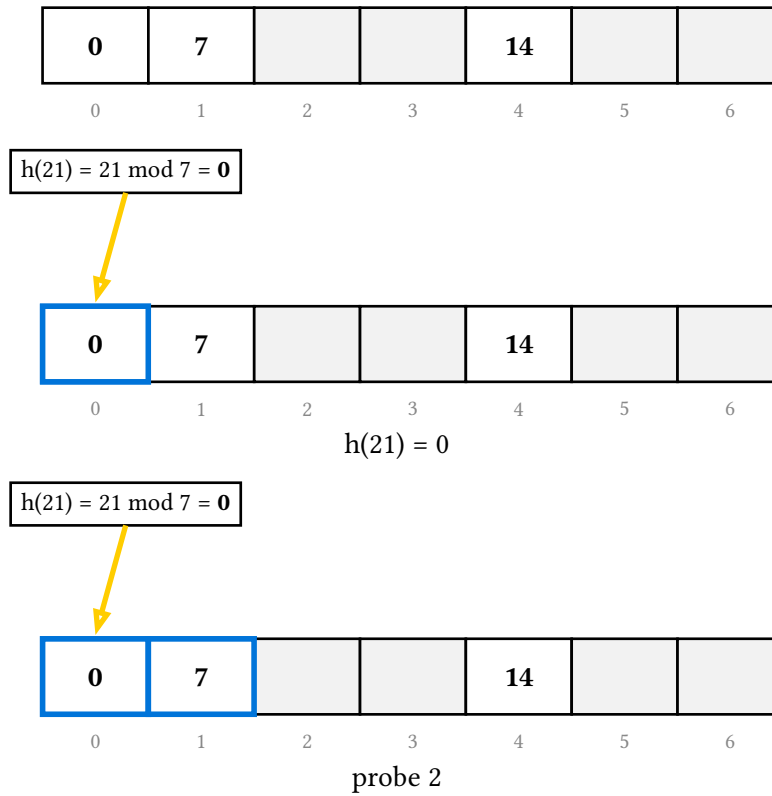


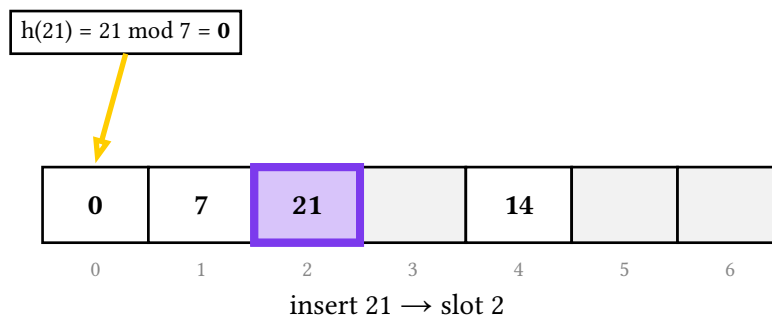
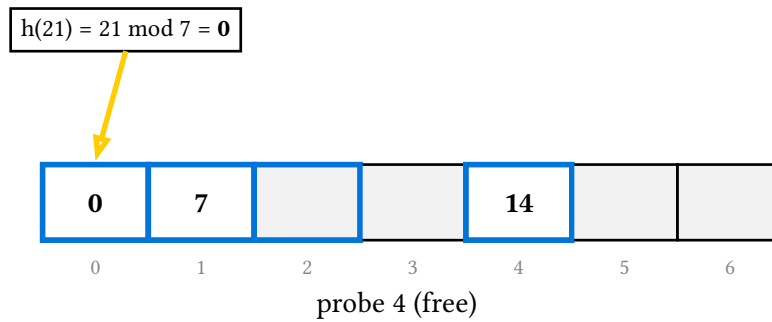
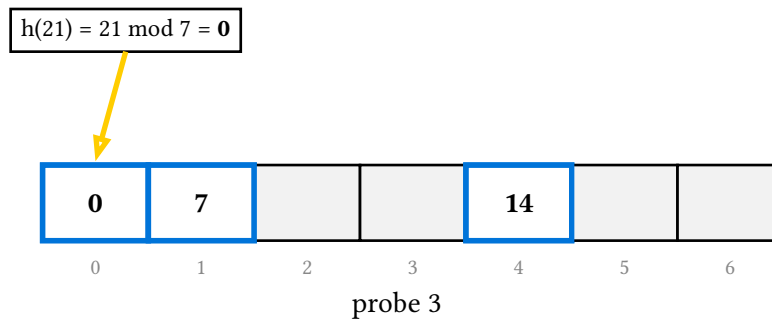
14	21	7				
0	1	2	3	4	5	6

probe 4 (free)

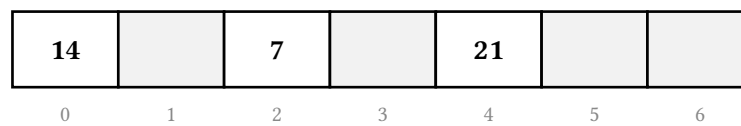


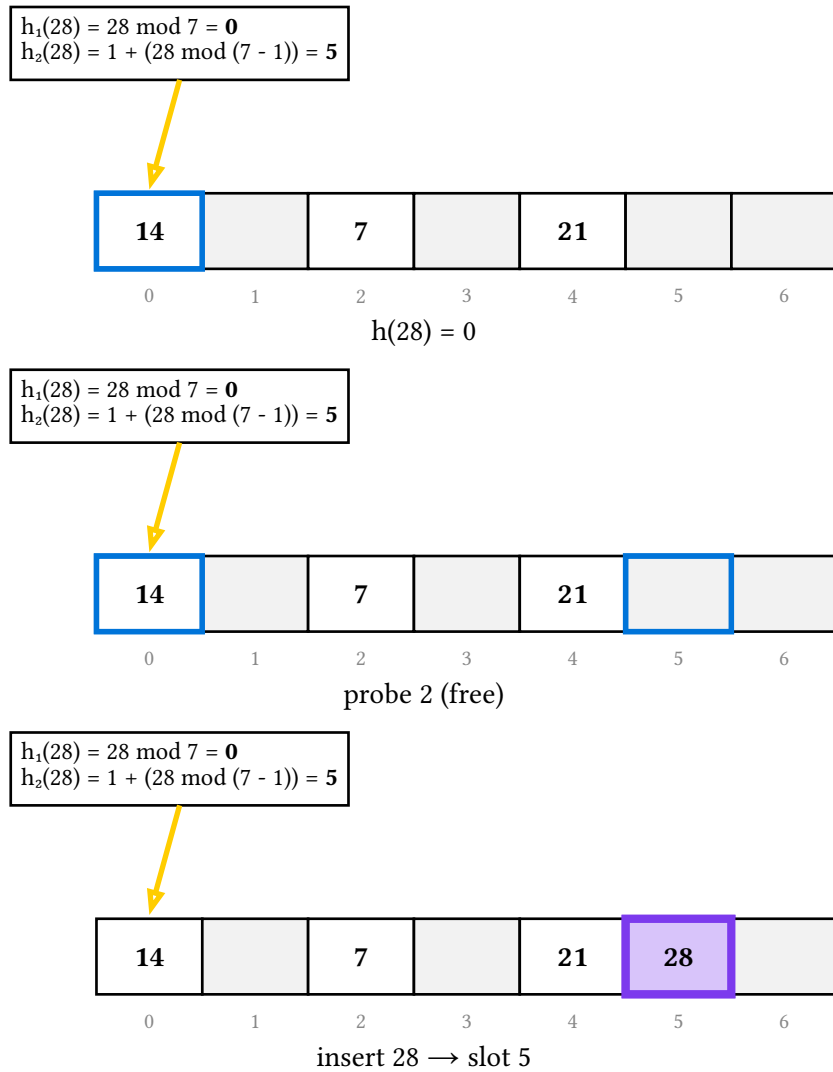
Quadratic probing spreads the probes out, which reduces the primary clustering linear probing suffers. The trade-off is that its sequence visits only a subset of the slots: it can report the table **full** while empty slots remain — a genuine teaching point, and a terminal frame of its own.





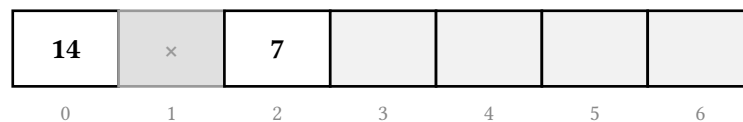
Double hashing takes the **step size** from a second hash, so two keys that collide at the same home slot generally walk different slots afterwards. The hash box shows both.

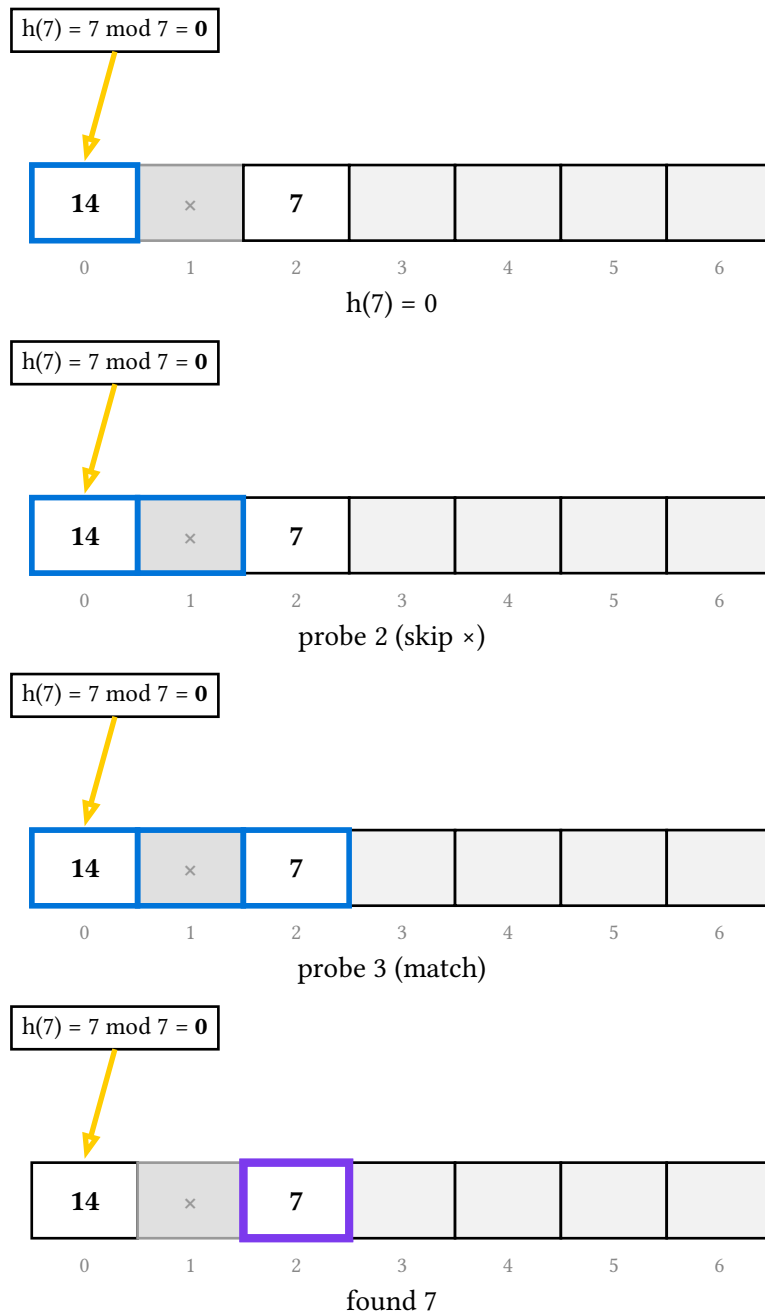




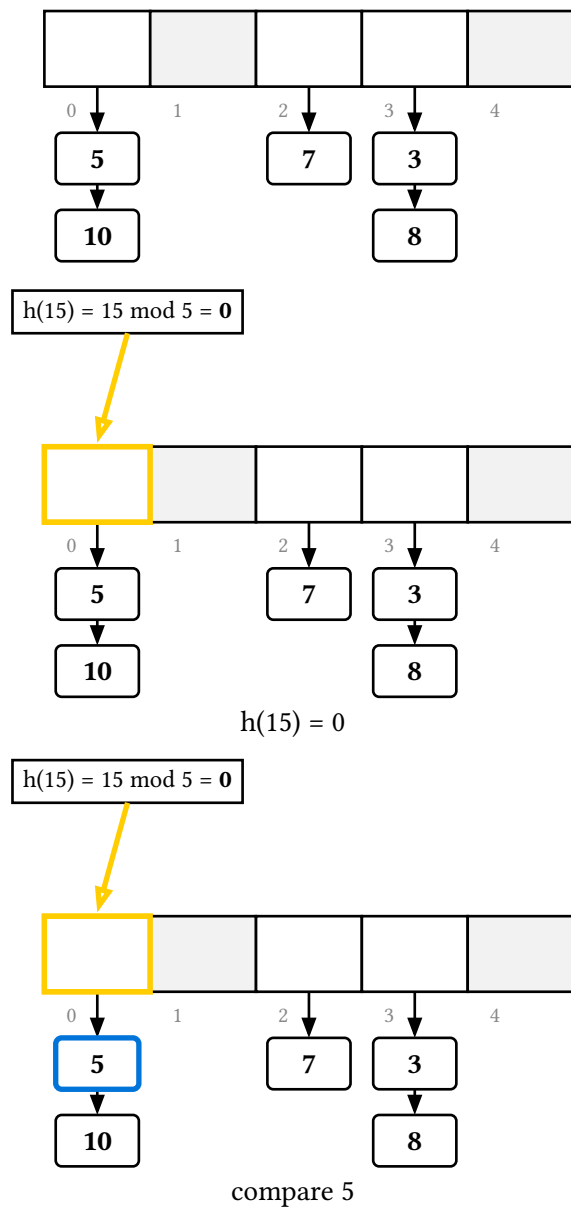
15.3. Search

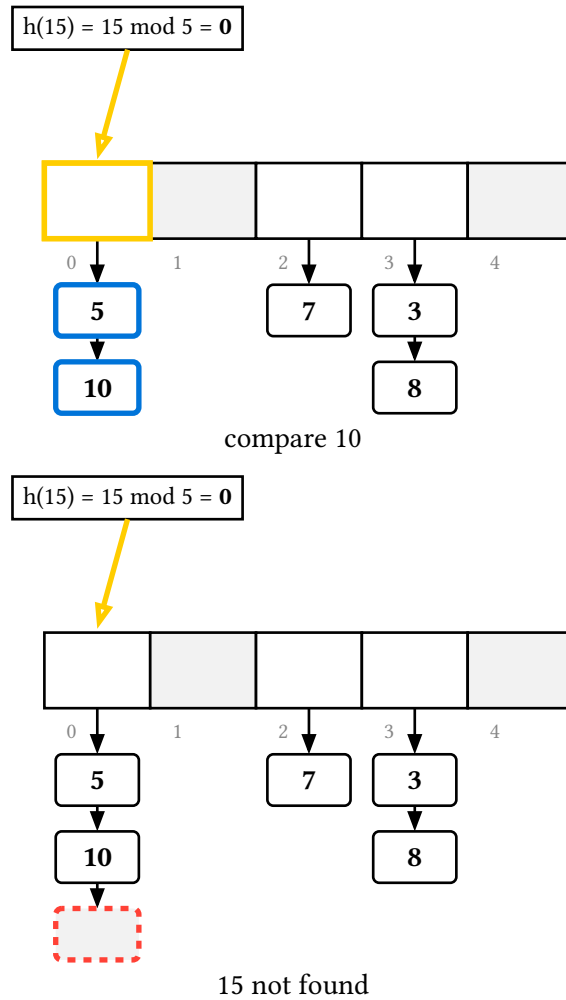
Open-addressing search **stops at the first empty slot** but **steps over tombstones**, so a lookup for a key sitting past a deleted entry still finds it:





A chaining miss walks to the end of the bucket and rings a **phantom** null cell hung one past the last entry — the slot the search fell off the chain into — rather than the keyless bucket header, which would read as if the bucket itself were the miss.

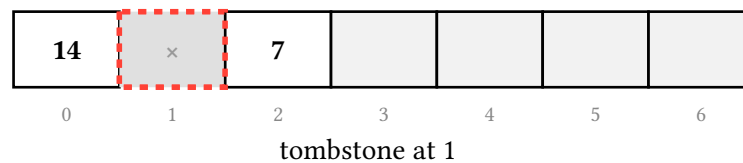
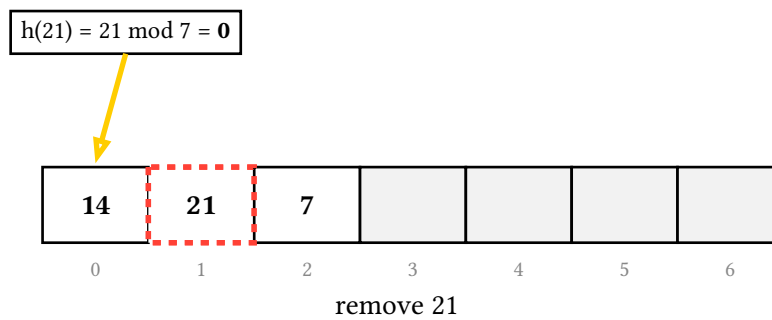
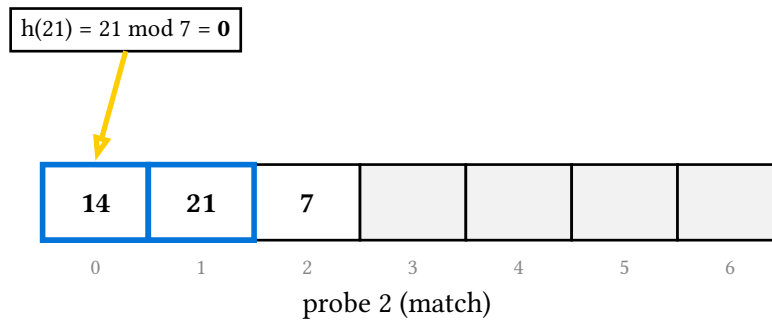
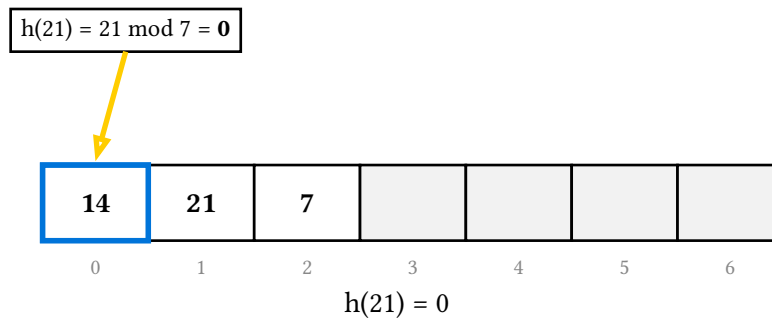




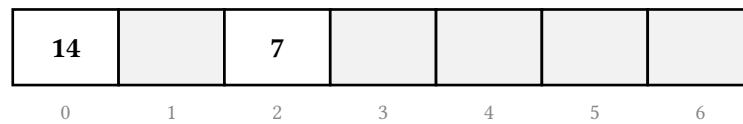
15.4. Deletion, tombstones, and a deliberate bug

Chaining unlinks the entry and re-links the chain. Open addressing cannot blank the slot — that would truncate every probe sequence running through it — so it writes a **tombstone** that search skips and insert may reuse.





To show **why**, delete and delete-display take tombstone: false: the naive deletion that clears the slot instead. Here 14, 21 and 7 all hash to slot 0, so clearing 21 severs the probe chain and a later search for 7 stops at the hole and wrongly reports a miss — even though 7 is still in slot 2.



$$h(7) = 7 \bmod 7 = 0$$



$$h(7) = 0$$

$$h(7) = 7 \bmod 7 = 0$$



probe 2 (free)

$$h(7) = 7 \bmod 7 = 0$$



7 not found

15.5. Resizing and rehashing

`resize-display(h, new-cap)` allocates a larger array and replays every live entry through the hash under the new capacity, one entry per frame, so the load factor drops and old collisions scatter:

14	21	7	3			
----	----	---	---	--	--	--

0

1

2

3

4

5

6

before (m = 7)

--	--	--	--	--	--	--	--	--	--	--

0

1

2

3

4

5

6

7

8

9

10

new array (m = 11)

$$h(14) = 14 \bmod 11 = 3$$

			14							
--	--	--	----	--	--	--	--	--	--	--

0

1

2

3

4

5

6

7

8

9

10

rehash 14

$$h(21) = 21 \bmod 11 = 10$$

			14							21
--	--	--	----	--	--	--	--	--	--	----

0

1

2

3

4

5

6

7

8

9

10

rehash 21

$$h(7) = 7 \bmod 11 = 7$$

			14				7			21
--	--	--	----	--	--	--	---	--	--	----

0

1

2

3

4

5

6

7

8

9

10

rehash 7

$$h(3) = 3 \bmod 11 = 3$$

			14	3			7			21
--	--	--	----	---	--	--	---	--	--	----

0

1

2

3

4

5

6

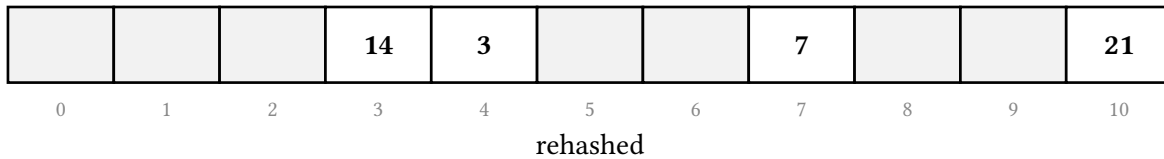
7

8

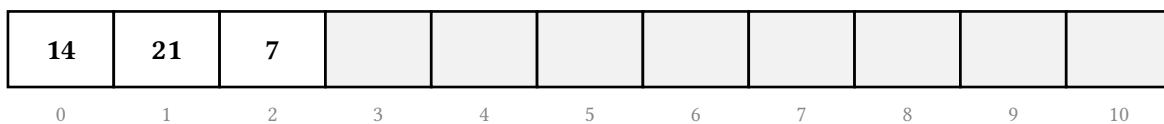
9

10

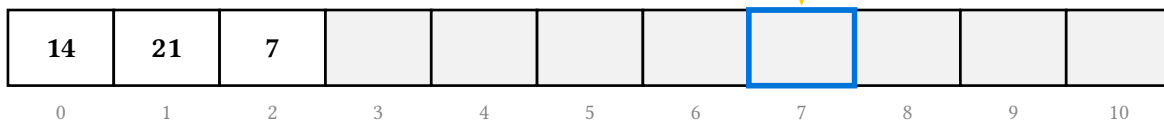
rehash 3



The matching teaching bug is rehash: `false`: grow the array but copy each entry to its **old** index. 14, 21 and 7 land back in slots 0, 1, 2, but $h(7) = 7 \bmod 11 = 7$ now — so a later search probes slot 7, finds it empty, and misses a key that is still in the table. (It needs `new-cap >= capacity`, so the old indices still fit.)

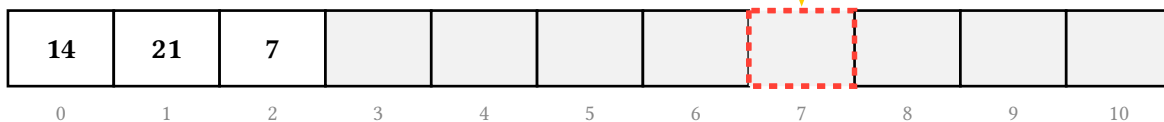


$$h(7) = 7 \bmod 11 = 7$$



$$h(7) = 7$$

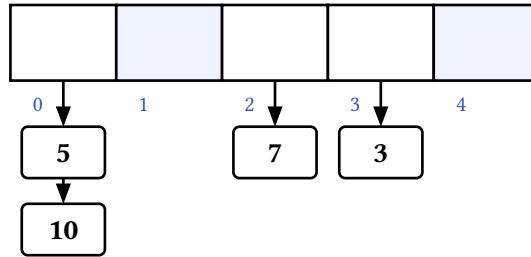
$$h(7) = 7 \bmod 11 = 7$$



7 not found

15.6. Palette

The hashmap theme section holds `empty-fill`, `index-fill`, `hash-box-fill / -stroke`, `tombstone-fill / -stroke`, and `chain-stroke`
chain-fill

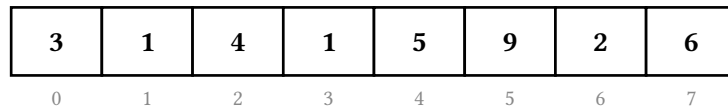


16. Linear sorts

`sort` animates the two **distribution** sorts — counting sort and LSD radix sort. They don't compare and swap; they read a value, compute an index, and write a cell, which is exactly the rows-of-boxes-with-arrows vocabulary the array backend draws.

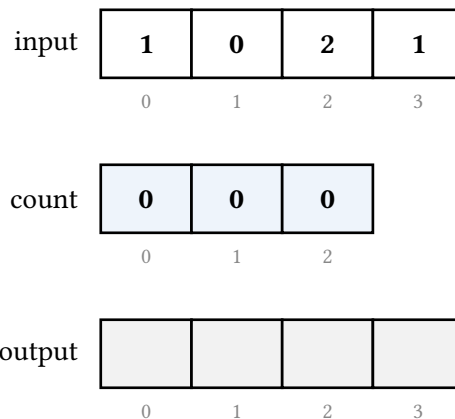
```
#let s = sort.new(3, 1, 4, 1, 5) // or sort.new((3, 1, 4, 1, 5))
```

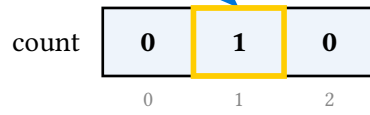
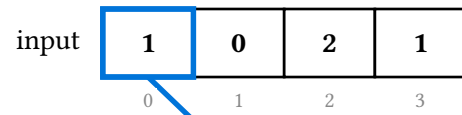
Keys must be non-negative integers, because the value **is** an index into the count array. To sort an enumeration you would rather show by name, give an element as (value: <int>, label: <content>): the integer orders it, the label is drawn, and the two may be mixed freely with bare integers. The pure `sort.counting(s, k: auto)` and `sort.radix(s, base: 10)` return the sorted keys (`sort.sorted(s)` is the built-in-sort oracle); the labels are a display concern.



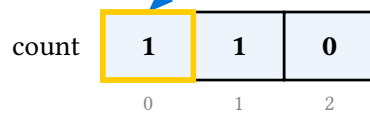
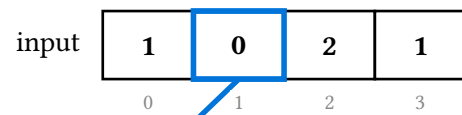
16.1. Counting sort

`counting-display(s)` animates the stable, prefix-sum counting sort: sweep the input to fill the **count** row (indexed **by value**, so a read arrow lands on the bucket the element names), turn the counts into cumulative end positions, then walk the input **right to left** — the stability radix sort depends on — placing each value at `output[count[value] - 1]` and decrementing.

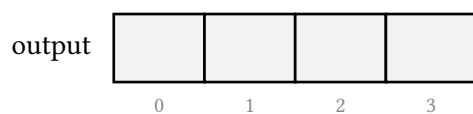
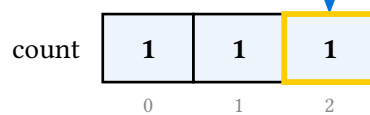
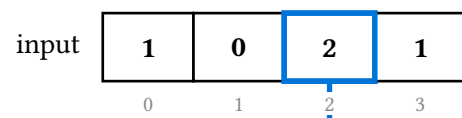




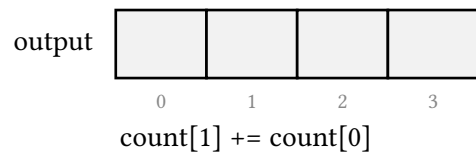
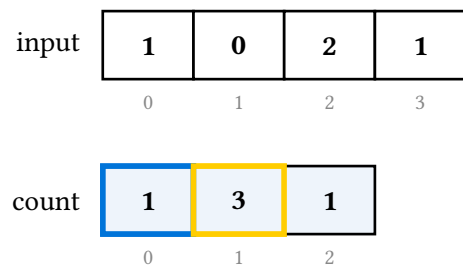
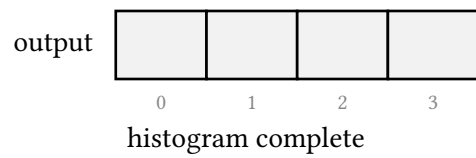
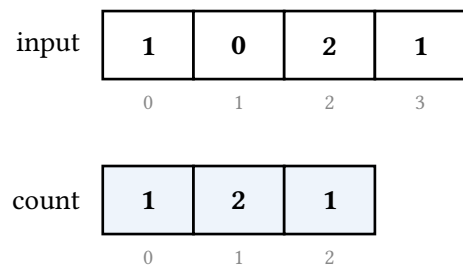
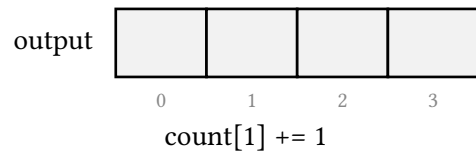
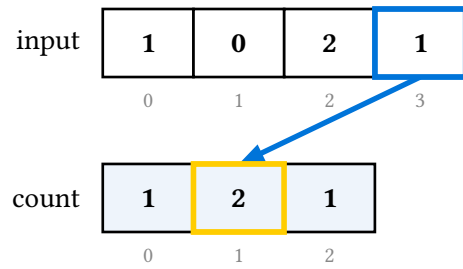
count[1] += 1



count[0] += 1



count[2] += 1



input

1	0	2	1
0	1	2	3

count

1	3	4
0	1	2

output

0	1	2	3

count[2] += count[1]

input

1	0	2	1
0	1	2	3

count

1	3	4
0	1	2

output

0	1	2	3

counts → end positions

input

1	0	2	1
0	1	2	3

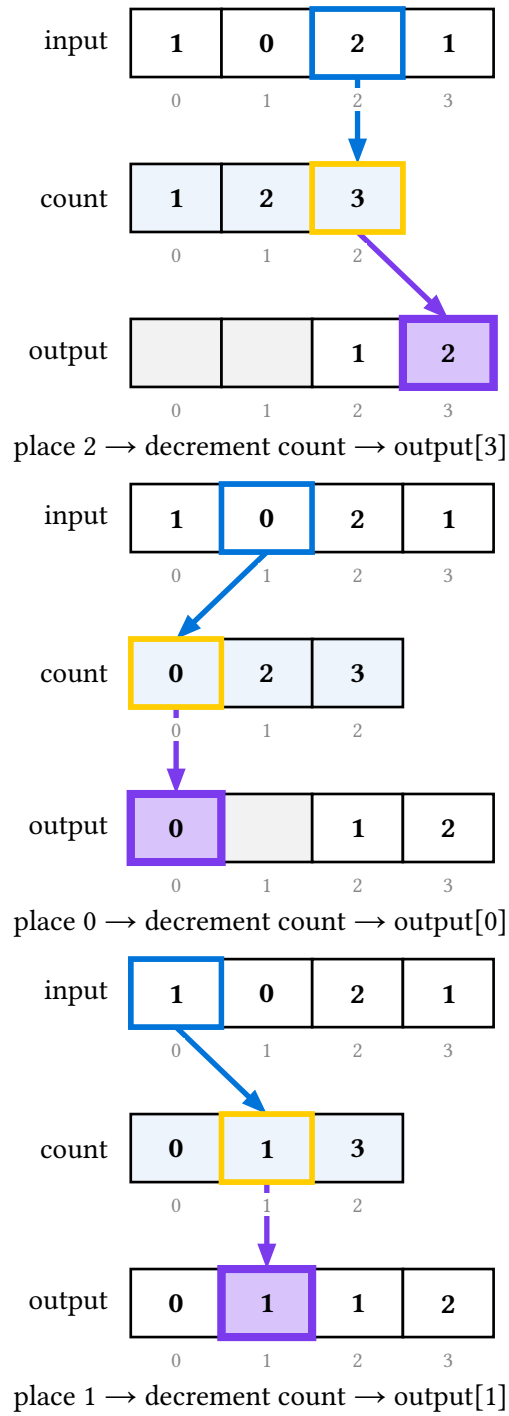
count

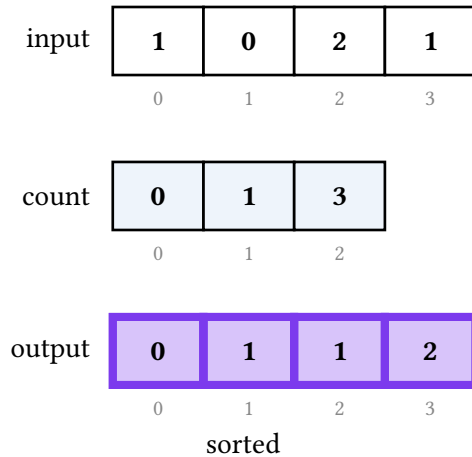
1	2	4
0	1	2

output

		1	
0	1	2	3

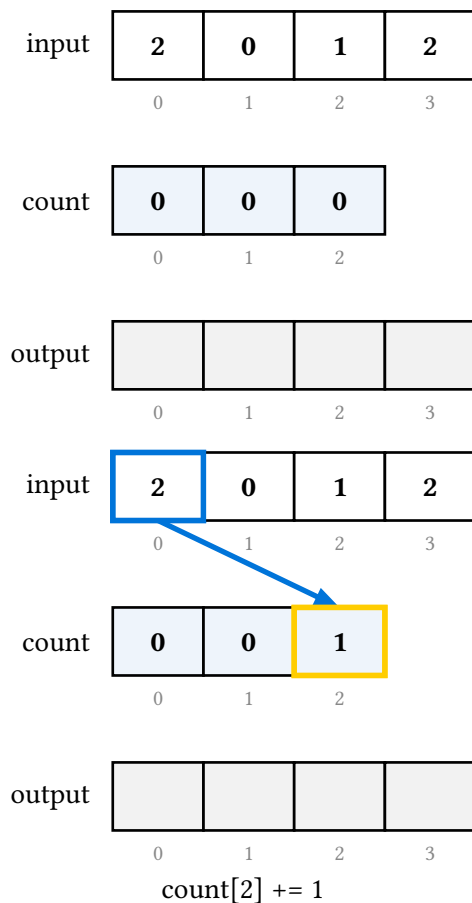
place 1 → decrement count → output[2]

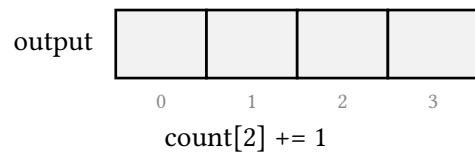
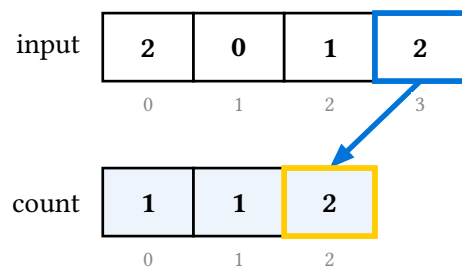
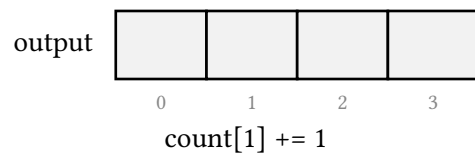
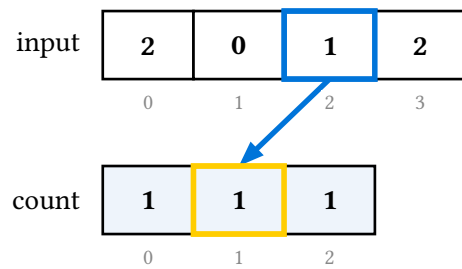
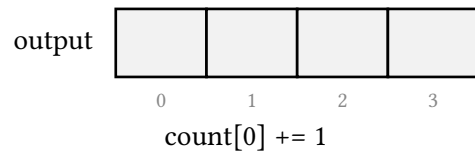
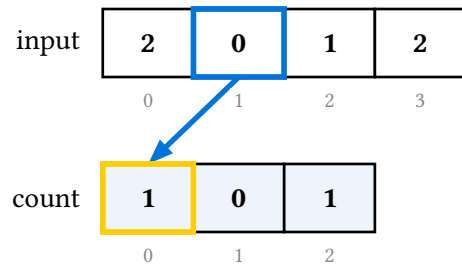


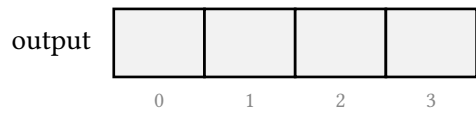
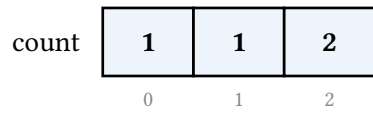
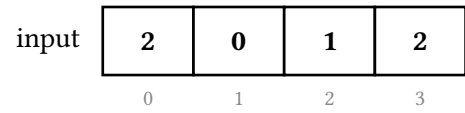


The count array is sized `max + 1` by default; pass `k`: to reserve a larger range. Three arguments change the telling:

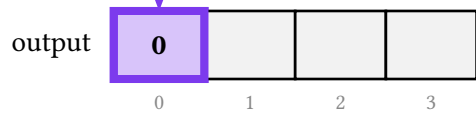
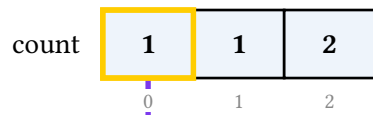
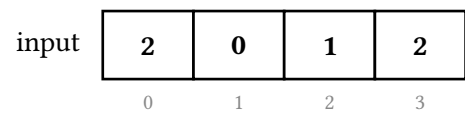
variant: "reconstruct" animates the intro version instead — build the histogram, then sweep the buckets and emit each value `count[v]` times. It sorts, but it uses neither the prefix sums nor a right-to-left pass, so it does not demonstrate stability. (It also rebuilds from the histogram alone, which is why it can only show the first-seen label per key: discarding element identity is precisely what makes it unstable.)



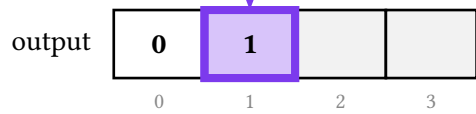
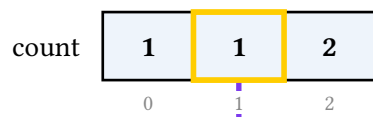
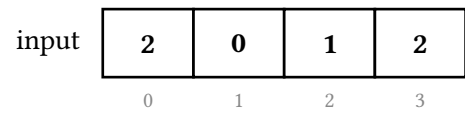




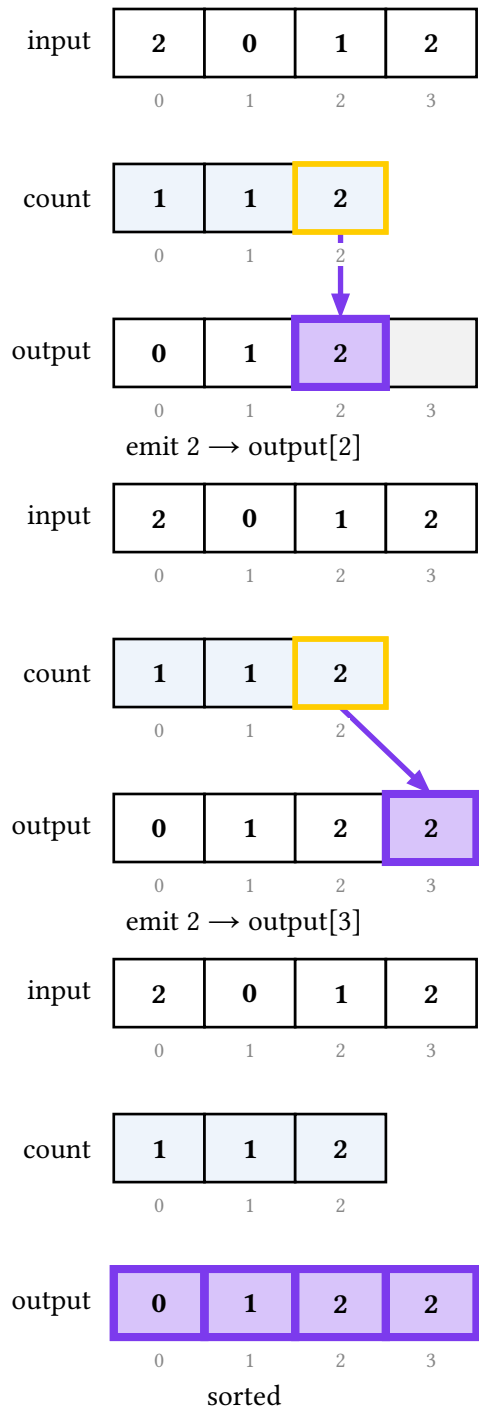
histogram complete



emit 0 $\rightarrow$ output[0]



emit 1 $\rightarrow$ output[1]



`separate-counts: true` keeps the raw histogram in the count row all pass and builds the cumulative sums in their own row below it, instead of overwriting the histogram in place. It costs a row and buys the clearer telling of **why** the prefix sum gives each value its slot.

input	1	0	2	1	3
	0	1	2	3	4

count	0	0	0	0
	0	1	2	3

cumulative				
	0	1	2	3

output					
	0	1	2	3	4

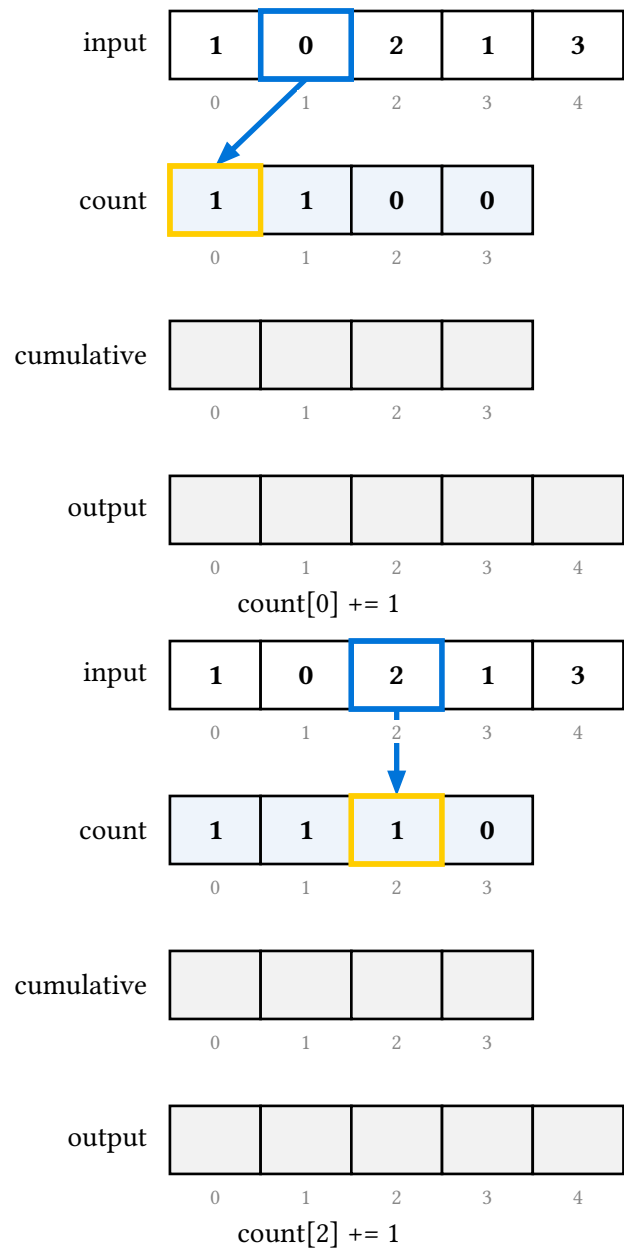
input	1	0	2	1	3
	0	1	2	3	4

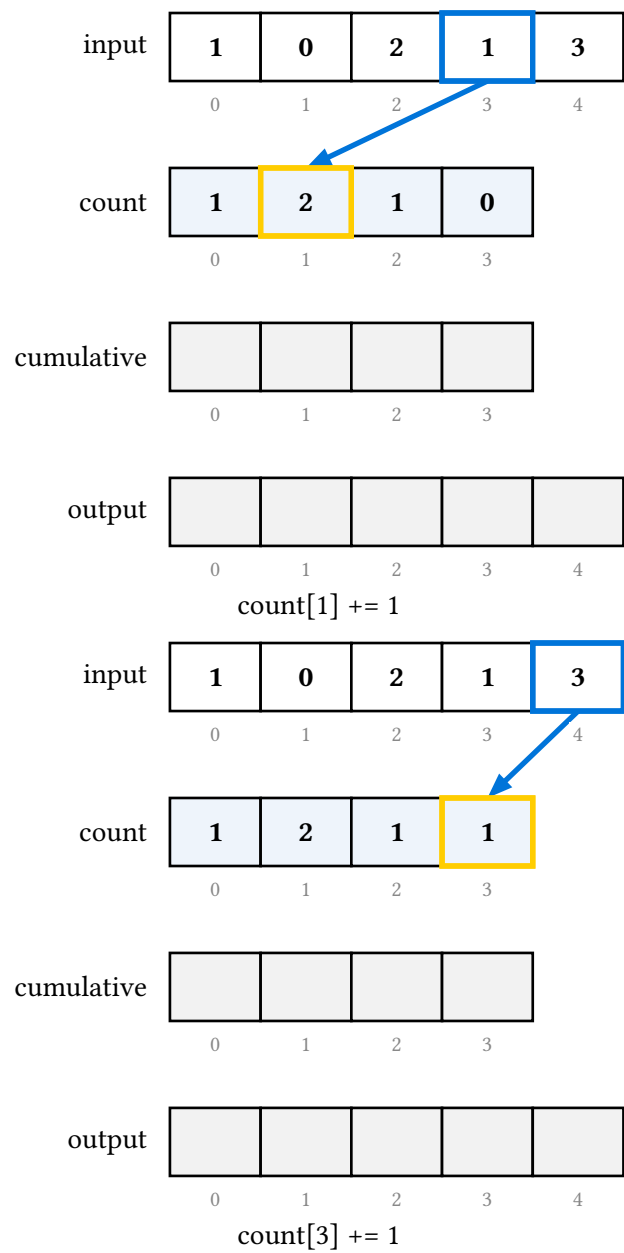
count	0	1	0	0
	0	1	2	3

cumulative				
	0	1	2	3

output					
	0	1	2	3	4

count[1] += 1





input

1	0	2	1	3
0	1	2	3	4

count

1	2	1	1
0	1	2	3

cumulative

0	1	2	3

output

0	1	2	3	4

histogram complete

input

1	0	2	1	3
0	1	2	3	4

count

1	2	1	1
0	1	2	3

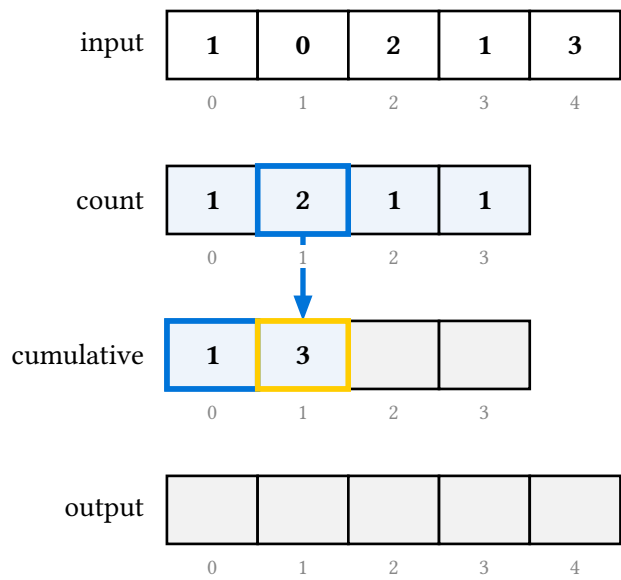
cumulative

1			
0	1	2	3

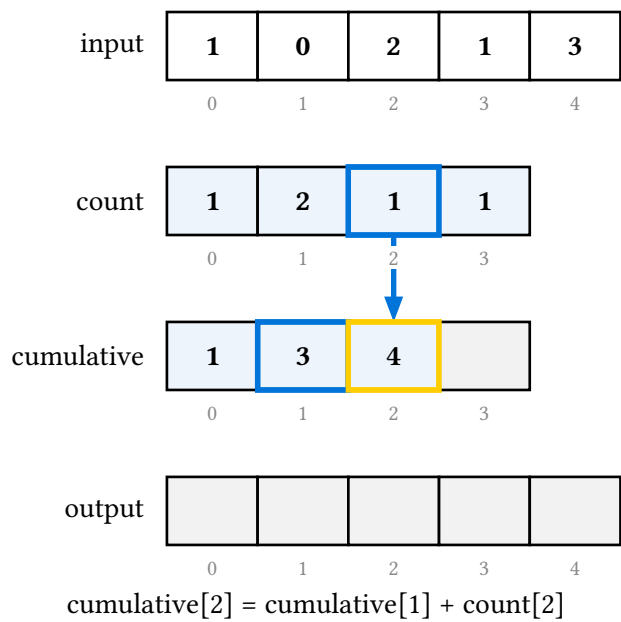
output

0	1	2	3	4

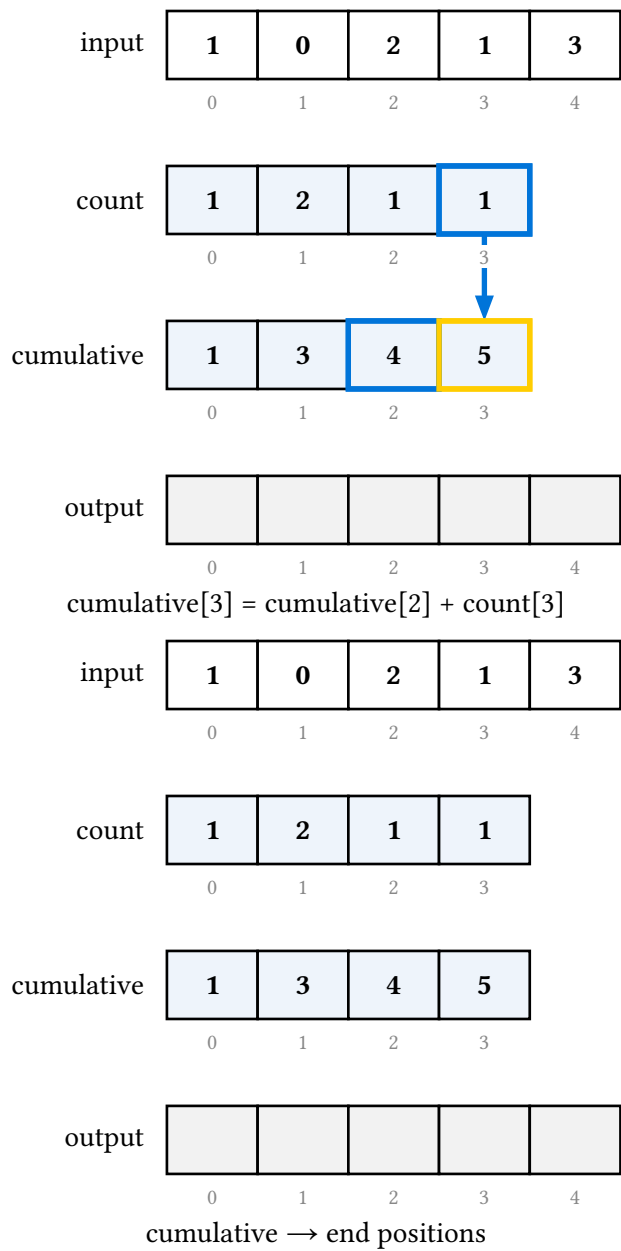
$\text{cumulative}[0] = \text{count}[0]$

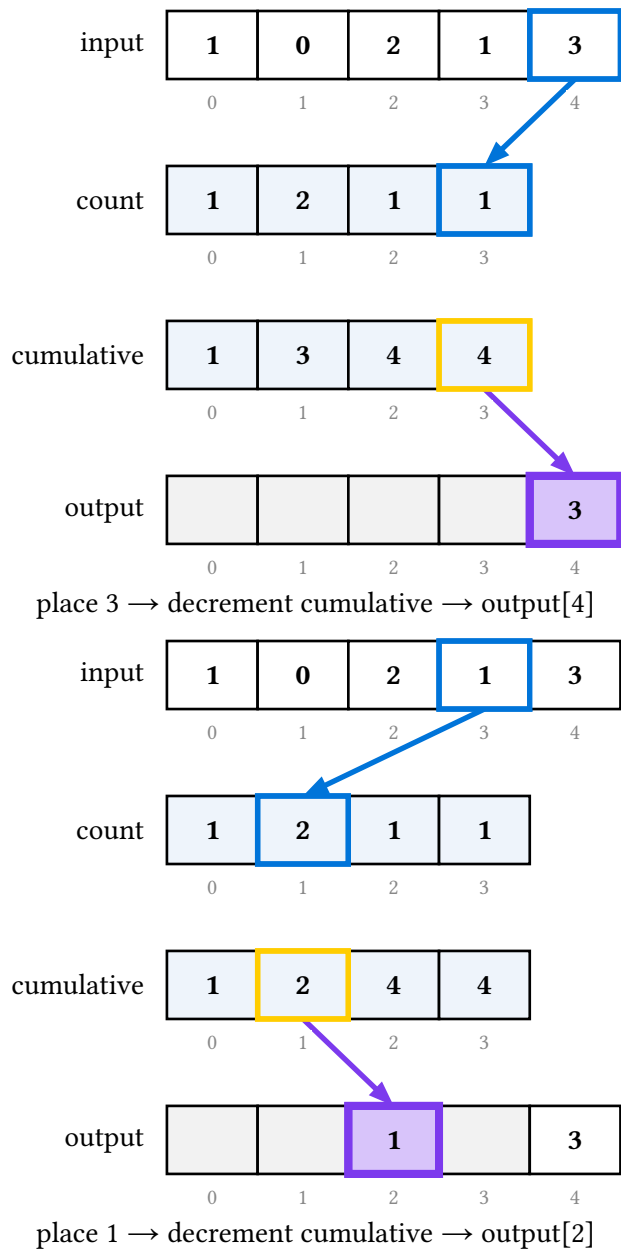


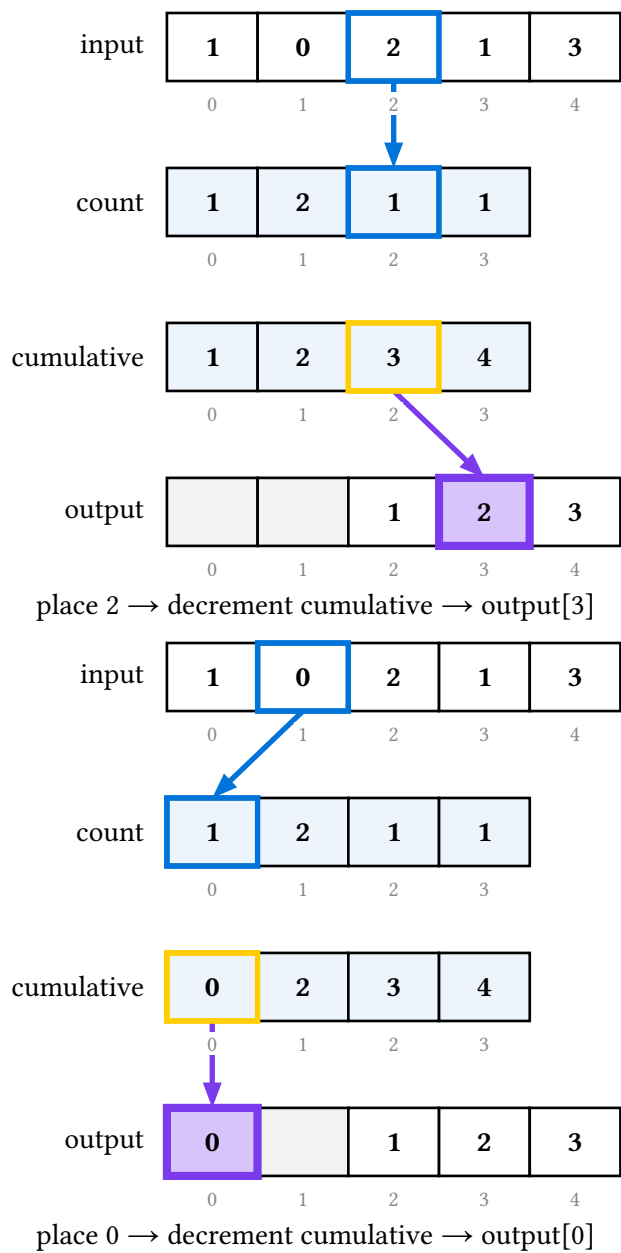
$$\text{cumulative}[1] = \text{cumulative}[0] + \text{count}[1]$$

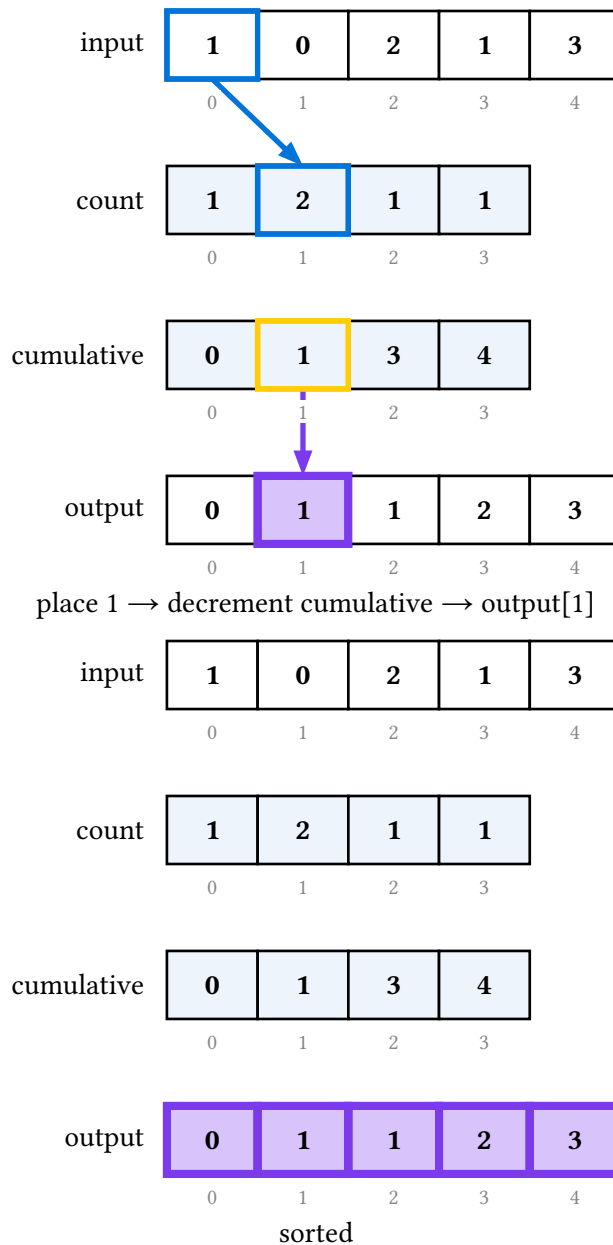


$$\text{cumulative}[2] = \text{cumulative}[1] + \text{count}[2]$$









variant: "buckets" draws the count array as a **chaining hash table** with the identity hash. Each element is copied into the tail of its value's chain, then the buckets are read left to right, each chain head to tail, into the output. It is deliberately space-inefficient — the whole element lives in the bucket — but it makes the mechanism concrete: sorting **is** distributing into value-indexed buckets and concatenating them. Tail-append plus head-first read means it is stable, and labels ride their chains faithfully.

input

Tue	Sun	Tue	Mon
0	1	2	3

buckets

0	1	2
----------	----------	----------

output

0	1	2	3

input

Tue	Sun	Tue	Mon
0	1	2	3

buckets

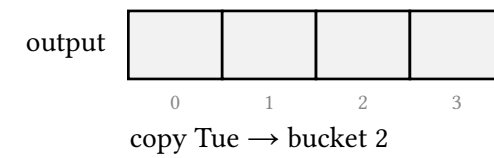
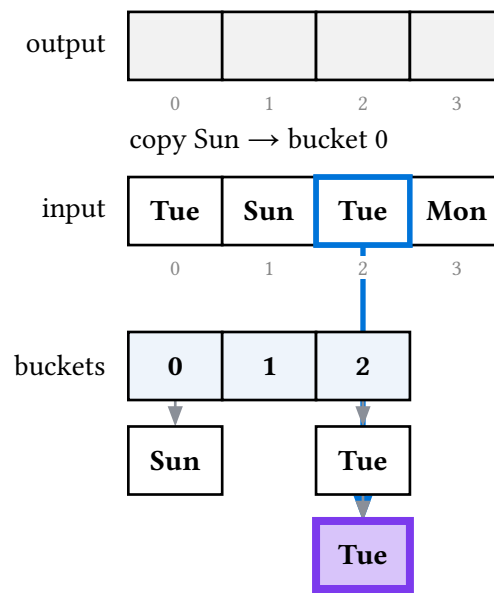
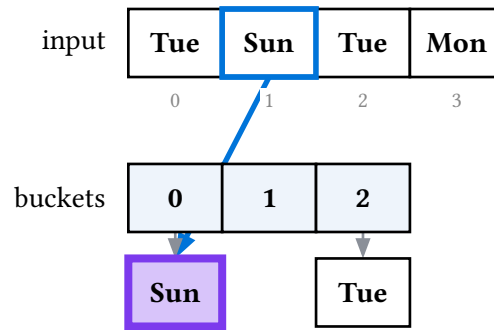
0	1	2
----------	----------	----------

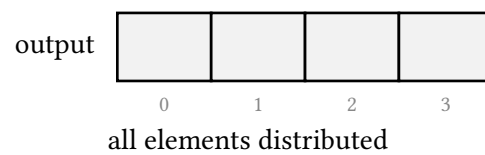
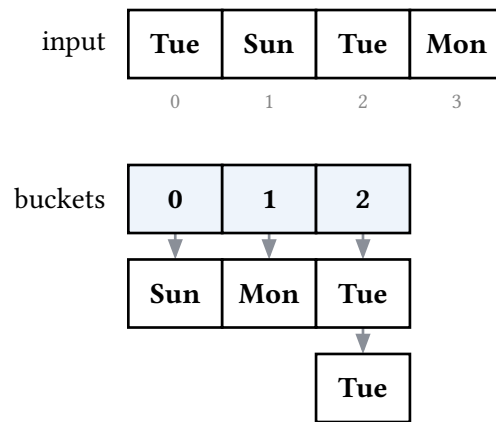
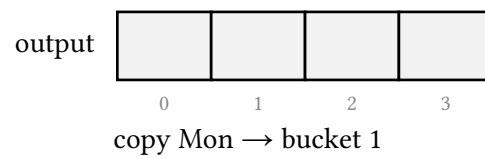
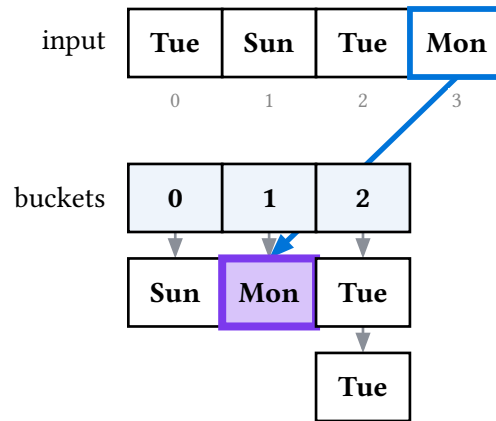
Tue

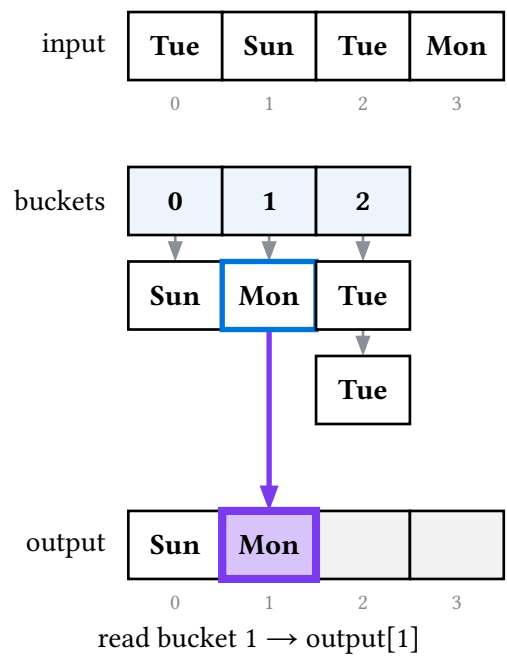
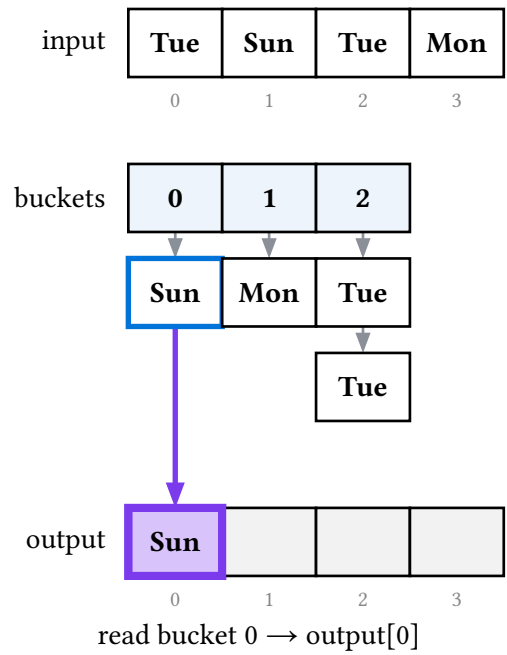
output

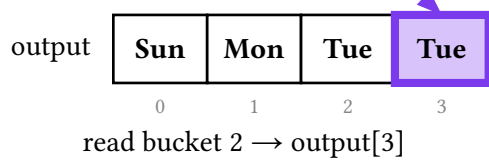
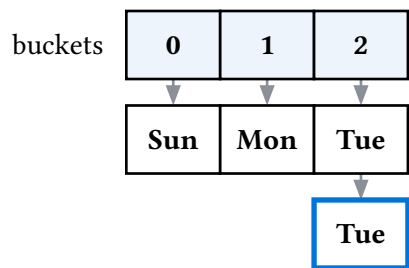
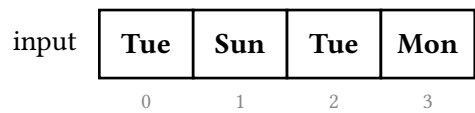
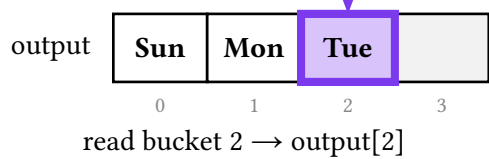
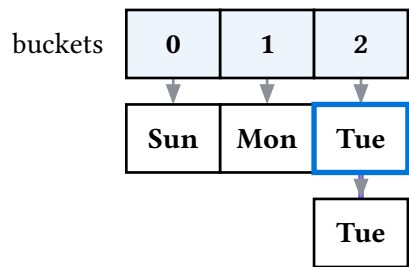
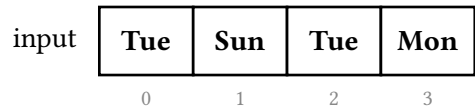
0	1	2	3

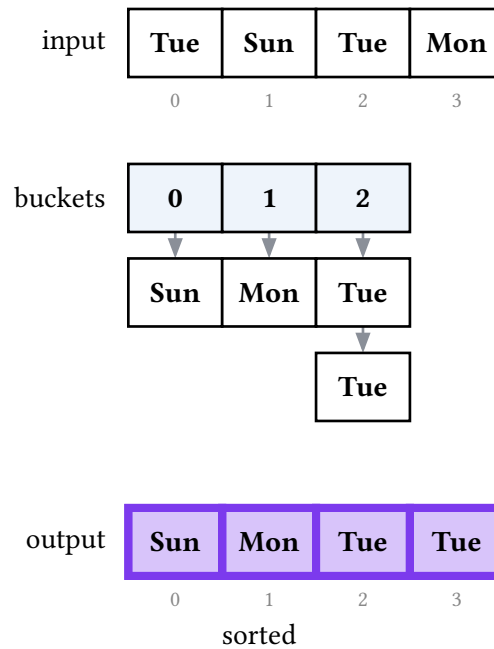
copy Tue → bucket 2





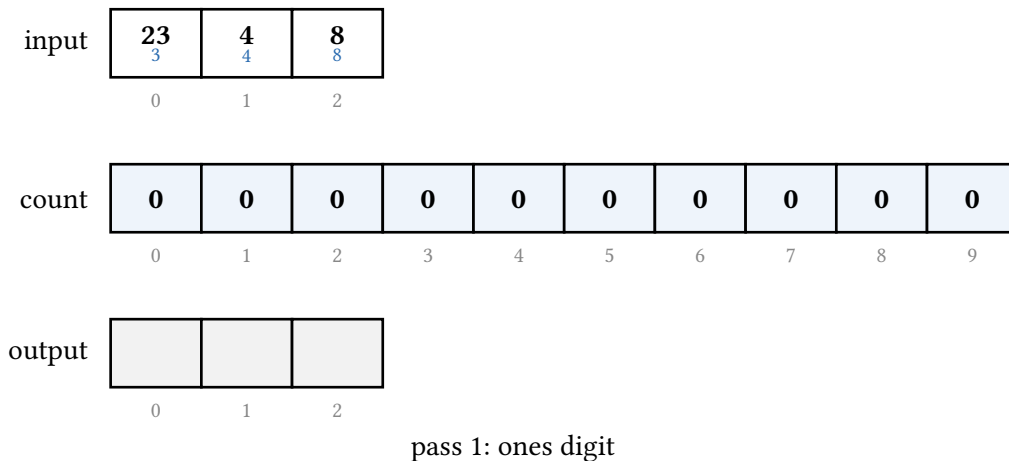


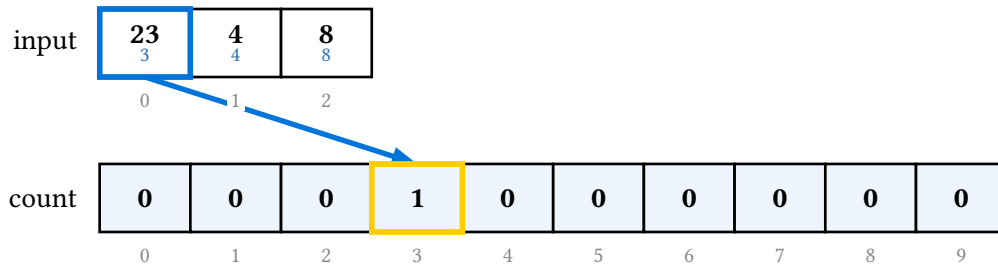




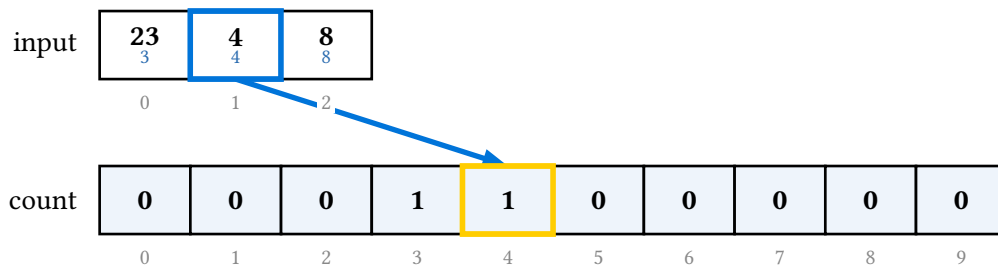
16.2. Radix sort

`radix-display(s, base: 10)` is LSD radix sort as **one stable counting-sort pass per digit place**, keyed on the extracted digit — so the count row has base buckets and each input cell wears its digit for the active pass as a subscript. Because each pass is the counting sort above, radix is a thin wrapper over the same engine.

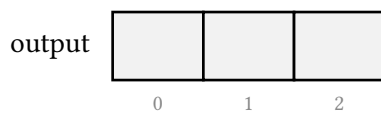
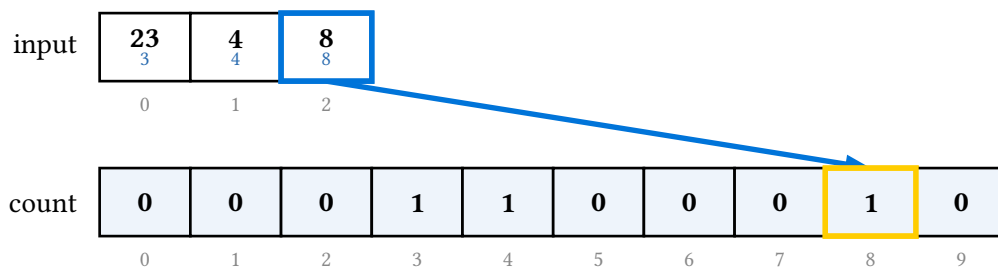




count[3] += 1



count[4] += 1



count[8] += 1

input

23 3	4 4	8 8
0	1	2

count

0	0	0	1	1	0	0	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

histogram complete

input

23 3	4 4	8 8
0	1	2

count

0	0	0	1	1	0	0	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[1] += count[0]

input

23 3	4 4	8 8
0	1	2

count

0	0	0	1	1	0	0	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[2] += count[1]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	1	0	0	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[3] += count[2]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	0	0	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[4] += count[3]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	2	0	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[5] += count[4]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	2	2	0	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[6] += count[5]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	2	2	2	1	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[7] += count[6]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	2	2	2	3	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[8] += count[7]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	2	2	2	3	3
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[9] += count[8]

input

23	4	8
3	4	8
0	1	2

count

0	0	0	1	2	2	2	2	3	3
0	1	2	3	4	5	6	7	8	9

output

0	1	2

counts → end positions

input

23	4	8
3	4	8
0	1	2

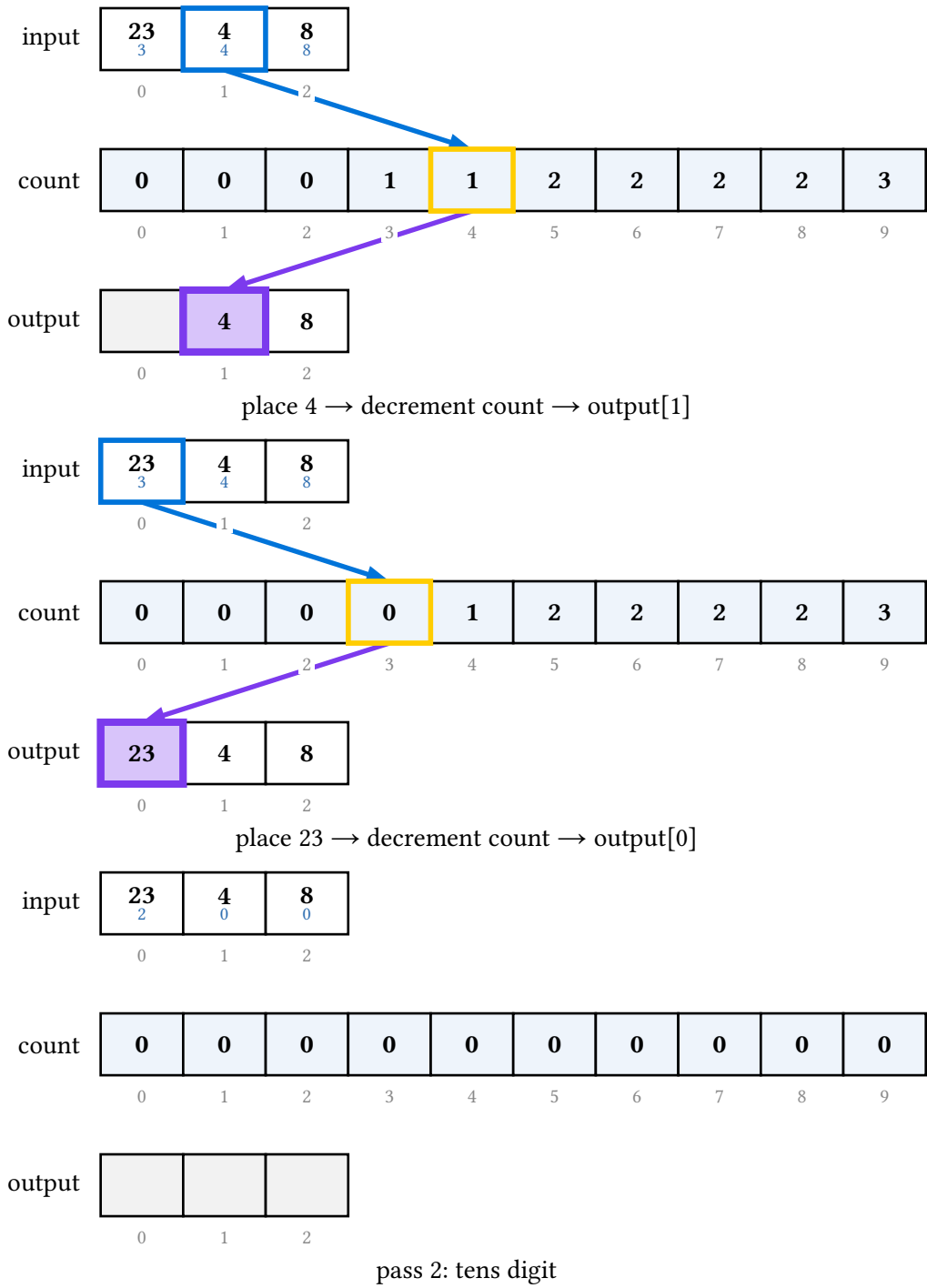
count

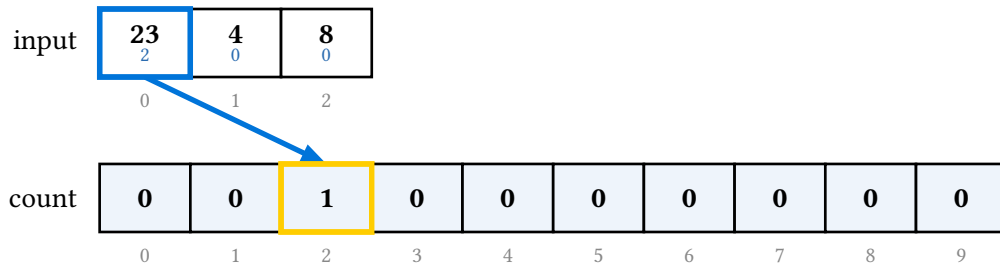
0	0	0	1	2	2	2	2	2	3
0	1	2	3	4	5	6	7	8	9

output

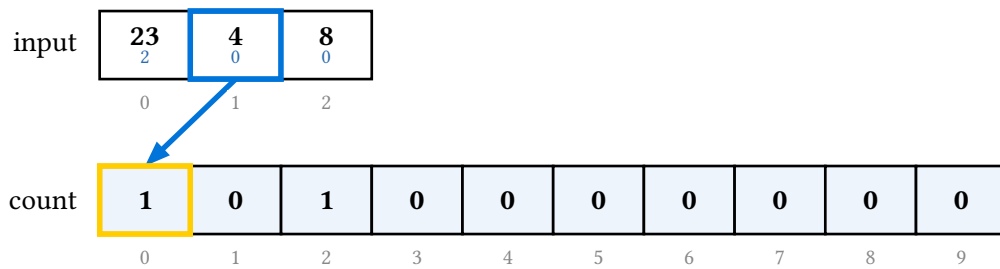
		8
0	1	2

place 8 → decrement count → output[2]

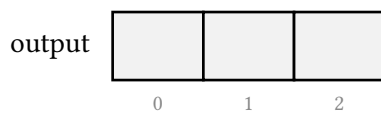
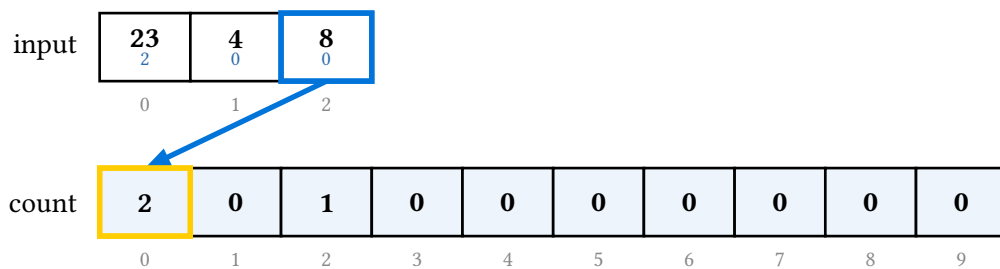




count[2] += 1



count[0] += 1



count[0] += 1

input

23 2	4 0	8 0
0	1	2

count

2	0	1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

histogram complete

input

23 2	4 0	8 0
0	1	2

count

2	2	1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[1] += count[0]

input

23 2	4 0	8 0
0	1	2

count

2	2	3	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[2] += count[1]

input

23	4	8
2	0	0

0 1 2

count

2	2	3	3	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[3] += count[2]

input

23	4	8
2	0	0

0 1 2

count

2	2	3	3	3	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[4] += count[3]

input

23	4	8
2	0	0

0 1 2

count

2	2	3	3	3	3	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[5] += count[4]

input

23	4	8
2	0	0
0	1	2

count

2	2	3	3	3	3	3	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[6] += count[5]

input

23	4	8
2	0	0
0	1	2

count

2	2	3	3	3	3	3	3	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[7] += count[6]

input

23	4	8
2	0	0
0	1	2

count

2	2	3	3	3	3	3	3	3	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[8] += count[7]

input

23	4	8
2	0	0
0	1	2

count

2	2	3	3	3	3	3	3	3	3
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[9] += count[8]

input

23	4	8
2	0	0
0	1	2

count

2	2	3	3	3	3	3	3	3	3
0	1	2	3	4	5	6	7	8	9

output

0	1	2

counts → end positions

input

23	4	8
2	0	0
0	1	2

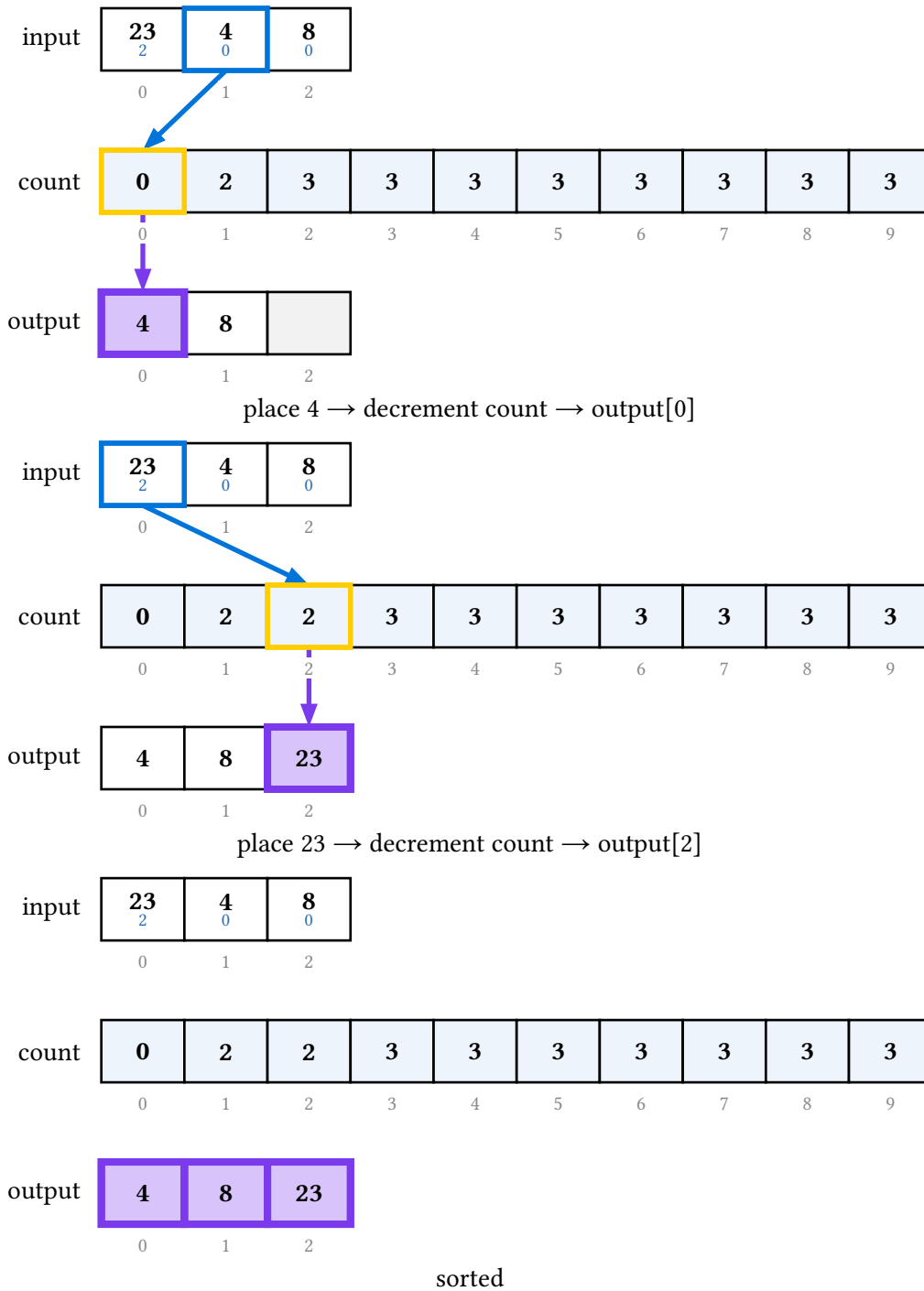
count

1	2	3	3	3	3	3	3	3	3
0	1	2	3	4	5	6	7	8	9

output

	8	
0	1	2

place 8 → decrement count → output[1]



Labels travel with their keys through every pass, so an enumeration comes out reordered by name:

input

23kg 3	4kg 4	8kg 8
0	1	2

count

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

pass 1: ones digit

input

23kg 3	4kg 4	8kg 8
0	1	2

count

0	0	0	1	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[3] += 1

input

23kg 3	4kg 4	8kg 8
0	1	2

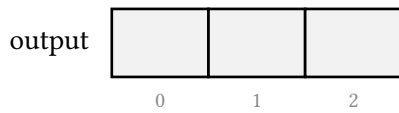
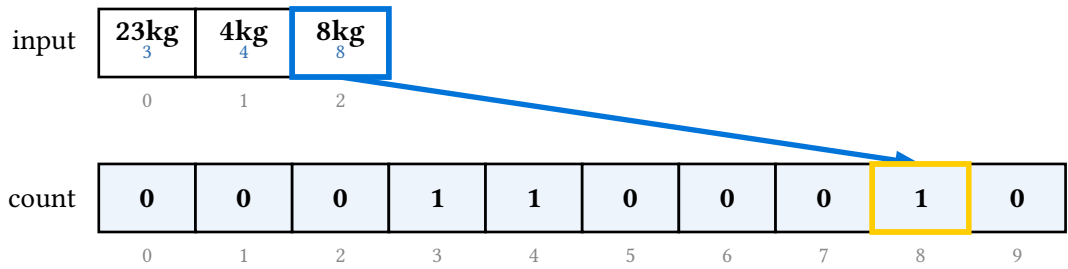
count

0	0	0	1	1	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

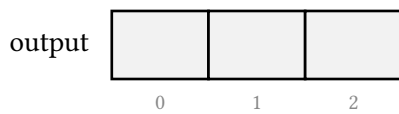
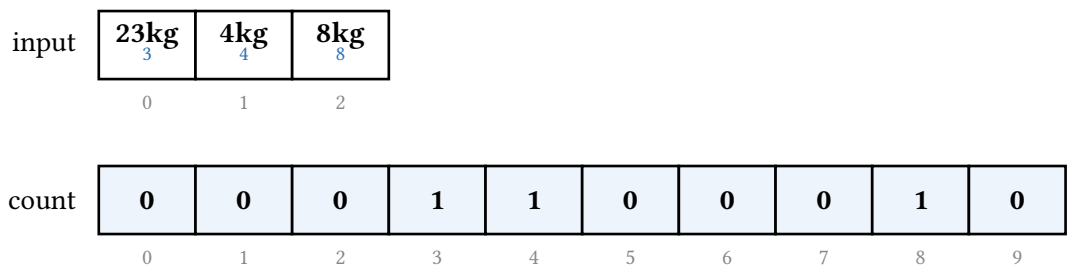
output

0	1	2

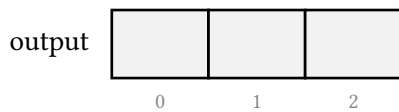
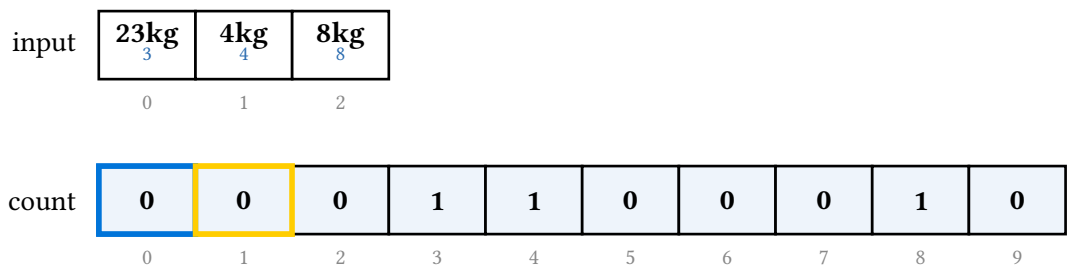
count[4] += 1



count[8] += 1



histogram complete



count[1] += count[0]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	1	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[2] += count[1]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	1	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[3] += count[2]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	0	0	0	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[4] += count[3]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	2	0	0	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[5] += count[4]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	2	2	0	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[6] += count[5]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	2	2	2	1	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[7] += count[6]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	2	2	2	3	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[8] += count[7]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	2	2	2	3	3
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[9] += count[8]

input

23kg 3	4kg 4	8kg 8
-----------	----------	----------

0 1 2

count

0	0	0	1	2	2	2	2	3	3
---	---	---	---	---	---	---	---	---	---

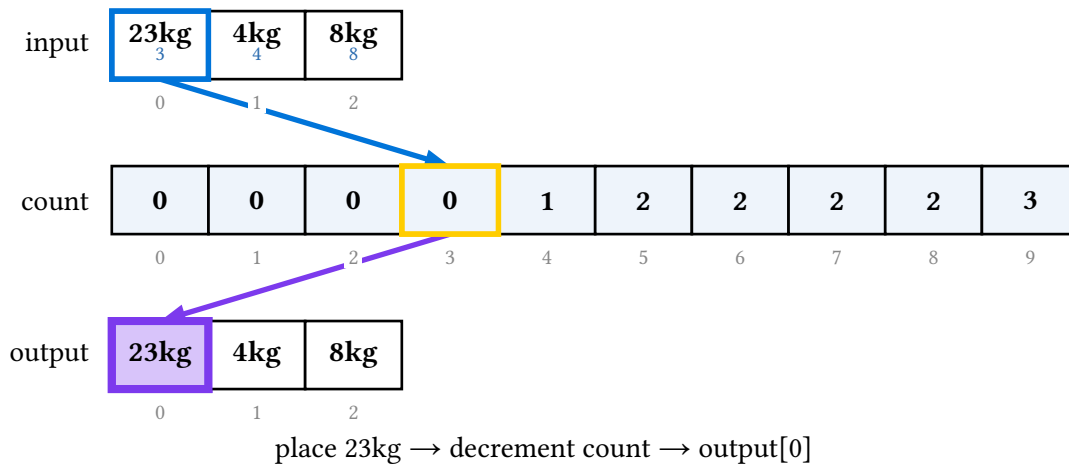
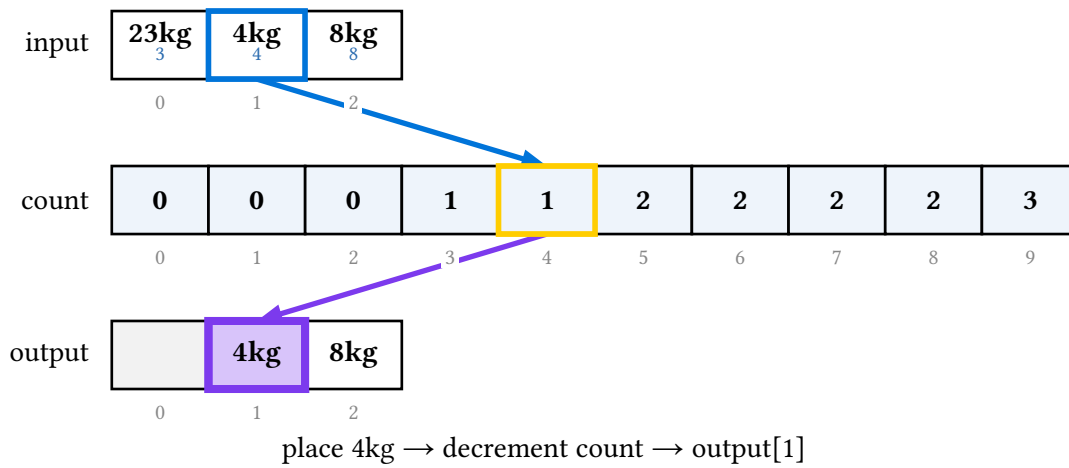
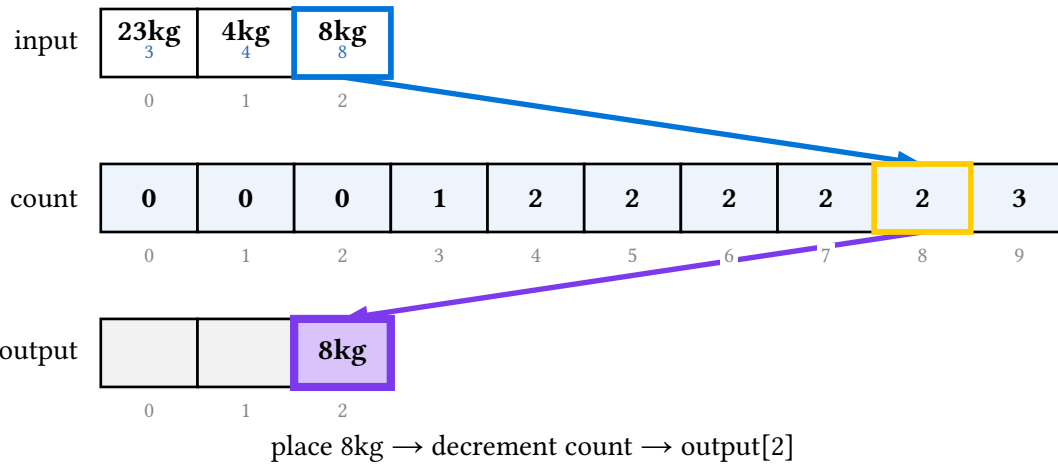
0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

counts → end positions



input

23kg 2	4kg 0	8kg 0
0	1	2

count

0	0	0	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

pass 2: tens digit

input

23kg 2	4kg 0	8kg 0
0	1	2

count

0	0	1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

output

0	1	2

count[2] += 1

input

23kg 2	4kg 0	8kg 0
0	1	2

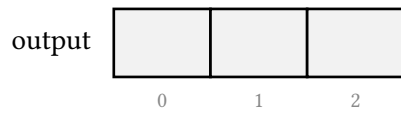
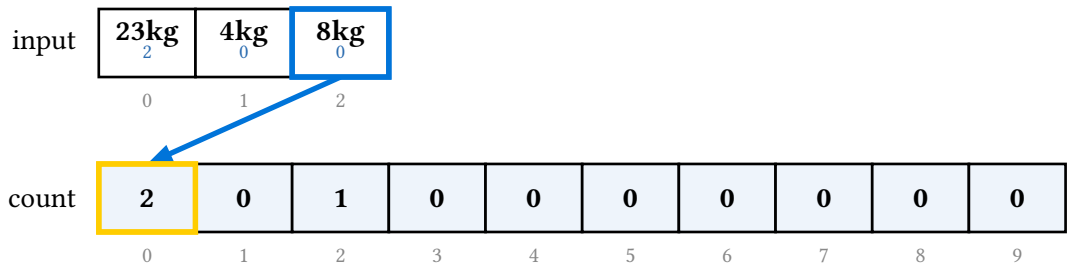
count

1	0	1	0	0	0	0	0	0	0
0	1	2	3	4	5	6	7	8	9

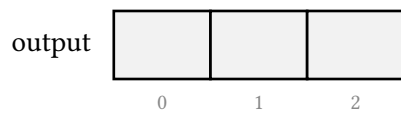
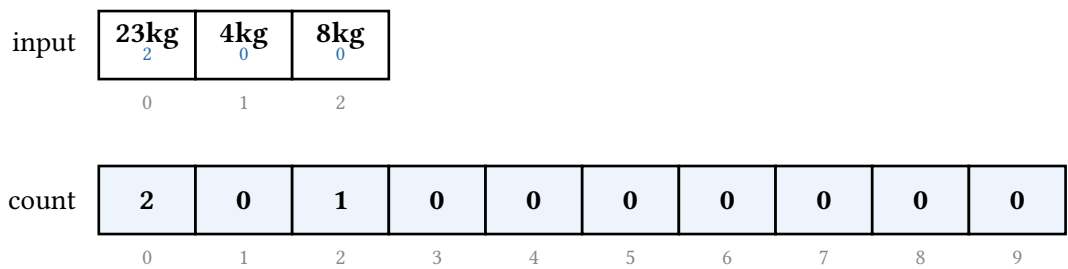
output

0	1	2

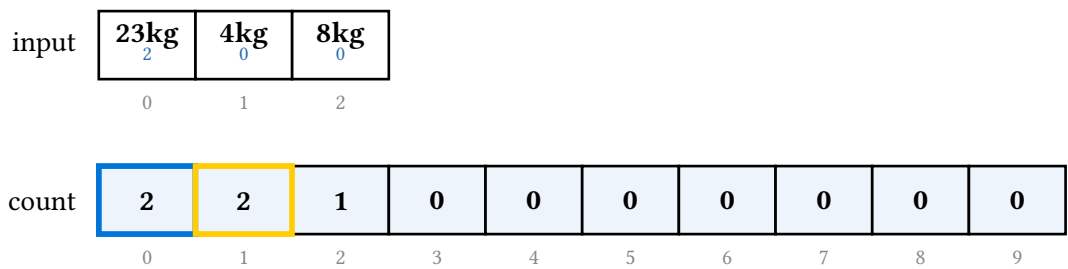
count[0] += 1



count[0] += 1



histogram complete



count[1] += count[0]

input

23kg 2	4kg 0	8kg 0
--------------------------	-------------------------	-------------------------

0 1 2

count

2	2	3	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[2] += count[1]

input

23kg 2	4kg 0	8kg 0
--------------------------	-------------------------	-------------------------

0 1 2

count

2	2	3	3	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[3] += count[2]

input

23kg 2	4kg 0	8kg 0
--------------------------	-------------------------	-------------------------

0 1 2

count

2	2	3	3	3	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[4] += count[3]

input

23kg 2	4kg 0	8kg 0
--------------------------	-------------------------	-------------------------

0 1 2

count

2	2	3	3	3	3	0	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[5] += count[4]

input

23kg 2	4kg 0	8kg 0
--------------------------	-------------------------	-------------------------

0 1 2

count

2	2	3	3	3	3	3	0	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[6] += count[5]

input

23kg 2	4kg 0	8kg 0
--------------------------	-------------------------	-------------------------

0 1 2

count

2	2	3	3	3	3	3	3	0	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[7] += count[6]

input

23kg 2	4kg 0	8kg 0
-----------	----------	----------

0 1 2

count

2	2	3	3	3	3	3	3	3	0
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[8] += count[7]

input

23kg 2	4kg 0	8kg 0
-----------	----------	----------

0 1 2

count

2	2	3	3	3	3	3	3	3	3
---	---	---	---	---	---	---	---	---	---

0 1 2 3 4 5 6 7 8 9

output

--	--	--

0 1 2

count[9] += count[8]

input

23kg 2	4kg 0	8kg 0
-----------	----------	----------

0 1 2

count

2	2	3	3	3	3	3	3	3	3
---	---	---	---	---	---	---	---	---	---

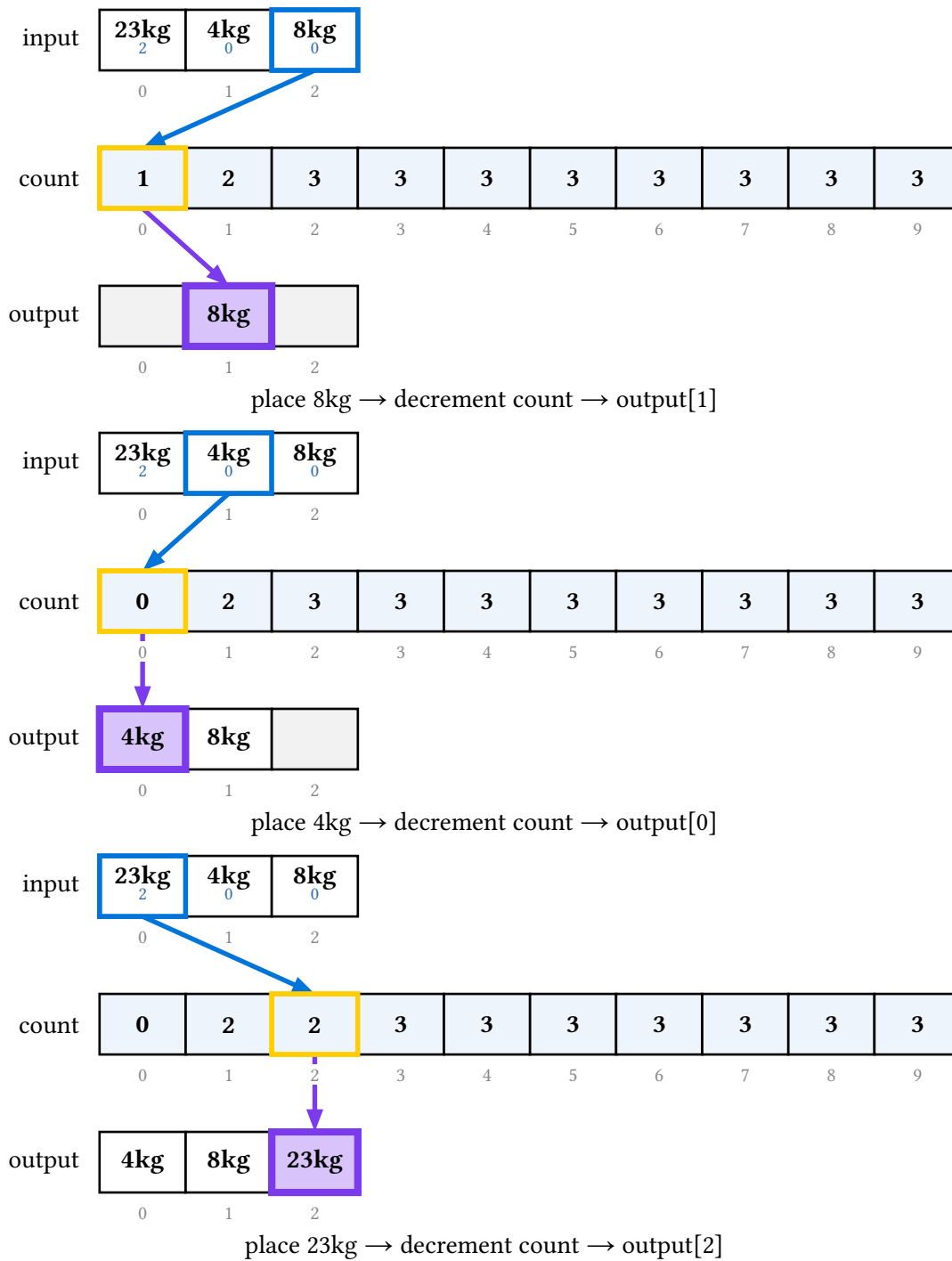
0 1 2 3 4 5 6 7 8 9

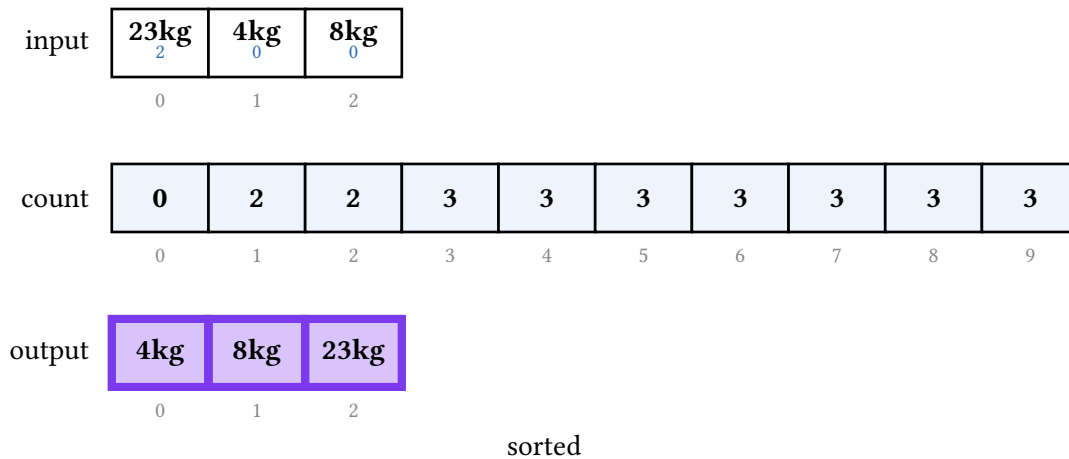
output

--	--	--

0 1 2

counts → end positions





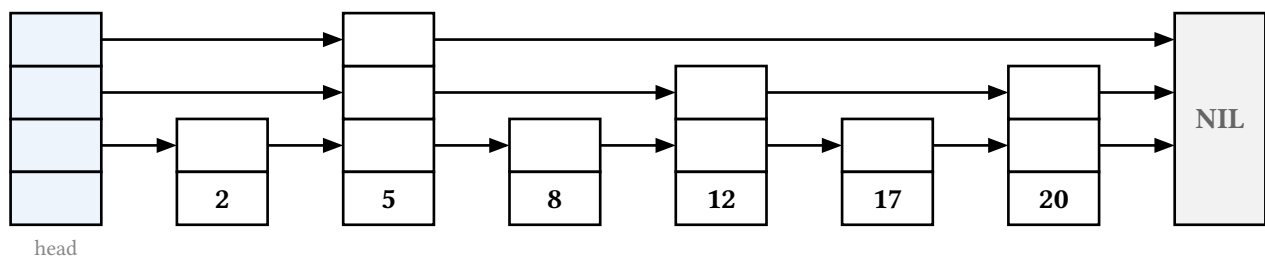
The sort theme section holds empty-fill, index-fill, row-label-fill, count-fill (the histogram and bucket tint), active-digit-fill (the radix subscript), and chain-stroke.

17. Skip lists

A skip list is a sorted set of non-negative integer keys stored as a probabilistic multi-level linked list: every key sits at level 0 and each also rises through a tower of express lanes that let a search skip ahead. It draws as a sparse grid — a header sentinel, one column per key, an optional NIL tail — with forward pointers at each level skipping the columns whose towers don't reach that high.

```
#let s = skiplist.new(3, 1, 4, 7, 5, seed: 7) // coin-flipped towers
#let s = skiplist.new( // pinned towers
  (value: 2, height: 1), (value: 5, height: 3),
  (value: 8, height: 1), (value: 12, height: 2),
)
```

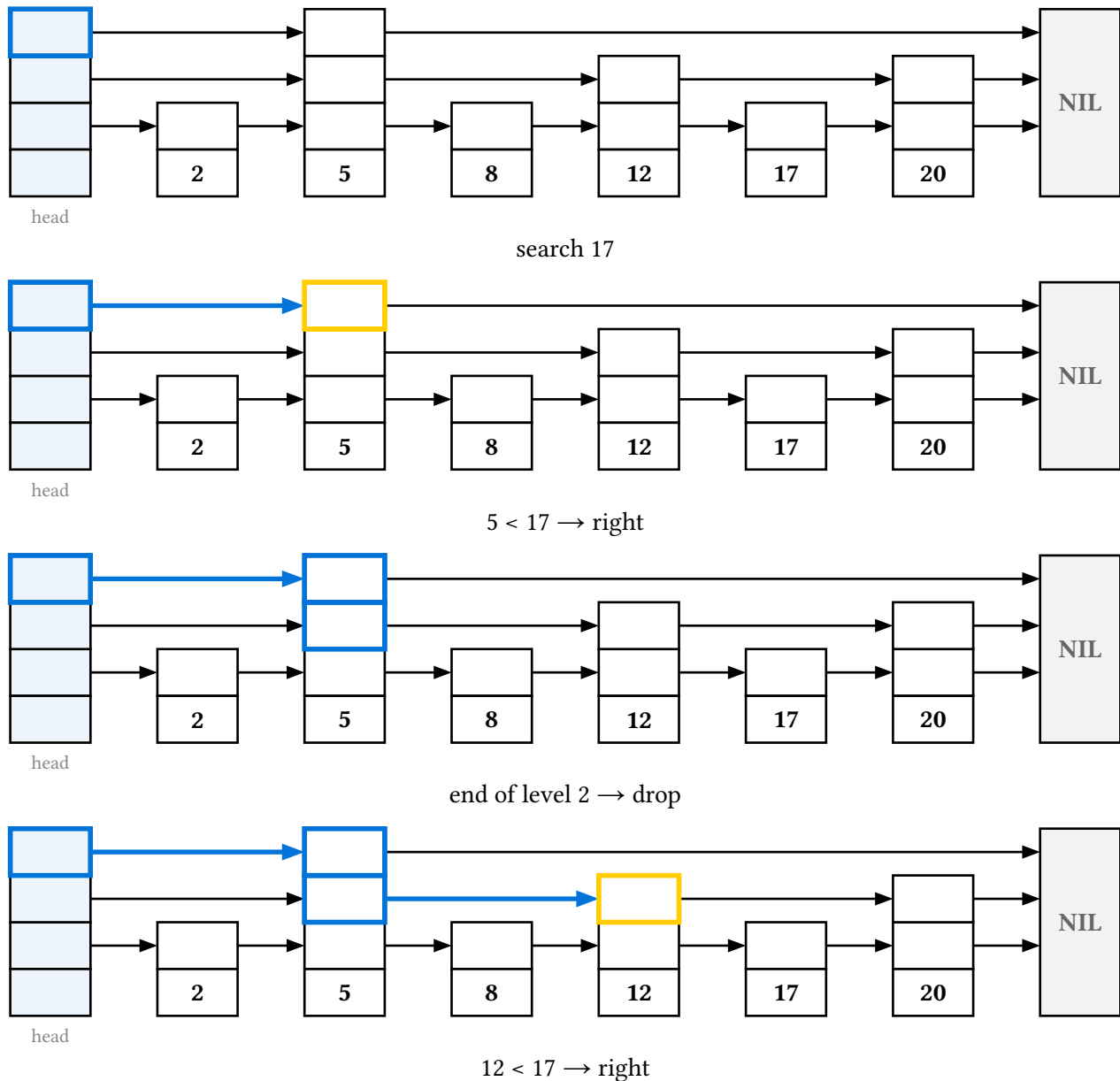
Tower heights come from an explicit height or from a **deterministic** seeded coin flip (grown with probability p , default one half, capped at max-level), so a document renders the same every time. Forward pointers are not stored: at level ℓ the list is exactly the subsequence of nodes whose towers reach that high, so the backend derives them.

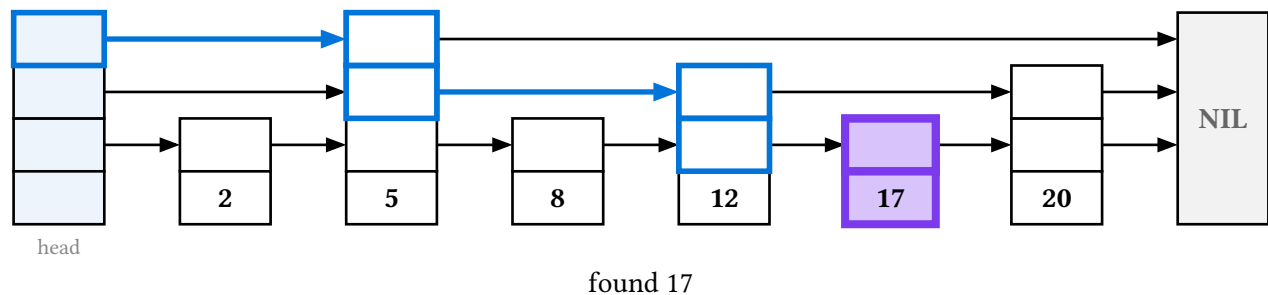
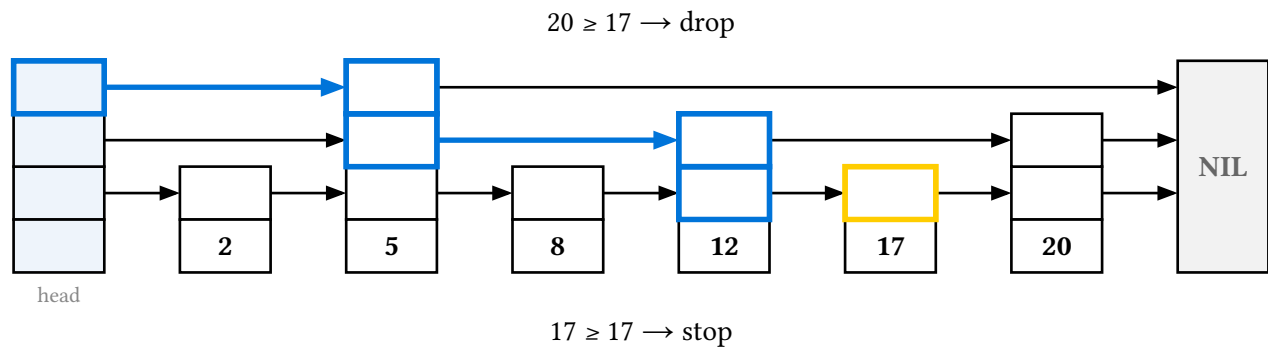
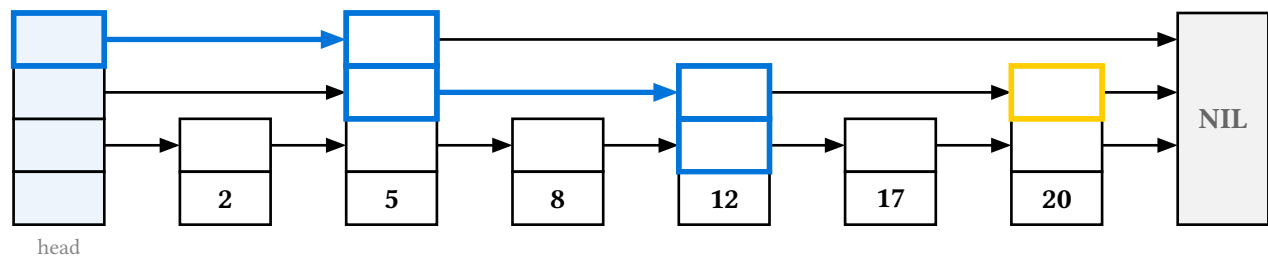


A node's tower is one flush column of **pointer cells**, one per level it reaches, sitting above a separate **data box** that holds the key. Because no forward pointer attaches to the data box, a link never crosses a key.

17.1. Search

`search-display(s, key)` animates the top-left descent: move right while the next key is below the target, drop a level the moment it would overshoot. The whole descent stays lit – every box stepped on and pointer followed accumulates into a trail, while the current comparison is picked out – so the finished animation reads as one continuous path.

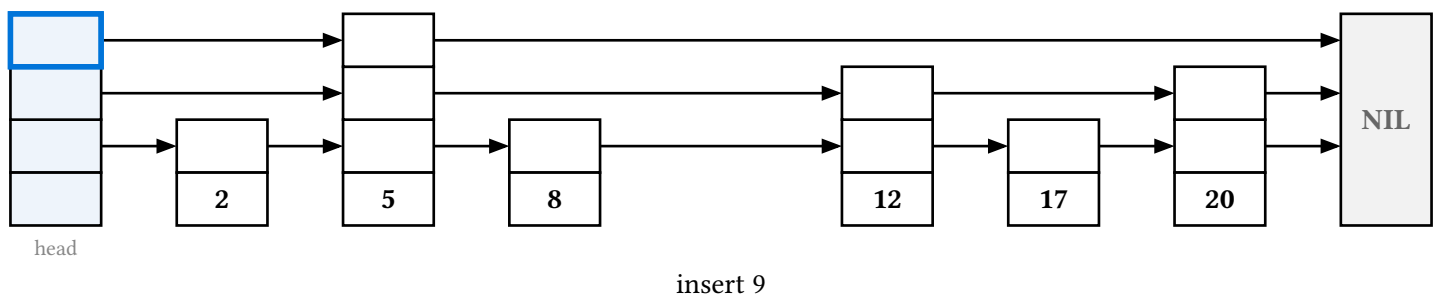


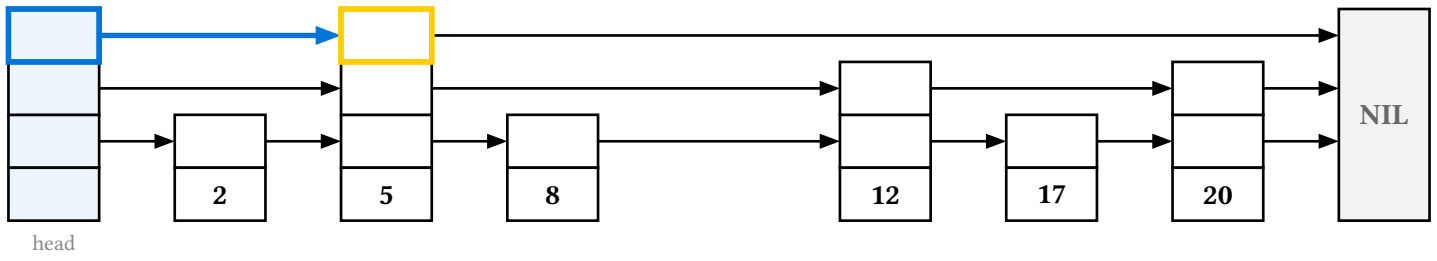


17.2. Insert and delete

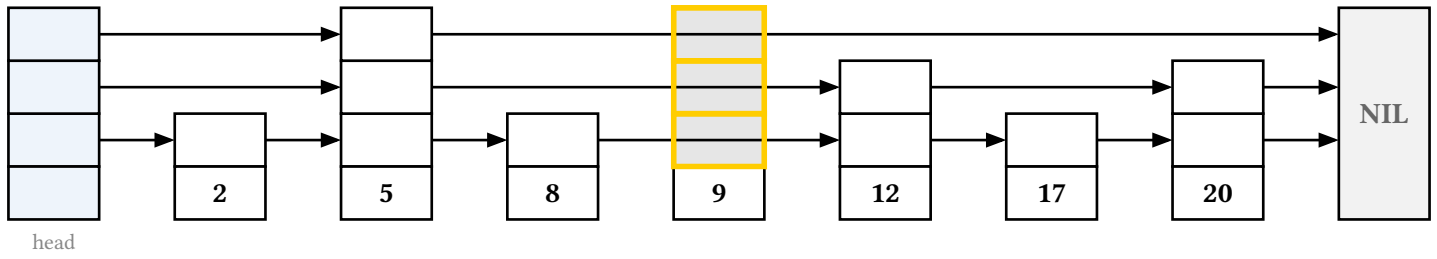
Both are **interleaved single passes**: the pointer surgery happens as the descent reaches each lane, because the descent lands on the target's predecessor on every lane it occupies. There is no separate splice phase and no backtracking.

On insert the new node stays a **ghost** — its column reserved, so the grid never shifts — until the descent first reaches its top lane, where it materializes; each lower lane is then woven in as the descent lands on it. A lane the node isn't linked on yet is drawn muted with the list's pointer running **over** it, so it is clear the list still skips past it there.

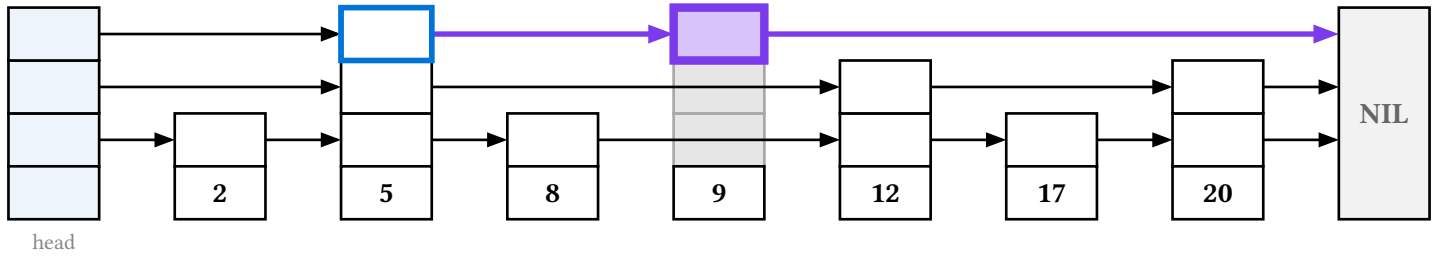




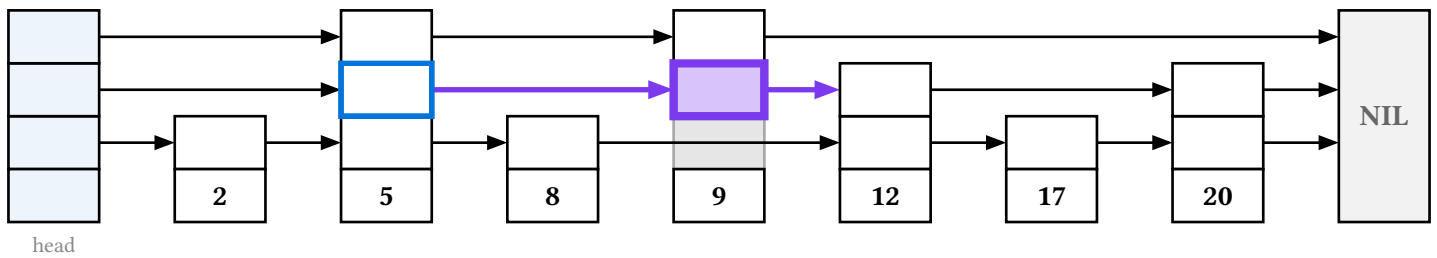
$5 < 9 \rightarrow \text{right}$



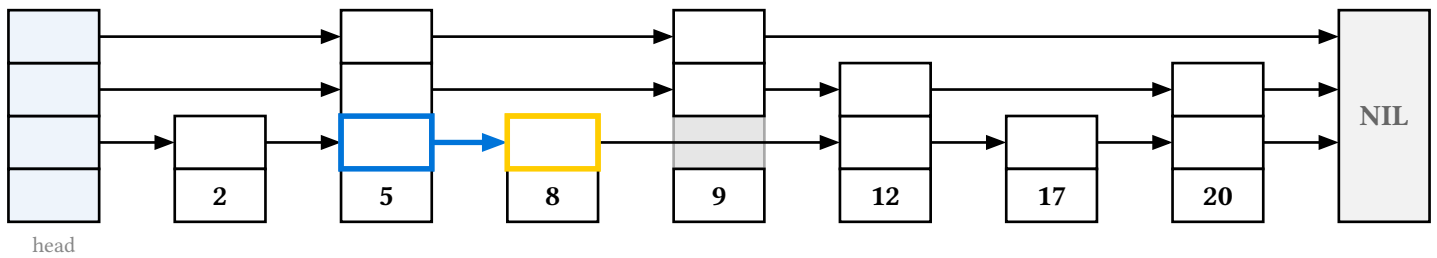
new node 9, height 3



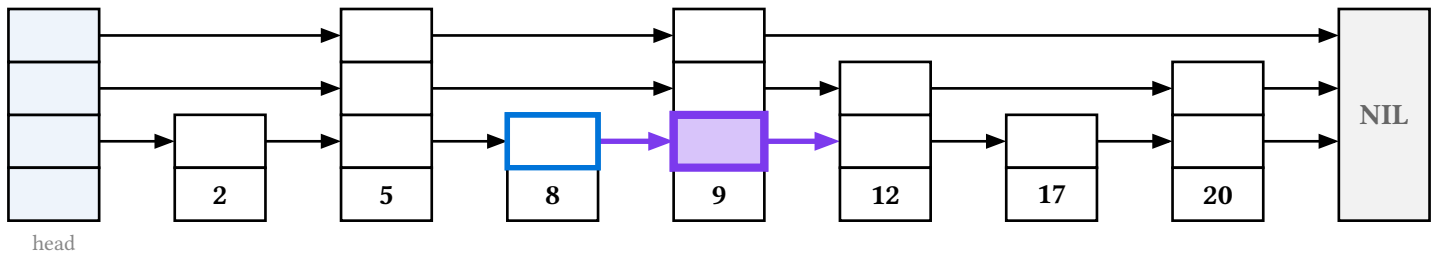
splice level 2



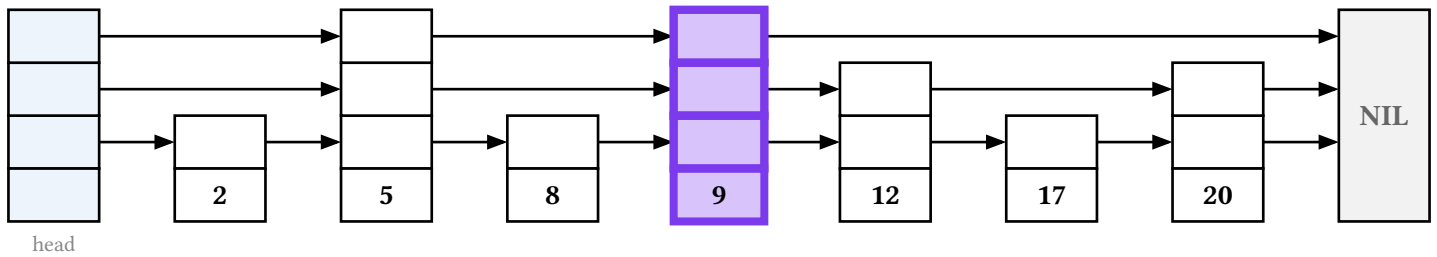
splice level 1



$8 < 9 \rightarrow \text{right}$

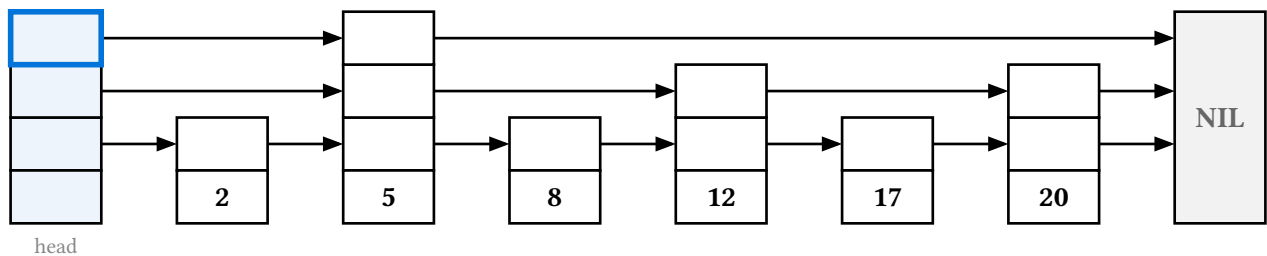


splice level 0

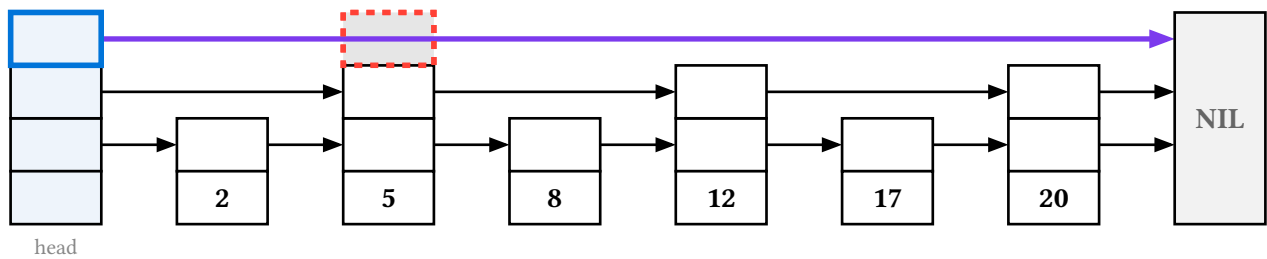


inserted 9

Delete is the mirror image: each lane is unlinked as the descent reaches it, the predecessor bypassing the target from the top down, and the node is left detached in place so it reads as removed rather than vanishing.

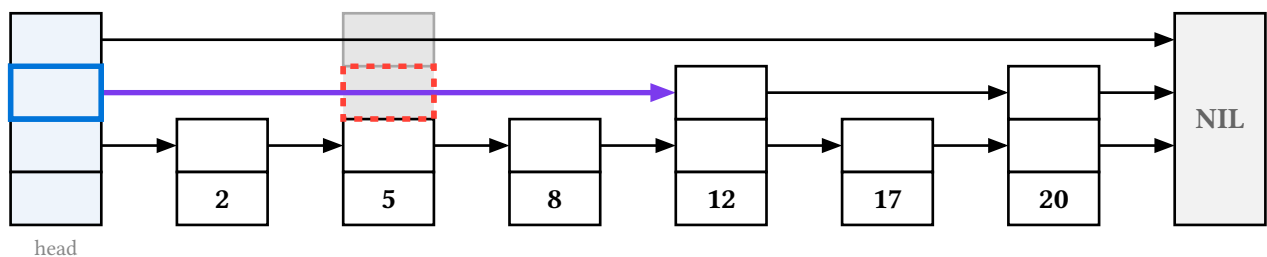


delete 5



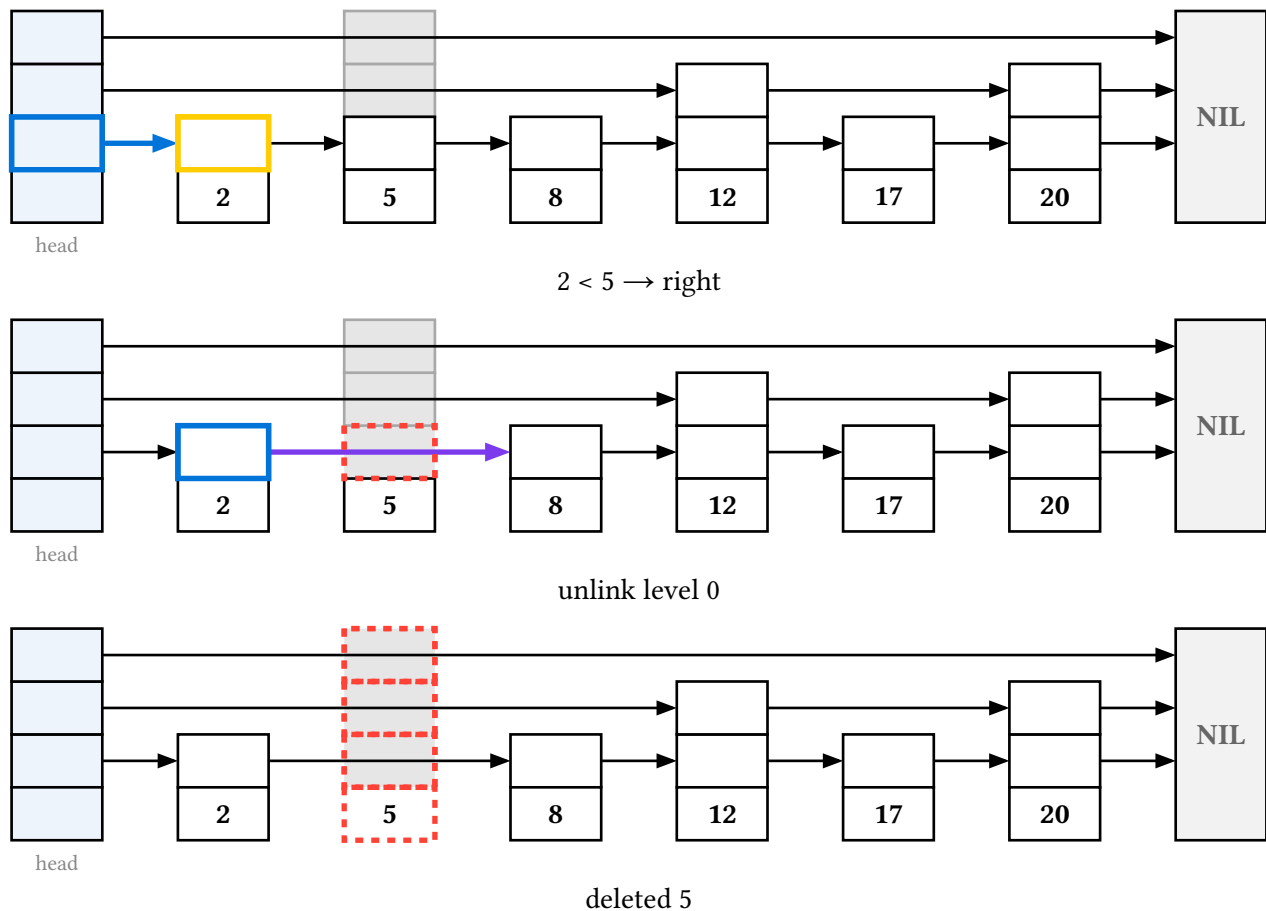
head

found 5 → unlink level 2



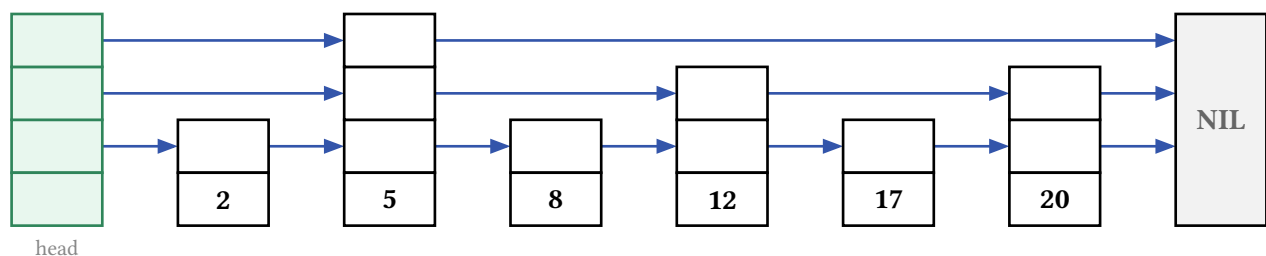
head

unlink level 1



`delete-display(s, key, search: false)` drops the pure navigation frames and keeps the surgery — “the pointer work without the walk that found it”.

The `skiplist` theme section holds the header and NIL palettes, `index-fill` (the head caption), `pointer-stroke`, and the `unlinked-*` trio that mutes a lane the node isn’t linked on.



18. Git commit graphs

The `git` namespace draws git histories — commits, branches, merges, tags, and HEAD/branch pointers. It is the one part of `starling` that does **not** ride the frame stack: rather than an immutable structure animated step by step, it is a **stateful, imperative cetz builder**. You call `commit` / `branch` / `merge` inside a `git-graph({ . . })` block and the verbs mutate `cetz`’s canvas context as they draw, so the presentation helpers do not apply — you place the block directly inside a `cetz.canvas`.

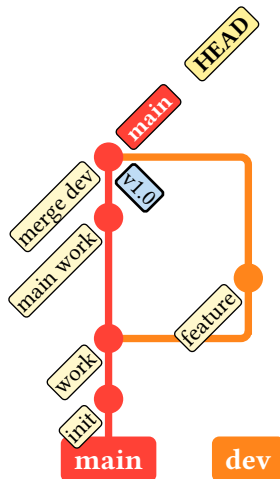
Its verbs have names too generic to sit in the flat layer, so they stay behind the namespace:

```

#import "@preview/cetz:0.5.2"
#import "@preview/starling:1.0.0" as starling
#import starling: git

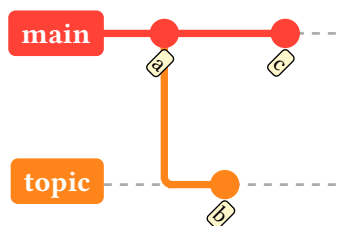
#cetx.canvas(git.git-graph({
  git.branch("main")
  git.commit("init")
  git.commit("work")
  git.branch("dev")
  git.commit("feature")
  git.checkout("main")
  git.commit("main work")
  git.merge("dev", message: "merge dev")
  git.branch-pointer("main")
  git.head_pointer()
  git.tag("v1.0")
}))

```



Create the initial branch before the first commit. `merge(branch)` takes the **branch name** to merge in; `commit`, `branch`, and `merge` accept a `name`: so later annotations survive reordering; `detached-commit(from, msg, name)` draws an orphan dot for `head-pointer(target: name)` to hang off; and `background-lanes()` rules each branch's lane.

Animation is touying-native: put `pause` or `alternatives(...)` markers inside the canvas body, or redraw a dot with `git-highlight`. Pass `direction: "left-to-right"` to lay commits out rightward instead of upward; the label angles and anchors switch with it.



Styling comes from the theme's git section — the branch colors palette plus the lane-style / graph-style / commit-style / tag-style / pointer-style sub-dicts — and a per-call theme: layers over the document theme exactly like any display's:

```
#cetz.canvas(git.git-graph(theme: (git: (colors: (teal, maroon, olive))), history))
```

Because those sub-styles are dicts, overriding one replaces it whole: pass a complete dict for the role you change. Behavioural arguments — direction:, commit-spacing:, lane-spacing: — are **not** theme; they stay arguments of git-graph.

19. Migrating from 0.3.x

Version 1.0.0 replaced typsy classes with plain dictionaries and namespaced the API per structure. There are no shims: old names are gone, and the compiler will tell you so. The mapping is mechanical.

0.3.x	1.0.0
bst(16, 11, 29)	bst.new(16, 11, 29) — plus bst.node / bst.leaf for literals
(t.insert)(5)	bst.insert(t, 5)
(t.insert-display)(5)	bst.insert-display(t, 5)
(t.insert-display)(5) then (t = (t.insert)(5))	let f = bst.insert-display(t, 5) then t = result(f)
(Op.StyleNode.new)(path: p, style: (fill: red))	style-node(p, fill: red)
(Op.Commit.new)(alt: a)	commit(alt: a)
(Op.Highlight.new)(path: p, color: c)	style-node(p, stroke: c + 2pt)
Op.ClearNotes	style-node(key, note: none) on the keys to clear
starling.make-renderer(t)	bst.renderer(t, sticky: true) — sticky now defaults to false
(r.render)()	render(r)
paint-rbt(make-renderer(t), t, bits: true)	rbt.renderer(t, bits: true)
paint-trie(r, t)	trie.renderer(t)
theme: (op-theme, on bst/avl/b24/graph)	theme: (op: (..))
theme: (palette, on rbt/trie/hashmap/sort/skiplist)	theme: (rbt: (..)) and so on
render-theme: (..)	theme: (render: (..))
set-op-theme(..), set-render-theme(..), set-rbt-theme(..),...	set-theme((op: .., render: .., rbt: ..)) — one state, one setter
default-op-theme.attention-stroke	default-theme.op.attention-stroke
path-anchor(p, tree-name: "t")	anchor(p, canvas: "t")

0.3.x	1.0.0
node-anchor(id), cell-anchor(i), array-cell-anchor(r, c), sl-box-anchor(c, l)	anchor(<the key>), via that module's key constructor
cell-key(i) / entry-key(i, j)	hashmap.cell-key(i) / hashmap.entry-key(i, j)
array-cell-key(row, col)	sort.cell-key(row, col)
array-arrow-key(id)	deleted — the id is the key
sl-box-key / sl-forward-key / sl-data-key	skiplist.box-key / .forward-key / .data-key
mst-prim-display / mst-kruskal-display	graph.prim-display / graph.kruskal-display
counting-sort-display / radix-sort-display	sort.counting-display / sort.radix-display; the pure ops are sort.counting / sort.radix
canvases-only(frames)	frames.map(f => canvas(f)) — or better, subslides(frames)
concat-frames, GraphNodeId, TreeRenderer, Op.Highlight, Op.ClearNotes	deleted

Four behaviour changes are worth knowing before you convert a deck.

Renderers no longer accumulate by default. sticky was true in the old tree-bound make-renderer; it is false now, so an op stream that expects each frame to build on the last must say sticky: true.

A per-call theme: layers over the document theme rather than replacing it. Before, passing theme: reset the palette to the defaults and skipped set-* -theme entirely; now naming two keys changes those two and leaves the rest of your theme alone.

Every animation's last frame carries step.result. The old pattern of writing each operation twice — once for the frames, once to advance the variable — is what result(frames) replaces.

Alt text is always set explicitly, and captions are always content (the sort and trie captions that used to be strings are not any more). Nothing derives alt from a caption.

Two smaller notes. import cetz.draw: * inside a canvas now shadows starling's anchor, since cetz has an anchor of its own — import selectively. And there is no Op enum: the constructors are top-level functions, each returning an array, so streams compose with +.

20. API reference

The rest of this manual is generated by [tidy](#) from the doc comments in the source. Names starting with _ are private and do not appear.

The nine data-structure namespaces come first, then the shared machinery they are built on, then the drawing layer. Where a module's names are reachable under a namespace, the heading says which — bst.insert, styles.ghost, git.commit.

20.1. bst — binary search trees

- [check-invariants\(\)](#)

- `delete()`
- `delete-display()`
- `describe()`
- `display()`
- `in-order-display()`
- `insert()`
- `insert-display()`
- `insert-many()`
- `leaf()`
- `level-order-display()`
- `new()`
- `node()`
- `post-order-display()`
- `pre-order-display()`
- `renderer()`
- `rotate()`
- `rotate-display()`
- `search-display()`

20.1.1. check-invariants

Check the search-tree ordering: an in-order walk must be non-decreasing (ties descend left, so equal values are legal). Returns `true` or panics naming the first violation.

Parameters

`check-invariants(tree: dictionary) -> bool`

tree dictionary

The tree.

20.1.2. delete

Delete `v`, returning a new tree (or `none` if the last node went). A node with two children is replaced by its in-order predecessor — value **and** label — which is then deleted from the left subtree.

Parameters

`delete(`
 `tree: dictionary ,`
 `v: int`
`) -> dictionary none`

tree dictionary

The tree.

v `int`

The ordering key to delete.

20.1.3. delete-display

Animate deleting *v*, dispatching on the target's children:

- a leaf is highlighted, its edge dashed, then it is gone;
- a one-child node dashes both edges, vanishes, and its child reattaches;
- a two-child node descends to its in-order predecessor, transfers that value into the target's slot, and settles.

With `search: true` the deletion is preceded by a search-style walk to the target; those comparison notes are cleared on the first deletion frame so they don't compete with the deletion highlights.

Parameters

```
delete-display(  
  tree: dictionary,  
  v: int,  
  search: bool,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree `dictionary`

The tree.

v `int`

The ordering key to delete.

search `bool`

Whether to precede the deletion with the walk to the target.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.1.4. describe

A recursive textual rendering of the tree, used in alt text.

Parameters

`describe(tree: dictionary) -> str`

tree dictionary

The tree.

20.1.5. display

The tree as a single static frame.

Parameters

```
display(  
    tree: dictionary,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.1.6. in-order-display

Animate an in-order (left, root, right) traversal — the sorted walk.

Parameters

```
in-order-display(  
  tree: dictionary ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.1.7. insert

Insert v , returning a new tree. Ties descend left.

Parameters

```
insert(  
    tree: dictionary,  
    v: int,  
    label: auto any  
) -> dictionary
```

tree dictionary

The tree.

v int

The ordering key to insert.

label auto or any

What to draw in the new node; auto draws `str(v)`.

Default: **auto**

20.1.8. insert-display

Animate inserting v : the descent to the insertion point, then the new leaf appearing on the grown tree.

Parameters

```
insert-display(  
    tree: dictionary,  
    v: int,  
    label: auto any,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to insert.

label `auto` or `any`

What to draw in the new node; `auto` draws `str(v)`.

Default: `auto`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.1.9. insert-many

Insert several values in order. Each is a bare key or a `(value, label)` pair.

Parameters

```
insert-many(  
  tree: dictionary,  
  ..vals: any  
) -> dictionary
```

tree `dictionary`

The tree.

..vals `any`

The values to insert.

20.1.10. leaf

A childless node — `node(value)` with the intent spelled out.

Parameters

```
leaf(  
    value: int,  
    label: auto any  
) -> dictionary
```

value `int`

The ordering key.

label `auto` or `any`

What to draw in the node; `auto` draws `str(value)`.

Default: `auto`

20.1.11. level-order-display

Animate a level-order (breadth-first) traversal.

Parameters

```
level-order-display(  
    tree: dictionary,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary  
) -> array
```

tree `dictionary`

The tree.

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme dictionary

Partial theme override for this call.

Default: (:)

20.1.12. new

Build a tree from a list of values: the first becomes the root, the rest are inserted in order. Each is a bare key or a (value, label) pair.

```
bst.new(4, 1, 7, 3, 6, 8)
bst.new((4, [FOUR]), 1, 7, (6, [SIX]))
```

Parameters

`new(..vals: any ) -> dictionary`

..vals any

The values. At least one is required — it becomes the root.

20.1.13. node

A node holding value, with either no children (a leaf) or both — pass none for a missing one.

```
bst.node(8, bst.leaf(3), bst.node(10, none, bst.leaf(14)))
```

Parameters

```
node(
  value: int,
  ..children: dictionary none,
  label: auto any
) -> dictionary
```

value int

The ordering key.

..children dictionary or none

Either nothing (a leaf) or exactly two children, left then right.

label `auto` or `any`

What to draw in the node; `auto` draws `str(value)`.

Default: `auto`

20.1.14. `post-order-display`

Animate a post-order (left, right, root) traversal.

Parameters

```
post-order-display(  
    tree: dictionary ,  
    node-style: dictionary ,  
    edge-style: dictionary ,  
    theme: dictionary  
) -> array
```

tree `dictionary`

The tree.

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.1.15. `pre-order-display`

Animate a pre-order (root, left, right) traversal.

Parameters

```
pre-order-display(  
  tree: dictionary,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.1.16. renderer

A Renderer over this tree, bound to the tree backend — the entry point for driving an animation yourself with the Op command stream. Pass `sticky: true` when you want each frame's styling to accumulate.

Parameters

```
renderer(  
  tree: dictionary,  
  node-style: dictionary,  
  edge-style: dictionary,  
  sticky: bool,  
  theme: dictionary  
) -> dictionary
```


tree dictionary

The tree.

node-style dictionary

Base node styling beneath every frame's snapshot.

Default: (:)

edge-style dictionary

Base edge styling beneath every frame's snapshot.

Default: (:)

sticky bool

Whether new frames inherit the previous frame's styling.

Default: false

theme dictionary

Partial theme override for this renderer.

Default: (:)

20.1.17. rotate

Rotate around child — anywhere in the tree, not only at the root. child is located by searching for its value, so its parent and the direction (left or right) are inferred.

Parameters

```
rotate(  
  tree: dictionary,  
  child: dictionary  
) -> dictionary
```

tree dictionary

The tree.

child dictionary

The node that should become the new subtree root.

20.1.18. rotate-display

Animate a rotation around `child` — anywhere in the tree, not only at the root. `child` is the node that should become the new subtree root; its parent and the direction are inferred from the search path.

Six frames: the pivots are marked, the edges that will move are broken, the tree restructures with those edges still hidden, they reconnect, and the highlights clear.

Parameters

```
rotate-display(  
  tree: dictionary ,  
  child: dictionary ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

child dictionary

The node that should become the new subtree root.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.1.19. search-display

Animate searching for *v*: one frame per comparison along the search path, each highlighting the visited node and drawing the comparison beside it. The walk ends at a match, or where the search runs off the tree.

Parameters

```
search-display(  
  tree: dictionary ,  
  v: int ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to search for.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.2. rbt — red-black trees

- [black\(\)](#)
- [check-invariants\(\)](#)
- [delete\(\)](#)
- [delete-display\(\)](#)
- [describe\(\)](#)
- [display\(\)](#)
- [double-black\(\)](#)
- [fixup-display\(\)](#)
- [in-order-display\(\)](#)
- [insert\(\)](#)
- [insert-display\(\)](#)
- [insert-many\(\)](#)
- [level-order-display\(\)](#)
- [new\(\)](#)
- [node\(\)](#)
- [paint-black\(\)](#)
- [paint-red\(\)](#)
- [post-order-display\(\)](#)
- [pre-order-display\(\)](#)
- [red\(\)](#)
- [renderer\(\)](#)
- [rotate\(\)](#)
- [search-display\(\)](#)

20.2.1. black

A black node — `node(..., red: false)`. With no children it is a black leaf.

Parameters

```
black(  
    value: int,  
    ..children: dictionary none,  
    label: auto any  
) -> dictionary
```

value int

The ordering key.

..children dictionary or none

Either nothing (a leaf) or exactly two children, left then right.

label auto or any

What to draw in the node; auto draws `str(value)`.

Default: **auto**

20.2.2. check-invariants

Check the search-tree ordering plus the four red-black invariants. Returns `true` or panics naming the first violation.

Parameters

`check-invariants(tree: dictionary) -> bool`

tree dictionary

The tree.

20.2.3. delete

Delete `v`, returning a new tree (or none if the last node went). A node with two children is replaced by its in-order predecessor — value **and** label — which is then excised from the left subtree.

Parameters

```
delete(  
  tree: dictionary,  
  v: int  
) -> dictionary none
```

tree dictionary

The tree.

v int

The ordering key to delete.

20.2.4. delete-display

Animate deleting v , following the CLRS decision tree.

The target is marked, a two-child target hands its slot to its in-order predecessor, the deletion-position node is excised, and then — if that node was black and nothing red was promoted in its place — the double-black fix-up runs, one pair of frames per case: the rotation first, so the shape change is visible, then the recolor.

With `search: true` the deletion is preceded by one frame per comparison on the way down, instead of a single frame showing the whole path.

Parameters

```
delete-display(  
  tree: dictionary ,  
  v: int ,  
  search: bool ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to delete.

search bool

Whether to walk to the target one comparison at a time.

Default: `false`

bits bool

Whether every node carries its black-height bit.

Default: `false`

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.2.5. describe

A recursive textual rendering of the tree, used in alt text. Each node's colour follows its label, separated by a space so a single-letter label doesn't fuse with it.

Parameters

```
describe(tree: dictionary) -> str
```

tree dictionary

The tree.

20.2.6. display

The tree as a single static frame, every node wearing its colour.

Parameters

```
display(  
    tree: dictionary ,  
    bits: bool ,  
    node-style: dictionary ,  
    edge-style: dictionary ,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

bits `bool`

Whether every node carries its black-height bit.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.2.7. double-black

Mark nodes as double-black: the bit 2 on the node and a filled dot on its incoming edge, the way the textbook draws a subtree one black short.

Parameters

`double-black(..keys: str) -> array`

..keys `str`

The paths to mark.

20.2.8. fixup-display

Animate the red-red fix-up on a hand-built tree that already has a violation at `violation-path` — the **lower** of the two reds.

The tree is deliberately not validated: the point is to show fix-up configurations that a single insert cannot produce, such as a red-red in the middle of a tree, or a multi-level Case 1 propagation that ends by blackening the root.

Parameters

```
fixup-display(  
  tree: dictionary,  
  violation-path: str,  
  bits: bool,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

violation-path str

The path of the lower of the two red nodes.

bits bool

Whether every node carries its black-height bit.

Default: `false`

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: `(:)`

theme dictionary

Partial theme override for this call.

Default: `(:)`

20.2.9. in-order-display

Animate an in-order (left, root, right) traversal — the sorted walk.

Parameters

```
in-order-display(  
  tree: dictionary ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

bits bool

Whether every node carries its black-height bit.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.2.10. insert

Insert v , returning a new tree with the invariants restored. Ties descend left, so equal values are legal and land in the left subtree.

Parameters

```
insert(  
  tree: dictionary ,  
  v: int ,  
  label: auto any  
) -> dictionary
```

tree dictionary

The tree.

v int

The ordering key to insert.

label auto or any

What to draw in the new node; auto draws `str(v)`.

Default: **auto**

20.2.11. insert-display

Animate inserting `v`: the search descent in one frame, the new red leaf appearing, then one frame per fix-up step — the red-red check, and whichever of Case 1 (recolor), Case 2 (straighten) and Case 3 (rotate and swap) applies — ending at a blackened root if the fix-up reached it.

Parameters

```
insert-display(  
  tree: dictionary ,  
  v: int ,  
  label: auto any ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to insert.

label `auto` or `any`

What to draw in the new node; `auto` draws `str(v)`.

Default: `auto`

bits `bool`

Whether every node carries its black-height bit.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.2.12. `insert-many`

Insert several values in order. Each is a bare key or a `(value, label)` pair.

Parameters

```
insert-many(  
  tree: dictionary,  
  ..vals: any  
) -> dictionary
```

tree `dictionary`

The tree.

..vals any

The values to insert.

20.2.13. level-order-display

Animate a level-order (breadth-first) traversal.

Parameters

```
level-order-display(  
  tree: dictionary ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

bits bool

Whether every node carries its black-height bit.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.2.14. new

Build a tree from a list of values: the first becomes the black root, the rest are inserted in order, so the CLRS fix-ups produce the final shape. Each is a bare key or a (value, label) pair.

```
rbt.new(8, 4, 12, 2, 6, 10, 14, 1)
rbt.new((8, [eight]), 4, (12, [XII]))
```

Parameters

`new(..vals: any) -> dictionary`

..vals `any`

The values. At least one is required — it becomes the root.

20.2.15. node

A node holding value, with either no children (a leaf) or both — pass none for a missing one.

```
rbt.node(8, rbt.red(4), rbt.black(12), red: false)
```

Parameters

```
node(
  value: int,
  ..children: dictionary none,
  red: bool,
  label: auto any
) -> dictionary
```

value `int`

The ordering key.

..children `dictionary` or `none`

Either nothing (a leaf) or exactly two children, left then right.

red `bool`

The colour bit: true red, false black.

Default: `false`

label `auto` or `any`

What to draw in the node; `auto` draws `str(value)`.

Default: `auto`

20.2.16. **paint-black**

Paint nodes black, with the black-height bit 1.

Parameters

```
paint-black(..keys: str) -> array
```

..keys `str`

The paths to paint.

20.2.17. **paint-red**

Paint nodes red, with the black-height bit 0.

```
apply-ops(r, rbt.paint-red("L", "RR") + commit())
```

Parameters

```
paint-red(..keys: str) -> array
```

..keys `str`

The paths to paint.

20.2.18. **post-order-display**

Animate a post-order (left, right, root) traversal.

Parameters

```
post-order-display(  
  tree: dictionary,  
  bits: bool,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

bits bool

Whether every node carries its black-height bit.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.2.19. pre-order-display

Animate a pre-order (root, left, right) traversal.

Parameters

```
pre-order-display(  
  tree: dictionary ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

bits `bool`

Whether every node carries its black-height bit.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.2.20. red

A red node — `node(..., red: true)`. With no children it is a red leaf.

```
rbt.black(4, rbt.red(2), rbt.red(6))
```

Parameters

```
red(  
  value: int,  
  ..children: dictionary or none,  
  label: auto any  
) -> dictionary
```

value `int`

The ordering key.

..children `dictionary` or `none`

Either nothing (a leaf) or exactly two children, left then right.

label `auto` or `any`

What to draw in the node; `auto` draws `str(value)`.

Default: `auto`

20.2.21. `renderer`

A `Renderer` over this tree, bound to the tree backend and pre-painted with the red-black palette — the entry point for driving an animation yourself with the `Op` command stream. Pass `sticky: true` so the palette (and your own highlights) carry from frame to frame.

Parameters

```
renderer(  
  tree: dictionary ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  sticky: bool ,  
  theme: dictionary  
) -> dictionary
```

tree `dictionary`

The tree.

bits `bool`

Whether every node carries its black-height bit.

Default: `false`

node-style `dictionary`

Base node styling beneath every frame's snapshot.

Default: `(:)`

edge-style `dictionary`

Base edge styling beneath every frame's snapshot.

Default: `(:)`

sticky `bool`

Whether new frames inherit the previous frame's styling.

Default: `false`

theme `dictionary`

Partial theme override for this renderer.

Default: `(:)`

20.2.22. rotate

Rotate around child — anywhere in the tree, not only at the root. child is located by searching for its value, so its parent and the direction are inferred.

This is a **structural** rotation: each rotated node keeps its colour, and the result need not satisfy the red-black invariants. The insert and delete fix-ups do not go through it.

Parameters

```
rotate(  
  tree: dictionary ,  
  child: dictionary  
) -> dictionary
```

tree `dictionary`

The tree.

child `dictionary`

The node that should become the new subtree root.

20.2.23. search-display

Animate searching for v: one frame per comparison along the search path, each highlighting the visited node and drawing the comparison beside it. The walk ends at a match, or where the search runs off the tree.

Parameters

```
search-display(  
  tree: dictionary ,  
  v: int ,  
  bits: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to search for.

bits bool

Whether every node carries its black-height bit.

Default: `false`

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: `(:)`

theme dictionary

Partial theme override for this call.

Default: `(:)`

20.3. avl — AVL trees

- [balance-factor\(\)](#)
- [check-invariants\(\)](#)
- [delete\(\)](#)
- [delete-display\(\)](#)

- `describe()`
- `display()`
- `fixup-display()`
- `imbalance-case()`
- `in-order-display()`
- `insert()`
- `insert-display()`
- `insert-many()`
- `leaf()`
- `level-order-display()`
- `new()`
- `node()`
- `post-order-display()`
- `pre-order-display()`
- `renderer()`
- `rotate()`
- `rotate-display()`
- `search-display()`
- `unbalanced()`

20.3.1. `balance-factor`

The signed balance factor of the node at path, formatted the way the factors: layer draws it — "+1", "0", "-1".

Parameters

```
balance-factor(
  tree: dictionary,
  path: str
) -> str
```

tree `dictionary`

The tree.

path `str`

The L/R path of the node.

20.3.2. `check-invariants`

Check the search-tree ordering, that every stored height agrees with the recomputed one, and that $|bf| \leq 1$ everywhere. Returns `true` or panics naming the first violation.

Parameters

```
check-invariants(tree: dictionary) -> bool
```

tree dictionary

The tree.

20.3.3. delete

Delete v , returning a new tree (or none if the last node went). A node with two children is replaced by its in-order predecessor — value **and** label. Rebalancing runs on the whole climb back to the root, so one deletion may rotate several times.

Parameters

```
delete(  
    tree: dictionary,  
    v: int  
) -> dictionary none
```

tree dictionary

The tree.

v int

The ordering key to delete.

20.3.4. delete-display

Animate deleting v : the target is marked, a two-child target hands its slot to its in-order predecessor, the deletion-position node is excised, and the climb rebalances all the way to the root — a delete can shorten a subtree, so unlike an insert it may rotate several times.

With `search: true` the deletion is preceded by one frame per comparison on the way down, instead of a single frame showing the whole path.

Parameters

```
delete-display(  
    tree: dictionary,  
    v: int,  
    search: bool,  
    factors: bool,  
    heights: bool,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to delete.

search bool

Whether to walk to the target one comparison at a time.

Default: **false**

factors bool

Whether every node carries its signed balance factor.

Default: **false**

heights bool

Whether every non-root edge carries the height of the subtree below it.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.3.5. describe

A recursive textual rendering of the tree, used in alt text. Each node's height follows its label after a colon.

Parameters

```
describe(tree: dictionary) -> str
```

tree dictionary

The tree.

20.3.6. display

The tree as a single static frame.

Parameters

```
display(  
    tree: dictionary,  
    factors: bool,  
    heights: bool,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

factors bool

Whether every node carries its signed balance factor.

Default: **false**

heights bool

Whether every non-root edge carries the height of the subtree below it.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.3.7. fixup-display

Animate the fix-up climb on a hand-built tree that is already imbalanced, starting from `violation-path` — usually the path of whatever was just modified by hand.

The tree is deliberately not validated: the point is to show configurations a single insert or delete cannot produce, such as several imbalances on one spine, or one case shown in isolation.

Parameters

```
fixup-display(  
  tree: dictionary ,  
  violation-path: str ,  
  factors: bool ,  
  heights: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

violation-path str

The deepest path the climb starts from.

factors bool

Whether every node carries its signed balance factor.

Default: `false`

heights `bool`

Whether every non-root edge carries the height of the subtree below it.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.3.8. imbalance-case

Which imbalance case applies at n — "LL", "LR", "RR", "RL", or none when the node is balanced.

Parameters

`imbalance-case`(n : `dictionary` `none`) -> `str` `none`

n `dictionary` or `none`

The node.

20.3.9. in-order-display

Animate an in-order (left, root, right) traversal — the sorted walk.

Parameters

```
in-order-display(  
  tree: dictionary,  
  factors: bool,  
  heights: bool,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

factors bool

Whether every node carries its signed balance factor.

Default: **false**

heights bool

Whether every non-root edge carries the height of the subtree below it.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.3.10. insert

Insert v , returning a new tree with the AVL invariant restored. Ties descend left, so equal values are legal and land in the left subtree.

Parameters

```
insert(  
  tree: dictionary,  
  v: int,  
  label: auto any  
) -> dictionary
```

tree dictionary

The tree.

v int

The ordering key to insert.

label auto or any

What to draw in the new node; auto draws `str(v)`.

Default: **auto**

20.3.11. insert-display

Animate inserting v : the search descent in one frame, the new leaf appearing, then the climb — one recompute frame per ancestor, and at the first node whose balance factor reaches ± 2 , a check naming the case, the straightening rotation if the case is LR or RL, and the rotation at the imbalanced node itself. The climb stops there: one rotation restores the subtree's pre-insertion height.

Parameters

```
insert-display(  
  tree: dictionary,  
  v: int,  
  label: auto any,  
  factors: bool,  
  heights: bool,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to insert.

label auto or any

What to draw in the new node; auto draws `str(v)`.

Default: **auto**

factors bool

Whether every node carries its signed balance factor.

Default: **false**

heights bool

Whether every non-root edge carries the height of the subtree below it.

Default: **false**

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: `(:)`

theme dictionary

Partial theme override for this call.

Default: `(:)`

20.3.12. insert-many

Insert several values in order. Each is a bare key or a (value, label) pair.

Parameters

```
insert-many(  
    tree: dictionary,  
    ..vals: any  
) -> dictionary
```

tree dictionary

The tree.

..vals any

The values to insert.

20.3.13. leaf

A childless node of height 1 — node(value) with the intent spelled out.

Parameters

```
leaf(  
    value: int,  
    label: auto or any,  
    height: auto or int  
) -> dictionary
```

value int

The ordering key.

label auto or any

What to draw in the node; auto draws str(value).

Default: auto

height auto or int

The stored height. auto is 1; an explicit integer builds a deliberately stale node.

Default: auto

20.3.14. level-order-display

Animate a level-order (breadth-first) traversal.

Parameters

```
level-order-display(  
  tree: dictionary ,  
  factors: bool ,  
  heights: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

factors bool

Whether every node carries its signed balance factor.

Default: **false**

heights bool

Whether every non-root edge carries the height of the subtree below it.

Default: **false**

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.3.15. new

Build a tree from a list of values: the first becomes the root, the rest are inserted in order, so the rebalancing produces the final shape. Each is a bare key or a (value, label) pair.

```
avl.new(4, 1, 7, 3, 6, 8)
avl.new((4, [FOUR]), 1, 7, (6, [SIX]))
```

Parameters

`new(..vals: any) -> dictionary`

..vals `any`

The values. At least one is required — it becomes the root.

20.3.16. node

A node holding value, with either no children (a leaf) or both — pass `none` for a missing one. The height is **computed** from the children, so a hand-built literal is as trustworthy as one grown by `insert` and `nobody` has to reimplement the height recursion.

```
avl.node(8, avl.leaf(3), avl.node(10, none, avl.leaf(14)))
```

The one reason to pass `height: explicitly` is to build a tree caught **mid-operation**, with a stale spine — which is exactly what `fixup-display()` animates. `avl.node(4, avl.node(2, none, avl.leaf(3)), none, height: 2)` is “a leaf was just grafted on and nothing has been recomputed yet”.

Parameters

```
node(
  value: int,
  ..children: dictionary none,
  label: auto any,
  height: auto int
) -> dictionary
```

value `int`

The ordering key.

..children `dictionary` or `none`

Either nothing (a leaf) or exactly two children, left then right.

label `auto` or any

What to draw in the node; auto draws `str(value)`.

Default: `auto`

height `auto` or `int`

The stored height. auto computes it from the children; an explicit integer builds a deliberately stale node.

Default: `auto`

20.3.17. post-order-display

Animate a post-order (left, right, root) traversal.

Parameters

```
post-order-display(  
  tree: dictionary,  
  factors: bool,  
  heights: bool,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

factors `bool`

Whether every node carries its signed balance factor.

Default: `false`

heights `bool`

Whether every non-root edge carries the height of the subtree below it.

Default: `false`

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.3.18. pre-order-display

Animate a pre-order (root, left, right) traversal.

Parameters

```
pre-order-display(  
  tree: dictionary ,  
  factors: bool ,  
  heights: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

factors bool

Whether every node carries its signed balance factor.

Default: false

heights `bool`

Whether every non-root edge carries the height of the subtree below it.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.3.19. **renderer**

A Renderer over this tree, bound to the tree backend and pre-painted with whichever tag layers you ask for — the entry point for driving an animation yourself with the `Op` command stream. Pass `sticky: true` so the tags (and your own highlights) carry from frame to frame.

Parameters

```
renderer(  
  tree: dictionary ,  
  factors: bool ,  
  heights: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  sticky: bool ,  
  theme: dictionary  
) -> dictionary
```

tree `dictionary`

The tree.

factors `bool`

Whether every node carries its signed balance factor.

Default: `false`

heights `bool`

Whether every non-root edge carries the height of the subtree below it.

Default: `false`

node-style `dictionary`

Base node styling beneath every frame's snapshot.

Default: `( : )`

edge-style `dictionary`

Base edge styling beneath every frame's snapshot.

Default: `( : )`

sticky `bool`

Whether new frames inherit the previous frame's styling.

Default: `false`

theme `dictionary`

Partial theme override for this renderer.

Default: `( : )`

20.3.20. rotate

Rotate around `child` — anywhere in the tree, not only at the root. `child` is located by searching for its value, so its parent and the direction are inferred.

This is a **structural** rotation: the two rotated nodes get fresh heights and the spine above is refreshed, but the AVL invariant is not restored. The insert and delete fix-ups do not go through it.

Parameters

```
rotate(  
  tree: dictionary ,  
  child: dictionary  
) -> dictionary
```

tree dictionary

The tree.

child dictionary

The node that should become the new subtree root.

20.3.21. rotate-display

Animate a rotation around child — anywhere in the tree, not only at the root. child is the node that should become the new subtree root; its parent and the direction are inferred from the search path.

Six frames: the pivots are marked, the edges that will move are broken, the tree restructures with those edges still hidden, they reconnect, and the highlights clear.

Parameters

```
rotate-display(  
  tree: dictionary ,  
  child: dictionary ,  
  factors: bool ,  
  heights: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

child dictionary

The node that should become the new subtree root.

factors `bool`

Whether every node carries its signed balance factor.

Default: `false`

heights `bool`

Whether every non-root edge carries the height of the subtree below it.

Default: `false`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.3.22. search-display

Animate searching for v : one frame per comparison along the search path, each highlighting the visited node and drawing the comparison beside it. The walk ends at a match, or where the search runs off the tree.

Parameters

```
search-display(  
  tree: dictionary ,  
  v: int ,  
  factors: bool ,  
  heights: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The ordering key to search for.

factors bool

Whether every node carries its signed balance factor.

Default: `false`

heights bool

Whether every non-root edge carries the height of the subtree below it.

Default: `false`

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: `(:)`

theme dictionary

Partial theme override for this call.

Default: `(:)`

20.3.23. unbalanced

Ring nodes as imbalanced — the way a check frame marks the node whose balance factor has hit ± 2 — optionally naming the case that applies in the operation-note slot.

```
apply-ops(r, avl.unbalanced("L", case: "LR") + commit())
```

Parameters

```
unbalanced(  
  ..keys: str,  
  case: str or none  
) -> array
```

..keys str

The paths to mark.

case str or none

The imbalance case ("LL", "LR", "RR", "RL"), drawn in the note slot; none draws no note.

Default: none

20.4. b24 — 2-3-4 trees

- [by-value\(\)](#)
- [check-invariants\(\)](#)
- [contains\(\)](#)
- [delete\(\)](#)
- [delete-display\(\)](#)
- [describe\(\)](#)
- [display\(\)](#)
- [in-order\(\)](#)
- [in-order-display\(\)](#)
- [insert\(\)](#)
- [insert-display\(\)](#)
- [insert-many\(\)](#)
- [leaf\(\)](#)
- [level-order\(\)](#)
- [level-order-display\(\)](#)
- [new\(\)](#)
- [node\(\)](#)
- [path-to\(\)](#)
- [post-order\(\)](#)
- [post-order-display\(\)](#)
- [pre-order\(\)](#)
- [pre-order-display\(\)](#)
- [renderer\(\)](#)
- [resolve\(\)](#)
- [search-display\(\)](#)

20.4.1. by-value

The compartment path "<node-path>#<key-idx>" of the key holding v. Panics when v is absent.

Parameters

```
by-value(  
  tree: dictionary ,  
  v: int  
) -> str
```

tree dictionary

The tree.

v int

The key to look for.

20.4.2. check-invariants

Check all four 2-3-4 invariants: 1 to 3 keys per node, strictly increasing, keys + 1 children at an internal node, and every leaf at the same depth. Returns true or panics naming the first violation.

Parameters

```
check-invariants(tree: dictionary) -> bool
```

tree dictionary

The tree.

20.4.3. contains

Whether v is in the tree.

Parameters

```
contains(  
  tree: dictionary ,  
  v: int  
) -> bool
```

tree dictionary

The tree.

v int

The key to look for.

20.4.4. delete

Delete v , returning a new tree (or none if the last key went).

strategy picks between refilling an under-full node on the way down ("top-down") and merging on the way back up ("bottom-up").

Parameters

```
delete(  
    tree: dictionary,  
    v: int,  
    strategy: str  
) -> dictionary | none
```

tree dictionary

The tree.

v int

The key to delete.

strategy str

"top-down" or "bottom-up".

Default: "top-down"

20.4.5. delete-display

Animate deleting v under the chosen strategy.

Top-down refills every under-full node it passes on the way down — by borrowing from a sibling or merging with one — so the leaf it reaches can always afford to lose a key; bottom-up removes first and repairs on the way back up.

With search: false the descent is dropped entirely — no comparison frames, and no comparison highlights inherited by the frames that remain — so the animation is only the structural work.

Parameters

```
delete-display(  
  tree: dictionary,  
  v: int,  
  search: bool,  
  strategy: str,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The key to delete.

search bool

Whether to show one frame per comparison on the way down.

Default: `true`

strategy str

"top-down" or "bottom-up".

Default: `"top-down"`

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: `(:)`

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.6. describe

A recursive textual rendering of the tree, used in alt text.

Parameters

`describe(tree: dictionary) -> str`

tree dictionary

The tree.

20.4.7. display

The tree as a single static frame.

Parameters

```
display(  
    tree: dictionary ,  
    node-style: dictionary ,  
    edge-style: dictionary ,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.8. in-order

The compartment paths of every key, in left-to-right sorted order — the in-order walk threads each key between its flanking subtrees.

Parameters

`in-order`(`tree`: dictionary) -> array

tree dictionary

The tree.

20.4.9. in-order-display

Animate an in-order traversal — the sorted walk, threading each key between its flanking subtrees.

Parameters

`in-order-display`(
 `tree`: dictionary ,
 `node-style`: dictionary ,
 `edge-style`: dictionary ,
 `theme`: dictionary
) -> array

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.10. insert

Insert `v`, returning a new tree.

`strategy` picks between splitting full nodes preventively on the way down ("top-down") and splitting a transient overflow on the way back up ("bottom-up"). Both are correct; the shapes may differ.

Parameters

```
insert(  
  tree: dictionary ,  
  v: int ,  
  label: auto any ,  
  strategy: str  
) -> dictionary
```

tree dictionary

The tree.

v int

The key to insert.

label auto or any

What to draw in the new compartment; `auto` draws `str(v)`.

Default: `auto`

strategy str

"top-down" or "bottom-up".

Default: "top-down"

20.4.11. insert-display

Animate inserting v under the chosen strategy.

Top-down splits every full node it passes on the way down, so the leaf it reaches always has room; bottom-up walks straight to the leaf, lets it overflow to 4 keys, and splits back up. The two can land the key in different places — running both is the point.

Parameters

```
insert-display(  
    tree: dictionary ,  
    v: int ,  
    label: auto any ,  
    strategy: str ,  
    node-style: dictionary ,  
    edge-style: dictionary ,  
    theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The key to insert.

label auto or any

What to draw in the new compartment; auto draws `str(v)`.

Default: `auto`

strategy str

"top-down" or "bottom-up".

Default: `"top-down"`

node-style dictionary

Base node styling.

Default: `(:)`

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.12. insert-many

Insert several keys in order, top-down. Each is a bare key or a (key, label) pair.

Parameters

```
insert-many(  
  tree: dictionary ,  
  ..vals: any  
) -> dictionary
```

tree dictionary

The tree.

..vals any

The keys to insert.

20.4.13. leaf

A childless node — node(keys) with the intent spelled out. Keys are variadic here, since a leaf has no children to disambiguate them from.

```
b24.leaf(3, 7)  
b24.leaf((3, [three]), 7)
```

Parameters

```
leaf(..keys: int array) -> dictionary
```


..keys `int` or `array`

The keys, each a bare key or a (key, label) pair.

20.4.14. level-order

The compartment paths of every key, breadth-first by node.

Parameters

`level-order(tree: dictionary) -> array`

tree `dictionary`

The tree.

20.4.15. level-order-display

Animate a level-order (breadth-first) traversal.

Parameters

`level-order-display(
 tree: dictionary,
 node-style: dictionary,
 edge-style: dictionary,
 theme: dictionary
) -> array`

tree `dictionary`

The tree.

node-style `dictionary`

Base node styling.

Default: (:)

edge-style `dictionary`

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.16. new

Build a tree from a list of keys: the first seeds the root, the rest are inserted in order. Each is a bare key or a (key, label) pair.

```
b24.new(4, 1, 7, 3, 6, 8)
b24.new((4, [FOUR]), 1, 7, (6, [SIX]))
```

Parameters

`new(..vals: any ) -> dictionary`

..vals any

The keys. At least one is required — it seeds the root.

20.4.17. node

A node holding keys, with either no children (a leaf) or exactly `keys.len() + 1` of them.

`keys` is one key or an array of them; each entry is a bare key or a (key, label) pair, so labels can be mixed in exactly as `new` allows.

```
b24.node((10, 20), b24.leaf(3, 7), b24.leaf(15), b24.leaf(25, 30))
```

Parameters

```
node(
  keys: int array,
  ..children: dictionary
) -> dictionary
```

keys int or array

One key, or an array of keys / (key, label) pairs.

..children dictionary

Nothing (a leaf) or exactly `keys.len() + 1` children.

20.4.18. path-to

Every comparison made searching for v , in order. Each record is (path, key-idx, key-value, key-label, cmp, found). Panics when v is absent — `search-display()` is the form that narrates a miss.

Parameters

```
path-to(  
  tree: dictionary,  
  v: int  
) -> array
```

tree dictionary

The tree.

v int

The key to look for.

20.4.19. post-order

The compartment paths of every key, children before node.

Parameters

```
post-order(tree: dictionary) -> array
```

tree dictionary

The tree.

20.4.20. post-order-display

Animate a post-order traversal — a node's keys after its children.

Parameters

```
post-order-display(  
  tree: dictionary,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.21. pre-order

The compartment paths of every key, node before children.

Parameters

`pre-order`(`tree`: dictionary) -> array

tree dictionary

The tree.

20.4.22. pre-order-display

Animate a pre-order traversal — a node's keys before its children.

Parameters

```
pre-order-display(  
  tree: dictionary ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.4.23. **renderer**

A Renderer over this tree, bound to the tree backend and carrying the subdivided-rectangle node shape — the entry point for driving an animation yourself with the Op command stream. Pass `sticky: true` when you want each frame's styling to accumulate.

Parameters

```
renderer(  
  tree: dictionary ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  sticky: bool ,  
  theme: dictionary  
) -> dictionary
```

tree dictionary

The tree.

node-style dictionary

Base node styling beneath every frame's snapshot.

Default: (:)

edge-style dictionary

Base edge styling beneath every frame's snapshot.

Default: (:)

sticky bool

Whether new frames inherit the previous frame's styling.

Default: **false**

theme dictionary

Partial theme override for this renderer.

Default: (:)

20.4.24. resolve

The subtree at a node-path (no # suffix).

Parameters

```
resolve(  
  tree: dictionary ,  
  path: str  
) -> dictionary
```

tree dictionary

The tree.

path str

The digit path.

20.4.25. search-display

Animate searching for v: one frame per comparison, each ringing the compared compartment and hanging the comparison beside its node. A miss is animated too — the walk ends on the last key checked before the search would run off the tree.

Parameters

```
search-display(  
  tree: dictionary,  
  v: int,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary  
) -> array
```

tree dictionary

The tree.

v int

The key to search for.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.5. trie — tries

- [check-invariants\(\)](#)
- [contains\(\)](#)
- [delete\(\)](#)
- [delete-display\(\)](#)
- [describe\(\)](#)
- [display\(\)](#)
- [has-prefix\(\)](#)
- [insert\(\)](#)
- [insert-display\(\)](#)
- [insert-many\(\)](#)

- `new()`
- `path-to()`
- `renderer()`
- `resolve()`
- `search-display()`
- `words()`

20.5.1. `check-invariants`

Check the structural invariants: the root's char is none, every other node's is a single character, and children are strictly ascending by char. Returns true or panics naming the first violation.

Parameters

`check-invariants(``trie``:` `dictionary``) ->` `bool`

trie `dictionary`

The trie.

20.5.2. `contains`

Whether word is stored — a node exists at that prefix **and** a word ends there.

Parameters

`contains(`
`trie``:` `dictionary``,`
`word``:` `str`
`) ->` `bool`

trie `dictionary`

The trie.

word `str`

The word to look for.

20.5.3. `delete`

Delete word, returning a new trie with the dead branch pruned. Deleting a word that isn't stored changes nothing.

Parameters

```
delete(  
  trie: dictionary ,  
  word: str  
) -> dictionary
```

trie dictionary

The trie.

word str

The word to remove.

20.5.4. delete-display

Animate deleting word: the walk down to its word-end node, the mark coming off (the bit flips “1” to “0” and the shading goes), then the branch that has just died pruned one node per frame.

A node is pruned when it is a childless non-terminal — so deleting "card" from a trie that also holds "car" removes exactly one node, while deleting the only word removes the whole chain.

With search: false the walk is dropped and the animation starts at the unmark. Panics when word isn't stored.

Parameters

```
delete-display(  
  trie: dictionary ,  
  word: str ,  
  search: bool ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

trie dictionary

The trie.

word str

The word to remove.

search `bool`

Whether to walk down to the word before removing it.

Default: `true`

node-style `dictionary`

Base node styling.

Default: `(:)`

edge-style `dictionary`

Base edge styling.

Default: `(:)`

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.5.5. **describe**

A textual rendering of the stored set, used in alt text.

Parameters

`describe(`**trie**`:` `dictionary``) ->` `str`

trie `dictionary`

The trie.

20.5.6. **display**

The trie as a single static frame: word-end nodes shaded, letters on the edges, terminal bits in the nodes.

Parameters

```
display(  
    trie: dictionary ,  
    node-style: dictionary ,  
    edge-style: dictionary ,  
    theme: dictionary  
) -> array
```

trie dictionary

The trie.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme dictionary

Partial theme override for this call.

Default: (:)

20.5.7. has-prefix

Whether prefix is a path in the trie, whether or not a word ends there.

Parameters

```
has-prefix(  
    trie: dictionary ,  
    prefix: str  
) -> bool
```

trie dictionary

The trie.

prefix `str`

The prefix to look for.

20.5.8. insert

Insert word, returning a new trie. Inserting a word that is already stored changes nothing.

Parameters

```
insert(  
    trie: dictionary ,  
    word: str  
) -> dictionary
```

trie `dictionary`

The trie.

word `str`

The word to store.

20.5.9. insert-display

Animate inserting word: first the walk down the longest prefix that already exists, then the remaining suffix growing one node per frame, then the endpoint flipping to a word end.

A word that is already stored gets a single terminal frame saying so — the trie is unchanged, and that is worth showing rather than hiding.

Parameters

```
insert-display(  
    trie: dictionary ,  
    word: str ,  
    node-style: dictionary ,  
    edge-style: dictionary ,  
    theme: dictionary  
) -> array
```

trie `dictionary`

The trie.

word `str`

The word to store.

node-style `dictionary`

Base node styling.

Default: (:)

edge-style `dictionary`

Base edge styling.

Default: (:)

theme `dictionary`

Partial theme override for this call.

Default: (:)

20.5.10. insert-many

Insert several words in order.

Parameters

```
insert-many(  
    trie: dictionary,  
    ..words: str  
) -> dictionary
```

trie `dictionary`

The trie.

..words `str`

The words to store.

20.5.11. new

Build a trie from a list of words, inserted left to right. With no arguments the result is an empty trie — a root and nothing else.

```
trie.new("cat", "car", "card", "dog")
```

Parameters

```
new(..words: str) -> dictionary
```

..words `str`

The words to store.

20.5.12. path-to

Every prefix visited walking word, root first — ("", "c", "ca"). Panics when the word's path isn't present.

Parameters

```
path-to(  
  trie: dictionary,  
  word: str  
) -> array
```

trie `dictionary`

The trie.

word `str`

The word to walk.

20.5.13. renderer

A Renderer over this trie, bound to the tree backend and pre-painted with the terminal shading and the edge letters — the entry point for driving an animation yourself with the 0p command stream. Pass `sticky: true` so the painting (and your own highlights) carry from frame to frame.

Parameters

```
renderer(  
  trie: dictionary ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  sticky: bool ,  
  theme: dictionary  
) -> dictionary
```

trie dictionary

The trie.

node-style dictionary

Base node styling beneath every frame's snapshot.

Default: (:)

edge-style dictionary

Base edge styling beneath every frame's snapshot.

Default: (:)

sticky bool

Whether new frames inherit the previous frame's styling.

Default: false

theme dictionary

Partial theme override for this renderer.

Default: (:)

20.5.14. resolve

The node at prefix, or none when the prefix runs off the trie. "" returns the root.

Parameters

```
resolve(  
  trie: dictionary ,  
  prefix: str  
) -> dictionary none
```

trie dictionary

The trie.

prefix str

The prefix to walk.

20.5.15. search-display

Animate searching for word: one frame per query character, each lighting the edge it matched and the node it reached.

Three ways to end. A missing edge rings the last matched node in danger-stroke and stops. A full match on a word-end node rings it in settled-stroke. A full match on an interior node is flagged a prefix only — the distinction a trie exists to make.

Parameters

```
search-display(  
  trie: dictionary ,  
  word: str ,  
  node-style: dictionary ,  
  edge-style: dictionary ,  
  theme: dictionary  
) -> array
```

trie dictionary

The trie.

word str

The word to search for.

node-style dictionary

Base node styling.

Default: (:)

edge-style dictionary

Base edge styling.

Default: (:)

theme `dictionary`

Partial theme override for this call.

Default: `(:)`

20.5.16. words

Every stored word, lexicographically ordered.

Parameters

`words(`**trie**`:` `dictionary``) ->` `array`

trie `dictionary`

The trie.

20.6. Shared binary-tree operations

These live in `ds/tree-common.typ` and are re-exported by `bst`, `rbt`, and `avl` — `bst.contains`, `avl.in-order`, and so on. The `render-*` helpers are the shared animation bodies those modules build their displays from.

- `by-value()`
- `cmp-string()`
- `contains()`
- `descend-walk()`
- `describe()`
- `in-order()`
- `insert-many()`
- `insert-walk()`
- `level-order()`
- `parse-value()`
- `path-to()`
- `post-order()`
- `pre-order()`
- `render-rotate()`
- `render-search()`
- `render-traversal()`
- `resolve()`
- `search-walk()`
- `stamp-result()`

20.6.1. by-value

The L/R path of the node holding `v`. Panics when `v` is absent.

Parameters

```
by-value(  
  tree,  
  v  
) -> str
```

20.6.2. cmp-string

The comparison shown beside a node during a walk: "7 < 8", "7 = 7".

Parameters

```
cmp-string(  
  v,  
  node-value  
) -> str
```

20.6.3. contains

Whether v is in the tree.

Parameters

```
contains(  
  tree,  
  v  
) -> bool
```

20.6.4. descend-walk

The comparisons made descending to a value already known to be present — the search prefix of a delete.

Parameters

```
descend-walk(  
  tree,  
  v  
) -> array
```

20.6.5. describe

A recursive textual rendering of the tree, for alt text.

head builds one node's own description; the recursion and the (left: ..., right: ...) framing are shared. RBT passes a head that appends the colour, AVL one that appends the height.

Parameters

```
describe(  
  tree,  
  head  
) -> str
```

20.6.6. in-order

The paths of every node, in left-root-right order.

Parameters

```
in-order(tree) -> array
```

20.6.7. insert-many

Fold insert over a list of factory arguments (bare values or (value, label) pairs), returning the grown tree.

Parameters

```
insert-many(  
  tree,  
  insert,  
  vals,  
  who  
) -> dictionary
```

20.6.8. insert-walk

The comparisons an **insert** of *v* makes. Ties go left (matching insert), and the walk ends on the node that will get the new child.

Parameters

```
insert-walk(  
  tree,  
  v  
) -> array
```

20.6.9. level-order

The paths of every node, breadth-first.

Parameters

```
level-order(tree) -> array
```

20.6.10. parse-value

Parse one factory argument into a (value, label) pair: either a bare ordering value (label auto, so the backend draws str(value)) or an explicit (value, label) 2-tuple. who names the caller in errors.

Parameters

```
parse-value(  
    x,  
    who  
) -> dictionary
```

20.6.11. path-to

Every path visited while searching for v, ending at the node holding it. Panics when v is absent.

Parameters

```
path-to(  
    tree,  
    v  
) -> array
```

20.6.12. post-order

The paths of every node, in left-right-root order.

Parameters

```
post-order(tree) -> array
```

20.6.13. pre-order

The paths of every node, in root-left-right order.

Parameters

```
pre-order(tree) -> array
```

20.6.14. render-rotate

Animate a rotation around child — anywhere in the tree, not only at the root. child is the node that should become the new subtree root; its parent and the direction are inferred from the search path. after is the rotated tree, which the caller produces with its own rotate (AVL's recomputes heights, BST's does not).

Six frames: the pivots are marked, the edges that will move are broken, the tree restructures with those edges still hidden, they reconnect, and the highlights clear. step.kind runs "init", "pivots", "break", "restructure", "connect", "settled"; the last carries step.result.

base is the DS's structural painting — (theme) => snapshot — laid down beneath the rotation highlights, the same hook `render-search` takes. `after-base` is the same for the rotated tree, which a DS whose painting depends on the shape needs (AVL's heights move with the pivot); `auto` reuses `base`.

Parameters

```
render-rotate(  
  tree,  
  after,  
  child,  
  ds-name,  
  describe-text,  
  who,  
  base,  
  after-base,  
  node-style,  
  edge-style,  
  theme  
) -> array
```

20.6.15. render-search

Animate a search for `v`: one frame per comparison along the search path, each ringing the visited node in the theme's `search-stroke` and drawing the comparison beside it. The walk ends at a match, or where the search runs off the tree.

`base` is the DS's structural painting — (theme) => snapshot — laid down beneath the search highlights (the RBT palette, AVL's factor and height tags); none leaves the tree unpainted, which is the BST case.

`step.kind` is "init" on the opening frame, "compare" along the way, and "found" / "not-found" on the last one; that frame also carries `step.result` (the unchanged tree).

Parameters

```
render-search(  
  tree,  
  v,  
  ds-name,  
  describe-text,  
  base,  
  node-style,  
  edge-style,  
  theme  
) -> array
```

20.6.16. render-traversal

Animate a traversal: one frame per visit, the visited node filled from the theme's `traversal-palette` and badged with its 1-indexed position, the running output accumulating in the caption. The final frame

wears the whole traversal's colour signature, so last-rendering four of these gives the compare-the-traversals view.

`step.kind` is "init" on the opening frame and "visit" thereafter; the final frame carries `step.result` (the unchanged tree).

`base` is the DS's structural painting — (theme) => snapshot — laid down before the traversal highlights. AVL passes the balance-factor and height tags through it; BST has none.

Parameters

```
render-traversal(  
  tree,  
  paths,  
  name,  
  ds-name,  
  describe-text,  
  base,  
  node-style,  
  edge-style,  
  theme  
) -> array
```

20.6.17. resolve

The subtree at an L/R path, or none when the path runs off a missing child.

Parameters

```
resolve(  
  tree,  
  path  
) -> dictionary none
```

20.6.18. search-walk

The comparisons a **search** for `v` makes: descend while the matching child exists, stopping at a match or where the search runs off the tree.

Parameters

```
search-walk(  
  tree,  
  v  
) -> array
```

20.6.19. stamp-result

Stamp `step.result` onto the last spec in `specs` — the post-operation structure a display's final frame carries, so a caller can advance their variable from the frames instead of writing the operation twice.

Parameters

```
stamp-result(  
  specs,  
  after  
) -> array
```

20.7. graph — weighted graphs

- `add-edge()`
- `add-node()`
- `adjacency-list()`
- `adjacency-matrix()`
- `bfs-display()`
- `check-invariants()`
- `contains-edge()`
- `contains-node()`
- `describe()`
- `dfs-display()`
- `dijkstra-display()`
- `display()`
- `ek()`
- `kruskal-display()`
- `neighbors()`
- `new()`
- `positioned()`
- `prim-display()`
- `renderer()`
- `weight()`
- `with-position()`

20.7.1. add-edge

Add an edge between two existing nodes. `weight` drives the algorithms; `label`, when set, is what gets drawn in its place.

Parameters

```
add-edge(  
  g,  
  u,  
  v,  
  weight,  
  label  
) -> dictionary
```

20.7.2. add-node

Add a node. pos is its (x, y) in cetz units, or none to leave it unplaced (for auto-layout, or for positions passed to a display).

Parameters

```
add-node(  
    g,  
    id,  
    label,  
    pos  
) -> dictionary
```

20.7.3. adjacency-list

The adjacency list as placeable table content, pairing each node with its adjacency. A directed graph lists out-neighbours; an undirected one lists every incident neighbour (a self-loop appears once). weights: true appends each edge's display value in parentheses.

Parameters

```
adjacency-list(  
    g,  
    weights: bool,  
    empty-marker: str,  
    theme: dictionary  
) -> content
```

weights bool

Append each edge's display value to its neighbour.

Default: false

empty-marker str

What to draw for a node with no neighbours.

Default: "-"

theme dictionary

Partial theme override, merged over the ambient theme.

Default: (:)

20.7.4. adjacency-matrix

The adjacency matrix as placeable table content (not frames — wrap it in a figure yourself for a caption or alt text).

Cells default to 1/0 presence; `weights: true` shows each edge's display value and `none-marker` for non-edges. An undirected graph gives a symmetric matrix; a directed one reads row = source, column = target, with self-loops on the diagonal.

Parameters

```
adjacency-matrix(  
  g,  
  weights: bool,  
  none-marker: str,  
  theme: dictionary  
) -> content
```

weights `bool`

Show edge values instead of 1/0 presence.

Default: `false`

none-marker `str`

What to draw where there is no edge, when `weights: true`.

Default: `"."`

theme `dictionary`

Partial theme override, merged over the ambient theme.

Default: `(:)`

20.7.5. bfs-display

Animate a breadth-first traversal from start.

A `target:` turns it into a search that stops the moment that node is dequeued. `spanning-tree: true` renders the BFS tree instead of the palette walk, closing on the tree alone.

Parameters

```
bfs-display(  
  g,  
  start: str,  
  target: none or str,  
  sort-frontier: bool,  
  spanning-tree: bool,  
  positions,  
  scale,  
  node-style,  
  layout,  
  layout-unit,  
  theme  
) -> array
```

start `str`

Where the traversal begins.

target `none` or `str`

Stop as soon as this node is visited; none covers the whole reachable component.

Default: `none`

sort-frontier `bool`

Enqueue each node's unseen neighbours in ascending id order rather than edge-declaration order, for a deterministic visit sequence.

Default: `false`

spanning-tree `bool`

Render the BFS spanning tree instead of the palette traversal.

Default: `false`

20.7.6. check-invariants

Check the structural invariants: every edge's endpoints exist, and no two edges share a canonical key. Returns true or panics with the first violation. Positions are validated lazily, by `positioned`.

Parameters

```
check-invariants(g) -> bool
```

20.7.7. contains-edge

Whether an edge joins u and v. Undirected graphs answer the same either way round; directed ones do not.

Parameters

```
contains-edge(  
    g,  
    u,  
    v  
) -> bool
```

20.7.8. contains-node

Whether id is a node of this graph.

Parameters

```
contains-node(  
    g,  
    id  
) -> bool
```

20.7.9. describe

A one-line prose summary — the opening line of every alt text.

Parameters

```
describe(g) -> str
```

20.7.10. dfs-display

Animate a depth-first traversal from start (iterative, pre-order).

A target: turns it into a search that stops the moment that node is visited. spanning-tree: true renders the DFS tree instead of the palette walk, closing on the tree alone.

Parameters

```
dfs-display(  
  g,  
  start: str,  
  target: none str,  
  sort-frontier: bool,  
  spanning-tree: bool,  
  positions,  
  scale,  
  node-style,  
  layout,  
  layout-unit,  
  theme  
) -> array
```

start str

Where the traversal begins.

target none or str

Stop as soon as this node is visited; none covers the whole reachable component.

Default: none

sort-frontier bool

Explore each node's unseen neighbours in ascending id order rather than edge-declaration order, for a deterministic visit sequence.

Default: false

spanning-tree bool

Render the DFS spanning tree instead of the palette traversal.

Default: false

20.7.11. dijkstra-display

Animate Dijkstra's shortest paths from source.

With a target: the search stops the moment that node is polled. Add reconstruct: true to follow it with the ConstructShortestPath phase, which walks prev back from the end one hop per frame instead of lighting the whole route at once.

Parameters

```
dijkstra-display(  
  g,  
  source: str,  
  target: none str,  
  node-distances: bool,  
  reconstruct: bool,  
  positions,  
  scale,  
  node-style,  
  layout,  
  layout-unit,  
  theme  
) -> array
```

source str

The node distances are measured from.

target none or str

Stop as soon as this node is polled; none runs to completion.

Default: none

node-distances bool

Show each node's tentative distance in its note slot. Turn it off when the dist aux map already carries them.

Default: true

reconstruct bool

Append the path-reconstruction phase (requires a target:).

Default: false

20.7.12. display

The graph as a single static frame.

Parameters

```
display(  
    g,  
    positions: auto dictionary ,  
    scale: float ,  
    node-style: dictionary ,  
    layout: none str ,  
    layout-unit: length ,  
    theme: dictionary  
) -> array
```

positions auto or dictionary

Node positions, or auto for the graph's own.

Default: **auto**

scale float

Multiplier on every coordinate, spreading nodes apart at a fixed drawn size.

Default: **1**

node-style dictionary

One base node style applied beneath every per-frame style — where a whole-graph shape / autosize / rx / ry goes.

Default: **(:)**

layout none or str

A graphviz engine name (e.g. "neato") to lay the graph out internally, or none to use positions. Only a non-none value loads the optional diagraph- layout dependency.

Default: **none**

layout-unit length

The cetz-unit size of one graphviz layout unit.

Default: **36pt**

theme dictionary

Partial theme override, merged over the ambient theme.

Default: **(:)**

20.7.13. **ek**

This graph's canonical key for the edge between u and v — the key its snapshots and anchor use. `edge-key(u, v, directed:)` is the same function without a graph to read the directedness from.

Parameters

```
ek(  
    g,  
    u,  
    v  
) -> str
```

20.7.14. **kruskal-display**

Animate Kruskal's minimum spanning tree.

Parameters

```
kruskal-display(  
    g,  
    positions,  
    scale,  
    node-style,  
    layout,  
    layout-unit,  
    theme  
) -> array
```

20.7.15. **neighbors**

The out-neighbours of id , as an array of $(id, weight)$. Every incident edge contributes its other endpoint in an undirected graph; only edges leaving id do in a directed one.

Parameters

```
neighbors(  
    g,  
    id  
) -> array
```

20.7.16. **new**

Build a graph from a list of nodes plus a list of edges.

Each node is one of:

- a bare `id` string (unplaced — supply positions later),
- an `(id, label)` pair (a display label, still unplaced),
- an `(id, x, y)` triple (placed, label defaults to the `id`), or

- an (id, x, y, label) 4-tuple (both).

Each edge is (u, v) optionally followed by a weight and/or a label. The third slot dispatches by type: a number is the weight, anything else is a display label drawn in its place. (u, v, weight, label) sets both — the numeric weight still drives Dijkstra and the MSTs.

```
#let g = graph.new(
  (("A", 0, 0), ("B", 3, 1), ("C", 1.5, 2.4)),
  edges: (("A", "B", 7), ("B", "C", 2), ("A", "C", 4)),
)
```

Parameters

```
new(
  nodes: array,
  edges: array,
  directed: bool
) -> dictionary
```

nodes array

The nodes — see above for the four accepted forms.

edges array

The edges, each (u, v) plus an optional weight and/or label.

Default: ()

directed bool

Whether edges are one-way (arrowheads, ordered edge keys).

Default: false

20.7.17. positioned

The positioned-graph dict the backend consumes — the entry point for a hand-composed cetz canvas (draw-graph) or the op command stream. Uses the graph's own positions unless an explicit map is passed; panics if any node lacks one.

scale multiplies every coordinate about the origin, spreading nodes apart while their drawn size stays fixed — the manual-layout analog of auto-layout's unit:.

Parameters

```
positioned(  
  g,  
  positions,  
  scale  
) -> dictionary
```

20.7.18. prim-display

Animate Prim's minimum spanning tree, growing from start.

Parameters

```
prim-display(  
  g,  
  start: str,  
  positions,  
  scale,  
  node-style,  
  layout,  
  layout-unit,  
  theme  
) -> array
```

start **str**

The node the tree grows from.

20.7.19. renderer

A Renderer over this graph, bound to the graph backend — the entry point for driving an animation yourself with the op command stream. Pass sticky: true when each frame's styling should accumulate.

Parameters

```
renderer(  
  g,  
  positions,  
  scale,  
  layout,  
  layout-unit,  
  node-style,  
  edge-style,  
  sticky,  
  theme  
) -> dictionary
```

20.7.20. weight

The weight of the edge between u and v. Panics when there is none.

Parameters

```
weight(  
  g,  
  u,  
  v  
) -> any
```

20.7.21. with-position

Place (or move) a node.

Parameters

```
with-position(  
  g,  
  id,  
  pos  
) -> dictionary
```

20.8. hashmap — hash maps

- [check-invariants\(\)](#)
- [contains\(\)](#)
- [delete\(\)](#)
- [delete-display\(\)](#)
- [describe\(\)](#)
- [display\(\)](#)
- [get\(\)](#)
- [hash-of\(\)](#)
- [insert\(\)](#)
- [insert-display\(\)](#)
- [load-factor\(\)](#)
- [new\(\)](#)
- [positioned\(\)](#)
- [probe-seq\(\)](#)
- [renderer\(\)](#)
- [resize\(\)](#)
- [resize-display\(\)](#)
- [search-display\(\)](#)
- [size\(\)](#)

Variables

- [strategies](#)

20.8.1. check-invariants

Check the structural invariants: a positive capacity, a known strategy, a slots array the length of the capacity, and every slot shaped for the strategy. Returns true or panics naming the first violation.

Parameters

```
check-invariants(hm) -> bool
```

20.8.2. contains

Whether key is present.

Parameters

```
contains(  
  hm,  
  key  
) -> bool
```

20.8.3. delete

Delete key. Chaining unlinks the entry; open addressing writes a tombstone, never a plain empty, so later probes still traverse the slot. A no-op when the key is absent.

tombstone: false is the **naïve** open-addressing deletion: it clears the slot to empty instead. That is deliberately buggy — a later search whose probe sequence runs through the cleared slot now stops early and fails to find keys stored beyond it. It exists to **teach** that failure. The flag is a no-op for chaining, which has no tombstones.

Parameters

```
delete(  
  hm,  
  key,  
  tombstone  
) -> dictionary
```

20.8.4. delete-display

Animate deleting key: walk to it, then remove it — chaining unlinks the entry, open addressing writes a tombstone (×) so later probes still traverse the slot. tombstone: false animates the naïve deletion that clears the slot instead (see delete). search: false drops the leading walk and cuts straight to the removal.

Parameters

```
delete-display(  
  hm,  
  key,  
  orientation,  
  tombstone,  
  search,  
  theme,  
  cell-width  
) -> array
```

20.8.5. describe

A textual rendering of the table, used in alt text.

Parameters

```
describe(hm) -> str
```

20.8.6. display

The table as a single static frame.

Parameters

```
display(  
  hm,  
  orientation,  
  theme,  
  cell-width  
) -> array
```

20.8.7. get

The value stored for key, or default when absent.

Parameters

```
get(  
  hm,  
  key,  
  default  
) -> any
```

20.8.8. hash-of

The bucket index key hashes to.

Parameters

```
hash-of(  
  hm,  
  key  
) -> int
```

20.8.9. insert

Insert (or update) key. An existing key is replaced in place — this is a map, not a multiset. Panics only when an open-addressing table is full and the key can't be placed.

Parameters

```
insert(  
  hm,  
  key,  
  value,  
  label  
) -> dictionary
```

20.8.10. insert-display

Animate inserting key: the hash box, the probe or chain walk, and the landing.

Parameters

```
insert-display(  
  hm,  
  key,  
  value,  
  label,  
  orientation,  
  theme,  
  cell-width  
) -> array
```

20.8.11. load-factor

The load factor, n / m .

Parameters

```
load-factor(hm) -> float
```

20.8.12. new

Build a hash map with capacity slots, a collision strategy, a pluggable hash function ($\text{key}, m \Rightarrow \text{index}$), and a display hash-repr string in which k means the key and m the capacity (substituted as whole words — write them standalone, since "3k" will not substitute).

The "double" strategy takes a second hash hash2 for the probe step size (with its own hash2-repr); the other strategies ignore it. Seed the table by listing entries — each a bare key or a (key, value) pair.

Parameters

```
new(  
  capacity: int,  
  strategy: str,  
  hash: function,  
  hash-repr: str,  
  hash2: function,  
  hash2-repr: str,  
  entries: array  
) -> dictionary
```

capacity int

The number of array slots (m). Must be positive.

strategy str

"chaining", "linear", "quadratic", or "double".

Default: "chaining"

hash function

The hash function ($\text{key}, m \Rightarrow \text{index}$); the division method by default.

Default: $(k, m) \Rightarrow \text{calc.rem}(k, m)$

hash-repr str

The hash's display form, shown in the animation's hash box.

Default: " $k \bmod m$ "

hash2 function

The second hash ($\text{key}, m \Rightarrow \text{step}$, sizing each probe step for the "double" strategy. The step is normalized into $[1, m)$. The default is nonzero and — when m is prime — coprime to m , so the probe sequence visits every slot.

Default: $(k, m) \Rightarrow 1 + \text{calc.rem}(k, m - 1)$

hash2-repr `str`

The second hash's display form, shown on the hash box's second line.

Default: `"1 + (k mod (m - 1))"`

entries `array`

Items to seed the table with, inserted in order — each a bare key or a (key, value) pair.

Default: `()`

20.8.13. positioned

The positioned-table dict the renderer consumes — the entry point for a hand-composed cetz canvas (`draw-hashmap`) or the 0p command stream (`make-renderer(positioned(hm), draw-hashmap)`). orientation picks the layout; hash-box, when set to (key, expr, index[, expr2, step]), draws the hash-box overlay.

Parameters

```
positioned(  
  hm,  
  orientation,  
  hash-box,  
  cell-width  
) -> dictionary
```

20.8.14. probe-seq

The probe sequence for key: capacity indices in probe order for open addressing, or the single relevant bucket for chaining.

Parameters

```
probe-seq(  
  hm,  
  key  
) -> array
```

20.8.15. renderer

A Renderer over this table, bound to the hash-map backend — the entry point for driving an animation yourself with the 0p command stream. Pass `sticky: true` when you want each frame's styling to accumulate.

Parameters

```
renderer(  
  hm,  
  orientation,  
  hash-box,  
  cell-width,  
  node-style,  
  edge-style,  
  sticky,  
  theme  
) -> dictionary
```

20.8.16. **resize**

Grow (or shrink) to new-cap, rehashing every live entry through the new capacity in slot order. Tombstones are dropped.

rehash: false is the **naive** resize (buggy, for teaching): each live entry is copied verbatim into its OLD index of the larger array, without recomputing the hash. Because $h(k) \bmod \text{new-cap}$ then points elsewhere, a later search for a moved key probes the wrong slot and misses — the demonstration of why a real resize must rehash. It requires $\text{new-cap} \geq \text{capacity}$, so the old indices fit.

Parameters

```
resize(  
  hm,  
  new-cap,  
  rehash  
) -> dictionary
```

20.8.17. **resize-display**

Animate growing (or shrinking) to new-cap, rehashing every live entry through the new capacity one frame at a time. rehash: false animates the naive resize that copies entries to their old indices instead (see resize).

Parameters

```
resize-display(  
  hm,  
  new-cap,  
  rehash,  
  orientation,  
  theme,  
  cell-width  
) -> array
```


20.8.18. search-display

Animate looking key up — the same walk as insert, ending in a settled ring on a hit or a danger ring on a miss. Open addressing stops at the first empty slot; chaining at the chain's end.

Parameters

```
search-display(  
    hm,  
    key,  
    orientation,  
    theme,  
    cell-width  
) -> array
```

20.8.19. size

The number of live entries — empties and tombstones don't count.

Parameters

```
size(hm) -> int
```

20.8.20. strategies

The collision strategies this module understands.

20.9. sort — linear sorts

- `check-invariants()`
- `counting()`
- `counting-display()`
- `describe()`
- `display()`
- `len()`
- `new()`
- `positioned()`
- `radix()`
- `radix-display()`
- `renderer()`
- `sorted()`

Variables

- `variants`

20.9.1. check-invariants

Check the structural invariants: every value is a non-negative integer and the labels array is parallel to it. Returns true or panics.

Parameters

`check-invariants(s)` -> `bool`

20.9.2. counting

Counting-sort the keys into `k` buckets (default `max + 1`), returning the sorted integer keys. Labels are a display concern and do not come along.

Parameters

```
counting(  
    s,  
    k  
) -> array
```

20.9.3. counting-display

Animate counting sort.

`variant`: "prefix" (the default) is the stable count → prefix-sum → place version; "reconstruct" is the simpler histogram-then-emit intro version; "buckets" is the space-inefficient chaining-hash-table view.

`separate-counts`: `true` (prefix only) splits the histogram and the cumulative sums into two rows, so the raw counts stay visible while the end positions build in their own row.

Parameters

```
counting-display(  
    s,  
    k: auto int,  
    variant: str,  
    separate-counts: bool,  
    theme: dictionary,  
    cell-width: str auto float  
) -> array
```

k `auto` or `int`

The count-array size. `auto` is `max + 1`.

Default: `auto`

variant `str`

Which view: "prefix", "reconstruct", or "buckets".

Default: `"prefix"`

separate-counts `bool`

Give the cumulative sums their own row (prefix only).

Default: `false`

theme `dictionary`

Partial theme override, merged over the ambient theme.

Default: `(:)`

cell-width `str` or `auto` or `float`

Cell sizing: "fit", auto, or an exact width.

Default: `"fit"`

20.9.4. **describe**

A one-line prose summary — the opening line of every alt text.

Parameters

`describe(s)` -> `str`

20.9.5. **display**

The array as a single static frame — one row, no operation styling.

Parameters

```
display(  
    s,  
    theme,  
    cell-width  
) -> array
```

20.9.6. **len**

How many elements there are.

Parameters

`len(s)` -> `int`

20.9.7. new

Build a sort from a list of elements.

Each is either a bare non-negative integer (the common case — both the sort key and what is shown) or a (value: <int>, label: <content>) dict for sorting an **enumeration**: the integer value is the sort key (counting and radix index by it) and label is the content drawn in the cell. The two may mix.

Accepts a splat of elements (sort.new(3, 1, 4)) or a single array of them (sort.new((3, 1, 4))).

```
#let s = sort.new(3, 1, 4, 1, 5)
#let week = sort.new(
  (value: 2, label: [Tue]),
  (value: 0, label: [Sun]),
  (value: 1, label: [Mon]),
)
```

Parameters

```
new(..args: int dictionary array) -> dictionary
```

```
..args int or dictionary or array
```

The elements — a splat of ints and/or (value:, label:) dicts, or one array of them.

20.9.8. positioned

The positioned-table dict the backend consumes — the entry point for a hand-composed cetz canvas (draw-array) or the op command stream.

Parameters

```
positioned(
  s,
  cell-width
) -> dictionary
```

20.9.9. radix

LSD radix-sort the keys in base (default 10), returning them sorted.

Parameters

```
radix(
  s,
  base
) -> array
```

20.9.10. radix-display

Animate LSD radix sort: one stable prefix counting-sort pass per digit place in base. Each input cell shows the digit the active pass extracted as a subscript, and each pass starts from the previous pass's order.

Parameters

```
radix-display(  
  s,  
  base: int,  
  theme: dictionary,  
  cell-width: str | auto | float  
) -> array
```

base int

The radix. Ten gives the familiar ones / tens / hundreds passes.

Default: 10

theme dictionary

Partial theme override, merged over the ambient theme.

Default: (:)

cell-width str or auto or float

Cell sizing: "fit", auto, or an exact width.

Default: "fit"

20.9.11. renderer

A Renderer over this array, bound to the array backend — the entry point for driving an animation yourself with the op command stream. Pass `sticky: true` when each frame's styling should accumulate.

Parameters

```
renderer(  
  s,  
  cell-width,  
  node-style,  
  edge-style,  
  sticky,  
  theme  
) -> dictionary
```

20.9.12. sorted

The keys in sorted order, via Typst's builtin sort — the oracle a test checks counting and radix against.

Parameters

`sorted(s)` -> `array`

20.9.13. variants

The counting-sort variants this module understands.

20.10. skiplist — skip lists

- `check-invariants()`
- `contains()`
- `delete()`
- `delete-display()`
- `describe()`
- `display()`
- `get()`
- `insert()`
- `insert-display()`
- `keys()`
- `len()`
- `levels()`
- `new()`
- `positioned()`
- `renderer()`
- `search-display()`
- `search-walk()`

20.10.1. check-invariants

Check the structural invariants: keys are strictly ascending non-negative integers, and every tower height is in `1..max-level`. Returns `true` or panics.

Parameters

`check-invariants(sl)` -> `bool`

20.10.2. contains

Whether key is in the list — answered by the same descent the animation draws.

Parameters

```
contains(  
  sl,  
  key  
) -> bool
```

20.10.3. delete

Delete key, returning the list unchanged when it is absent.

Parameters

```
delete(  
    sl,  
    key  
) -> dictionary
```

20.10.4. delete-display

Animate deleting key: the descent unlinking each of the target's lanes as it reaches them, leaving the node detached in place. A miss ends on a single danger frame. search: false drops the navigation frames and shows only the pointer surgery.

Parameters

```
delete-display(  
    sl,  
    key,  
    search,  
    theme,  
    cell-width  
) -> array
```

20.10.5. describe

A one-line prose summary — the opening line of every alt text.

Parameters

```
describe(sl) -> str
```

20.10.6. display

The skip list as a single static frame.

Parameters

```
display(  
    sl,  
    theme,  
    cell-width  
) -> array
```

20.10.7. get

The label stored at key, or none when it is absent.

Parameters

```
get(  
    sl,  
    key  
) -> any
```

20.10.8. insert

Insert key, updating the label in place when it is already present. height is explicit (capped at max-level) or a seeded coin flip; the rng advances only when a flip actually happens.

Parameters

```
insert(  
    sl,  
    key,  
    label,  
    height  
) -> dictionary
```

20.10.9. insert-display

Animate inserting key: the descent, with the new node's column reserved as a ghost, then its tower materializing and splicing into one lane per frame. height is explicit or the seeded coin flip the pure insert would have picked.

Parameters

```
insert-display(  
    sl,  
    key,  
    label,  
    height,  
    theme,  
    cell-width  
) -> array
```

20.10.10. keys

The keys, in sorted order.

Parameters

```
keys(sl) -> array
```


20.10.11. len

How many keys the list holds.

Parameters

`len(sl)` -> `int`

20.10.12. levels

How many levels the list reaches — the tallest tower, at least one.

Parameters

`levels(sl)` -> `int`

20.10.13. new

Build a skip list from a set of keys.

Each element is either a bare non-negative integer (its own key, an auto label, and a seeded coin-flip tower) or a (value: <int>, label: <content>, height: <int>) dict — value required, the other two optional. A missing height is chosen by a deterministic coin flip off seed, so references stay stable; duplicate keys are rejected.

Accepts a splat of elements (`skiplist.new(3, 1, 4)`) or a single array of them.

```
#let s = skiplist.new(3, 1, 4, 5, seed: 7)           // seeded towers
#let t = skiplist.new((value: 3, height: 3), 1, 4)  // one height pinned
```

Parameters

```
new(
  ..args: int dictionary array,
  seed: int,
  max-level: int,
  p: float,
  nil: bool
) -> dictionary
```

..args `int` or `dictionary` or `array`

The keys — a splat of ints and/or (value:, label:, height:) dicts, or one array of them.

seed `int`

Coin-flip seed, giving deterministic towers for every element without an explicit height.

Default: `1`

max-level `int`

The tallest tower a coin flip may build.

Default: `4`

p `float`

The probability of growing one more level on each flip.

Default: `0.5`

nil `bool`

Whether to draw the nil tail sentinel.

Default: `true`

20.10.14. **positioned**

The positioned-table dict the backend consumes — the entry point for a hand-composed cetz canvas (draw-skiplist) or the op command stream.

Parameters

```
positioned(  
  sl,  
  cell-width  
) -> dictionary
```

20.10.15. **renderer**

A Renderer over this list, bound to the skip-list backend — the entry point for driving an animation yourself with the op command stream. Pass `sticky: true` when each frame's styling should accumulate.

Parameters

```
renderer(  
  sl,  
  cell-width,  
  node-style,  
  edge-style,  
  sticky,  
  theme  
) -> dictionary
```

20.10.16. search-display

Animate the top-left search descent for key, leaving the whole walk lit behind it.

Parameters

```
search-display(  
  sl,  
  key,  
  theme,  
  cell-width  
) -> array
```

20.10.17. search-walk

The classic skip-list search, shared by the pure operations and every animation. Descends from the top level, moving right while the next key is below key and dropping a level when it would overshoot.

links gives each node's **linked** height ($\text{links}[j] > L \iff \text{node } j \text{ participates at level } L$), which is what lets an insert's search ignore its own not-yet-spliced node by giving it link 0.

Returns found, the per-level update array (the node index whose level-L pointer precedes the search position, -1 = header), the animation path of advance / drop moves, the node index cand the search lands before at level 0, and how many levels it considered.

Parameters

```
search-walk(  
  nodes,  
  links,  
  key  
) -> dictionary
```

20.11. styles — the semantic style vocabulary

- [attention\(\)](#)
- [danger\(\)](#)
- [force-show\(\)](#)
- [ghost\(\)](#)
- [hidden\(\)](#)
- [nullify\(\)](#)
- [revealed\(\)](#)
- [search\(\)](#)
- [subtree\(\)](#)
- [success\(\)](#)

20.11.1. attention

Ring an element in the “look here” color — a rotation pivot, a deletion target, the entry being polled.

Parameters

```
attention(...keys)
```

20.11.2. **danger**

Ring an element as broken, removed, or rejected.

Parameters

`danger(...keys)`

20.11.3. **force-show**

Draw an edge even though one of its endpoints is a phantom — the stub edge into a nil position (the RBT double-black marker needs one).

Parameters

`force-show(...keys)`

20.11.4. **ghost**

Make elements invisible but keep their space: the layout is identical to the fully-drawn structure, so a progressive reveal doesn't shift what is already on screen. This is the one to reach for on a slide.

Parameters

`ghost(...keys)`

20.11.5. **hidden**

Drop elements from the drawing entirely. Unlike ghost, this releases their layout space, so everything around them moves.

Parameters

`hidden(...keys)`

20.11.6. **nullify**

Draw a nil child as a real $\emptyset$ node. Tree phantoms exist only where the layout generates a balance sibling or an edge sets `force-show: true`; materializing anywhere else has nothing to attach to.

Parameters

`nullify(...keys)`

20.11.7. **revealed**

Undo ghost or hidden on a sticky frame.

Parameters

`revealed(...keys)`

20.11.8. search

Ring an element as part of a search or insert walk.

Parameters

`search(...keys)`

20.11.9. subtree

Elide a subtree: draw the node as a gray triangle with its incoming edge landing on the apex, standing in for “any subtree of the right height”. The gray comes from the render theme’s `elided-fill` / `elided-stroke`, so it stays distinguishable from a real node under any palette.

Parameters

`subtree(...keys)`

20.11.10. success

Mark an element as settled: the success fill plus the terminal ring.

Parameters

`success(...keys)`

20.12. Presentation helpers

Exported flat: `last`, `stacked`, `figures`, `subslides`, `canvas`.

- `canvas()`
- `figures()`
- `last()`
- `stacked()`
- `subslides()`

20.12.1. canvas

One frame’s bare canvas, with no caption and **no alt text** — for hand-built layouts where you place the canvas yourself and own the accessibility. Reach for `last()` or `figures()` unless you are doing exactly that.

Parameters

```
canvas(  
  frame: dictionary ,  
  theme: dictionary  
) -> content
```

frame dictionary

A single frame.

theme dictionary

Partial theme override for this render, merged over the ambient theme.

Default: (:)

20.12.2. figures

An array of figure content, one per frame — splat it into `touying's alternatives(...)` to give each frame its own subslide.

Unlike `last()` and `stacked()` this cannot collapse the theme read to one: each figure is laid out independently (touying splits them across subslides), so each resolves the theme in its own context. In a deck that relies on `set-theme`, prefer a per-call `theme:` argument on the display to skip state entirely.

Parameters

```
figures(  
  frames: array dictionary ,  
  caption: bool ,  
  caption-spacing: length ,  
  alt: auto str  
) -> array
```

frames array or dictionary

The frame array from a `*-display`, or a single frame.

caption bool

Whether to stack each frame's caption below its canvas in the figure.

Default: `true`

caption-spacing length

Spacing between canvas and caption when `caption: true`.

Default: `0.5em`

alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's own.

Default: `auto`

20.12.3. `last`

The final frame as one piece of content — a static rendering for print, or “just show me the end state”.

Accepts either the frame array a `*-display` returns or a single frame, so picking one step out of an animation needs no re-wrapping:

```
#last(bst.search-display(t, 7).at(3))
```

The in-canvas annotations already say what the final step did, so the textual caption is suppressed by default.

Parameters

```
last(  
    frames: array dictionary,  
    caption: bool,  
    spacing: length,  
    alt: auto str  
) -> content
```

frames `array` or `dictionary`

The frame array from a `*-display`, or a single frame.

caption `bool`

Whether to stack the step's caption below the canvas.

Default: `false`

spacing `length`

Vertical spacing between canvas and caption when `caption: true`.

Default: `0.5em`

alt `auto` or `str`

Alt-text override. `auto` uses the frame's own `alt`.

Default: `auto`

20.12.4. stacked

Every frame stacked vertically as one block — the handout form, showing a whole animation in one figure. Captions are on by default, since a stack reads as a sequence and the caption says which step each image is.

Each frame is wrapped in its own alt-tagged figure, so the narration stays per-step.

Parameters

```
stacked(  
  frames: array dictionary ,  
  caption: bool ,  
  spacing: length ,  
  caption-spacing: length ,  
  alt: auto str  
) -> content
```

frames `array` or `dictionary`

The frame array from a `*-display`, or a single frame.

caption `bool`

Whether to include each frame's caption below its canvas.

Default: `true`

spacing `length`

Vertical spacing between frames.

Default: `1em`

caption-spacing `length`

Spacing between each canvas and its caption.

Default: `0.5em`

alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's own.

Default: `auto`

20.12.5. `subslides`

One piece of composed content per frame — canvas, the algorithm's auxiliary state, and the step's caption, laid out together and ready to splat into `touying's alternatives(..)`:

```
#alternatives(..subslides(graph.bfs-display(g, "A"), aux: "right"))
```

This is the whole point of the frame model on a slide: the same drawing re-rendered per step, with the queue or the priority queue beside it and the narration beneath. Composing it by hand (a grid of canvas + aux-strip + f.caption, each wrapped in its own `alternatives`) both loses the alt text and lets the pieces drift apart from subslide to subslide; this keeps one composition per step and wraps all of it in the frame's alt figure.

Starling still has no touying dependency — the return value is a plain array of content.

Parameters

```
subslides(  
  frames: array dictionary ,  
  caption: bool ,  
  aux: none str ,  
  aux-view: auto str ,  
  aux-size: length ,  
  fit: none ratio array ,  
  alt: auto str  
) -> array
```

frames `array` or `dictionary`

The frame array from a `*-display`, or a single frame.

caption `bool`

Whether to place each frame's caption below the composition.

Default: `true`

aux `none` or `str`

Where to put the auxiliary strip, if anywhere: `none`, `"right"`, `"left"`, or `"below"`. Only the graph algorithm displays carry the state a strip needs.

Default: `none`

aux-view `auto` or `str`

Which auxiliary view to show when a frame carries several (Kruskal carries two, Dijkstra three). `auto` stacks them all, exactly as `aux-strip` does on its own.

Default: `auto`

aux-size `length`

Text size for the auxiliary strip. Strips carry a lot of small cells, so they read better a little below body size.

Default: `0.8em`

fit `none` or `ratio` or `array`

Scale the compositions down (or up) to fit: a `ratio` scales by exactly that much, and a `(width, height)` pair measures every frame and picks the one factor that fits the largest of them in that box — so the drawing keeps a constant size across the subslides instead of resizing under each step.

Default: `none`

alt `auto` or `str`

Alt-text override applied to every frame. `auto` uses each frame's own.

Default: `auto`

20.13. aux — auxiliary state strips

`aux-strip` is also exported flat.

- `aux-strip()`
- `aux-view-title()`

20.13.1. aux-strip

Render the auxiliary bookkeeping captured for one animation frame as a placeable strip of boxes — a teaching aid so students can track the algorithm's own state alongside the canvas. Pass a frame's `step`:

```
#let frames = graph.bfs-display(g, "A")
#grid(columns: 2, canvas(frames.at(2)), aux-strip(frames.at(2).step))
```

What each display stashes:

- `bfs-display` / `dfs-display` — the BFS queue / DFS stack (the stack is shown faithfully, duplicates and all, since a node may be pushed more than once before an earlier copy is popped).
- `prim-display` — the frontier priority queue of crossing edges, min first, the chosen edge ringed.
- `kruskal-display` — two views: the sorted edge list with a cursor and per-edge status, plus the disjoint-set partition.
- `dijkstra-display` — three views: the priority queue of `(node, dist)` entries (min first, the polled entry ringed), and the `dist` and `prev` maps.

Returns plain content, not a Frame, so it drops anywhere. When a step carries several views they stack vertically; pass `view:` to render just one for separate placement.

Parameters

```
aux-strip(  
  step: dictionary,  
  labels: dictionary,  
  view: auto or str,  
  title: auto or bool,  
  theme: dictionary  
) -> content
```

step dictionary

One frame's step metadata dict.

labels dictionary

Optional id-to-content map giving each node a display label (to match custom node labels; also used for the endpoints of edge elements). Ids not present fall back to the id string itself.

Default: `(:)`

view auto or str

Select a single view by its kind when the step carries more than one (e.g. "partition" for Kruskal's disjoint sets). `auto` renders every view the step holds, stacked.

Default: `auto`

title auto or bool

Whether to stamp each view's title above it. `auto` shows titles only when several views are stacked, so a single view: is untitled; `true` forces one on, `false` suppresses it even for a stack.

Default: `auto`

theme dictionary

Partial theme override, merged over the ambient theme. Elements are colored from `theme.op` (by status) and `theme.render` (structurally).

Default: `(:)`

20.13.2. aux-view-title

The human-readable name of one view kind — the same heading `aux-strip()` stamps above each view when it stacks several.

Exposed so a slide that places views separately (via `aux-strip(step, view: ..)`) can render each view's title in its own style. The kinds a frame carries are the kind fields of its `step.aux-views`.

Parameters

`aux-view-title(kind: str) -> str`

kind str

A view kind: "queue" / "stack" / "pq" / "edge-list" / "partition" / "dist-pq" / "dist-map" / "prev-map".

20.14. The op command stream

Exported flat: `style-node`, `style-edge`, `annotate`, `commit`, `set-alt`, `set-caption`, `set-step`, `apply-ops`.

- `annotate()`
- `apply-ops()`
- `commit()`
- `set-alt()`
- `set-caption()`
- `set-step()`
- `style-edge()`
- `style-node()`

20.14.1. annotate

Attach an operation note to a node — the transient annotation drawn beside it in the canvas.

Parameters

```
annotate(  
    key,  
    note  
)
```

20.14.2. **apply-ops**

Fold a stream of ops into a renderer, returning the updated renderer. Nested arrays are flattened, so +-composed helper output needs no unwrapping.

The trailing in-progress frame is kept — no closing commit is needed after the last batch — but give it alt text with `set-alt` so it is as accessible as the committed frames.

Parameters

```
apply-ops(  
  renderer,  
  ops  
)
```

20.14.3. **commit**

Close the in-progress frame — attaching any caption, step metadata, and alt text given here — and open a fresh one. Arguments left none leave whatever the frame already has.

Parameters

```
commit(  
  caption,  
  step,  
  alt  
)
```

20.14.4. **set-alt**

Set the in-progress frame's alt text without committing. Use it for the trailing frame, which has no following commit to carry its text.

Parameters

```
set-alt(text)
```

20.14.5. **set-caption**

Set the in-progress frame's caption without committing — the caption counterpart of `set-alt`, for the same trailing frame.

Parameters

```
set-caption(caption)
```

20.14.6. set-step

Set the in-progress frame's step metadata without committing. A hand-driven renderer needs this on its last frame, since `result(frames)` reads `step.result` from there.

Parameters

`set-step(step)`

20.14.7. style-edge

Style one or more edges. Positional arguments are element keys; named arguments are the edge style applied to each of them. An edge is keyed by its child (trees) or by `edge-key(u, v)` (graphs).

Parameters

`style-edge(...args)`

20.14.8. style-node

Style one or more nodes. Positional arguments are element keys; named arguments are the node style applied to each of them.

```
style-node("L", "RR", fill: red, tag: [0])
```

Parameters

`style-node(...args)`

20.15. Frames and renderers

Exported flat: `make-renderer`, `render`, `overlay`, `result`. The rest is what a custom draw backend or a hand-built display talks to.

- `frame()`
- `make-canvas()`
- `make-frames()`
- `make-renderer()`
- `overlay()`
- `patch()`
- `push-frame()`
- `render()`
- `result()`
- `with-alt()`
- `with-caption()`
- `with-step()`

20.15.1. frame

Build one frame.

- `make` is `(theme, extra) => content`, where `theme` is the **full** resolved nested theme dict and `extra` the overlay commands to draw inside the canvas.
- `builder` is the field callers use: `(theme) => content`, `make` partially applied to the frame's current `extra`. overlay rebuilds it when it appends commands, which is why the frame keeps `make` around.
- `caption` is content (or none) — never a bare string.
- `step` is free-form metadata; each display documents its `step.kind` vocabulary, and every display's final frame carries `step.result`.
- `alt` is accessible text, always an explicitly-set string.

Parameters

```
frame(
  make,
  caption,
  step,
  alt,
  extra
)
```

20.15.2. **make-canvas**

Draw one snapshot of a structure and wrap it in a `cetz.canvas`.

This is the single choke point where theme references are resolved: the base node/edge style layers and every per-element style in the snapshot are run through `style.resolve-refs` against the resolved theme just before the backend sees them. That is what lets a style built long before render time (an op stream, a `styles.attention(...)` helper) follow the theme that is actually active.

`extra` is an array of additional `cetz` commands appended **inside** the same canvas, so the backend's element anchors are in scope for them (this is how `overlay` adds a callout). Each entry is either `cetz` content or a function `(theme) => cetz content`.

Parameters

```
make-canvas(
  structure,
  snapshot,
  node-style,
  edge-style,
  theme,
  draw,
  extra
)
```

20.15.3. **make-frames**

Build an array of frames from an array of **specs**.

Each spec describes one frame:

```
(
  structure: <the structure to draw for this frame>,
  build: (theme) => <snapshot>, // this frame's styling
  caption: none | content,
  step: (:), // metadata; final frame carries `result`
  alt: "",
  node-style: auto, // `auto` = the make-frames default
  edge-style: auto,
  extra: (), // cetz commands inside the canvas
)
```

Carrying the structure per spec is what lets one animation span a shape change (a rotation, a split, a chain growing) without stitching several renderers together. Carrying build per spec is what keeps the work linear: each frame builds only its own snapshot.

theme is the display's per-call override, merged over the ambient theme the builder is handed at render time.

Parameters

```
make-frames(
  specs,
  draw,
  theme,
  node-style,
  edge-style
)
```

20.15.4. make-renderer

Build a renderer seeded with one blank frame, bound to a draw backend.

This is the low-level, accumulate-as-you-go path behind the 0p command stream — and the extension point for a custom draw backend. The data structures' `renderer(..)` functions wrap it with their own backend and structural painting.

With `sticky: true` each new frame starts from the previous frame's styling, so highlights accumulate; with `sticky: false` (the default) every frame starts blank.

theme is a partial nested override merged over the ambient theme at render time.

Parameters

```
make-renderer(
  structure,
  draw,
  node-style,
  edge-style,
  sticky,
  theme
)
```


20.15.5. overlay

Append cetz commands to one frame's canvas, returning the new frame array. Use it for callouts that need the backend's element anchors:

```
#let frames = overlay(bst.search-display(t, 7), at: -1, draw: theme => {  
  import cetz.draw: line, content  
  line(anchor("LR"), (rel: (1, 1)))  
})
```

at indexes the frame array and accepts negative indices (-1 is the last frame). draw is either cetz content or a function (theme) => cetz content, so an overlay can follow the active theme.

Parameters

```
overlay(  
  frames,  
  at,  
  draw  
)
```

20.15.6. patch

Replace the in-progress frame's snapshot with fn(snapshot).

Parameters

```
patch(  
  r,  
  fn  
)
```

20.15.7. push-frame

Open a new frame. Its styling starts from the previous frame's when the renderer is sticky, blank otherwise.

Parameters

```
push-frame(r)
```

20.15.8. render

Zip a renderer into an array of frames — one per accumulated snapshot.

Parameters

```
render(r)
```

20.15.9. result

The post-operation structure a display's final frame carries in `step.result`. This is what replaces writing every operation twice — once for the frames and once to advance the variable:

```
#let frames = bst.insert-display(t, 5)
#let t = result(frames)
```

For a search or traversal the result is the unchanged input structure.

Parameters

```
result(frames)
```

20.15.10. with-alt

Set the in-progress frame's alt text.

Parameters

```
with-alt(
  r,
  a
)
```

20.15.11. with-caption

Set the in-progress frame's caption.

Parameters

```
with-caption(
  r,
  c
)
```

20.15.12. with-step

Set the in-progress frame's step metadata.

Parameters

```
with-step(
  r,
  s
)
```

20.16. Snapshots

Exported flat: `blank-snapshot`, `apply-snapshot`.

- `apply-snapshot()`
- `blank-snapshot()`
- `clear-notes()`
- `note-edge()`
- `note-node()`
- `with-edge()`
- `with-node()`

20.16.1. `apply-snapshot`

Fold styling ops into a snapshot, returning the new snapshot.

Accepts `style-node` / `style-edge` / `annotate ops` (see `core/ops.typ`) and nested arrays of them; frame-level ops (`commit`, `alt`) panic, because a snapshot has no frames to commit — fold those into a renderer with `apply-ops` instead.

Parameters

```
apply-snapshot (
  snap,
  ops
)
```

20.16.2. `blank-snapshot`

A snapshot with no overrides.

Parameters

```
blank-snapshot ( )
```

20.16.3. `clear-notes`

Drop every operation note, keeping the structural styling. Sticky animations use this between phases so notes don't pile up.

Parameters

```
clear-notes (snap)
```

20.16.4. `note-edge`

Attach an operation note to an edge.

Parameters

```
note-edge(  
  snap,  
  key,  
  note  
)
```

20.16.5. note-node

Attach an operation note to a node — the transient annotation slot drawn in-canvas beside the element.

Parameters

```
note-node(  
  snap,  
  key,  
  note  
)
```

20.16.6. with-edge

Merge style into the edge styling for key.

Parameters

```
with-edge(  
  snap,  
  key,  
  style  
)
```

20.16.7. with-node

Merge style into the node styling for key. key-styles merges index-wise (see style.merge-key-styles); everything else is replaced.

Parameters

```
with-node(  
  snap,  
  key,  
  style  
)
```

20.17. Styles and style keys

Exported flat: theme-ref, role. The node- and edge-style key allowlists below are the complete vocabulary a snapshot may use.

- [assert-any-of\(\)](#)
- [assert-edge-style\(\)](#)
- [assert-keys\(\)](#)
- [assert-node-style\(\)](#)
- [is-theme-ref\(\)](#)
- [merge-into\(\)](#)
- [merge-key-styles\(\)](#)
- [merge-style\(\)](#)
- [resolve-ref\(\)](#)
- [resolve-refs\(\)](#)
- [resolve-refs-map\(\)](#)
- [role\(\)](#)
- [strip-notes\(\)](#)
- [theme-ref\(\)](#)

Variables

- [node-style-keys](#)
- [edge-style-keys](#)

20.17.1. **assert-any-of**

Panic unless value is one of options. who names the caller.

Parameters

```
assert-any-of(
  value,
  options,
  who
)
```

20.17.2. **assert-edge-style**

Panic unless d is a valid edge-style dict.

Parameters

```
assert-edge-style(
  d,
  who
)
```

20.17.3. **assert-keys**

Panic unless every key of dict appears in allowed. who names the caller in the message (e.g. "node style", "skiplist.new").

Parameters

```
assert-keys(  
    dict,  
    allowed,  
    who  
)
```

20.17.4. assert-node-style

Panic unless d is a valid node-style dict.

Parameters

```
assert-node-style(  
    d,  
    who  
)
```

20.17.5. is-theme-ref

Whether v is a theme reference.

Parameters

```
is-theme-ref(v)
```

20.17.6. merge-into

Shallow dict merge: every key of override wins over base.

Parameters

```
merge-into(  
    base,  
    override  
)
```

20.17.7. merge-key-styles

Index-wise merge of two key-styles arrays. Position i in the result is merge-into(old.at(i), new.at(i)), so per-compartment overrides accumulate across sticky frames instead of the second call wiping out the first. Out-of-range positions on either side are the empty dict.

Parameters

```
merge-key-styles(  
  old,  
  new  
)
```

20.17.8. merge-style

Merge one style dict over another. Like merge-into, except that key-styles merges index-wise (see merge-key-styles) rather than being replaced wholesale.

Parameters

```
merge-style(  
  base,  
  override  
)
```

20.17.9. resolve-ref

Look one theme reference up in a resolved theme dict.

Parameters

```
resolve-ref(  
  ref,  
  theme  
)
```

20.17.10. resolve-refs

Replace every theme reference in a value by its value in theme. Recurses through nested dicts and arrays, so a reference inside a key-styles entry or a partial stroke dict resolves too. Values that are not references are returned untouched.

Parameters

```
resolve-refs(  
  style,  
  theme  
)
```

20.17.11. resolve-refs-map

Resolve theme references in a whole key-to-style map.

Parameters

```
resolve-refs-map(  
  styles,  
  theme  
)
```

20.17.12. role

Shorthand for the common case: a reference into the op section (the operation-semantic roles — search-stroke, attention-stroke, ...).

Parameters

```
role(key)
```

20.17.13. strip-notes

Drop the note / note-fill slots from every style in a key-to-style dict. The operation-note slot is transient; sticky animations clear it between phases while keeping the structural highlights.

Parameters

```
strip-notes(styles)
```

20.17.14. theme-ref

A placeholder for theme.<section>.<key>, resolved when the frame is drawn. Usable anywhere a style value is expected.

Parameters

```
theme-ref(  
  section,  
  key  
)
```

20.17.15. node-style-keys

The recognised node-style keys.

Honored by every backend: fill, stroke, text-fill, note, note-fill, hide, ghost, tag, label.

Tree backend only: shape ("circle" | "triangle" | "rectangle" | "btree-node"), materialize (draw an otherwise-phantom slot as a real node), key-styles (per-compartment styling for a "btree-node", an array of dicts merged index-wise across sticky frames).

Graph backend only — node geometry, all in cetz units: `r` (circle radius), `rx/ry` (ellipse/rectangle half-extents), `autosize` (fit the node to its label via `measure`), `pad-x/pad-y` (padding for that fit). The graph backend also honors shape with its own smaller dispatch (`"circle"` | `"ellipse"` | `"rectangle"/"square"`).

`ghost: true` is “invisible but space-reserving”: the element still contributes its exact normal footprint to the canvas bounds and still emits its anchors, but nothing is painted. Contrast `hide: true`, which drops the element from layout entirely (and so shifts everything around it).

20.17.16. edge-style-keys

The recognised edge-style keys.

Honored by every backend: `stroke`, `note`, `note-fill`, `tag`, `mark`, `hide`.

Tree backend only: `parent-anchor` / `child-anchor` (a cetz anchor name overriding the default fractional-distance endpoint — how an edge lands flush on a non-circular shape, e.g. `child-anchor: "north"` for a triangle’s apex) and `force-show` (draw an edge even when one endpoint is a phantom).

Graph backend only: `bend` (cetz units; curve the edge into an arc whose control point is the straight midpoint pushed perpendicular by this amount, positive = left of the $u \rightarrow v$ direction) and `label-offset` (signed cetz units; seat a straight edge’s weight/label off the line, sign picks the side).

`tag` is a small persistent annotation near the edge (drawn in the render theme’s `edge-tag-fill`) — AVL subtree heights, trie letters, graph edge weights. `note` is the transient gold operation slot.

20.18. Theme

Exported flat: `default-theme`, `set-theme`.

- [merge-theme\(\)](#)
- [resolve-theme\(\)](#)
- [set-theme\(\)](#)

Variables

- [default-theme](#)

20.18.1. merge-theme

Merge a partial theme override over base, one section at a time. The merge is two levels deep and no deeper: overriding a key replaces its value wholesale, so a sub-style dict (e.g. a `stroke`) must be given complete. Unknown sections or keys panic.

Parameters

```
merge-theme(  
  base,  
  override  
)
```

20.18.2. resolve-theme

The active theme: default-theme, with the document's set-theme overrides merged over it, with override merged over that. Pass (:) for no override. Reads state, so it must be called inside a context block — the slides.typ helpers open exactly one context per call and pass the result into every frame's builder.

Parameters

`resolve-theme`(override)

20.18.3. set-theme

Override theme keys for the rest of the document. Pass a partial nested dict — only the sections and keys you list change:

```
#set-theme((op: (search-stroke: (paint: teal, thickness: 3pt)),
             render: (note-bg: rgb("#fdf6e3"))))
```

Unknown section names or keys panic. Note that reading and writing a Typst state in one document forces a second layout pass; a per-call theme: argument on a display avoids that cost.

Parameters

`set-theme`(overrides)

20.18.4. default-theme

The full default theme. Every section is complete; per-call overrides and set-theme take partial dicts merged over this.

20.19. Drawing utilities

Exported flat: anchor. The rest is shared geometry and measurement the backends use, and what a custom backend should reuse.

- `anchor()`
- `haloed()`
- `measure-max()`
- `muted()`
- `resolve-dims()`
- `stroke-paint()`
- `text-fill-for()`

Variables

- `PAD-X`

20.19.1. anchor

The cetz element name for a snapshot key.

A "#<int>" suffix addresses one compartment of a multi-key node (B24) and becomes a cetz sub-anchor: anchor("01#1") is "el-01.key-1". Pass canvas: to qualify the name with the enclosing group's name, which is what a backend's name: argument creates.

Parameters

```
anchor(  
  key,  
  canvas  
)
```

20.19.2. haloed

A small piece of content on a filled halo, so index labels and edge tags stay legible over any connector running beneath them. theme is the full resolved theme; the halo uses theme.render.note-bg.

Parameters

```
haloed(  
  draw,  
  pos,  
  body,  
  theme,  
  anchor  
)
```

20.19.3. measure-max

The largest rendered width and height (in cetz units) over a set of cells, as (w, h). body-fn(cell) builds the cell's content; cells whose body is none are skipped. Returns (w: 0, h: 0) when nothing is measurable.

measure-cells, when non-empty, is measured **instead of** cells. An animation passes the global set of bodies across all its frames here, so every frame measures the same superset and the canvas never jumps as a running count grows wider than the widest value.

Parameters

```
measure-max(  
  cells,  
  body-fn,  
  measure-cells  
)
```

20.19.4. muted

A de-emphasized version of a theme color, for the parts of a drawing that should read second: an absent adjacency-matrix cell, an empty auxiliary structure, a rejected element's label.

Parameters

`muted(c)`

20.19.5. resolve-dims

Resolve a box footprint for one render, as (w, h) in cetz units.

The shared implementation behind the hash-map, array, and skip-list backends' cell sizing — see the mode table above. floor-w / floor-h are the fixed footprint each backend falls back to (and never shrinks below); pad-x / pad-y are added on each side of a measured body.

Parameters

```
resolve-dims(  
  cells,  
  body-fn,  
  cell-width,  
  floor-w,  
  floor-h,  
  pad-x,  
  pad-y,  
  measure-cells  
)
```

20.19.6. stroke-paint

The paint of a stroke spec — used to fill an arrowhead so it matches its line. Accepts a color, dict, or stroke; falls back to black for auto / none or a stroke with no explicit paint.

Parameters

`stroke-paint(s)`

20.19.7. text-fill-for

A readable text fill for a given background, chosen from the background's oklab lightness. Keeps a label legible across a gradient palette or a component-colored node.

Parameters

`text-fill-for(bg)`

20.19.8. PAD-X

Padding per side, in cetz units, between a label and its box edge when sizing to fit.

20.20. Alt-text helpers

The string builders every display's alt text goes through, so the narration reads the same across structures.

- `alt-describe()`
- `alt-intro()`
- `alt-key-label()`
- `alt-label()`
- `display-value()`

20.20.1. alt-describe

Alt text for a static display() frame: "<Structure>: <describe>."

Parameters

```
alt-describe(  
    ds-name,  
    describe-text  
)
```

20.20.2. alt-intro

Alt text for the first frame of an operation: "<Structure>: <describe>. About to <action>."

Every display's opening frame goes through this, so a screen-reader user gets the whole structure once as a baseline and only the per-step change thereafter.

Parameters

```
alt-intro(  
    ds-name,  
    describe-text,  
    action  
)
```

20.20.3. alt-key-label

The n-ary case: a node carries parallel keys / labels arrays, so a reference names one compartment *i* (B24).

Parameters

```
alt-key-label(  
    node,  
    i  
)
```

20.20.4. alt-label

The binary-tree case of display-value: a node carrying a value / label pair (BST, RBT, AVL).

Parameters

`alt-label` (node)

20.20.5. display-value

The string used to refer to an element in captions and alt text.

An element visually displays its label whenever that is a plain string (the backends fall back to the ordering value only for auto), so text referring to it should read the same way. auto and arbitrary content can't be spliced into a string, so those fall back to the ordering value.

value-key names the field holding that ordering value — "value" for the trees, "key" for the sort and skip-list element dicts.

Parameters

```
display-value(  
    elem,  
    value-key  
)
```

20.21. Draw backends

Each of these is exported flat (`draw-tree`, `draw-graph`, ...) and can be called inside a `cetz.canvas` of your own, with no context needed.

20.21.1. draw-tree

- `draw-tree()`

20.21.2. draw-tree

Emit the `cetz` draw commands for one styled snapshot of `tree`, *without* wrapping them in a `cetz.canvas` — the caller owns the canvas, which is what lets you add your own annotations alongside the tree and anchor them with `anchor(<path>)`.

Shape dispatch is on the node: a `terminal` field means a trie, a `children` field means an n-ary tree, otherwise `left` / `right` means a binary tree.

Parameters

```
draw-tree(  
    tree: dictionary,  
    snapshot: dictionary,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary,  
    name: none str,  
    grow: float,  
    spread: float  
) -> content
```

tree dictionary

The tree to render. Three accepted shapes:

- Binary: value, label, left, right. A label of auto falls back to `str(value)`; any other value (string, image, content) is drawn as the node's label.
- N-ary: keys (array of key values), optional labels (parallel array of overrides), and children (empty for leaves). Pair it with `shape: "btree-node"` in `node-style` for the canonical subdivided rectangle.
- Trie: char, terminal, children.

snapshot dictionary

Style overlay for this snapshot; `blank-snapshot()` draws it unstyled.

node-style dictionary

Base node styling, applied beneath the snapshot's per-node styles.

Default: `(:)`

edge-style dictionary

Base edge styling, applied beneath the snapshot's per-edge styles.

Default: `(:)`

theme dictionary

The full resolved theme. The backend reads `theme.render`.

Default: `default-theme`

name `none` or `str`

Wrap the whole tree in a `cetz` group of this name, qualifying every element anchor under it — `anchor(path, canvas: name)`. `none` draws into the enclosing canvas directly.

Default: `none`

grow float

Depth-direction layout factor passed to `cetz-tree`. Below 1 shortens the edges between levels, above 1 stretches them. Node sizes are unaffected.

Default: `1`

spread `float`

Sibling-direction layout factor passed to `cetz-tree`. Below 1 pulls siblings together, above 1 pushes them apart. Node sizes are unaffected.

Default: `1`

20.21.3. `draw-graph`

- `draw-graph()`
- `edge-key()`

20.21.4. `draw-graph`

Emit the `cetz` draw commands for one styled snapshot of a positioned graph, *without* wrapping them in a `cetz.canvas` — the caller owns the canvas, so you can add your own annotations and anchor them with `anchor(<id>)` or `anchor(edge-key(u, v))`.

Edges are drawn first and the filled node glyphs on top, so the line ends are cleanly occluded by the nodes. Each edge's intrinsic weight/label is drawn near its midpoint in `theme.render.edge-tag-fill`; a snapshot edge tag overrides that text, and a snapshot note (gold) draws an operation annotation on the edge. Directed graphs get filled arrowheads (the mark fill follows the edge stroke) unless an edge sets its own mark; filling them keeps a mutual pair from showing each line through the other's tip. A self-edge (`u == v`) draws as a teardrop loop above the node rather than a zero-length line.

Two edge-style keys shape an edge's geometry. `bend` (`cetz` units, positive = left of the `u->v` direction) curves it into an arc; giving a mutual directed pair the same `bend` fans the two arcs to opposite sides, so they read as distinct edges instead of overlapping into one line. `label-offset` (signed `cetz` units, default `0.12`) seats a **straight** edge's weight/label off the line: the sign picks the side (same convention as `bend`) and the magnitude the gap. A bent edge places its label on the convex side and ignores `label-offset`.

Parameters

```
draw-graph(  
  pg: dictionary,  
  snapshot: dictionary,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary,  
  name: none str  
) -> content
```

pg `dictionary`

The positioned graph — see the module header for its shape.

snapshot `dictionary`

Style overlay for this snapshot; `blank-snapshot()` draws it unstyled.

node-style dictionary

Base node styling, applied beneath the snapshot's per-node styles.

Default: (:)

edge-style dictionary

Base edge styling, applied beneath the snapshot's per-edge styles.

Default: (:)

theme dictionary

The full resolved theme. The backend reads `theme.render`.

Default: `default-theme`

name `none` or `str`

Wrap the whole graph in a `cetz` group of this name, qualifying every element anchor under it — `anchor(id, canvas: name)`. `none` draws into the enclosing canvas directly.

Default: `none`

20.21.5. edge-key

The canonical snapshot key for the edge between `u` and `v`.

A directed graph keys edges “`u->v`” (order significant); an undirected graph sorts the endpoints into “`u-v`”, so the same edge has one key however it is named.

Parameters

```
edge-key(  
    u: str,  
    v: str,  
    directed: bool  
) -> str
```

u `str`

First endpoint id.

v `str`

Second endpoint id.

directed `bool`

Whether the graph is directed.

Default: `false`

20.21.6. draw-hashmap

- `cell-key()`
- `draw-hashmap()`
- `entry-key()`

20.21.7. cell-key

The canonical snapshot key for the array cell (slot) at index `i`.

Parameters

`cell-key(i: int) -> str`

`i` `int`

Slot index.

20.21.8. draw-hashmap

Emit the `cetz` draw commands for one styled snapshot of a hash table, *without* wrapping them in a `cetz.canvas` — the caller owns the canvas, so you can add your own annotations and anchor them with `anchor(cell-key(i))` or `anchor(entry-key(i, j))`.

The array of cells is drawn first, then (for chaining) the chains hanging off each bucket, then the hash-box overlay on top. Each cell's structural state comes from the table dict (empty / occupied / tombstone / chain); per-frame highlights come from the snapshot — the probe walk in `search-stroke`, the landing in `success-fill`, and so on.

Parameters

```
draw-hashmap(  
    tbl: dictionary,  
    snapshot: dictionary,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary,  
    name: none str  
) -> content
```

`tbl` `dictionary`

The positioned table — see the module header for its shape.

snapshot dictionary

Style overlay for this snapshot; `blank-snapshot()` draws it unstyled.

node-style dictionary

Base cell styling, applied beneath the snapshot's per-cell styles.

Default: `(:)`

edge-style dictionary

Base link styling, applied beneath the snapshot's per-link styles.

Default: `(:)`

theme dictionary

The full resolved theme. The backend reads `theme.render`, `theme.hashmap`, and `theme.op.attention-stroke` (the hash-box arrow).

Default: `default-theme`

name `none` or `str`

Wrap the whole table in a `cetz` group of this name, qualifying every element anchor under it — `anchor(cell-key(i), canvas: name)`. `none` draws into the enclosing canvas directly.

Default: `none`

20.21.9. entry-key

The canonical snapshot key for the chaining entry at depth `j` in bucket `i` (`0` = the head entry). Also the key of the link **into** that entry.

Parameters

```
entry-key(  
    i: int,  
    j: int  
) -> str
```

i int

Bucket (slot) index.

j `int`

Depth in the chain (0 = head).

20.21.10. draw-array

The backend behind `sort`; `sort.cell-key` and `sort.entry-key` are re-exported from here.

- `cell-key()`
- `draw-array()`
- `entry-key()`

20.21.11. cell-key

The canonical snapshot key for the cell at column `col` of row `row` — `cell-key("count", 5)` is `"count:5"`.

Parameters

```
cell-key(  
  row: str,  
  col: int  
) -> str
```

row `str`

Row id.

col `int`

Column index within the row.

20.21.12. draw-array

Emit the `cetz` draw commands for one styled snapshot of a multi-row array, *without* wrapping them in a `cetz.canvas` — the caller owns the canvas, so you can add your own annotations and anchor them with `anchor(cell-key(row, col))`.

Arrows are drawn first, so the cells drawn afterwards occlude the line where it enters a box and leave a clean arrowhead in the inter-row gap. Then come the cell rectangles, then a re-stroke pass for highlighted cells (contiguous cells share edges, so a neighbour's border can clip a highlight — redraw it on top), then the row and index labels.

Parameters

```
draw-array(  
  tbl: dictionary,  
  snapshot: dictionary,  
  node-style: dictionary,  
  edge-style: dictionary,  
  theme: dictionary,  
  name: none or str  
) -> content
```

tbl dictionary

The positioned table — see the module header for its shape.

snapshot dictionary

Style overlay for this snapshot; `blank-snapshot()` draws it unstyled.

node-style dictionary

Base cell styling, applied beneath the snapshot's per-cell styles.

Default: `(:)`

edge-style dictionary

Base arrow styling, applied beneath the snapshot's per-arrow styles.

Default: `(:)`

theme dictionary

The full resolved theme. The backend reads `theme.render` and `theme.sort`.

Default: `default-theme`

name none or str

Wrap the whole table in a `cetz` group of this name, qualifying every element anchor under it — `anchor(cell-key(row, col), canvas: name)`. `none` draws into the enclosing canvas directly.

Default: `none`

20.21.13. entry-key

The canonical snapshot key for the chain entry at depth j ($0 = \text{the head}$) below bucket header i of row row — the “buckets” row kind. `entry-key("buckets", 2, 0)` is `"buckets:2:0"`. The entry box is styled through the snapshot’s nodes slot, just like a cell; its three parts are what distinguish it from a two-part `cell-key`.

Parameters

```
entry-key(  
  row: str,  
  i: int,  
  j: int  
) -> str
```

row `str`

Row id (a “buckets” row).

i `int`

Bucket (column) index within the row.

j `int`

Depth in the chain ($0 = \text{head}$).

20.21.14. draw-skiplist

- `box-key()`
- `data-key()`
- `draw-skiplist()`
- `forward-key()`

20.21.15. box-key

The canonical snapshot key for the lane box at level of column `col` ($0 = \text{header}$, $1..n = \text{data}$, $n+1 = \text{nil}$) — `box-key(2, 0)` is `"b2:0"`. Styled through the snapshot’s nodes slot like any node.

Parameters

```
box-key(  
  col: int,  
  level: int  
) -> str
```

col `int`

Column index ($0 = \text{header}$).

level `int`

Level within the tower (0 = bottom).

20.21.16. data-key

The canonical snapshot key for the DATA box of column `col` — the box holding the node's key, drawn once **below** the whole tower, where no forward pointer attaches. `data-key(2)` is "d2".

Parameters

`data-key(col: int) -> str`

col `int`

Column index (0 = header).

20.21.17. draw-skiplist

Emit the `cetz` draw commands for one styled snapshot of a skip list, *without* wrapping them in a `cetz.canvas` — the caller owns the canvas, so you can add your own annotations and anchor them with `anchor(box-key(col, level))`.

The draw order is layered so the occlusion tells the linked/unlinked story:

1. the UNLINKED lane boxes — a data column's boxes at levels outside its [`link-min`, `link-height`) range — in the muted palette;
2. the forward pointers, so they run OVER those grayed boxes: the cue that the list **skips past** such a node rather than stopping on it;
3. the linked lane boxes, the header/nil sentinels, the data boxes, and the ghost reserves, which occlude the pointer line ends into clean arrowheads (a linked box's intact border versus a grayed box's pointer-crossed one is itself a cue);
4. a re-stroke pass for highlighted lane boxes, so a just-unlinked grayed box keeps its ring above the pointer running through it;
5. the labels, on top, so nothing occludes a key.

Parameters

```
draw-skiplist(  
    tbl: dictionary,  
    snapshot: dictionary,  
    node-style: dictionary,  
    edge-style: dictionary,  
    theme: dictionary,  
    name: none str  
) -> content
```

tbl dictionary

The positioned table — see the module header for its shape.

snapshot dictionary

Style overlay for this snapshot; `blank-snapshot()` draws it unstyled.

node-style dictionary

Base box styling, applied beneath the snapshot's per-box styles.

Default: `(:)`

edge-style dictionary

Base pointer styling, applied beneath the snapshot's per-pointer styles.

Default: `(:)`

theme dictionary

The full resolved theme. The backend reads `theme.render` and `theme.skiplist`.

Default: `default-theme`

name `none` or `str`

Wrap the whole list in a `cetz` group of this name, qualifying every element anchor under it — `anchor(box-key(c, l), canvas: name)`. `none` draws into the enclosing canvas directly.

Default: `none`

20.21.18. forward-key

The canonical snapshot key for the forward pointer LEAVING column `col` at level — `forward-key(0, 2)` is `"f0:2"`. Its target, the next node present at that level, is derived by the backend. Styled through the snapshot's edges slot like a graph edge.

Parameters

```
forward-key(  
    col: int,  
    level: int  
) -> str
```


col `int`

Source column index.

level `int`

Level of the pointer (0 = bottom).

20.22. Graph auto-layout

Exported flat: auto-layout.

- [auto-layout\(\)](#)

20.22.1. auto-layout

Compute node positions for `g` with graphviz and return a `(id: (x, y))` map in cetz units, suitable for the positions: argument of any Graph *-display method (or Graph.positioned).

diagraph-layout returns node centers in points (y-up, matching cetz); each coordinate is divided by unit to convert to cetz units. The default 36pt (½ inch) gives spacing that suits the renderer's radius-0.6 nodes; lower it to spread the graph out, raise it to pack it tighter. Edge spline points from graphviz are ignored — starling draws straight edges.

sizes controls how much room graphviz reserves per node: none uses graphviz's default node size (fine for short, circular nodes); auto estimates each node's box from its label's character count so wide labels get spread apart; a `(id: (width, height))` dict supplies explicit lengths. The auto estimate is intentionally rough — it is computed eagerly (no measure, so auto-layout stays callable outside a context) while draw-graph sizes the drawn node **exactly** by measuring the label. The two only approximate each other, but graphviz's node margins absorb the slack; nudge unit/the display's scale: if spacing looks off. The *-display methods pass sizes: auto automatically when you use their layout: argument.

Parameters

```
auto-layout(  
    g: Graph,  
    engine: str,  
    unit: length,  
    sizes: none auto dictionary  
) -> dictionary
```

g Graph

The Graph to lay out. Its positions (if any) are ignored; only nodes, edges, and directedness are used.

engine `str`

graphviz layout engine — "dot" (layered, the default), "neato", "fdp", "sfdp", "circo", or "twopi".

Default: "dot"

unit `length`

Points per cetz unit used to scale graphviz's output.

Default: `36pt`

sizes `none` or `auto` or dictionary

Per-node box sizing fed to graphviz: none (graphviz default), auto (estimate from label length), or a (id: (width, height)) dict of explicit lengths.

Default: `none`

20.23. git — the commit-graph DSL

- `background-lanes()`
- `branch()`
- `branch-pointer()`
- `checkout()`
- `commit()`
- `detached-commit()`
- `git-graph()`
- `git-highlight()`
- `head-pointer()`
- `merge()`
- `tag()`

20.23.1. background-lanes

Rule a dashed lane along every branch, from its label to just past the last commit — the backdrop that makes which lane a commit sits on obvious. Call it after the history, so every branch exists. Styled by the theme's `git.lane-style`.

Parameters

`background-lanes()`

20.23.2. branch

Start a branch and check it out. The first call must come before the first commit; later ones fork from the current tip (or from `from:`, a cetz anchor). `color:` pins this branch's colour, `colors:` replaces the palette it is drawn from (the theme's `git.colors` by default), and `edge-name:` names the fork line for later annotation.

Parameters

```
branch(  
    name,  
    color,  
    colors,  
    edge-name,  
    from  
)
```

20.23.3. branch-pointer

Draw a branch pointer — the branch's name in its own colour — at the tip of name. It mirrors the commit message through the dot: the same tilt, but extending away from it. The label registers the cetz anchor `branch-pointer-<name>`, which is what `head-pointer` stacks against.

Parameters

```
branch-pointer(  
    name,  
    padding,  
    anchor,  
    angle  
)
```

20.23.4. checkout

Make branch the active one, so subsequent commits land on its lane.

Parameters

```
checkout(branch)
```

20.23.5. commit

Draw a commit on the active branch, with message beside its dot. `branch`: checks that branch out first; `name`: gives the dot a cetz anchor name of your choosing (otherwise it is `commit-id-<n>`), which is what later annotations and merge refer to.

Parameters

```
commit(  
    message,  
    branch,  
    name,  
    edge-name  
)
```

20.23.6. detached-commit

Draw an orphan commit dot offset from from (a cetz anchor — a commit-id-<n> or a commit's own name:), joined back to it by a line in the same colour (grey by default, to read as unreachable). The dot is registered as name, so head-pointer(target: name) can hang a detached HEAD off it. Unlike commit it joins no lane, advances no counter, and is not a merge target.

Parameters

```
detached-commit(  
  from,  
  message,  
  name,  
  offset,  
  edge-name,  
  color  
)
```

20.23.7. git-graph

The block every other verb is called inside: it seeds cetz's canvas context with the resolved theme and layout defaults, then draws graph. Place it directly in a cetz.canvas.

theme: is a partial nested theme override (theme: (git: (..))), layered over the document's theme like any display's. ..style takes the behavioural knobs — direction: ("bottom-to-top", the default, or "left-to-right"), commit-spacing:, lane-spacing: — which are deliberately not theme. name: wraps the drawing in a named cetz group so its anchors are reachable from outside, at the cost of swallowing touying pause markers inside the body.

Parameters

```
git-graph(  
  graph,  
  name,  
  theme,  
  ..style  
)
```

20.23.8. git-highlight

Redraw a commit's dot in fill, on a layer above the original, to pick it out. Pair it with touying's alternatives(..) to flip a highlight on and off across subslides.

Parameters

```
git-highlight(  
  commit-name,  
  fill  
)
```

20.23.9. head-pointer

Draw the HEAD label at the tip of `branch`: (default: the active one). It stacks past that branch's pointer label when one was drawn; pass `after: none` to anchor it at the dot instead, or `after: "<branch>"` to stack past a different branch's pointer. Pass `target: (any cetz anchor name)` to point HEAD at an arbitrary commit — a detached HEAD.

Parameters

```
head-pointer(  
  branch,  
  target,  
  after,  
  padding,  
  anchor,  
  angle  
)
```

20.23.10. merge

Merge `commit-id` — a branch name, or a commit's name: — into the active branch, drawing the merge commit and the incoming edge. `fast-forward: true` draws the tip as a fast-forward instead of a merge dot.

Parameters

```
merge(  
  commit-id,  
  message,  
  name,  
  edge-name,  
  fast-forward  
)
```

20.23.11. tag

Hang a tag label off the current tip, in the theme's `git.tag-style`.

Parameters

```
tag(message)
```